

סיכום למידת מכונה - תשפ"ו 2026

28 ביולי 2026

גיא יער-און

הסיכום נכתב במהלך סמסטר ב' תשפ"ו (2026), ומבוסס על הרצאותיהם של פרופסור גל צצ'יק, ד"ר אורי שוהם וד"ר אורן גליקמן, לכן יתכן שנפלו טעויות בסיכום, כך שהשימוש בו על אחריותכם בלבד.

תוכן עניינים

4	1	הרצאה 1	1
4	1.1	רקע מתמטי לקורס	
8	1.2	הגרדיאנט (<i>Gradient</i>)	
10	2	הרצאה 2	2
10	2.1	למידה מונחית (<i>Supervised learning</i>)	
12	2.1.1	איך נוכל להפחית <i>overfitting</i> ?	
12	2.2	תאוריית ההחלטה של בייס - <i>Bayes decision theory</i>	
16	2.3	<i>The Bias-Variance Trade-off</i>	
18	2.4	<i>Overfitting</i> ו <i>Underfitting</i>	
19	2.5	<i>Cross Validation</i>	
19	3	הרצאה 3	3
19	3.1	Parameter estimation (הערכת פרמטרים)	
20	3.2	Maximum Likelihood (ML)	
21	3.3	Maximum A Posteriori (MAP) (הסתברות פוסטרירורית מקסימלית)	
22	3.4	Bayesian Estimation (הערכה בייאסיאנית)	
23	3.5	Loss-Based View	
24	3.6	סיכום	
24	3.7	PAC Learning	
25	3.7.1	מטרת העל של תהליך הלמידה	
25	3.7.2	Realizability	
26	3.7.3	Empirical Risk Minimization (ERM)	
26	3.7.4	Version Space and Bad Hypotheses	
28	3.7.5	אי שוויון הופדינג	
29	3.7.6	הטרייד אוף בין השונות לביאס	
30	4	הרצאה 4	4
30	4.1	מבוא	
30	4.2	רגרסיה לינארית: הגדרת הבעיה	
30	4.3	הגדרה מטריציונית	
31	4.4	מציאת הפתרון האופטימלי	
32	4.5	$\hat{y}$ הוא הטלה אורתוגונלית ובפרט ממזער את השגיאה	
33	4.6	רגרסיה פולינומיאלית	
33	4.7	Ridge Regression	
34	4.8	בפרספקטיבה של <i>SVD</i>	
35	4.9	Binary classification	
35	4.10	Linear-threshold neuron	
36	4.11	The perceptron algorithm (אלגוריתם הפרספטרון)	

37	Gradient Descent	4.12	
38	$\mathbb{R}^d$ במימד גבוה GD	4.13	
38	Stochastic Gradient Descent (SGD)	4.14	
38	הטריידאוף בין GD ל SGD	4.15	
39	Linear separability	4.16	
40	Correct Classification - הצלחה בסיווג	4.17	
40	דוגמאות	4.18	
41	Hinge loss function ו SGD באמצעות	4.19	
41	רגרסיה לוגיסטית	4.20	
44	מדוע רגרסיה לוגיסטית נחשבת למסווג לינארי?	4.21	
44	תרגול 4	4.22	
47	רגרסיה לינארית עם GD	4.23	
47	הרצאה 5	5	
47	הקדמה	5.1	
47	vs-All-1 (אחד כנגד כולם) - פתרון נאיבי	5.2	
48	Softmax regression	5.3	
49	הגדרה מטריציונית ל $softmax$	5.4	
49	פרקטיקה - איזה בעיות יהיו לנו?	5.5	
50	כמה המודל שלנו טוב?	5.6	
50	Cross Entropy Loss	5.7	
50	כלל העדכון ל SGD	5.8	
52	SVM - הקדמה	5.9	
52	Maximum margin	5.10	
54	Hard SVM	5.11	
54	The penalty method	5.12	
56	Soft SVM	5.13	
57	סיכום	5.14	
57	אופטימיזציה תחת אילוצים	5.15	
58	maximum entropy	5.16	
59	Evaluation Metrics (מדדי הערכה)	5.17	
60	הרצאה 6	6	
60	SVM kernels	6.1	
61	הגדרה פורמלית	6.2	
61	ומה עושים לאחר ההתמרה?	6.3	
62	The curse of explicit mapping (קללת המיפוי המפורש)	6.4	
62	Kernel Trick	6.5	
63	The radial Basic function (RBF)	6.6	
64	מתי קרנל נחשב חוקי?	6.7	
64	Decision Trees	6.8	
66	כיצד מוצאים את עץ ההחלטה הכי טוב?	6.9	
66	אלגוריתם ה $TDIDT$ ($ID3$)	6.10	
67	בחירת הפיצ'ר הכי טוב - Antropy	6.11	
68	Information Gain - כלל הבחירה	6.12	
68	CART Algorithm	6.13	
68	סיכום	6.14	
69	Boosting	6.15	
69	אלגוריתם $XGBoost$	6.15.1	
72	Naïve Bayes	6.16	
72	(K nearest neighbors) KNN	6.17	
73	הרצאה 7 - Deep Learning	7	
73	Multi Layer Perceptrons	7.1	
75	פונקציות אקטיבציה לא לינאריות וגזירות	7.1.1	
75	פונקציות הפסד	7.1.2	
75	פעפוע לאחור של הגרדיאנט (Backpropagation)	7.1.3	

76	 Convolution Neural networks	7.2	
77	 Max and Average pooling	7.2.1	
78	 Multilayer perceptron	7.3	
80	 הרצאה 8	8	
80	 שיפור האופטימיזציה וההכללה	8.1	
80	 BatchNorm	8.2	
82	 ResNet	8.3	
83	 אתחול המשקולות	8.3.1	
83	 Generalization in deep networks (הכללה ברשתות עמוקות)	8.4	
84	 Implicit Regularization - הרחבת נתונים	8.4.1	
84	 MixUp - Implicit Regularization	8.4.2	
85	 CNN - תרגול	8.5	
85	 Flatten	8.6	
86	 Weight Decay	8.7	
86	 Optimizers - אלגוריתמי אופטימיזציה	8.8	
86	 SGD - אופטימיזר ראשון	8.8.1	
87	 Momentum - אופטימיזר שני	8.8.2	
87	 AdaGrad - אופטימיזר שלישי	8.8.3	
87	 RMSprop - אופטימיזר רביעי	8.8.4	
88	 Adam - אופטימיזר חמישי	8.8.5	
88	 AdamW - אופטימיזר שישי	8.8.6	
88	 הרצאה 9 -	9	
88	 Unsupervised Learning	9.1	
89	 Clustering	9.2	
90	 אלגוריתם K-Means (1957)	9.3	
92	 Gaussian Mixture Model (GMM) (מודל תערובת גאוסיאנים)	9.4	
93	 EM אלגוריתם ה	9.4.1	
94	 Principal Component Analysis (PCA)	9.5	
100	 Autoencoder	9.6	
100	 תרגול - PCA	9.7	
101	 PCA סיווג ו	9.8	
101	 הרצאה 10	10	
101	 Self-supervised learning	10.1	
102	 תרגול- Autoencoders	10.2	
103	 סוגים של Autoencoders	10.2.1	
104	 Diffusion Models	10.3	
105	 DropOut	10.4	
106	 הרצאה 11	11	
106	 Word Embeddings	11.1	
108	 Transformers	11.2	
108	 Tokenization: המעבר מקלט טוקונים לוקטורים - כקלט לEncoder	11.2.1	
109	 Self-Attention - הוקטורים שיצאו מהטוקוניזציה נכנסים למנגנון זה.	11.2.2	
110	 Feed Forward Layer - שלב עיבוד לאחר הAttention	11.2.3	
110	 Add and Norm	11.2.4	
111	 בעיות נוספות בתהליך	11.2.5	
111	 סיכום	11.2.6	
112	 הרצאה 12	12	
112	 Biases, Fairness	12.1	

1 הרצאה 1

1.1 רקע מתמטי לקורס

הגדרה 1. תהי $A \in \mathbb{R}^{d \times d}$. הוקטורים $v \in \mathbb{R}^d$ $0 \neq v$ כך שהפעלת A עליהם לא תשנה את הכיוון שלהם נקראים **וקטורים עצמיים**. כלומר, מתקיים $Av = \lambda v$, כך ש $\lambda \in \mathbb{C}^d$ נקרא **ערך עצמי** ול $\frac{v}{\|v\|}$ נקרא וקטור עצמי המתאים ל λ .

הערה 2. תמיד ננרמל את הוקטורים העצמיים.

הערה 3. בפועל, וקטור עצמי הוא וקטור שהפעלת המטריצה עליו רק משנה את גודל הוקטור, ולא את הכיוון. המטריצה משנה את הגודל בהתאם ל λ , הערך העצמי.

הערה 4. באופן כללי: כאשר מטריצה A פועלת על וקטור v , מתקבל הוקטור Av . A משנה את v באופן לינארי: סיבוב + הזזה.

משפט 5. תהי $A \in \mathbb{R}^{d \times d}$ סימטרית. ויהי λ ע"ע של A . אזי, λ בהכרח ממשי.

הערה 6. לכל מטריצה $A \in \mathbb{R}^{d \times d}$ ישנם d ע"ע מעל $\mathbb{C}$ (הם לא בהכרח שונים זה מזה, ויכולים להיות מרוכבים).

הגדרה 7. המכפלה הסקלרית הסטנדרטית בהינתן $v_1 = (x_1, \dots, x_n), v_2 = (y_1, \dots, y_n) \in \mathbb{R}^n$ מוגדרת כך:

$$\langle v_1, v_2 \rangle = v_1^T v_2 = \sum_{i=1}^n x_i y_i$$

הגדרה 8. יהיו שני וקטורים $v, u \in \mathbb{R}^n$.

א. נאמר כי u, v הם **אורתוגונליים** אם, ורק אם $\langle v, u \rangle = v^T u = 0$
 ב. נאמר כי u, v הם **אורתונורמליים** אם הם אורתוגונליים וגם $\sqrt{\sum_{i=1}^n v_i^2} = 1$ ו $\sqrt{v^T v} = \|v\|$ $\forall v$.

משפט 9. תהי $A \in \mathbb{R}^{d \times d}$ סימטרית, ויהיו $v_1, v_2 \in \mathbb{R}^d$ ו $0 \neq v_1, v_2$ השייכים לע"ע המתאימים $\lambda_1 \neq \lambda_2$. אזי, v_1 ו v_2 אורתוגונליים (ולכן אורתונורמליים) זה לזה.

הוכחה: מתקיים $Av_1 = \lambda_1 v_1 \wedge Av_2 = \lambda_2 v_2$ ולכן:

$$\lambda_2 v_1^T v_2 = v_1^T \lambda_2 v_2 =_{Av_2 = \lambda_2 v_2} v_1^T Av_2 =_{A=A^T} v_1^T A^T v_2 = (Av_1)^T v_2 = (\lambda_1 v_1)^T v_2 = v_1^T \lambda_1 v_2$$

$$\implies (\lambda_1 - \lambda_2) v_1^T v_2 = 0$$

כיוון ש $\lambda_1 \neq \lambda_2$ הרי ש $\lambda_1 - \lambda_2 \neq 0$ ולכן $v_1^T v_2 = 0$ כלומר $\langle v_1, v_2 \rangle = 0$, ולכן v_1, v_2 אורתוגונליים. כנדרש.

■

משפט 10. תהי $A \in \mathbb{R}^{d \times d}$ סימטרית. אזי,

א. A לכסינה.

ב. קיים לה בסיס אורתונורמלי של ו"ע של $\mathbb{R}^d$.

משפט 11. תהי $A \in \mathbb{R}^{d \times d}$ סימטרית בעלת הע"ע $\{\lambda_i\}_{i=1}^n$ כך ש $\forall 1 \leq i \leq n : \lambda_i \geq 0$, אזי לכל וקטור $z \in \mathbb{R}^d$ מתקיים $z^T A z \geq 0$.

הוכחה: A סימטרית ולכן קיים לה בסיס אורתונורמלי של וקטורים עצמיים (לפי משפט לעיל), נסמנו $\{v_1, \dots, v_d\}$. מכאן, כל וקטור z ניתן לכתוב כצירוף לינארי של הו"ע כך: $z = \sum_{i=1}^d c_i v_i$, באשר $\{c_i\}_{i=1}^d$ המקדמים. כעת:

$$Az = A \times \sum_{i=1}^d c_i v_i = \sum_{i=1}^d c_i \times Av_i = \sum_{i=1}^d c_i \lambda_i v_i$$

כעת, נעבור לביטוי $z^T A z$:

$$z^T A z = \left(\sum_{i=1}^d c_i v_i \right)^T \left(\sum_{j=1}^d c_j \lambda_j v_j \right) \implies \sum_{i=1}^d \sum_{j=1}^d c_i c_j \lambda_j (v_i^T v_j)$$

לפי הגדרת בסיס אורתונורמלי, אם $i \neq j$ הרי ש $v_i^T v_j = 0$ ואחרת $v_i^T v_i = 1$ (נורמלי). כלומר, כל האיברים בהם $i \neq j$ מתאפסים ונשאר עם האיברים בהם $i = j$, ולכן:

$$\Rightarrow \sum_{i=1}^d c_i^2 \lambda_i \geq 0$$

שכן, $\{\lambda_i\}_{i=1}^n \geq 0$, ולכן סה"כ קיבלנו $z^T A z \geq 0$, כנדרש.

■

הגדרה 12. תהי $A \in \mathbb{R}^{d \times d}$. אזי,

1. נאמר כי A היא PSD (Positive Semi Definite) אם לכל $z \in \mathbb{R}^d$ מתקיים $z^T A z \geq 0$
2. נאמר כי A היא PD (Positive Definite) אם לכל $z \in \mathbb{R}^d$ מתקיים $z^T A z > 0$

משפט 13. תהי $A \in \mathbb{R}^{d \times d}$ סימטרית. אזי, כל הע"ע של A הם אי שליליים אם A היא PSD . בדומה, כל הע"ע של A הם חיוביים ממש אם A היא PD .

הוכחה:

$\Leftarrow$ לפי משפט 10.

$\Rightarrow$ יהי λ ע"ע עם ו"ע תואם לו v המקיים $\|v\| = 1$ (מנורמל). אזי,

$$0 \leq v^T A v = v^T \lambda v = \lambda v^T v = \lambda \|v\|^2 = \lambda \cdot 1 = \lambda$$

כנדרש.

■

14. ÷ä14. מסקנה ממשפט 13: מטרית PSD היא גם סימטרית, וגם כל הערכים העצמיים שלה אי שליליים.

15. äöäâ. תהי מטריצה $A \in \mathbb{R}^{n \times n}$. נאמר כי A אורתוגונלית אם $A^T A = A A^T = I$.

16. äöäâ. בפועל, במטריצה אורתוגונלית הכוונה כי כל זוג שורות ועמודות במטריצה הם אורתוגונליים זה לזה ואורתונורמליים (מאונכים, בגודל וקטור יחידה).

17. הערה. אם נפעיל וקטור על מטריצה אורתוגונלית וגם PSD - מה יקרה? נזכר כי הע"ע של מטריצה אורתוגונלית הם ± 1 . כיוון שהיא PSD , אזי רק 1 הוא ערך עצמי. לכן לא יקרה כלום.

משפט 18. (כלל בייס). יהיו A, B זוג מאורעות. אזי, $Pr(A|B) = \frac{Pr(B|A)Pr(A)}{Pr(B)}$

הגדרה 19. יהיו 2 משתנים מקריים, X ו Y . כך ש X מקבל ערכים $x_1, \dots, x_k$ עם ההסתברות $Pr(X = x_i)$ ו Y מקבל ערכים $y_1, \dots, y_k$ עם ההסתברות $Pr(Y = y_i)$. אזי, נגדיר את ההתפלגות המשותפת שלהם $Pr(X = x_i, Y = y_i)$. נוכל להכליל זאת עבור d משתנים מקריים $\{X_i\}_{i=1}^d$, כך שכל X_i מקבל ערכים $X_i^1, \dots, X_i^{k_i}$ וההתפלגות המשותפת שלהם הינה:

$$Pr(X_1 = x_1, \dots, X_d = x_d)$$

כמו כן, נוכל להגדירו כמשתנה מקרי יחיד רב ממדי, וקטור $X = (X_1, \dots, X_d)$ ולשאול מתי הוא מקבל את הערך $(x_1, \dots, x_d)$.

הגדרה 20. נגדיר את התוחלת של משתנה מקרי רב ממדי להיות וקטור רב ממדי של התוחלות כך:

$$\mu = \mathbb{E}[X] = (\mathbb{E}[X_1], \dots, \mathbb{E}[X_d])$$

הגדרה 21. יהיו X, Y זוג משתנים מקריים. נגדיר את $Covariance$ שלהם להיות השונות המשותפת שלהם, בצורה הבאה:

$$Cov(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$$

מדד זה בודק כמה ישנו קשר בניהם, בפרט אם $Cov(X, Y) > 0$ אזי ישנו קשר חיובי בין השניים (עולים יחד), אם $Cov(X, Y) < 0$ אזי ישנו קשר שלילי בין השניים (אחד עולה, השני יורד) ואם $Cov(X, Y) = 0$ אזי אין קשר לינארי בניהם.

משפט 22. כל ההבאים מתקיימים:

א. $Cov(X, X) = Var(X)$

ב. $Cov(X, Y) = Cov(Y, X)$

ג. $Var(X + Y) = Var(X) + Var(Y) + 2Cov(X, Y)$

ד. אם X, Y ב"ת אזי הם בלתי מתואמים.

הגדרה 23. נגדיר את מטריצת ה- $Covariance$ באופן הבא:

$$\begin{pmatrix} Cov(X, X) & Cov(X, Y) \\ Cov(Y, X) & Cov(Y, Y) \end{pmatrix}$$

ונבחין כי היא מטריצה סימטרית.

הגדרה 24. יהי $X = (X_1, \dots, X_d)$ משתנה מקרי רב ממדי, נגדיר את מטריצת ה- $Covariance$ כך:

$$\Sigma = Cov(X) = \begin{pmatrix} Cov(X_1, X_1) & \dots & \dots & Cov(X_1, X_d) \\ & Cov(X_2, X_2) & & \\ \vdots & & \ddots & \vdots \\ Cov(X_1, X_{d-1}) & & & Cov(X_d, X_d) \end{pmatrix} = \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])^T]$$

הערה 25. לכל $i, j \in [n]$ מתקיים כי $\Sigma_{ij} = Cov(X_i, X_j)$ כאשר בתא זה של המטריצה ישנו ערך ה- $Covariance$ בין שני המשתנים.

הערה 26. המטריצה Σ היא מטריצה סימטרית, לכן היא לכסינה אורתוגונלית, כמו כן, המטריצה היא PSD ולכן כל הע"ע שלה הם אי שליליים לפי משפט שראינו קודם.

משפט 27. מטריצת Σ היא PSD , כלומר לכל וקטור $z \in \mathbb{R}^d$ שונה מאפס מתקיים $z^T \Sigma z \geq 0$.

הוכחה:

$$z^T \Sigma z =_{def} z^T \mathbb{E}[(X - \mu)(X - \mu)^T] z$$

כעת, כיוון ש- z הוא וקטור קבוע, הרי מתקיים עבור קבוע $\mathbb{E}[aX] = a\mathbb{E}[X]$ ותכונה זו נכונה גם עבור וקטור קבוע, ולכן נוכל להכניסו:

$$= \mathbb{E}[z^T (X - \mu)(X - \mu)^T z]$$

נבחין כי $z^T (X - \mu)$ הוא מכפלה סקלרית (וקטור שורה, כפול וקטור עמודה), לכן תוצאתו סקלר. הביטוי השני, הוא בדיק טרנספוז של סקלר זה. כיוון שטרנספוז של סקלר שווה לסקלר עצמו, הרי שנוכל לכתוב כי:

$$= \mathbb{E}[(z^T (X - \mu))(z^T (X - \mu))^T] = \mathbb{E}[(z^T (X - \mu))^2] \geq 0$$

שהרי המעבר האחרון נובע מכך שאם m הוא אי שלילי, גם התוחלת שלו היא אי שלילי. כנדרש. ■

הגדרה 28. יהי $X \sim N(\mu, \sigma^2)$ משתנה מקרי גאוסיאני (נורמלי), אזי פונקציית הצפיפות שלו מקיימת:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2} \cdot \left(\frac{x-\mu}{\sigma}\right)^2}$$

הגדרה 29. יהי X וקטור רב ממדי, כך ש- $X \in \mathbb{R}^d$, המתפלג גאוסיאני $X \sim N(\mu, \Sigma)$, אזי פונקציית הצפיפות שלו מקיימת:

$$f(x) = \frac{1}{(2\pi)^{\frac{d}{2}} \sqrt{\det(\Sigma)}} e^{-\frac{1}{2} \cdot (x-\mu)^T \cdot \Sigma^{-1} \times (x-\mu)}$$

באשר x הוא וקטור הקלט, μ הוא וקטור התוחלת, Σ היא מטריצת ה- $Covariance$ (קובעת את "צורת" הפעמון), $\det(\Sigma)$ היא הדטרמיננטה של המטריצה. הביטוי במעריך מחליף את הריבוע בנוסחה המקורית, והוא מודד כמה רחוק x מהמרכז μ , אך הוא מתחשב ב- $Covariance$. מרחק זה נקרא Mahalanobis Distance.

הערה 30. מאוד חשוב לשים לב - משתנה רב ממדי מתפלג נורמלית כאשר Σ מייצגת את השונות, נבחין כי למשתנה רב ממדי אין שונות כערך מספרי בודד, אלא מטריצת $Covariance$ שעל האלכסון מייצגת את השונות של הערכים $\{X_i\}_{i=1}^d$.

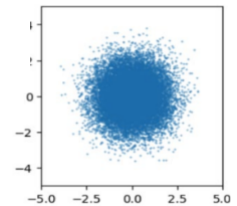
ישנם כמה מקרים פרטיים שנרצה לדון בהם:

1. **מקרה ראשון** - $\Sigma = I$: במקרה זה יתקיים,

$$f(x) = \frac{1}{(2\pi)^{\frac{d}{2}}} e^{-\frac{1}{2} \cdot \|X - \mu\|^2}$$

כאן כל השונות שוות לאחד, וכן כל זוג מימדים $x_i \neq x_j$ בלתי מתואמים, כלומר $Cov(X_i, X_j) = 0$. לבסוף, קיבלנו כי ההסתברות של הוקטור הנ"ל תלויה אך ורק במרחק הגאומטרי הישיר שלה מהמרכז (μ) . יש צפיפות גבוהה מאוד סביב התוחלת, ואם נחפש את כל הנקודות שיש להן את אותה הסתברות נקבל את המשוואה $\|X - \mu\|^2 = c$ באשר במרחב דו ממדי מדובר במעגל, ובמרחב רב ממדי על כדור.

לדוגמה: נקודות שנלקחות מתוך התפלגות המקיימת $\Sigma = I$, ענן הנקודות נראה כמו עיגול סימטרי, הסיבה היא כי על האלכסון יש רק 1 ולכן השונות של x שווה בדיוק לשונות של y : הפיזור לגובה שווה לפיזור לרוחב. כיוון שהערכים מחוץ לאלכסון הם אפס, אין קשר בין המשתנים, לכן הם בלתי מתואמים ולכן ענן הנקודות לא נוטה באלכסון. כמו כן, רוב הנקודות קרובות ל $(0,0)$, כיוון שכאן מתקיים $\mu = (0,0)$.



2. **מקרה שני** - Σ היא מטריצה אלכסונית: כלומר: $\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & \dots & 0 \\ \vdots & \sigma_2^2 & & \vdots \\ 0 & & \ddots & 0 \\ 0 & \dots & 0 & \sigma_d^2 \end{pmatrix}$ נפתח אלגברית מעט:

$$f(x) = \frac{1}{(2\pi)^{\frac{d}{2}} \sqrt{\det(\Sigma)}} e^{-\frac{1}{2} \cdot (x-\mu)^T \cdot \Sigma^{-1} \times (x-\mu)} = \frac{1}{(2\pi)^{\frac{d}{2}} \sqrt{\prod_{i=1}^d \sigma_i^2}} e^{-\frac{1}{2} \cdot (x-\mu)^T \cdot \begin{pmatrix} \frac{1}{\sigma_1^2} & 0 & \dots & 0 \\ \vdots & \frac{1}{\sigma_2^2} & & \vdots \\ 0 & & \ddots & 0 \\ 0 & \dots & 0 & \frac{1}{\sigma_d^2} \end{pmatrix} \times (x-\mu)}$$

$$= \prod_{i=1}^d \frac{1}{\sqrt{2\pi\sigma_i^2}} \cdot \exp\left(-\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right)$$

נתעכב להסביר את החלק הכי פחות טריוואלי, המעריך. הרי $x - \mu$ הוא וקטור, נוכל לסמן את ערכיו על מנת שיהיו לנו חיים קלים ע"י

$$\begin{pmatrix} \frac{1}{\sigma_1^2} & 0 & \dots & 0 \\ \vdots & \frac{1}{\sigma_2^2} & & \vdots \\ 0 & & \ddots & 0 \\ 0 & \dots & 0 & \frac{1}{\sigma_d^2} \end{pmatrix} \times (x - \mu) = \begin{pmatrix} \frac{x_1 - \mu_1}{\sigma_1^2} \\ \vdots \\ \vdots \\ \frac{x_d - \mu_d}{\sigma_d^2} \end{pmatrix}, \text{ מכאן, } \{x_i - \mu_i\}_{i=1}^d$$

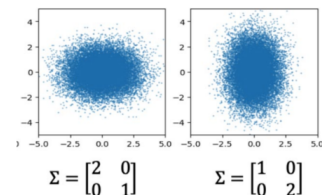
שמקבל את הערכים $\{x_i - \mu_i\}_{i=1}^d$, מכאן מדובר במכפלה סקלרית:

$$(x_1 - \mu_1, \dots, x_d - \mu_d) \times \begin{pmatrix} \frac{x_1 - \mu_1}{\sigma_1^2} \\ \vdots \\ \vdots \\ \frac{x_d - \mu_d}{\sigma_d^2} \end{pmatrix} = \prod_{i=1}^d \frac{(x_i - \mu_i)^2}{\sigma_i^2}$$

לאחר מכפלה ב $-\frac{1}{2}$, מקבלים את הדרוש.

שוב, תהיה צפיפות סביב μ גבוהה מאוד, אך הפעם בודו מימד מדובר בצורת אליפסה (ולא במעגל) וברב מימד צורות מעניינות אף יותר ככל שהמימד ימשיך לגדול.

דוגמה: כאן, אכן מדובר במטריצה אלכסונית. בדוגמה משמאל נראה כי שונות x היא 2, והשונות בציר y היא 1 ולכן ציר x נראה מתוח יותר לצדדים וקיבלנו אליפסה אופקית. בדוגמה מימין, זה בדיוק הפוך, קיבלנו אליפסה אנכית כיוון שהשונות בציר y גדולה מהשונות בציר x .



משפט 31. יהי $X = (X_1, \dots, X_d)$ משתנה מקרי רב ממדי גאוסיאני $X \sim N(\mu, \Sigma)$. אזי, $X_i | X_j$ גאוסיאן (כלומר, גם אם יודעים ערך של אחד מ $\{X_i\}_{i=1}^d$ אזי עדיין המשתנה הרב ממדי ממימד קטן יותר הוא גאוסיאן, וכן $\int_{-\infty}^{\infty} f_X(x_1, \dots, x_d) dx_1$ גאוסיאן גם כן).

משפט 32. אם מטריצת Σ Covariance היא מטריצה אלכסונית (כלומר לכל $i \neq j$ מתקיים $Cov(X_i, X_j) = 0$) אזי $\{X_i\}_{i=1}^n$ הם ב"ת.

הערה 33. לא כטענה אך לשים לב: כל תזוזה לינארית (כולל הכפלה במטריצה וכו') של גאוסיאן היא גאוסיאן.

1.2 הגרדיאנט (Gradient)

הערה 34. מהו הגרדיאנט? מדובר בוקטור, ש"זוכר" את כל הנגזרות החלקיות של פונקציה בכמה משתנים. במקום להסתכל רק על כיוון אחד, הגרדיאנט מייצג את השיפוע של הפונקציה בכל הכיוונים בבת אחת. הוא אומר לנו באיזה כיוון הפונקציה עולה הכי מהר ובאיזו עוצמה. ופורמלית:

הגדרה 35. תהי פונקציה $f(x_1, x_2, \dots, x_d)$ המוגדרת בתחום $f: \mathbb{R}^d \rightarrow \mathbb{R}$. נגדיר את הגרדיאנט של f כך:

$$\nabla f(x) = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_d} \right]$$

הגדרה 36. תהי f פונקציה, נגדיר את **קצב השינוי של הפונקציה** f בנקודה x לאורך כיוון u **כנגזרת הכיוונית** של f בכיוון u (כאשר u הוא וקטור יחידה כלומר $\|u\| = 1$) כך:

$$D_u f(x) = \lim_{h \rightarrow 0} \frac{f(x + hu) - f(x)}{h}$$

משפט 37. תהי $f: \mathbb{R}^d \rightarrow \mathbb{R}$ פונקציה גזירה בנקודה $x \in \mathbb{R}^d$. אזי, קצב השינוי של הפונקציה f בנקודה x בכיוון וקטור יחידה u הוא מקסימלי כאשר u בכיוון הגרדיאנט.

הוכחה: תהי f פונקציה גזירה, אזי הקירוב עבורה מסדר ראשון הינו:

$$f(x + hu) \approx f(x) + \nabla f(x) \cdot (hu) + \varepsilon(h)$$

באשר $\varepsilon(h)$ מסמלת שארית ששואפת לאפס כאשר $h \rightarrow 0$. מכאן, נציב בהגדרת הנגזרת הכיוונית:

$$D_u f(x) = \lim_{h \rightarrow 0} \frac{f(x) + \nabla f(x) \cdot (hu) + \varepsilon(h) - f(x)}{h} = \lim_{h \rightarrow 0} \frac{\nabla f(x) \cdot (hu) + \varepsilon(h)}{h} =$$

$$\lim_{h \rightarrow 0} (\varepsilon + \nabla f(x) \cdot u) = \nabla f(x) \cdot u$$

מסקנה: קצב השינוי של הפונקציה בכיוון u הוא המכפלה הסקלרית $\nabla f(x) \cdot u$. נעזר באי שוויון שורץ: $a \cdot b \leq \|a\| \cdot \|b\|$ ונקבל:

$$D_u f(x) = \nabla f(x) \cdot u \leq \|\nabla f(x)\| \cdot \|u\|$$

אנו מניחים כי u וקטור יחידה והרי $\|u\| = 1$ ומכאן:

$$D_u f(x) \leq \|\nabla f(x)\|$$

כלומר, הערך המקסימלי של $D_u f(x)$ חסום ע"י $\nabla f(x)$ ויש שוויון לפי א"ש שזורץ רק כאשר הוקטורים תלויים זה בזה, מכאן: הפונקציה משתנה הכי מהר בכיוון $\nabla f(x)$.

■

משפט 38. נגדיר את הפונקציה $f(x) = x^T x = \|x\|^2$, אזי, $\nabla f(x) = 2x$.
הוכחה: נבחין כי אם נסמן $x = (x_1, \dots, x_d)$ הרי שמתקיים לכל $1 \leq i \leq d$ כי:

$$\frac{\partial}{\partial x_i} x^T x = \frac{\partial}{\partial x_i} \left(\sum_{j=1}^d x_j^2 \right) = 2x_i$$

$$\nabla f(x) = (2x_1, \dots, 2x_d) = 2x, \text{ ולכן,}$$

■

הגדרה 39. מטריצת הסיאן (*Hessian*) היא מטריצה ריבועית שמגדירה את הנגזרות החלקיות מסדר שני. כלומר,

$$H_f = \nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial^2 x_1} & & \frac{\partial^2 f}{\partial x_1 \partial x_d} \\ \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1} & \dots & \dots & \frac{\partial^2 f}{\partial x_d^2} \end{pmatrix}$$

משפט 40. נגדיר את התבנית הריבועית $f(x) = \frac{1}{2} x^T A x$ כאשר A היא מטריצה סימטרית ממיד d . אזי, $\nabla^2 f = A$.
הוכחה: נשלב ראשון נראה כי:

$$\frac{\partial f}{\partial x_k} \left(\frac{1}{2} \cdot \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j \right) =$$

$$\frac{\partial f}{\partial x_k} \left(\frac{1}{2} \left(\sum_{i, j \neq k} a_{ij} x_i x_j + \sum_{i=k, j \neq k} a_{kj} x_k x_j + \sum_{j=k, i \neq k} a_{ik} x_i x_k + a_{kk} x_k^2 \right) \right) =$$

$$\frac{1}{2} \left(\sum_{j \neq k} a_{kj} x_j + \sum_{i \neq k} a_{ik} x_i + 2a_{kk} x_k \right) = \frac{1}{2} \left(\sum_{i=1}^d a_{ik} x_i + \sum_{j=1}^d a_{ki} x_i \right) = \sum_{i=1}^d a_{ik} x_i = (Ax)_k$$

המעבר * נכון כיוון ש A סימטרית ולכן $a_{ik} = a_{ki}$. מכאן,

$$\nabla f = \begin{bmatrix} (Ax)_1 \\ (Ax)_2 \\ \vdots \\ (Ax)_d \end{bmatrix} = Ax$$

$$H_f = \nabla^2 f = \frac{\partial^2 f}{\partial^2 x} (\nabla f(x)) = \frac{\partial^2 f}{\partial^2 x} (Ax) = A, \text{ ומכאן,}$$

■

הגדרה 41. פונקציה היא קמורה אם הקטע שמחבר כל זוג נקודות על גרף הפונקציה נמצא מעל או על הגרף עצמו. פורמלית, לכל $0 \leq t \leq 1$ ולכל x, y :

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$$

הערה 42. אנחנו נעדיף פונקציות הפסד קמורות, כיוון שבהן כל מינימום מקומי הוא בהכרח מינימום גלובלי. כלומר - אם מצאת נקודה בה השיפוע הוא אפס (נקודת מינימום מקומית), היא בהכרח תהיה הנקודה הנמוכה ביותר בכל הפונקציה.

משפט 43. תהי f פונקציה גזירה פעמיים בתחום קצור. f היא פונקציית קמורה אם ורק אם מטריצת ההסיאן שלה $H_f(x)$ היא PSD.

2 הרצאה 2

2.1 למידה מונחית (Supervised learning)

המטרה שלנו תהיה לנסות ללמוד איזושהי "חוקיות" מהעולם. בשביל שנוכל ללמוד חוקיות - אנו זקוקים למידע, $data$.

נתוני האימון - (Training Data): בהינתן n זוגות של דגימות מהצורה $\{(x_i, y_i)\}_{i=1}^n$, כך ש $x_i \in X, y_i \in Y$, $\forall 1 \leq i \leq n$, כך ש x_i הוא הקלט שלקוח מעולם קלטים X וכן y_i הוא הפלט (או התווית) מתוך עולם של פלטים שנקרא Y . לדוגמה: x_i הוא תמונה, y_i יכול להיות התיאור "חתול". ישנה הנחת יסוד לפיה קיימת התפלגות כלשהי $P(X, Y)$ אינה ידועה לנו שממנה מגיעים זוגות אלו. התפלגות זו מייצגת פונקציה $f: X \rightarrow Y$ שאותה נרצה למצוא.

המטרה: נרצה למצוא פונקציה $h: X \rightarrow Y$ שתקרא היפותזה, היא תהיה ההיפותזה שלנו: היא "תחקה" את הפונקציה הנסתרת f בצורה הכי טובה שאפשר. לשם כך נגדיר את המדד הבא -

הגדרה 44. מדד ההפסד ($Loss$): נגדיר מדד הפסד ונסמנו $\ell(h(x), y)$ - מדד זה מודד חוסר התאמה בין מה שחזינו ($h(x)$) לבין המציאות (y). בפועל, נחפש פונקציה h שתביא למינימום את Expected Loss - הממוצע המשוקלל של הטעויות שלנו על פני כל המקרים האפשריים לפי ההתפלגות $P(x, y)$ ומוגדר כך:

$$\mathbb{E}_{P(X,Y)}[\ell(h(x), y)] = \sum_{(x,y)} \ell(h(x), y) \cdot p(x, y)$$

הגדרה 45. (מרחב ההיפותזות): נסמן ב $\mathcal{H}$ את מרחב ההיפותזות - מדובר במשפחה מוגדרת של פונקציות שממנה אנחנו "בוחרים" את פונקציית ההפסד. כל פונקציה $h \in \mathcal{H}$ במרחב זה מאופיינת ע"י פרמטרים שנקראים w, θ אנו מחפשים $h \in \mathcal{H}$ הטובה ביותר שקיימת במרחב ההיפותזות.

את תהליך הלמידה נוכל לחלק לשני חלקים מרכזיים, נפרדים ומשלמים:

1. **Training Phase (שלב האימון):** זהו השלב שבו המחשב "לומד". אנו נותנים לאלגוריתם הלמידה את נתוני האימון שלנו - זוגות $\{(x_i, y_i)\}_{i=1}^n$. כפלט, האלגוריתם מוציא מודל אותו נסמן ב h_w . המשמעות היא שה h_w מייצג את הפרמטרים שהאלגוריתם מצא כדי שהפונקציה h תתאים בצורה הטובה ביותר לנתונים.

2. **inference Phase (שלב ההסקה/יישום):** זהו השלב שבו אנחנו משתמשים במה שלמדנו על נתונים חדשים שלא ראינו קודם לכן. אנו לוקחים את המודל הנלמד h_w ומפעילים אותו על קלט חדש x , אנו חוזים את y המתאים למעשה ע"י חישוב פשוט של הפונקציה $h_w(x)$.

הערה 46. בעיות של למידה מונחית מחולקות לרוב לפי סוג הפלט (Y). כאשר סוג הפלט הוא משתנה קטגורי בדיד נקרא לבעיה מסוג Classification (בעיה מסוג סיווג). כאשר סוג הפלט הוא משתנה רציף נקרא לבעיה מסוג Regression (רגרסיה).

לצורך דוגמה, נוכל לבחור באופן ספציפי את מרחב ההיפותזות $\mathcal{H}$ להיות פולינומים מדרגה d . פורמלית, נגדיר את $h_w(x)$ כסכום ממושקל של חזקות x , כך:

$$h_w(x) = w_0 + w_1x + w_2x^2 + \dots + w_dx^d$$

כאשר w 'ים (המשקולות) הם הפרמטרים של המודל שאנו מחפשים בשלב האימון. למה שנרצה לבחור את משפחת הפולינומים? מדובר בסט עשיר מאוד - שיכול לקרב כמעט כל פונקציה שהיא רציפה בצורה טובה. כלומר, הקירוב יהיה טוב. נוכל למצוא פרמטרים $\{w_i\}_{i=1}^d$ שיהיו קרובים מספיק לכלל. כעת, לאחר שבידנו (למשל) פונקציה כ"ל $h_w(x)$. כיצד נוכל לבדוק שהיא אכן מתאימה לנתונים שבידינו?

הגדרה 47. נגדיר מדד הפסד ($Loss$) בשם **Squared Loss** (**הפסד ריבועי**) שיעבוד בצורה הבאה: עבור דגימה בודדת (x_i, y_i) נגדיר את **ההפסד האמפירי** (**Empirical Loss**) ע"י הפונקציה:

$$\ell(w, x_i, y_i) = (h_w(x_i) - y_i)^2$$

ההפרש בניהם מועלה בריבוע על מנת שכל טעות תהיה חיובית, ועל מנת לתת קנס" על טעויות גבוהות יותר. נגדיר את **מודל הטעות הכולל** (**The Overall Error**) כממוצע הטעויות הריבועיות, ונסמן פונקציה זו ב- $Err(w)$ (או ב- MSE):

$$Err(w) = \frac{1}{n} \sum_{i=1}^n (h_w(x_i) - y_i)^2$$

כיצד נמדוד איכות של פתרון? נרצה לבדוק כמה המודל (ההיפותזה $\mathcal{H}$) מתאימה לנתונים. ראשית נתבונן במקרה פשוט, בו המודל הוא מדרגה $d = 0$. דהיינו, $h_w(x) = w_0$. נראה כי הפרמטר של המודל הוא יחיד w_0 . נרצה לפתור את בעיית האופטימיזציה הבאה:

$$\min_{w_0} \{Err(w_0)\} = \min_{w_0} \left\{ \frac{1}{n} \sum_{i=1}^n (w_0 - y_i)^2 \right\}$$

נראה שמדובר בפונקציה ריבועית ב- w_0 , ונחפש את המינימום שלה. לכן נגזור לפי w_0 :

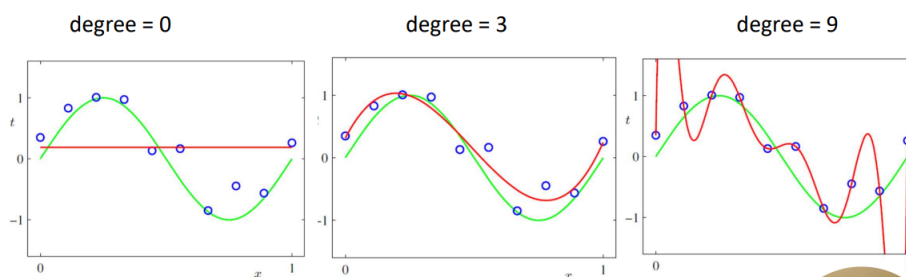
$$\frac{\partial Err(w)}{\partial w_0} = \frac{1}{n} \sum_{i=1}^n 2(w_0 - y_i) \Rightarrow$$

נשווה לאפס, ונקבל: $\sum_{i=1}^n (w_0 - y_i) = 0$, כלומר קיבלנו $n \cdot w_0 - \sum_{i=1}^n y_i = 0$ ולכן:

$$w_0 = \frac{\sum_{i=1}^n y_i}{n}$$

כלומר, קיבלנו כי **הערך האופטימלי עבור מודל מדרגה 0 הוא הממוצע האריתמטי של כל $\{y_i\}_{i=1}^n$** .

נראה, כי בסבירות גבוהה שמודל מדרגה 0 לא יביא למינימום את $Loss$ של המודל. לכן, מה שנרצה הוא להמשיך ולהעלות את חזקת הפולינום. בצורה הבאה:



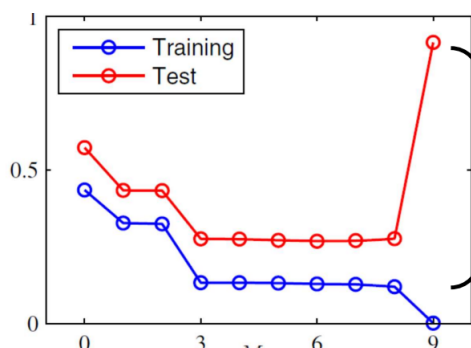
נבחין כי פולינום מדרגה 0 מביא שגיאה שונה מפולינום מדרגה גבוהה מדי (כמו 9). פולינום מדרגה 0 מביא פשוט ערך קבוע - סוג השגיאות שהוא עושה הן קבועות, נראה כי לכל x משמאל הוא חוזה ערך נמוך מדי, ולכל x מימין הוא חוזה ערך גדול מדי. ישנה הטיה ($Bias$). עם זאת, השגיאות במעלה גבוהה הרבה יותר מעניינות - המודל מביא שגיאה מזערית. אך למה אינטואיטיבית הוא מרגיש לנו מודל "לא טוב"? כי אנחנו רואים שהקו האדום שונה מהקו הירוק. נבחין כי אנחנו נקבל במקרה זה שונות גבוהה מאוד! יתרה מזאת - השגיאות שהמודל הזה עושה תלויות ברעש של הדגימה (כמה "רעש" נכנס לנתונים). מה הבעיה? המודל הנ"ל למד יותר מדי את הרעש מהנתונים - הוא משנן את הרעש שיש בנתונים, הוא מסוגל לחזות את הנתונים שהוא כבר ראה באופן מושלם, אבל זה נובע מכך שהוא שינן ($Overfitting$) את הרעש ולא רק את המידע עצמו. זו תופעה מעניינת מאוד - שחושפת לנו את ההבדל בין שגיאת האימון לבין סוג אחר של שגיאה - **Generalization Error**: "טעות ההכללה" שמוגדרת כך:

$$Err_{GEN}(w) = \mathbb{E}_{P(x,y)}[\ell(h_w; x, y)]$$

אנחנו הרי לא מעוניינים להקטין את $Loss$ על סט הנקודות עליהם אנחנו "מתאמנים", אלא נרצה להקטין את תוחלת ה- $Loss$ על כלל ערכי (x, y) מתוך ההתפלגות. הרי - המטרה שלנו היא שהוא יצליח לחזות היטב לא רק את הנקודות שאנחנו נתנו לו, אלא כל זוג נתונים שהוא יקבל בעתיד מתוך $\mathcal{H}$. כלומר, אם נסכם:

אנו לא מכירים את $P(x, y)$ ולכן לא יכולנו למזער את כלל האוכלוסייה - זה שניסינו למזער את נתוני האימון (הערכים שידועים לנו) מוביל לשתי בעיות מרכזיות:

1. **ביצועים ירודים** - המודל עלול ללמוד "רעש" או תכונות ספציפיות שקיימות רק ב- n הדגימות שהבאנו לו ולא קיימות בחוקיות האמיתית של העולם.
2. **אופטימיות יתר** - טעות האימון תהיה תמיד נמוכה יותר מטעות ההכללה, אנו חושבים שהמודל שלנו מצויין כיוון שהוא חוזה מעולה את נתוני האימון אך הוא עלול להכשל על נתונים חדשים. כלומר הבעיה שלנו היא שהביצועים שלנו גרועים - ואנחנו לא יודעים זאת!



נתבונן בדוגמה הבאה בהמשך לדוגמה על הפולינומים:

נראה כי ישנן 2 סוגי טעויות. טעות המבחן מסומנת באדום, ושגיאת האימון בכחול. בעוד שטעות האימון תמיד תמיד לרדת ככל שהמודל נעשה מורכב יותר (במקרה זה, עם הפולינומים!), טעות המבחן תתחיל לעלות בשלב מסוים - העליה הזו בטעות המבחן היא הסימן לכך שהמודל הפסיק ללמוד חוקיות כללית ופשוט התחיל לשנן את נתוני האימון. כלומר: טעות המבחן עוזרת לנו לזהות *overfitting*.

הערה 48. אם נניח כי למודל היו 10 נקודות: $\{(x_i, y_i)_{i=1}^{10}\}$, אזי לפולינום מדרגה 9 ישנן 10 משקולות: כלומר, יש לנו 10 נקודות ועלינו למצוא 10 מקדמים - לכן אפשר למצוא ממש את הפולינום עצמו, ולכן נקבל בגרף כי שגיאת האימון כאשר דרגת הפולינום 9 הוא אפס, כי הצלחנו לחזות את הפולינום בדיוק.

Test: כפי שאמרנו, נרצה שהמודל שלנו ידע להתמודד עם נתונים שלא ראה. זהו שלב המבחן שלו. פעמים רבות נחזיק $data_{test}$, דטה שמיועד למען בדיקה של המודל. אנחנו משתמשים במבחן על מנת להעריך כיצד המודל יתפקד במציאות.

2.1.1 איך נוכל להפחית *overfitting*?

נציע כמה וכמה פתרונות.

1. **להוסיף עוד $data$** : כשיש לנו מעט דגימות, "רעש" מקרי יכול להראות כמו חוקיות אמיתית. ככל שנוסיף יותר דוגמאות הרעש יתחיל להתקזז והחוקיות האמיתית תבלוט יותר. יתרה מזאת - אם נתן למודל 3 נקודות הפולינום יוכל להתפתל בניהן באלפי דרכים מוזרות. אם נתן לו 1000 נקודות, הוא יהיה חייב "להתיישר" לפי המגמה הכללית של כולן כדי לא לצבור שגיאה גדולה מדי.

2. **Regularization**: כאשר אנו מבצעים *overfitting* המודל מנסה לעבור בדיוק דרך כל נקודת רעש בנתוני האימון. כדי לעשות את היגזים האדירים הללו המקדמים של הפולינום הופכים להיות גדולים מאוד בערכם המוחלט. לכן, הרעיון הוא להוסיף מעין קנס למודל על כך שמקדמיו גבוהים מדי. אנו משנים את פונקציית המטרה שלנו כך שהיא לא רק תנסה למזער את הטעות, אלא גם תדאג לשמור על המקדמים קטנים. כך:

$$Err(w) = \sum_{i=1}^n (h_w(x_i) - y_i)^2 + \frac{\lambda}{2} \cdot ||w||^2$$

החלק השני הוא למעשה $||w||^2 = w^T w = \sum_{i=1}^d w_i^2$. החלק הנ"ל מעוניין שהמקדמים יהיו כמה שיותר קרובים לאפס. תפקיד ה- λ הוא להיות "הבורר": אם λ קטן מאוד, אנו כמעט ולא קונסים את המודל - הוא ימשיך לעשות *overfitting* על מנת להוריד את $Loss$. אם λ גדול מאוד - הקנס הופך להיות ממש דומיננטי: המודל יפחד לקבוע מקדמים גבוהים ויעדיף לקבל טעות אימון קצת גבוהה יותר, העיקר שהמקדמים ישארו קטנים.

2.2 תאוריית ההחלטה של בייס - Bayes decision theory

לרוב בלמידת מכונה, נרצה לנחש את הקשר בין הקלט לפלט, x ו- y מתוך נתונים ידועים. נניח לרגע ויש לנו ידע אלוהי - ובו אנו יודעים בדיוק את פונקציית ההסתברות המשותפת של x ו- y . כלומר אנו יודעים את $D = P(x, y)$. בהינתן ידע מלא על הבעיה, מהי האסטרטגיה האופטימלית לפעולה? כלומר, איך נחליט החלטות שיביאו לכמה שפחות טעויות.

ואפילו מקרה מעניין יותר: אנו יודעים את ההתפלגות - אך איננו יודעים את הפרמטרים של ההתפלגות. למשל, אנו יודעים כי הנתונים שלנו מתפלגים נורמלית אך לא יודעים את μ, σ^2 . לכן: נרצה לבצע הערכה (estimate) של הפרמטרים.

איך לא, נתחיל בדוגמה. דילמה: אתה שוקל האם לקחת היום מטרייה, או שלא. אתה לא יודע בוודאות אם ירד או שלא ירד היום גשם! **זוהי דילמה.** מצד אחד, גשם+אין מטרייה: אתה תרטיב ותהיה מעוצבן. מצד שני, אין גשם+יש מטרייה: אתה תהיה מעוצבן קצת (כי סחבת מטרייה לחינם). אך, נניח וקיבלת שבריר של מידע. הסתכלת דרך החלון וראית עננים שחורים - כעת החלטת: בעקבות התצפית, אקח מטרייה. כמובן שמטריות לא מעניינות אותנו. נרצה להכליל דוגמה זו **להסקה סטטיסטית**. בהינתן $data$, דגימה אמפירית, נרצה להכליל ממנה ולהסיק על "העולם". נפרמל זאת:

הגדרה 49. על מנת לתאר את תאוריית ההחלטה של ביס נגדיר:

1. **world states:** "מצבי עולם", מצבים $\{\omega_i\}_{i=1}^n$ שהם התרחישים האפשריים במציאות. מצבים אלו זרים, כלומר לכל $i \neq j \in [n]$ מתקיים $\omega_i \neq \omega_j$ וכן אם Ω הוא מרחב שלנו אזי $\Omega = \bigcup_{i=1}^n \omega_i$.
2. **תצפיות אמפיריות:** $S_n = \{x_1, \dots, x_n\}$. דגימות אמפיריות אשר אספנו מהשטח. n מייצג את מס' הדגימות.
3. **מודל הסתברותי:** יש לנו ידע מלא על ההתפלגויות. בפרט, $P(\omega)$ - ההסתברות לפני שראינו בכלל את הנתונים (למשל: מהי ההסתברות שירד גשם ביום זה), וכן $P(S_n|\omega)$ - הסיכוי לראות את התצפיות הללו, באשר שהעולם נמצא במצב מסוים (למשל: מה הסיכוי לראות עננים שחורים אם אכן ירד גשם?).
4. **פעולות אפשריות:** קבוצה $A = \{\alpha_1, \dots, \alpha_k\}$ של החלטות אפשריות שאנו יכולים לקבל. (קודם לכן בדוגמה, α_1 היא לקחת מטרייה ו- α_2 היא שלא לקחת).
5. **עלויות:** נסמן $\Lambda = \{\lambda(\alpha_k|\omega_j)\}$ - המדד שקובע כמה נשלם על ההחלטה שלנו. כלומר, המחיר של נקיטת פעולה α_k כאשר המצב האמיתי בעולם הוא ω_j . למשל: $\lambda(Umbrella|rain)$ יהיה בעל מחיר גדול מאוד: העצבים שנרטבים ואין מטרייה.

משפט 50. יהי ω מצב כלשהו בעולם Ω שאינו ידוע. תהי x תצפית מהשטח. אזי, לפי חוק ביס מתקיים:

$$\underbrace{Pr(\omega|x)}_{\text{Posterior}} = \frac{Pr(x|\omega)}{Pr(x)} \cdot \underbrace{Pr(\omega)}_{\text{Prior (belief)}}$$

הסבר הנוסחה:

- $Pr(\omega)$: ה"אמונה", זהו הידע הקודם. זוהי ההסתברות של המצב ω עוד לפני שראינו את הנתונים.
 - $Pr(x|\omega)$: זו ה"נראות", כלומר, בהינתן שאנו יודעים כי העולם במצב ω מה הסיכוי לקבל x .
 - $Pr(x)$: מדובר בראייה. זוהי ההסתברות לראות את התצפית x ללא קשר למצב העולם כרגע.
 - $Pr(\omega|x)$: מדובר ביעד הסופי, ה"הסתברות" $Pr(\omega|x)$: ההסתברות כי מצב העולם הוא ω לאחר שכבר ראינו את התצפית x . כלומר, אחרי שראית את התצפית (העננים), מה הסיכוי שבאמת יהיה גשם?
- הערה 51.** זהו המקום שבו אנחנו מגלים שנוסחה בהסתברות כל כך תמימה כל כך חשובה - אנו למעשה לוקחים ידע קודם, משלבים אותו עם נתונים חדשים ($Likelihood$) ומקבלים מסקנה מעודכנת.
- מהי המטרה שלנו?** מטרתנו לתכנן אסטרטגיה, שתסומן כפונקציה $\alpha(S_n)$. כקלט: היא מקבלת את התצפיות שלנו, וכפלט היא קובעת איזו פעולה α_i באשר $i \in [k]$ עלינו לבצע. הקריטריון להצלחה - היא תביא למינימום של עלויות/הפסדים. כלומר - נרצה כמה שפחות נזקים בממוצע בהתחשב במחירי הטעות λ .

הגדרה 52. הסיכון המותנה (Conditional risk) מסומן כ- $R(\alpha_k|x)$ הוא העלות המצופה שלנו אם נבחר בפעולה α_k לאחר שראינו את התצפית x . מתקיים:

$$R(\alpha_k|x) = \mathbb{E}_{P(\omega|x)}[\lambda(\alpha_k|\omega)] = \sum_{\omega_j \in \Omega} \lambda(\alpha_k|\omega_j) Pr(\omega_j|x)$$

הסבר: מדובר סה"כ במעבר על כל מצבי העולם, חישוב מחיר הפעולה בהינתן המצב הספציפי הזה, ומכפילים בהסתברות של מצב העולם הזה בהינתן שאנחנו יודעים את התצפית x .

נבחין כי בנוסחה מוזכר $Pr(\omega_j|x)$. כיצד נחשבו? כלומר - בהינתן התצפית x , מהי ההסתברות למצב העולם ω_j ? נוכל למצוא את ההסתברות באמצעות חוק ביס ונוסחת ההסתברות השלמה:

$$Pr(\omega_j|x) = \frac{Pr(x|\omega_j)}{Pr(x)} \cdot Pr(\omega_j) = \frac{Pr(x|\omega_j) \cdot Pr(\omega_j)}{\sum_{\omega_i \in \Sigma} Pr(x|\omega_i) \cdot Pr(\omega_i)}$$

כעת, יש לנו מס' סיכון לכל פעולה α_k אפשרית. כיצד נבחר את האסטרטגיה האופטימלית? ובכן, נגדיר:

$$\alpha^*(S_n) = \operatorname{argmin}_{\alpha} R(\alpha|x)$$

כלומר, מעבר על כל α האפשריות ונקח את α שנתנה את התוצאה המינימלית עבור R (מספר הסיכון) - כיוון שנרצה כמה שפחות עלות.

הגדרה 53. (Overall risk). אסטרטגיה $\alpha(S_n)$ היא החוק שקובע איזו פעולה נבחרה עבור כל תצפית x_i אפשרית. נרצה להגדיר את העלות הממוצעת של חוק זה על פני כל התצפיות:

$$R(\alpha(S_n)) = \sum_{x_i \in S_n} R(\alpha(x_i)|x_i) \cdot Pr(x_i)$$

הערה 54. אסטרטגיה היא פונקציה, כך שבהינתן x_i היא אומרת מראש - בצע $\alpha(x_i)$. לכן בהגדרה הקודמת, רצינו לחשב את הסיכון של האסטרטגיה. לכל פעולה ישנה אסטרטגיה שנבחרה בפונקציית האסטרטגיה, $\alpha(x_i)$. נרצה לבדוק את הסיכון של אותה פעולה נבחרת, כלומר $R(\alpha(x_i)|x_i)$, ולשקלל זאת בהסתברות לקבל את אותו הקלט. כך נחשב למעשה את הסיכון המשוקלל של האסטרטגיה.

כעת נציג כמה וכמה דוגמאות חשובות:

דוגמה 55. (Zero One cost) נניח כי אנחנו רוצים לנחש נכון את מצבי העולם, וכל טעות עולה לנו אותו דבר. נניח כי הפעולות $\{\alpha_i\}_{i=1}^k$ הן הניחושים שלנו לגבי מצב העולם האמיתי $\{\omega_i\}_{i=1}^k$. (לדוגמה: מצבי העולם הם התמונה היא תפוז, והתמונה היא תפוח. α_1 זה ניחוש שהתמונה תפוז α_2 ניחוש שהתמונה תפוח. ישנה התאמה על ניחוש מצב העולם האמיתי). כמו כן, נשלם 1 על כל טעות ו-0 אם צדקנו. לשם כך, נשתמש בפונקציה דלתא δ_{kj} כך שמקיימת:

$$\delta_{kj} := \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases}$$

לכן, נגדיר את העלות בצורה הבאה: $\lambda(\alpha_k|\omega_j) = 1 - \delta_{kj}$ (הרי אם הניחוש α_k מתאים למצב ω_j אזי $k = j$ ובמקרה זה $\delta_{kj} = 1$ ולכן $\lambda(\alpha_k|\omega_j) = 1 - 1 = 0$), אחרת נקבל כי $\lambda(\alpha_k|\omega_j) = 1$. הרי שאם צדקנו בניחוש, לא נרצה לשלם על כך אך אם טעינו נשלם בדיוק אחד). כעת, נחשב את הסיכון המותנה R :

$$R(\alpha_k|x) = \sum_{j=1}^K \lambda(\alpha_k|\omega_j) Pr(\omega_j|x)$$

נבחין כי העלות היא 0 שצדקנו ו-1 שטעינו, לכן הגורמים מתאפסים בכל המקרים בהם צדקנו. כלומר ה' Σ 'ל היא סכום המקרים בהם טעינו.

$$R(\alpha_k|x) = \sum_{j \neq k} Pr(\omega_j|x)$$

אך אנחנו יודעים כי סכום ההסתברויות הללו, של כל מצבי העולם הוא 1. כלומר, $\sum_{j \neq k} Pr(\omega_j|x) + Pr(\omega_k|x) = 1$ ולכן:

$$R(\alpha_k|x) = 1 - Pr(\omega_k|x)$$

כעת, נרצה $\alpha^*(S_n) = \operatorname{argmin}\{R(\alpha_k|x)\}$. על מנת ש- $1 - Pr(\omega_k|x)$ יהיה הכי קטן, נרצה כי $Pr(\omega_k|x)$ יהיה גדול ככל הניתן. כלומר, להמר על מצב העולם שיש לו את ההסתברות הגבוהה ביותר בהינתן התצפיות שראינו. לסיכום: כשכל טעות עולה לנו אותו המחיר בדיוק, הדרך הכי זולה במקרה זה היא להמר על הסוס הכי חזק (זה עם ההסתברות הכי גבוהה). והערה נוספת: למעשה הדרך שלנו לשלם הכי פחות היא באמת אכן $1 - Pr(\omega_k|x)$. אנחנו תמיד נשלם 1, פרט למקרה בו צדקנו, ואז נשלם פחות. הדרך הכי טובה שלנו לשלם פחות - להמר על זה שנהיה צודקים לגבי ההסתברות הגבוהה ביותר.

■

דוגמה 56. כעת, נניח כי העלויות לא בהכרח 0 ו-1. נתח מקרה עם 2 מצבי עולם ו-2 פעולות. נתבונן במטריצת עלויות מגודל 2×2 . נסמן $\lambda_{kj} = \lambda(\alpha_k | \omega_j)$. הסיכון לכל פעולה מוגדר כך:

$$R(\alpha_1 | x) = \lambda_{11} Pr(\omega_1 | x) + \lambda_{12} Pr(\omega_2 | x)$$

$$R(\alpha_2 | x) = \lambda_{21} Pr(\omega_1 | x) + \lambda_{22} Pr(\omega_2 | x)$$

כעת, נשאל את עצמנו מתי נבחר ב- α_1 ? כאשר הסיכון שלה קטן יותר. דהיינו,

$$\lambda_{11} Pr(\omega_1 | x) + \lambda_{12} Pr(\omega_2 | x) < \lambda_{21} Pr(\omega_1 | x) + \lambda_{22} Pr(\omega_2 | x)$$

$$(\lambda_{12} - \lambda_{22}) Pr(\omega_2 | x) < (\lambda_{21} - \lambda_{11}) Pr(\omega_1 | x)$$

$$\frac{Pr(\omega_1 | x)}{Pr(\omega_2 | x)} > \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}$$

כעת, נעזר בחוק בייס:

$$\frac{\frac{Pr(x|\omega_1)}{Pr(x)} \cdot Pr(\omega_1)}{\frac{Pr(x|\omega_2)}{Pr(x)} \cdot Pr(\omega_2)} > \frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}}$$

$$\underbrace{\frac{Pr(x|\omega_1)}{Pr(x|\omega_2)}}_{\text{Likelihood ratio}} > \underbrace{\frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}} \cdot \frac{Pr(\omega_2)}{Pr(\omega_1)}}_{\text{Decision boundary}}$$

מאגף שמאל, קיבלנו את Likelihood ration: יחס בין הסתברויות, שעונה על השאלה: פי כמה התצפית שראינו x מתאימה למצב 1 יותר מאשר למצב 2?
מצד ימין, קיבלנו את Decision boundry: מדובר בערך מספרי שקובע את הרף ומורכב מ-2 חלקים:
1. יחס $Priors$: כמה מצב 2 נפוץ יותר לעומת מצב 1 ביומיום.
2. יחס העלויות: כמה יותר כואב לטעות בסוג אחד של טעות לעומת השני.
המספר הזה הוא הגבול - אם הראיות מהשטח (לייקהוד רטיו) חזקות יותר מערך זה - נבחר ב- α_1 . אחרת: נבחר ב- α_2 .
במקום לחשב הסתברויות מסובכות בכל שלב, המודל הנ"ל אומר: תשווה בין שתי האפשרויות, תראה אם הראיה מספיק חזקה לך על מנת לעבור את הסף.

■

עד כה התעסקנו במצב בו יש לנו תצפית אחת, x . ומה אם יש לנו n תצפיות? כלומר תצפיות $\{x_i\}_{i=1}^n$? במצב זה:

$$\underbrace{\frac{Pr(x_1, \dots, x_n | \omega_1)}{Pr(x_1, \dots, x_n | \omega_2)}}_{\text{Likelihood ratio}} > \underbrace{\frac{\lambda_{12} - \lambda_{22}}{\lambda_{21} - \lambda_{11}} \cdot \frac{Pr(\omega_2)}{Pr(\omega_1)}}_{\text{Decision boundary}} = \Theta$$

כעת, נניח כי התצפיות שלנו $\{x_i\}_{i=1}^n$ הן בלתי תלויות זו בזו (דרישה הגיונית), בהינתן למצב העולם ω_i . אזי, לפי חוקי הסתברות יתקיים כעת:

$$Pr(x_1, \dots, x_n | \omega_1) = \prod_{i=1}^n Pr(x_i | \omega_1), Pr(x_1, \dots, x_n | \omega_2) = \prod_{i=1}^n Pr(x_i | \omega_2)$$

ולכן אי השוויון יהפוך להיות: $\frac{\prod_{i=1}^n Pr(x_i | \omega_1)}{\prod_{i=1}^n Pr(x_i | \omega_2)} > \Theta$. נבחין כי בעולם של הסתברויות, להכפיל הרבה הסתברויות זה לא רעיון חכם במיוחד. למה לא? כי יתכן וההסתברויות הנ"ל מאוד קטנות וקרובות לאפס, מכפלתן זו בזו תהיה עוד יותר קטנה וקרובה לאפס, ומבחינת המחשב זה ייצור מספרים אפסיים שהוא לא ידע כיצד להתמודד איתם. לכן, נרצה להעזר במתמטיקה על מנת למנוע מצב כזה: נשתמש ב $\log$ שהיא פונקציה מונוטונית עולה ולכן לא תשפיע על אי השוויון. נגדיר $\Theta' = \log(\Theta)$ ויתקיים:

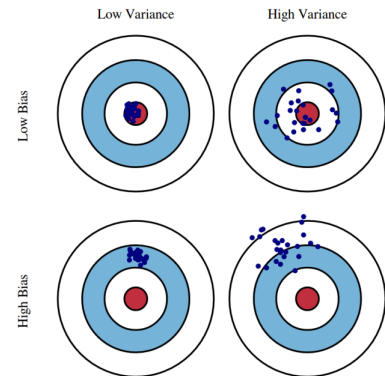
$$\log\left(\frac{\prod_{i=1}^n Pr(x_i | \omega_1)}{\prod_{i=1}^n Pr(x_i | \omega_2)}\right) > \Theta' \iff \sum_{i=1}^n \log\left(\frac{Pr(x_i | \omega_1)}{Pr(x_i | \omega_2)}\right) > \Theta'$$

מעבר למשמעות המתמטית, במקום מבחן אחד עכשיו יש לנו צבירת ראיות. לכל תצפית x_i כך $i \in [n]$ נחשב את Log-Likelihood ration שלה. אם התצפית תומכת ב ω_1 אזי $\log$ יהיה חיובי ואחרת שלילי (לפי חוקי $\log$, הרי תמיכה משמעותה $\frac{Pr(x_i | \omega_1)}{Pr(x_i | \omega_2)} > 1$ ובמקרה זה $\log$ חיובי ואי תמיכה משמעותה $\frac{Pr(x_i | \omega_1)}{Pr(x_i | \omega_2)} < 1$ ובמקרה זה $\log$ שלילי). אנו סוכמים את הראיות, לכן נבחר את α_1 אם סכום התצפיות הכולל היה גדול מהגבול, בדומה, על α_2 .

הערה 57. כפי שראינו בדוגמאות קודמות, הנחנו כי אנחנו יודעים את התפלגות הנתונים. תאוריית בייס מניחה שיש לנו ידע אלוהי על התפלגויות הנתונים. בפועל, בחיים האמיתיים אף אחד לא נותן לנו את התפלגות הנתונים. אז מה נעשה בקורס? הקורס ילמד אותנו כיצד לגשר בין הגישות.

2.3 The Bias-Variance Trade-off

Bias היא הטייה, זו שגיאה שנובעת מהנחות יסוד פשטניות מדי של המודל. שונות זו השגיאה שנובעת מרגישות יתר של המודל לשינויים בנתוני האימון. ננסה להבין מה הקשר בין השניים, באמצעות התמונה הבאה:



נחלק למקרים:

1. **שונות והטיה נמוכים (שמאל למעלה):** המודל פוגע בדיוק במרכז, ובכל פעם שניתן לו סט חדש של נתונים יפגע בדיוק באותו מקום - עקבי, זהו היעד שלנו.
2. **שונות נמוכה, הטיה גבוהה (שמאל למטה):** המודל מאוד עקבי (תמיד פוגע באותה נקודה!) כי השונות נמוכה, אבל הנקודה הנ"ל רחוקה מהמרכז: כי יש הטיה. המודל טועה בגלל הנחה שגויה, אך טועה בצורה עקבית.
3. **שונות גבוהה, הטיה נמוכה (ימין למעלה):** בממוצע, הנקודות נמצאות יחסית קרובות למרכז כי ההטייה נמוכה אך הן מפוזרות זו מזו בגלל שהשונות גבוהה. המודל מבין את הכיוון הכללי - אך נורא מפוזר.
4. **שונות והטיה גבוהים (ימין למטה):** המצב הכי גרוע. גם הטיה גבוהה - ולכן רחוק מהיעד, וגם הנתונים מפוזרים כי השונות גבוהה. המודל טועה גם בממוצע וגם לא עקבי.

נרצה לפרמל את המושגים עליהם דנו כאן, ולפתח קשר מתמטי.

הגדרה 58. יהיו $S_n = \{(x_i, y_i)_{i=1}^n\}$ נתוני האימון שלנו, המגיעים מעולם $\mathcal{D}$ כך שהדגימות ב"ז זו בזו. נרצה להעריך את y (שלא ידוע) באמצעות פונקציית היפותזה $\mathcal{H}$: $h(x) = h(x; S_n)$ שנבחין שתלוי בנתונים עליה התאמנה. מטרתנו לחזות את y עבור נק' מבחן חדשה $(x, y) \sim \mathcal{D}$ שלא נמצאת בנתוני האימון שלנו. נגדיר את MSE (Mean Squared Error) כתוחלת של ריבוע ההפרש בין הערך האמיתי לחזוי:

$$MSE = \mathbb{E}_{(x,y) \sim \mathcal{D}}[(y - h(x))^2]$$

מטרתנו תהיה להבין מדוע המודל שלנו טועה - האם זה בגלל חוסר גמישות (Bias) או מרגישות יתר לנתונים (Variance). MSE מתפרק לשניהם -

- $Bias$: המידה שבה החיזוי של המודל מתאים לפונקציית המטרה בממוצע.
- $Variance$: המידה שבה החיזור עבור נקודה בודדת משתנה בממוצע.

לפני כן, נגדיר שני סוגי תוחלות:

1. תוחלת על נתונים: תוחלת ע"פ כל הנקודות מהעולם $\mathcal{D}$. כאשר נבדוק טעות מבחן, נשתמש בתוחלת זו.

$$\mathbb{E}_{(x,y)}[\cdot] = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\cdot]$$

2. תוחלת על סטי אימון: תוחלת מעל סטי האימון. התוחלת הנ"ל מדגימה בממוצע כיצד המודל מתנהג בממוצע כשמחליפים לו את הדטה עליו התאמן.

$$\mathbb{E}_{S_n}[\cdot] = \mathbb{E}_{S_n \sim \mathcal{D}^n}[\cdot]$$

משפט 59. בהינתן התפלגות נתונים $\mathcal{D}$, מדגם אימון $S_n \sim \mathcal{D}^n$, והיפoteזה $h(x, S_n)$ השגיאה הריבועית הממוצעת (MSE) ניתנת לפירוק לשני רכיבים חיוביים:

$$MSE = Bias^2 + Variance$$

באשר:

$$Bias = \mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)])^2] \text{ ו} Variance = \mathbb{E}_{(x,y)}[\mathbb{E}_{S_n}[(h(x) - \mathbb{E}_{S_n}[h(x)])^2]]$$

הוכחה:

$$\mathbb{E}_{(x,y) \sim \mathcal{D}}[(y - h(x))^2] = \mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)] + \mathbb{E}_{S_n}[h(x)] - h(x))^2] =$$

$$\mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)])^2] + \mathbb{E}_{(x,y)}[(\mathbb{E}_{S_n}[h(x)] - h(x))^2] + 2\mathbb{E}_{(x,y)}[(\mathbb{E}_{S_n}[h(x)] - h(x)) \cdot (y - \mathbb{E}_{S_n}[h(x)])]$$

נרצה לנתח את הביטוי $2\mathbb{E}_{(x,y)}[(\mathbb{E}_{S_n}[h(x)] - h(x)) \cdot (y - \mathbb{E}_{S_n}[h(x)])]$

$$2\mathbb{E}_{(x,y)}[(\mathbb{E}_{S_n}[h(x)] - h(x)) \cdot (y - \mathbb{E}_{S_n}[h(x)])] =$$

$$2\mathbb{E}_{S_n, (x,y)}[(y - \mathbb{E}_{S_n}[h(x)])(\mathbb{E}_{S_n}[h(x)] - h(x))] = 2\mathbb{E}_{(x,y)}[\mathbb{E}_{S_n}[(y - \mathbb{E}_{S_n}[h(x)])(\mathbb{E}_{S_n}[h(x)] - h(x))]]$$

$$= 2\mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)])(\mathbb{E}_{S_n}[\mathbb{E}_{S_n}[h(x)] - h(x)])] =$$

$$2\mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)])(\mathbb{E}_{S_n}[h(x)] - \mathbb{E}_{S_n}[h(x)])] = 0$$

מכאן קיבלנו:

$$MSE = \mathbb{E}_{(x,y)}[(y - \mathbb{E}_{S_n}[h(x)])^2] + \mathbb{E}_{(x,y)}[(\mathbb{E}_{S_n}[h(x)] - h(x))^2]$$

■

הערה 60. למעשה, $Bias$ הוא כמה המודל $h(x)$ חוזה טוב את המטרה בתוחלת. אם יש $Bias$ גבוה - המודל עקום מהיסוד. **כיצד נקטין את $Bias$?** צריך להפוך את המודל לגמיש יותר או לחלופין למורכב יותר. כלומר: נרצה ש $h(x)$ תבחר מסת רחב יותר של פונקציות - למשל פולינומי מדרגות גבוהות ולא רק לינארי.

כיצד נקטין את השונות? אפשרות אחת היא להגדיל את הדטה של האימון (ככה נעריך טוב יותר את $h(x)$), **אפשרות אחרת** היא להגביל מראש את המודל לקבוצה קטנה יותר של פונקציות - הרי אם נכריח אותו לבחור מתוך סט קטן (למשל מתוך קווים לינאריים בלבד) הוא יהיה הרבה יותר "קשיח": ההגיון הוא כי מודל פשוט הוא מודל עקבי - קו ישר לא יכול להתפתל כדי לעבור דרך רעש בנתונים ולכן אם נחליף את נתוני האימון הקו הישר שנקבל יהיה דומה לקודם. התוצאה היא שהשונות תרד משמעותית אך ההטייה עלולה לעלות.

2.4 $Overfitting$ ו $Underfitting$

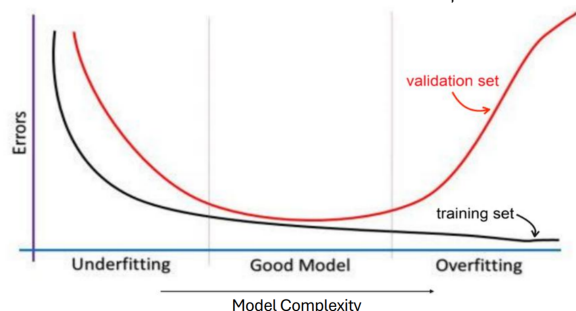
כיצד בפועל נראה אימון של מודל? נחלק לשלבים:

Training loop (לולאת האימון): זהו השלב שבו המודל לומד. המודל מעדכן את הפרמטרים שלו, כדי למזער את השגיאה על סט האימון S_n . המטרה בשלב זה היא להקטין את $Bias$. ככל שנרוץ יותר פעמים על נתוני האימון, המודל יתאים את עצמו אליהם יותר.

Evaluate the loss on the validation set: זהו שלב ה"בקרה" שקורה תוך כדי האימון. מדוע זה נחוץ? סט האימון לא יכול להגיד לנו אם התחלנו לעשות $overfitting$, ולכן אנחנו בודקים את הביצועים על נתונים שהמודל לא התאמן עליהם ישירות. אם השגיאה באימון יורדת אבל השגיאה בוולדיאציה מתחילה לעלות: זהו סימן מובהק לכך שהמודל התחיל ללמוד רעש ועלינו לעצור.

Evaluate on Test Set: זהו שלב המבחן הסופי שקורה רק פעם אחת בסוף התהליך. סט המבחן חייב להיות טהור - אסור לנו לקבל החלטות על המודל כמו לשנות פרמטרים בהתבסס על סט המבחן - זה לרמות את עצמנו.

כיוון שאנחנו כל הזמן עושים בקרה - שגיאת האימון שלי תמיד ירדה, כי אני מתקן את עצמי תוך כדי הבקרה. אך השגיאה של ה $validation$ לא בהכרח תמיד תרד! כמו בדוגמה בתמונה מטה. אם נראה מצב שהשגיאה של ה $validation$ עולה: אנחנו נבין שיש כאן סכנה ל $overfitting$. אם ראינו מצב כזה - צריך לעצור ולתת למודל שוב להתאמן. לבסוף, כשנראה ששגיאת הוולדיאציה מתיישרת עם שגיאת האימון (שירידת), נוכל לעבור לשלב המבחן הסופי.



הגדרה 61. Underfitting הוא מצב של תת התאמה. זהו מצב שבו המודל פשוט מדי בכדי לתפוס את המבנה של הנתונים. סימנים לכך:

1. שגיאה גבוהה בסט האימון וגם בסט ההערכה.
 2. מצב של $Bias$ גבוה, המודל לא מצליח ללמוד אפילו את הנתונים שהוא ראה: לכן ברור שלא יצליח לחזות חדשים. למשל, נסיון להתאים קו לינארי לנתונים שמתנהגים בצורה ריבועית של פרבולה.
- מדוע זה קורה? המודל פשוט מדי. איך לתקן?** להגדיל את מורכבות המודל או לאפסמז יותר טוב.

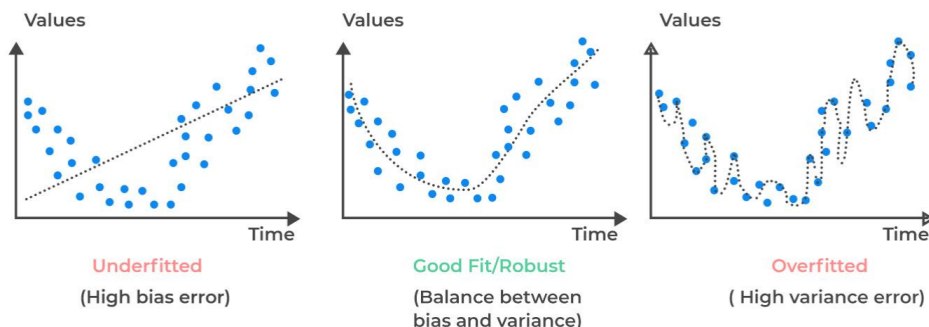
הגדרה 62. Good fit היא התאמה טובה, זהו המצב אליו אנחנו שואפים בתהליך האימון. סימנים לכך:

1. שגיאה נמוכה בסט האימון וגם בסט ההערכה.
2. מצאנו איזון בין השונות להטיה: $Bias$ נמוך מספיק בשביל להבין את החוקיות, והשונות נמוכה מספיק בשביל לא להיות מושפעים מהרעש. המודל מבצע הכללה (Generalization) טובה.

הגדרה 63. $overfitting$ הוא מצב שבו המודל שינן את נתוני האימון במקום ללמוד את החוקיות שלהם. סימנים לכך:

1. שגיאה נמוכה מאוד בסט האימון, אך שגיאה גבוהה בסט ההערכה.
 2. מצב של שונות גבוהה מאוד - המודל גמיש מדי וחודר לרעש של מדגם ספציפי. פוגע בנתוני האימון אך מתרסק על נתונים חדשים.
- מדוע זה קורה?** סט נתונים קטן מדי או מודל מורכב. **כיצד לתקן?** לקחת מודל פחות מורכב, לקחת דטה-סט גדול יותר.

הערה 64. בתמונה, דוגמאות לכל אחד מהמקרים:



אם נרצה לתאר מהם שונות והטיה, נוכל לומר כך: Bias מתאר למעשה כמה הנתונים שלי רחוקים מהמצב האמיתי (y), בעוד Variance מתאר כמה בכל שלב ושלב החיזויים שלי יהיו שונים זה מזה. כעת נרצה לשאול:

מהו Bias גבוה? בממוצע, אנחנו רחוקים מהחיזוי האמיתי. המודל לא הבין את מורכבות הדטה.

מהו Variance גבוה? המודל הופך להיות רגיש מדי לכל תנועה בנתוני האימון - במקום ללמוד חוקיות כללית שמושפעת לכל הנתונים בעולם הוא מתחיל להתייחס לכל רעש או סטיות מקריות בסט האימון הספציפי. מדוע לפונקציה מורכבת יש שונות גבוהה? המודל בונה פונקציה מורכבת - שעוברת בדיוק דרך כל נקודה: אם יש נקודה אחת שהיא טעות או רעש - המודל יתעקם כדי להגיע אליה. ההתעקמויות האלו הם יוצרים וריאנס גבוה: המודל איבד את יכולת ההכללה כי בנה את עצמו בהתאם לנתונים.

2.5 Cross Validation

כפי שראינו, אי אפשר להשתמש ב-Test set על מנת לקבל החלטות על המודל לכן צריך את סט Validation. תפקידו: עזרה בבחירת הגדרות המודל (אלו ה-Hyper-parameters): מס' הפרמטרים, בחירת אלגוריתם אופטימיזציה, קבועי רגולציה λ וכו'.

מה הבעיה שנוצרת מכך? אם ה-dataset שלנו קטן, אם נפריש חלק משמעותי ממנו ל-Validation ישאר קצת מאוד דטה עבור האימון עצמו. (זה יעלה את Bias כי המודל לא למד מספיק)

אם נפריש חלק קטן ממנו ל-Validation הערכת הביצועים שלנו תהיה תלויה מאוד בנקודות הספציפיות שנפלו בסט הוולידציה. בחירה מקרית מזלזלת או מחמירה מדי של סט וולידציה תשפיע על כל החלטות המדגם.

לשם כך, נשתמש בשיטה בשם Cross Validation:

1. נקח את כל הנתונים שלנו ונחלקם ל- k קבוצות שוות בגודלן. (לרוב משתמשים $k = 5, 10$)
2. מריצים אימון למשך k פעמים: בכל איטרציה $1 \leq i \leq k$:
- א. Training set: משתמשים ב- $k - 1$ מהחלקים כדי לאמן את המודל.
- ב. Validation set: משתמשים בחלק i שנותר בצד כדי לבדוק את הביצועים.
- המשמעות: בכל פעם חלק אחר של הנתונים משמש כ"בוחר".
3. בסוף הולאה יש לנו k ערכי שגיאה (אחד לכל קבוצה) ונחשב את הממוצע שלהם: $Error = \frac{1}{k} \sum_{i=1}^k Error_i$. הממוצע הנ"ל הוא Performance Estimation שלנו, הרבה יותר יציב מהערכה על סט וולידציה בודד כי הוא ראה את כל הנתונים בתפקיד הוולידציה.
4. בחר את הפרמטרים עם performance הגבוה ביותר. אנחנו למעשה חוזרים על התהליך הנ"ל כולו עבור קבוצות שונות של היפר פרמטרים (פולינומים דרגה 2, דרגה 3 וכו'...). נבחר את הפרמטרים שהשיגו את הממוצע השגיאות הנמוך ביותר.

הערה 65. אם בחרנו בשיטה זו, אנחנו קודם נבצע cross validation, ואז נעבור לשלב האימון - ללא ולדיציה, עם הפרמטרים שנבחרו, ולבסוף את שלב המבחן. לעומת זאת: אם לא בחרנו בשיטה זו - הוולידציה קורית במקביל לאימון, ולבסוף מתבצע המבחן.

הערה 66. כיצד בחירת k משפיעה על ההטיה והשונות?

נבחין כי ככל ש- k יהיה גדול יותר, Bias יהיה קטן יותר: כיוון שככל ש- k גדול, כך סט האימון בכל קבוצה גדול יותר וקרוב יותר לגודל N המקורי, מודל שמאומן על יותר נתונים מייצג טוב יותר את המודל הסופי, ולכן השגיאה שמודדים ב- CV היא הערכה קרובה יותר.

כמו כן, ככל ש- K גדול יותר - Variance יהיה גדול יותר: כיוון שככל ש- k גדל, סטי האימון בכל איטרציה הופכים להיות כמעט זהים (לצורך ההמשלה: אם $N = 100$ וכן $k = 2$, אזי החפיפה תהיה אפסית שכן אפשר לחלק ל-2 קבוצות בגודל 50. אך אם $N = 100$ ו- $k = 10$, במקרה זה בכל איטרציה יהיו 90 דגימות - ויהיה הרבה יותר חפיפה בין הדגימות במקרה זה), כאשר המדגמים עליהם נרץ את K האיטרציות מאוד דומים - הממוצע שייחושב בסוף יהיה רגיש יותר לנתונים חריגים ולכן השונות תגדל.

מסקנה: שוב ישנו טרייד-אוף בין השניים.

3 הרצאה 3

3.1 Parameter estimation (הערכת פרמטרים)

תחת ההנחה של Bayes Decision Theory, התייחסנו למצב בו כל הנתונים הסטטיסטיים ידועים לנו. כלומר, $\forall i Pr(\omega_i)$ היה ידוע לנו ה-Priors, וגם הצפיפות המותנית $Pr(x|\omega_i)$. אם הנתונים הללו אכן בידיו - אנו יכולים לבנות מסווג אופטימלי (Bayes Optimal classifier) שממזער את

טעות הסיווג בצורה מתמטית. אך - בעולם האמיתי זה לא ככה. אנחנו לא באמת יודעים את ההתפלגויות האמיתיות. יש לנו אינטואיציה כיצד הדברים אמורים להראות - ויש לנו סט אימון. במקום להניח שאנחנו יודעים את הסטטיסטיקה של העולם, אנחנו נשתמש בדגימות שבידינו על מנת להעריך את אותן הסתברויות.

למה שנעבור לגישה זו? למה שלא פשוט נעבור פונקציה פונקציה וננחש אותן ונבדוק אם היא מתאימה? ניסיון להעריך פונקציה כזו קשה מהסיבות הבאות:

- צריך כמות עצומה של דגימות שלרוב אין לנו
- קללת המימד: ככל של x יש יותר מאפיינים (מימד גבוה יותר) המרחב הופך לריק יותר, ונצטרך כמות דגימות שגדלה אקספוננציאלית.
- לכן - **אנחנו מעט נרמה. נניח מראש כי אנחנו יודעים את הצורה של הפונקציה: אך אנחנו לא יודעים את ערך הפרמטרים שלה.** למשל, אם נניח כי $Pr(x|\omega_i) \sim N(\mu_i, \Sigma_i)$ נצטרך רק ללמוד מהדטה את Σ_i, μ_i .

3.2 Maximum Likelihood (ML)

- התפיסה: הפרמטר θ קבוע אך לא ידוע. אין התפלגות - מדובר במס' שצריך למצוא.
- התוצר: הערכה נקודתית $\hat{\theta}_{ML}$ של הפרמטר.
- הפרשנות: נבחר את θ שיתן את הסיכוי הגבוה ביותר לראות את הדטה שבאמת ראינו. כלומר - אם זה מה שקרה בפועל: איזה θ הכי סביר שהוביל לזה לקרות?

יהיו הנתונים $\mathcal{D} = S_n = \{x_1, \dots, x_n\}$. מייצגים את קבוצת הdataset. כל x_i היא דגימה בודדת. אנו מניחים כי הנתונים נוצרו ע"י תהליך סטטיסטי שיש לו פרמטר לא ידוע θ . אנחנו נבחר את θ שגורם לדטה שראינו להיות הכי סביר (Most Likely). כלומר - אם נסתכל על כל ה- θ האפשריים בעולם, נשאל איזה מהם נותן את ההסתברות הגבוהה ביותר להיות $x_1, \dots, x_n$? נגדיר:

$$\hat{\theta}_{ML} = \operatorname{argmax}_{\theta} Pr(S_n|\theta)$$

כאומד θ לפי ML , כאשר $Pr(S_n|\theta)$ היא פונקציית הנראות. היא מודדת כמה הנתונים סבירים ביחס לפרמטר מסויים θ . כיוון שהדגימות iid (מגיעות מהתפלגות זהה וב"ת) הרי שמתקיים:

$$Pr(S_n|\theta) = \prod_{i=1}^n Pr(x_i|\theta)$$

כיוון ש- $\log$ פונקצייה מונוטונית עולה יתקיים:

$$\hat{\theta}_{ML} = \operatorname{argmax}_{\theta} Pr(S_n|\theta) = \operatorname{argmax}_{\theta} \sum_{i=1}^n Pr(x_i|\theta)$$

דוגמה 67. מודל התפלגות בינומית. נניח כי אנחנו מטילים מטבע כך ש: $Pr(H) = p, Pr(T) = 1 - p$. לפי נוסחה, ההסתברות ל- k הצלחות בהינתן n נסיונות ניתנת לפי:

$$Pr(X = k|p) = \binom{n}{k} p^k (1-p)^{n-k}$$

נתייחס ל- (n, k) כקבועים ונאמוד את ערך הפרמטר p כך שימקסם את ההסתברות. נפעיל $\log$:

$$\ln(\ell(p)) = \ln\left(\binom{n}{k}\right) + k \ln(p) + (n-k) \ln(1-p)$$

כעת נגזור ונשווה לאפס:

$$\nabla_p = \frac{k}{p} - \frac{n-k}{1-p} = 0 \implies \frac{k}{p} = \frac{n-k}{1-p} \implies k - kp = pn - kp \implies \hat{p}_{ML} = \frac{k}{n}$$

כלומר ערך הפרמטר p שימקסם את ההסתברות יהיה השכיחות היחסית של k בהינתן הדטה n .



3.3 Maximum A Posteriori (MAP) (הסתברות פוסטרירית מקסימלית)

- התפיסה: הפרמטר θ הוא משתנה אקראי, יש לנו ידע מוקדם עליו ($Prior$) לפני שראינו את הנתונים.
- התוצר: הערכה נקודתית $\hat{\theta}_{ML}$ של הפרמטר.
- הפרשנות: נבחר את θ הכי סביר אחרי ($Post$) ששקללנו את הדטה עם הידע המוקדם שלנו. פשרה בין מה שחשבנו מראש - למה שהנתונים מראים.

Map למעשה הוא השכיח של התפלגות $Posterior$. נגדיר:

$$\hat{\theta}_{MAP} = \operatorname{argmax}_{\theta} Pr(\theta|\mathcal{D}) \propto \operatorname{argmax}_{\theta} Pr(\mathcal{D}|\theta)Pr(\theta)$$

- נזכר כי $Pr(\theta|\mathcal{D}) = \frac{Pr(\mathcal{D}|\theta)Pr(\theta)}{Pr(\mathcal{D})}$, כיוון ש $Pr(\mathcal{D})$ הוא ערך קבוע שאיננו תלוי ב θ , אנחנו יכולים לומר שפורפורצינלית ($\propto$) אם נרצה למקסם את $Pr(\theta|\mathcal{D})$ זה שקול למקסם אך ורק את המונה. כלומר - MAP הוא הנקודה שבה הדטה $\times$ הידע המוקדם - הכי גבוה.
- הערה: אם $Prior$ שלנו היא פונקציה אחידה (כלומר $Pr(\theta) = c$ עבור c כלשהו) אזי $MAP = MLE$, שכן במקרה זה נוכל גם לומר כי:

$$\hat{\theta}_{MAP} = \operatorname{argmax}_{\theta} Pr(\theta|\mathcal{D}) \propto \operatorname{argmax}_{\theta} Pr(\mathcal{D}|\theta)Pr(\theta) \propto \operatorname{argmax}_{\theta} Pr(\mathcal{D}|\theta) = \hat{\theta}_{ML}$$

לכן אם נסכם, MAP בניגוד ל MLE גם מייחס חשיבות לערך $Prior$, וכן הוא פחות רגיש לרעשים והוא מתקן את MLE לכיוון $prior$.

דוגמה 68. נדון שוב בהטלת מטבע. נסמן θ כהסתברות ל H . $\mathcal{D}$ הוא הדטה שראינו בפועל: k פעמים H וכן $n - k$ פעמים T . נניח כי הידע המוקדם שלנו על המטבע מתנהג לפי התפלגות בטא, כאשר:

$$Pr(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$$

(מדוע פורפורצינלית? ישנם קבועים נוספים בהתפלגות בטא, אך הם לא רלוונטים כשנרצה לחשב את המקסימום).

אם כן, **אנו יודעים שני ערכים:**

- נראות - $Likelihood$: מהדטה נתון כי ראינו k ראשים ו $n - k$ פלי, לכן כיוון שיש אי תלות :

$$Pr(\mathcal{D}|\theta) = \theta^k(1-\theta)^{n-k}$$

- ידע מוקדם - $Prior$: לפי הגדרת פונקציית בטא:

$$Pr(\theta) \propto \theta^{\alpha-1}(1-\theta)^{\beta-1}$$

מכאן:

$$\hat{\theta}_{MAP} \propto \operatorname{argmax}_{\theta} Pr(\mathcal{D}|\theta)Pr(\theta) \propto \theta^k(1-\theta)^{n-k} \cdot \theta^{\alpha-1}(1-\theta)^{\beta-1} = \theta^{\alpha-1+k}(1-\theta)^{n-k+\beta-1}$$

נפעיל $\ln$:

$$(\alpha - 1 + k)\ln(\theta) + (n - k + \beta - 1)\ln(1 - \theta) \approx x\ln(\theta) + y\ln(1 - \theta)$$

נגזור:

$$\frac{x}{\theta} - \frac{y}{1-\theta} = 0 \implies \theta y = x - x\theta \implies \theta = \frac{x}{x+y} = \frac{\alpha + k - 1}{n + \beta + \alpha - 2}$$

וסה"כ קיבלנו כי $\hat{\theta}_{MAP} = \frac{\alpha+k-1}{n+\beta+\alpha-2}$

דוגמה 69. נניח כי $\{x_i\}_{i=1}^n$ הם זמני המתנה לתור. נניח $i.i.d$. אם כן, ידוע כי: $\forall i \in [n] : Pr(x_i|\theta) = \theta e^{-\theta x_i}$. לפי הגדרת הנראות,

$$Pr(\mathcal{D}|\theta) = \prod_{i=1}^n \theta e^{-\theta x_i} = \theta^n e^{-\theta \sum_{i=1}^n x_i}$$

נניח כי הידע המוקדם, $prior$ הוא $Pr(\theta) = e^{-\theta}$. מכאן, לפי הנוסחה:

$$Pr(\theta|\mathcal{D}) \propto \theta^n e^{-\theta \sum_{i=1}^n x_i} \cdot e^{-\theta} = \theta^n e^{-\theta(\sum_{i=1}^n x_i + 1)}$$

מכאן, נפעיל ln ונגזור לפי θ . נקבל כי: $\hat{\theta}_{MAP} = \frac{n}{\sum_{i=1}^n x_i + 1}$.

לצורך ההשוואה, אם היינו פועלים לפי MLE היינו מקבלים $\hat{\theta}_{ML} = \frac{n}{\sum_{i=1}^n x_i}$. מה ההבדל? ה-1, הוא הנציג של $prior$, אם ראינו מעט מאוד דטה, כלומר n קטן יחסית, ה-1 הזה ישפיע מאוד לטובת $prior$. ככל שנוסיף עוד דטה - ה-1 יתפוך לזניח ויתקיים $\hat{\theta}_{ML} \approx \hat{\theta}_{MAP}$. ■

3.4 Bayesian Estimation (הערכה בייאסיאנית)

- התפיסה: הפרמטר θ הוא משתנה אקראי.
- התוצר: התפלגות פוסטריורית מלאה $Pr(\theta|\mathcal{D})$, לא מקבלים ערך אחד אלא פונקציית שאומרת מה ההסתברות לכל ערך אפשרי של θ .
- הפרשנות: לא מהמרים על ערך אחד של θ . מחשבים ומעדכנים את כל האי ודאות שלנו לגבי הפרמטר. אנחנו לא מבצעים כאן אופטימיזציה של הפרמטר - אלא אינטגרציה / חישוב של התפלגות מלאה.

כעת נתייחס ל- θ כמשתנה אקראי, ולא כערך קבוע. הרעיון כאן הוא כהליך של עדכון. אנו מביאים איתנו ידע קודם. $Pr(\theta)$ זו ההתפלגות המוקדמת - היא מייצגת את מה שאנחנו חושבים על הפרמטר לפני שראינו ולו דגימה אחת מהדטה. (למשל: אני חושב שהמטבע הוגן אזי $pr(H) = \frac{1}{2}$). בעת שמגיע דטה $\mathcal{D}$, נשתמש בחוק בייס לעדכן את האמונה שלנו - המטרה למצוא את ה- $posterior$ (ההסתברות בדיעבד). באופן הבא:

$$Pr(\theta|\mathcal{D}) = \frac{Pr(\mathcal{D}|\theta)Pr(\theta)}{Pr(\mathcal{D})}$$

כאן, בניגוד ל- MLE שם חיפשנו רק נקודת אחת, אנחנו מחשבים את כל הפונקציה $p(\theta|\mathcal{D})$.

דוגמה 70. (לא מההרצאה). בידינו מטבע. איננו יודעים את ערך הפרמטר. נניח כי $prior$ שלנו הוא $Pr(\theta) = 6\theta(1-\theta)$. נניח כי ראינו דטה $\mathcal{D}$ של הטלה אחת H . אם כן, $Pr(\mathcal{D}|\theta) = \theta$ שכן θ מעריך את ההסתברות לקבל H . אם כן, כעת נחשב:

$$Pr(\mathcal{D}|\theta)Pr(\theta) = \theta \cdot 6\theta(1-\theta) = 6\theta^2(1-\theta)$$

זו פונקציה חדשה שמתאימה לדטה שראינו. נבחין כי על מנת שיהיה מדובר בפונקציית הסתברות - השטח מתחתה חייב להיות שווה לאחד. אך,

$$\int_0^1 6\theta^2(1-\theta) = \int_0^1 6\theta^2 - 6\theta^3 = [2\theta^3 - 1.5\theta^4]_0^1 = 0.5$$

זו לא פונקציית הסתברות תקינה. אך, לפי בייס: צריך לחלק ב- $Pr(\mathcal{D})$. והרי, $Pr(\mathcal{D}) = \int P(\mathcal{D}|\theta)Pr(\theta)d\theta = 0.5$ לפי נוסחת ההסתברות השלמה. מכאן, נקבל כי:

$$Pr(\theta|\mathcal{D}) = \frac{6\theta^2(1-\theta)}{0.5} = 12\theta^2(1-\theta)$$

זו פונקציית ההסתברות החדשה, לאחר שראינו את ערך הפרמטר. ■

3.5 Loss-Based View

כעת, מהשאלה: "מה הערך הסביר ביותר?", נעבור לשאלה: "מי הערך שיגרום לי להכי פחות נזק אם אטעה?". נניח $P(x|\theta)$ כמודל ההסתברות, $Pr(\theta)$ כ-*prior* וכן $\mathcal{D}$ אלו הנתונים שראינו. כמו כן, תהי $\lambda(\hat{\theta}, \theta^*)$ פונקציית ההפסד. היא מקבלת את הפרמטר שניבאנו $\hat{\theta}$ ואת האמיתי θ^* ומחזירה מס' (קנס). נרצה למצוא אומד $\hat{\theta}$ שיביא למינימום את ההפסד הצפוי. למה צפוי? כי איננו יודעים את θ^* . מה בידינו? בידינו ה-*posterior* ($Pr(\theta|\mathcal{D})$) שאומר לנו כמה כל θ הוא סביר בהינתן הנתונים. לכן, נעשה ממוצע (תוחלת) של הנזק ע"פ כל האפשרויות של θ . כך:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \mathbb{E}_{\theta|\mathcal{D}}[\lambda(\hat{\theta}, \theta^*)]$$

על מנת לחשב את תוחלת זו, נעזר באינטגרל:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \int \lambda(\hat{\theta}, \theta^*) Pr(\theta|\mathcal{D}) d\theta$$

בעברית - עבור על כל ערך של θ אפשרי בעולם, בדוק כמה נזק הוא יגרום לך בעת שסיפקת לו $\hat{\theta}$ שאמור למזער את הנזק, הכפל בסיכוי שזה אכן ה- θ האמיתי. סכום את הכל: מי שינצח הוא זה שיתן את הערך הנמוך ביותר.

דוגמה 71. הפסד ריבועי.

נניח כי בידינו פונקציית הפסד ריבועי, $\lambda(\hat{\theta}, \theta^*) = (\hat{\theta} - \theta^*)^2$. נרצה למצוא:

$$\hat{\theta}_{SE} = \operatorname{argmin}_{\theta} \int (\hat{\theta} - \theta^*)^2 Pr(\theta|\mathcal{D}) d\theta$$

נגזור את הביטוי לפי $\hat{\theta}$. אם כן:

$$\frac{d}{d\hat{\theta}} \int (\hat{\theta} - \theta)^2 Pr(\theta|\mathcal{D}) d\theta = 0 \implies \int 2(\hat{\theta} - \theta) Pr(\theta|\mathcal{D}) d\theta = 0$$

$$\implies \int 2\hat{\theta} Pr(\theta|\mathcal{D}) d\theta - \int 2\theta Pr(\theta|\mathcal{D}) d\theta = 0 \implies \int \hat{\theta} Pr(\theta|\mathcal{D}) d\theta = \int \theta Pr(\theta|\mathcal{D}) d\theta$$

כעת נבחין, בתוך האינטגרל המשתנה שרץ הוא θ , לכן ניתן להוציא את $\hat{\theta}$. מכאן:

$$\hat{\theta} \int Pr(\theta|\mathcal{D}) d\theta = \int \theta Pr(\theta|\mathcal{D}) d\theta$$

לפי הגדרה, $\int Pr(\theta|\mathcal{D}) d\theta = 1$ שהרי זהו סכום הסתברויות ה-*Posterior*. ערך זה שווה אחד מהגדרת פונקציית הסתברות. ולכן נקבל:

$$\hat{\theta} = \int \theta Pr(\theta|\mathcal{D}) d\theta = \mathbb{E}_{\theta|\mathcal{D}}[\theta]$$

עד כה, MAP חיפשנו את הערך הסביר ביותר ביחס ל-*priors* - זהו ה-*MODE*. אך, ראינו כי אם פונקציית ההפסד ריבועית, דווקא נרצה את הממוצע/ תוחלת - לא השיא. אינטואיטיבית, במצב של הענשה ריבועית נעדיף להיות בממוצע על מנת "לחטוף כמה שפחות".

■

דוגמה 72. בניגוד לדוגמה הקודמת, נראה כיצד MAP דווקא הוא הכי טוב כשעובדים עם פונקציית Zero to one. אם כן:

$$\hat{\theta}_{0-1} = \operatorname{argmin}_{\theta} \sum_{\theta} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D})$$

מכאן, לפי ההגדרה הרי שהעלות היא 0 כאשר צודקים:

$$\begin{aligned}\sum_{\theta} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D}) &= \sum_{\theta=\hat{\theta}} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D}) + \sum_{\theta \neq \hat{\theta}} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D}) = \sum_{\theta=\hat{\theta}} 0 \cdot Pr(\theta|\mathcal{D}) + \sum_{\theta \neq \hat{\theta}} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D}) \\ &= \sum_{\theta \neq \hat{\theta}} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D}) = 1 - \sum_{\theta=\hat{\theta}} \lambda(\theta, \hat{\theta}) Pr(\theta|\mathcal{D})\end{aligned}$$

לכן,

$$\hat{\theta}_{0-1} = \operatorname{argmin}_{\theta} (1 - Pr(\hat{\theta}|\mathcal{D}))$$

אם כן, על מנת להביא ביטוי זה למינימום נרצה להביא את $Pr(\hat{\theta}|\mathcal{D})$ למקסימום. הערך שמביא את $Posterior$ למקסימום - הוא בדיוק MAP . מכאן ש:

$$\hat{\theta}_{0-1} = \hat{\theta}_{MAP}$$

■

3.6 סיכום

$Posterior$ היא פונקציה שמכילה המון מידע. כדי לקבל החלטה - עלינו לשלוף ממנה מס' אחד $\hat{\theta}$. איזה מס' לשלוף? זה תלוי בקנס שאנו מפחדים ממנו. ישנם 3 מקרים עיקריים:

- הפסד ריבועי: אם תטעה קצת, המשמעות שהקנס קטן. אם תטעה הרבה, הקנס עצום. לכן - האומד האופטימלי הוא הממוצע שכן הממוצע הוא מרכז המסה.
- הפסד מוחלט: $|\theta - \hat{\theta}|$. הטעות היא למעשה רק המרחק. האומד האופטימלי כאן הוא החציון, מדוע? זהו הערך שמזער את המרחקים המוחלטים.
- הפסד 0-1: לא משנה כמה טעות, אם לא פגעת בול זה נחשב קנס מקסימלי. במקרה זה: האומד האופטימלי הוא השכיח - MAP : הכי הגיוני לבחור את הערך שיש לו את ההסתברות הגבוהה ביותר לקרות, על מנת לא לקבל קנס, זה השיא של הפונקציה ולפי הגדרה בדיוק MAP .

לכן:

כשנרצה להמנע מטעויות ענקיות - נשתמש בפונקציית הפסד ריבועי והערך שנשלוף יהיה הממוצע.
כשנרצה להיות באמצע האפשרויות - נשתמש בהפסד מוחלט והערך שנשלוף יהיה החציון.
כשנרצה לפגוע בול בסיכויי הכי גבוה - נשתמש ב-0-1 והערך שנשלוף יהיה השכיח.

כפי שכבר דנו, תאוריית ההחלטה של בייס מניחה שאנחנו יודעים הכל. בפועל - זהו לא המצב. כאן נכנסת למידת מכונה לתמונה. ישנן 2 אסטרטגיות עיקריות של למידת העולם:

1. אסטרטגיה גנרטיבית (Generative): לומדים בנפרד איך נראה כלב, איך נראה חתול - כשמגיעה תמונה חדשה: משתמשים בחוק בייס על מנת לחשב למי היא דומה יותר.
2. אסטרטגיה דיסקרימינטיבית (Discriminative): בגישה זו לא מעניין כיצד נוצרים חתולים או כלבים. מעניין רק להבדיל ביניהם. לומדים ישירות פונקצייה (חוק החלטה) שמקבלת תמונה ומחזירה "כלב" או "חתול". המטרה - מינימום טעויות על הדטה.

לבסוף, נכנסת לסיפור שאלה של Generalization:

- Empirical risk: הטעות של המודל על הדטה שבו השתמשת לאימון.
 - True risk: הטעות של המודל על כל העולם (הדטה שלא ראית).
- תמיד נמדוד את עצמנו על הסיכון האמפירי - אך המטרה שלנו היא שהסיכוי האמיתי יהיה נמוך. השאלה "כמה דטה מספיק?" - הוא הבסיס לתאוריה של למידת מכונה.

3.7 PAC Learning

בהרצאה הקודמת למדנו על דרכים להורדת overfitting. אחת הדרכים לעשות זאת, היא להגדיל את כמות הדטה. כעת ננסה לענות על השאלה: כמה דטה צריך מודל בשביל להיות טוב?

נניח כעת את היסודות ל-Supervised learning (למידה מונחית). נגדיר:

- התפלגות הנתונים $\mathcal{D} = P(x, y)$. נניח שקיים מנגנון סטטיסטי בטבע שמייצר את הדוגמאות שלנו, מנגנון זה יקרא להתפלגות המשותפת. x הוא הקלט, y הוא הפלט או התווית. איננו יודעים את ההתפלגות. כלומר כל דגימת אימון היא וקטור שמורכב מ-2 חלקים.

y הוא תגית שמוצמדת ל- x . נזכר כי מתקיים: $Pr(x, y) = Pr(x) \cdot Pr(y|x)$. לכן, אפשר להסתכל על כל הנקודות בעולם כהליך ייצור דו שלבי:

1. שלב הדגימה - המשתנה x נוצר באופן ב"ת בתגית שלו - קודם מגיע האובייקט.
2. לאחר x כבר קיים, המנגנון מצמיד לו תווית y שנקבעת בהיתן שידוע כבר הערך של x .

3.7.1 מטרת העל של תהליך הלמידה

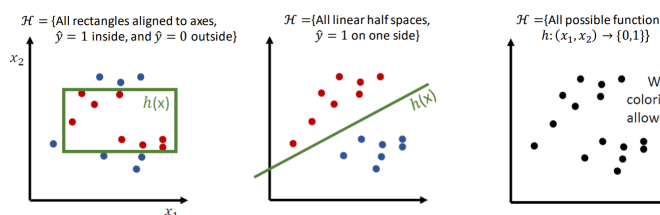
אנו מעוניינים לבנות מודל. **פורמלית:** נחפש $h: X \rightarrow \hat{Y}$ היפותזה, כך שהקלט x הוא וקטור תכונות והפלט $\hat{y}$ היא התגית הנחזית. המודל צריך להסכים עם h האמיתי. לשם כך, מוגדרת הloss function: $l(y, h(y))$ שמודדת את המחיר של הטעות.

המטרה: h הנ"ל תמזער את $\mathbb{E}_{p(x,y)}[l(y, h(y))]$.

נתונים: מדגם $S = \{(x_i, y_i)\}_{i=1}^n$ כך שהדגימות הן *i.i.d*.

כאשר אנחנו מתכננים אלגוריתם למידה, אנחנו מגבילים אותו לבחור היפותזות ממחלקה מסוימת $\mathcal{H}$. לפנינו כמה דוגמאות:

- מרחב כל חצאי המרחבים הליניאריים (קיים קו מפריד במרחב), ומסווגים נקודות לפי המיקום שלהם (מעל או מתחת לקו)
- מרחב כל הפונקציות $h(x_1, x_2) \rightarrow \{0, 1\}$. - מה הבעיה כאן? יש לנו את כל הפונקציות האפשריות ואין לנו דרך לסווג נקודות. המודל עלול לשנן את נקודות האימון - אך יכולת ההכללה שלו לא תהיה קיימת על דוגמאות שלא ראה באימון.
- מרחב כל הלבנים שמקבילים לצירים.



כעת נגדיר פורמלית את מדידת הטעות:

הגדרה 73. (Zero-One loss) פונקציית הפסד דיסקרטית למשימת סיווג. מוגדרת כך:

$$\ell_{0-1}(h(x), y) = \begin{cases} 1 & h(x) \neq y \\ 0 & h(x) = y \end{cases}$$

מכאן, השגיאה תוגדר באופן הבא:

$$Err_{\mathcal{D}}(h) = \mathbb{E}_{P(x,y)}[\ell_{0-1}(h(x), y)]$$

כלומר: שגיאת ההיפותזה h היא התוחלת מעל מרחב ההתפלגות המשותפת של פונקציית ההפסד. מה הבעיה? איננו יודעים את $\mathcal{D}$. לכן אי אפשר לחשב את המספר הזה! לכן, נפנה למדגם S שבידינו. ונגדיר:

$$Err_S(h) = \frac{1}{n} \sum_{i=1}^n \ell_{0-1}(h(x_i), y)$$

כאחוז שגיאות המודל על סט האימון.

3.7.2 Realizability

נרצה לשאול שאלה לגיטימית. האם בכלל הפתרון המושלם נמצא בתוך מה שאני מחפש? כלומר, ישנה הנחה גורסת לפיה הקשר בין x ל- y אינו סתם רעש אקראי: אלא ממש חוקיות מוחלטת שניתן לתאר במדויק. לכן נגדיר מחלקת השערות $\mathcal{H}$ - קבוצת כל הפונקציות שהאלגוריתם שלנו יכול לבחור מתוכן. הנחת ה-Realizability קיימת $h^* \in \mathcal{H}$ שהיא המתייג האמיתי של הנתונים.

אם הנחת ה-Realizability אכן מתקיימת - זה גורר תוצאה דרמטית על $Err_{\mathcal{D}}$. קיים לפחות אחד $h^* \in \mathcal{H}$ כך ש- $Err_{\mathcal{D}}(h^*) = 0$. כלומר - מודל זה חוזה את כל התגיות בעולם. המשמעות האמיתית: אם הרצנו אלגו' שמוצא $h \in \mathcal{H}$ שמקיים $Err_S(h) = 0$, ואנחנו תחת הנחת ה-Realizability, אזי אנחנו יודעים שקיים פתרון שחזה והשאלה שנשאלת כעת נותרה רק: עד כמה h קרוב ל- h^* האמיתי.

3.7.3 Empirical Risk Minimization (ERM)

מזער סיכונים אמפירי. מדובר בכלל החלטה - כיוון שאיננו מכירים את העולם $\mathcal{D}$, נרצה לבחור את המודל שידוע להצליח הכי טוב עם הנתונים שיש לנו ביד. כלומר,

$$ERM(S) = \operatorname{argmin}_{h \in \mathcal{H}} Err_S(h)$$

כלומר, נחפש את ההיפותזה h שתתן את הערך המינימלי. (נבחין - ERM לבסוף היא פונקציית היפותזה).

♡ אם אנחנו מניחים ריאליזביליות, בהכרח קיים $h^* \in \mathcal{H}$ עבורו $Err_D(h^*) = 0$. בפרט, הוא מושלם גם על תת קבוצה של דגימות מהעולם (המדגם S) ולכן נקבל כי בהכרח:

$$\min_{h \in \mathcal{H}} Err_S(h) = 0$$

המשמעות: במצב ריאליזבילי, אלגוריתם ERM יחזיר תמיד פונקציה עם אפס שגיאות אימון. אם לא מצאת כזו - או שהאלג' לא סרק את כל $\mathcal{H}$ או שההנחה שגויה.

הערה 74. נשים לב - יתכנו מס' פונקציות $h_1, \dots, h_k$ עבורן $Err_S(h_i) = 0 \forall i \in [k]$. אך, יתכן שהן מאוד שונות אחת מהשנייה על נקודות שלא ראינו במדגם - וזהו לב ליבו של נושא ההכללה. איך נדע אילו מבין פונקציות ההיפותזה שואף ל h^* האמיתי?

משפט 75. השגיאה האמפירית $Err_S(h)$ היא אמד חסר הטיה ($Unbiased$) של $Err_D(h)$.
הוכחה:

$$\mathbb{E}[Err_S(h)] = \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n \ell_{0-1}(h(x_i), y_i)\right] = \frac{1}{n} \cdot n \cdot Err_D(h) = Err_D(h)$$

שכן, כיוון שהדגימות לקוחות מאותה התפלגות לכולם אותה התוחלת, וכן נעזרנו בליניאריות התוחלת.

הערה 76. המשמעות של המשפט הקודם - אם היינו לוקחים אנסוף מדגמים שונים בגודל n , ומחשבים לכל אחד את $Err_S(h)$, ממוצע השגיאות הללו היה השגיאה האמיתית.

3.7.4 Version Space and Bad Hypotheses

הגדרה 77. נגדיר את $Version Space$ באופן הבא:

$$VS_{\mathcal{H}, S} = \{h \in \mathcal{H} : Err_S(h) = 0\}$$

כלומר, כל ההיפותזות כך ששגיאת המדגם עליהם היא אפס.

במבט ריאליזבילי, $VS_{\mathcal{H}, S} \neq \emptyset$ בהכרח - אך יתכן שישנם שם "טרמפיסטים".

הגדרה 78. יהי $\varepsilon > 0$. ותהי היפותזה $h \in \mathcal{H}$. תקרא השערה רעה אם $Err_D(h) > \varepsilon$.

מתי היפותזה רעה שורדת בתוך VS ? אם למרות שהיא היפותזה רעה, יצא במקרה שהיא לא טעתה אף פעם על המדגם הספציפי S . מתי דבר שכזה יכול לקרות? כלומר, מהי ההסתברות שקיימת $h \in \mathcal{H}$ כך ש: $Err_D(h) > \varepsilon$ וגם $Err_S(h) = 0$. בעת דגימת מדגם אקראי, תמיד ישנו סיכוי δ שהמדגם הנ"ל מטעה את ERM לבחור היפותזה גרועה. כלומר,

$$Pr[\exists h \in VS : Err_D(h) > \varepsilon] \leq \delta$$

לכן, בהסתברות של לפחות $1 - \delta$ נצליח. מכאן: כל הצהרה שניתן על הדיוק של המודל תמיד תהיה מלווה בסייג מסויים (בהסתברות של לפחות $1 - \delta$).

הגדרה 79. תהי $\mathcal{H}$ מחלקת היפותזות. $\mathcal{H}$ היא PAC-learnable אם קיים אלגוריתם A שמוצא השערה $h = A(S_n)$ עבור מדגם S_n כך שמתקיים:

- לכל $\varepsilon, \delta > 0$ קיים $N_{\varepsilon, \delta}$ (אשר חייב להיות פולינומי ב $\frac{1}{\delta}, \frac{1}{\varepsilon}$) כך שלכל $n > N_{\varepsilon, \delta}$ ולכל התפלגות $\mathcal{D}$ יתקיים:

$$P(\text{Err}_{\mathcal{D}}(A(S_n)) < \varepsilon) > 1 - \delta$$

כאשר ההסתברות מעל מרחבי S_n האפשריים.

אם כן, ישנם שני רכיבים שקובעים האם למידה היא מוצלחת:

רכיב ראשון: δ - קובע את הבטחון. אנחנו לא דורשים כי האלגוריתם יצליח ב 100% מהמקרים. מרשים לו להכשל בהסתברות קטנה δ .

רכיב שני: ε - קובע את רמת השגיאה. לא דורשים מודל שיהיה מושלם בעולם האמיתי, ולכן מוכנים לקבל מודל כל עוד השגיאה שלו היא לכל היותר ε קבוע.

סיבוכיות: N הנ"ל חייב להיות פולינומי. מדוע? כי אם בשביל להגדיל את הדיוק פי 2 צריך פי 2^{1000} דגימות - הלמידה הופכת להיות לא פרקטית. PAC דורש שהגידול בכמות המידע יהיה סביר - פולינומי.

לכן, בשורה התחתונה: **עם מספיק נתונים $n > N$, אנחנו כנראה (בהסתברות $1 - \delta$ לפחות) נוכל למצוא השערה שהיא נכונה בקירוב (שגיאה קטנה מ ε).**

משפט 80. חסם הדגימה למחלקת היפותזות סופית. בהינתן הנחת הריאליזביליות, לכל $\varepsilon, \delta > 0$ אם מס' הדגימות n מקיים:

$$n > \frac{1}{\varepsilon} \ln\left(\frac{|\mathcal{H}|}{\delta}\right)$$

אזי לכל התפלגות $\mathcal{D}$ ולכל מדגם S_n מתקיים:

$$P(\text{Err}_{\mathcal{D}}(A(S_n)) < \varepsilon) > 1 - \delta$$

הוכחה: נסמן ב $\mathcal{H}_{bad}(D, \varepsilon) = \{h \in \mathcal{H} : \text{Err}_{\mathcal{D}}(h) > \varepsilon\}$ את מרחב ההיפותזות הרעות. נרצה לחסום את ההסתברות שה ERM יבחר השערה מתוך קבוצה זו:

$$P(\text{Err}_{\mathcal{D}}(ERM(S_n)) > \varepsilon) = Pr(ERM(S_n) \in \mathcal{H}_{bad}(D, \varepsilon))$$

כיוון שאנחנו מניחים ריאליזביליות, ה ERM תמיד יחזיר השערה המקיימת $\text{Err}_{S_n}(h) = 0$. לכן, כדאי שה ERM יבחר השערה רעה, חייבת להיות לפחות השערה אחת ששרדה את המדגם עם אפס טעויות. נחזור על זה שוב: מה ה ERM עושה? הוא מחפש במרחב ההיפותזות השערה שיש לה 0 טעויות על המדגם S_n . בגלל הנחת הריאליזביליות, ישנה אחת כזו לפחות. לכן הוא תמיד יחזיר אחת כזו כך ש $\text{Err}_{S_n}(h) = 0$. מתי הוא נכשל? כאשר הוא בחר בחירה רעה גדולה מאפסילון בעולם האמיתי. נבחין כי המאורע משמאל מוכל במאורע מימין. מדוע? נניח כי $ERM(S_n) \in \mathcal{H}_{bad}$. אזי, ה ERM גם בחר השערה רעה, וגם הטעות שלה היא אפס. בפרט - הטעות שלה היא אפס. לכן, אותה השערה נמצאת גם במאורע מימין. סה"כ המאורע משמאל מוכל בימין לכן נקבל אי שוויון:

$$P(ERM(S_n) \in \mathcal{H}_{bad}) \leq P(\exists h \in \mathcal{H}_{bad} : \text{Err}_{S_n}(h) = 0)$$

הקיים הופך לאיחוד כמובן. דהיינו, $P(\exists h \in \mathcal{H}_{bad} : \text{Err}_{S_n}(h) = 0) = Pr(\bigcup_{h \in \mathcal{H}_{bad}} \text{Err}_{S_n}(h) = 0)$ מכאן לפי חסם האיחוד יתקיים:

$$Pr\left(\bigcup_{h \in \mathcal{H}_{bad}} \text{Err}_{S_n}(h) = 0\right) \leq \sum_{h \in \mathcal{H}_{bad}} Pr(\text{Err}_{S_n}(h) = 0)$$

כעת, נרצה לחשב את ההסתברות עבור השערה רעה בודדת h . ההסתברות שהיא לא תטעה על אף דגימה, וכיוון שהדגימות הן $i.i.d$ היא:

$$\begin{aligned} Pr(\text{Err}_{S_n}(h)) &= Pr\left(\bigcap_{i=1}^n h(x_i) = y_i\right) = \prod_{i=1}^n Pr(h(x_i) = y_i) = \prod_{i=1}^n 1 - Pr(h(x_i) \neq y_i) \\ &\leq \prod_{i=1}^n (1 - \varepsilon) = (1 - \varepsilon)^n \end{aligned}$$

שכן, כיוון ש h רעה, ההסתברות שהיא תצדק על דגימה בודדת היא לכל היותר $1 - \varepsilon$ (מדוע? אנחנו מניחים פונקציית הפסד 0-1 בנושא זה. וכן מתקיים ש $\mathbb{E}[\ell] = P(\ell(x) = y) = 1 - \varepsilon$ בפונקציה זו). אם כן, נחזור:

$$\sum_{h \in \mathcal{H}_{bad}} Pr(Err_{S_n}(h) = 0) \leq |\mathcal{H}_{bad}| \cdot (1 - \varepsilon)^n$$

קיבלנו:

$$P(Err_{\mathcal{D}}(ERM(S_n)) > \varepsilon) \leq |\mathcal{H}_{bad}| \cdot (1 - \varepsilon)^n \leq |\mathcal{H}_{bad}| \cdot e^{-n\varepsilon}$$

באשר המעבר האחרון נכון מאי השוויון $1 - x \leq e^{-x}$. על מנת שהחסם לטעות יהיה לכל היותר δ נדרוש: $|\mathcal{H}_{bad}| \cdot e^{-n\varepsilon} < \delta$. כלומר אם נפעיל $\ln$ על שני האגפים:

$$-n\varepsilon + \ln(|\mathcal{H}_{bad}|) < \ln(\delta) \iff n > \frac{1}{\varepsilon} \ln\left(\frac{|\mathcal{H}_{bad}|}{\delta}\right)$$

כנדרש. ■

מסקנה 81. כמה מסקנות שנוכל להסיק ממשפט חסם הדגימה:

- כל מחלקה $\mathcal{H}$ סופית היא PAC-Lernable.
- אם n גדל, ε קטן.
- אם δ קטן יותר (רמת בטחון גבוהה יותר) אזי נאלץ להתפשר על ε גדול יותר (שגיאה גדולה יותר שאנחנו נכניס לתחום).
- ככל שיש יותר השערות, כלומר ככל ש $|\mathcal{H}|$ גדול יותר אזי השגיאה ε גדלה. (די אינטואיטיבי: השגיאה תגדל אם אני צריך לבחור מפקוס גדול יותר כי הסיכוי של פונקציה לא טובה להבחר גדול יותר).

הנחנו ריליזיבלי - וזה נחמד, אך זה לא בהכרח המצב. לכן נגדיר:

הגדרה 82. Agnostic הוא מצב שונה מריליזיבלי. קודם לכן הנחנו שקיימת פונקציה מושלמת, כעת נניח שאין כזו. ייתכן שכל ההשערות h יטעו. אנחנו כעת לא מנסים לגרום לטעות ε להתקרב כמה שיותר לאפס אלא להתקרב כמה שיותר להשערה הכי טובה שיש לנו במרחב ההיפותזות. היעד: h^* שיש לה את השגיאה האמיתית הנמוכה ביותר. לא יודעים מיהי, אך שואפים שה ERM ימצא משהו שמתקרב לביצועיה. פורמלית:

$$h^* = \operatorname{argmin}_{h \in \mathcal{H}} Err_{\mathcal{D}}(h)$$

3.7.5 אי שוויון הופדינג

בהינתן משתנים $Z_1, \dots, Z_n$ שהם ה"הפסדים" שחוונו בכל דגימה, דגימות $i.i.d$ בלתי תלויות, וכן משתנים בטווח סגור בין 0 ל 1, אזי: אם נסמן $\bar{Z} = \frac{1}{n} \sum_{i=1}^n Z_i$ (במונחי למידה, מדובר ב Err_S). מתקיים:

$$Pr[|\bar{Z} - \mathbb{E}[Z]| > \varepsilon] \leq 2e^{-2n\varepsilon^2}$$

משפט 83. לכל $\varepsilon, \delta > 0$ וכן $n > \frac{1}{\varepsilon^2} \ln\left(\frac{2|\mathcal{H}|}{\delta}\right)$ והתפלגות $\mathcal{D}$ מתקיים:

$$Pr[Err_{\mathcal{D}}(ERM(S_n)) - Err_{\mathcal{D}}(h^*) < \varepsilon] > 1 - \delta$$

כלומר, ההפרש בין השגיאה שהאלגוריתם מצא לבין השגיאה של ההשערה הטובה בתאוריה הוא קטן מאפסילון בהסתברות טובה. **הוכחה:**

הערה. ההוכחה לא נראתה בהרצאה, אלא סקיצה שלה. נסמן $\hat{h} = ERM(S_n)$ כממזער השגיאה האמפירית. תהי $h^* = \arg \min_{h \in \mathcal{H}} Err_{\mathcal{D}}(h)$ ההיפותזה הטובה ביותר תאורטית.

תחילה, לכל $h \in \mathcal{H}$ קבועה מתקיים

$$Err_S(h) = \frac{1}{n} \sum_i 1[h(x_i) \neq y_i]$$

כלומר אלו n משתני ברנולי, ולקוחת הממוצע שלהם. התוחלת של המשתנים הללו היא שגיאת האמת של ההיפותזה. לכן,

$$\mathbb{E}[Err_S(h)] = Err_D(h)$$

לפי אי שוויון הופדינג נקבל:

$$Pr\left(|Err_S(h) - Err_D(h)| > \frac{\varepsilon}{2}\right) \leq 2 \exp\left(-2n \cdot \left(\frac{\varepsilon}{2}\right)^2\right) = 2 \exp\left(\frac{-n\varepsilon^2}{2}\right)$$

עם זאת, האלגוריתם שלנו לא בוחר היפותזה אחת אלא מסתכל על כל ההיפותזות ב $\mathcal{H}$ ובוחר את הטובה ביותר. נגדיר מאורע "רע" ככזה שקיימת לפחות היפותזה אחת שסטתה מ $\frac{\varepsilon}{2}$. לפי Union Bound נקבל:

$$Pr\left(\exists h \in \mathcal{H} : |Err_S(h) - Err_D(h)| > \frac{\varepsilon}{2}\right) \leq \sum_{h \in \mathcal{H}} 2e^{-\frac{n\varepsilon^2}{2}} = 2|\mathcal{H}| \exp\left(\frac{-n\varepsilon^2}{2}\right)$$

$$\text{נרצה שיתקיים } \delta \geq 2|\mathcal{H}| \exp\left(\frac{-n\varepsilon^2}{2}\right) \Rightarrow \frac{n\varepsilon^2}{2} \geq \ln\left(\frac{2|\mathcal{H}|}{\delta}\right) \Rightarrow n \geq \frac{2}{\varepsilon^2} \cdot \ln\left(\frac{2|\mathcal{H}|}{\delta}\right)$$

$$n \geq \frac{2}{\varepsilon^2} \cdot \ln\left(\frac{2|\mathcal{H}|}{\delta}\right)$$

כנדרש. ■

3.7.6 הטרייד אופ בין השונות לביאס

משפט 84. בהסתברות של לפחות $1 - \delta$ מתקיים:

$$Err_D(ERM(S_n)) < \min_{h \in \mathcal{H}} Err_S(h) + \sqrt{\frac{\ln(2|\mathcal{H}|/\delta)}{2n}}$$

מה המשמעות של המשפט לעיל? השגיאה האמיתית בעולם, היא לכל היותר השגיאה שמדדת במדגם שלך ועוד "קנס" על מורכבות. • איבר ה $bias: \min_{h \in \mathcal{H}} Err_S(h)$, זה כמה טוב המודל הצליח ללמוד את המדגם שנתנו לו. ככל ש $\mathcal{H}$ גדולה יותר, בהכרח $Bias$ יקטן. מדוע? כי בתוך קבוצה ענקית של פונקציות, קל למצוא אחת שמתאימה לנתונים שלך (לעומת זאת: במרחב קטן יתכן שאין אחת שתתאים). • איבר ה $Variance: \sqrt{\frac{\ln(2|\mathcal{H}|/\delta)}{2n}}$, מדובר בממדד הפחד שלנו. כמה אנחנו חוששים שהמדגם שיקר לנו. ככל ש $\mathcal{H}$ גדולה יותר, $Variance$ יגדל. מדוע? ככל ששי יותר פונקציות גדל הסיכוי שאחת מהן תצא תותחית על המדגם במקרה למרות שהיא גרועה בעולם האמיתי (יש יותר פונקציות, יתכן מאוד שהרבה מהן מתאימות לנתונים אלו בלבד).

לכן אנחנו בטרייד אופ - אם $\mathcal{H}$ גדולה ההטייה תהיה אפסית אבל השונות בשמיים. לעומת זאת, אם $\mathcal{H}$ קטנה השונות תקטן אבל ההטייה תזנק כי המודל פשוט מדי.

לכן המטרה היא: למצוא את גודל $\mathcal{H}$ בו הסכום של שניהם הכי נמוך. ובמילים פשוטות: אתה רוצה מודל מספיק חכם כדי להבין את החומר, אבל לא כזה חכם שהוא יתחיל להמציא תיאוריות קונספירציה על סמך כמה דגימות בודדות.

4 הרצאה 4

4.1 מבוא

- בהרצאה זו נדון במודל נפוץ בלמידת מכונה, בשם רגרסיה לינארית ועל הרחבה חשובה בשם Ridge Regression. לפני שנקפוץ למתמטיקה - נדון בשאלה חשובה: מדוע שנסתכל על מודלים לינאריים?
- לפי תאוריית התחום שלמדנו ב-PAC Learning, נעדיף מרחבי היפותזות פשוטים שכן נרצה לעבוד עם פונקציות שאינן מסובכות מדי. כמו כן, ככל שהמודל המתמטי פשוט יותר שגיאת ההכללה שלו "הדוקה יותר", כלומר: הפער בין השגיאה על האימון לבין השגיאה בעולם האמיתי יהיה קטן יותר מה שיוריד את הסיכון ל-Overfitting (למידת יתר של הרעש בנתונים במקום התבנית).
 - עבור נתונים רציפים, מרחב ההיפותזות הלינארי הוא המרחב הטבעי והפשוט ביותר עבור נתונים רציפים. אם יש רצף מספרים - קו ישר לינארי הוא הדרך הבסיסית לתאר קשר בניהם.
 - רגרסיה לינארית הוא המודל היסודי ביותר בלמידה מונחית (למידה מתוך דוגמאות מתוגות של זוגות $((X, y))$).
 - קל מאוד לחשב את המודל.

4.2 רגרסיה לינארית: הגדרת הבעיה

- בסיס הנתונים** - נגדיר קבוצת נתונים שנשמנה D :
- $\{(x_i, y_i)\}_{i=1}^n$ כ- n דוגמאות אימון, כל דוגמה היא זוג של קלט ותיג.
 - הקלט $x_i \in \mathbb{R}^d$ הוא וקטור תכונות, כאשר d מייצג את המימד (כמה מאפיינים יש לכל דוגמה: גובה, משקל, וכו').
 - הפלט: $y_i \in \mathbb{R}$, ערך רציף שאנחנו מנסים לחזות.
- המודל הלינארי**: המודל מורכב משני מאפיינים עיקריים -
- וקטור משקלים $w \in \mathbb{R}^d$, לכל תכונה ב- x יש משקל תואם w .
 - הטיה - Bias, מדובר בסקלר $b \in \mathbb{R}$ שנקרא גם Intercept ומהווה את נק' החיתוך עם הציר.

משוואת החיזוי: עבור דוגמה x_i החיזוי של המודל יסומן ב- $\hat{y}_i$ ויתקיים:

$$\hat{y}_i = f(x_i) = w^T x_i + b$$

מתמטית, נבחין כי ערך זה שקול ל:

$$\hat{y}_i = f(x_i) = \sum_{j=1}^d w_j x_{i,j} + b$$

לצורך דוגמה, נניח ואנחנו מעוניינים לחזות מחירו של בית בהינתן הרבה נתונים: שטח הבית, מס' החדרים, מס' חדרי השירותים, האם יש בריכה, כמה חניות יש? וכדומה, נקבל כל נתון כוקטור, נראה כי ערכים כמו "יש בריכה?" שתשובה אליהם תהיה כן/לא (בינארית) נרצה לסווג לערך נומרי - ע"י (למשל) לקבוע כי $yes = 1, no = 0$. זה די קשה להמיר זאת - ולכן נניח בקורס כי בידינו d פיצ'רים נומריים.

4.3 הגדרה מטריצינית

נרצה להעלים את b , על מנת להקל עלינו במתמטיקה בהמשך. לכן: נרצה "לבלוע" את Bias לתוך וקטור המשקלים. לכן, נשנה את x_i ע"י כך שנוסיף לו קורדינטה ראשונה להיות תמיד 1, וכעת יתקיים: $x_i \in \mathbb{R}^{d+1}$. וכן, נגדיר $w_1 = b$ בוקטור המשקלים וכעת יתקיים $w \in \mathbb{R}^{d+1}$ כאשר $w = (w_1, w_2, \dots, w_{d+1})^T$. כעת, נבחין כי ההגדרות אכן שקולות מתמטית ומתקיים:

$$f(x_i) = w^T x_i = \sum_{j=1}^{d+1} w_j x_{i,j} = w_1 x_{i,1} + \sum_{j=2}^{d+1} w_j x_{i,j} = b + \sum_{j=2}^{d+1} w_j x_{i,j}$$

כעת נקבל:

$$\hat{y}_i = w^T x_i$$

- כמו כן, במקום לחשב חיזוי לכל דוגמה $i \in [n]$ בנפרד נרצה לחשב את כל החיזויים למדגם בבת אחת. לכן נגדיר:
- X תהיה מטריצה מגודל $n \times (d+1)$, כאשר כל שורה במטריצה היא וקטור תכונות של דוגמה אחת $x_i \in \mathbb{R}^{d+1}$.

• יהיה וקטור החיזויים כך ש $\hat{Y}_i = \hat{y}_i = f(x_i)$ לכל $i \in [n]$. נקבל:

$$\hat{Y} = Xw$$

4.4 מציאת הפתרון האופטימלי

נבחין כי מטרתנו תהיה למזער את השגיאות של המודל, כלומר נרצה למזער את ערך פונקציית ההפסד, MSE . נזכר כי:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

בצורה מטריציונית, נכתוב זאת כך:

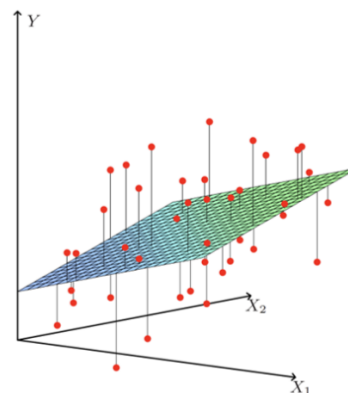
$$MSE = \frac{1}{n} (y - Xw)^T (y - Xw)$$

נחפש את וקטור המשקולות $\hat{w}$ כך שיתקיים:

$$\hat{w} = \operatorname{argmin}_w MSE(w)$$

נבחין כי MSE היא פונקציה קמורה וריבועית של w , כלומר יש לה ערך מינימום גלובלי יחיד. לכן, נקח את הגרדיאנט ונשווה לאפס. כלומר: $\nabla_w MSE = 0$.

מה המטרה שלנו בעצם? למצוא את הקו שסכום המרחקים מהקו יהיה מינימלי, כיצד זה נראה גאומטרית ממממד גדול יותר?



המישור שבתמונה הוא המישור שבו מרחב הפתרונות שלנו נמצא, ונרצה למצוא מישור שבו סכום ההטלות של כל הדגימות מהמישור יהיה **קטן ככל הניתן**.

ובכן, נתחיל בלפשט את הביטוי:

$$\begin{aligned} MSE(w) &= \frac{1}{n} (y - Xw)^T (y - Xw) = \frac{1}{n} (y^T - w^T X^T) (y - Xw) = \frac{1}{n} (y^T y - y^T Xw - w^T X^T y + w^T X^T Xw) \\ &= \frac{1}{n} (y^T y - 2w^T X^T y + w^T X^T Xw) \end{aligned}$$

באשר המעבר האחרון דורש הסבר. נבחין, $y^T Xw$ הוא סקלר. מדוע? $y^T \in \mathbb{R}^{1 \times (d+1)}$ וכן $Xw \in \mathbb{R}^{(d+1) \times 1}$ ולכן $y^T Xw \in \mathbb{R}$. באופן דומה, $w^T X^T y$ הוא סקלר. כיוון שלכל סקלר מתקיים $c = c^T$ אם נפעיל על הראשון טרנספוז נקבל: $(y^T Xw)^T = w^T X^T y$, ומכאן שהמעבר האחרון די ברור.

נגזר איבר איבר:

- $\nabla(y^T y) = 0$ שכן גזרים לפי w .
 - $\nabla(-2w^T X^T y) = -2X^T y$, שכן: אם נסמן $a = X^T y$ נקבל $-2w^T a$. נזכר כי $\nabla_w(w^T a) = a$ ונקבל שהנגזרת היא $-2X^T y$.
 - $\nabla(w^T X^T X w) = 2X^T X w$. שכן: עבור A סימטרית מתקיים $\nabla(w^T A w) = 2Aw$, וכן $X^T X$ תמיד מטריצה סימטרית לפי לינאריות, ומכאן שהנגזרת אכן $2X^T X w$.
- סה"כ קיבלנו:

$$\nabla_w(MSE) = \frac{2}{n}(X^T X w - X^T y) = 0 \iff X^T X w = X^T y$$

מדובר בפתרון הסופי. כעת, אם נניח כי $X^T X$ היא מטריצה הפיכה אכן נקבל ע"י הכפלה ב- $(X^T X)^{-1}$ ונקבל:

$$\hat{w} = (X^T X)^{-1} X^T y$$

כעת, כאשר נרצה לחזות את ערך y_{new} על דטה חדש, כלומר X_{new} כל שנעשה יהיה:

$$Y_{new} = X_{new} \hat{w} = X_{new} [(X^T X)^{-1} X^T y]$$

4.5 $\hat{y}$ הוא הטלה אורתוגונלית ובפרט ממזער את השגיאה

כעת נטען, כי החיזוי $\hat{y}$ הוא ההטלה האורתוגונלית של y על המישור של X .
הוקטור y הוא וקטור שמייצג את התשובה האמיתית שקיבלנו מהנתונים. הוא נמצא איפשהו במרחב. המטריצה X מגדירה באמצעות העמודות שלה מישור בתוך המרחב. החיזוי $\hat{y}$ הוא מוגדר לפי המודל $y = Xw$ ולכן בהכרח $\hat{y}$ הוא קומבינציה לינארית של עמודות X , כלומר $\hat{y}$ חייב להיות נקודה שיושבת במישור של X . מה הבעיה? בדרך כלל - הוקטור y לא נמצא בדיוק על המישור של X . המטרה שלנו בגרסיה היא למצוא נקודה בתוך המישור ($\hat{y}$) שתהיה הכי קרובה ל- y שנמצא מחוץ למישור.

כיצד נמצא את הנקודה הכי קרובה ל- y מחוץ למישור? מורידים אנך, בהכרח זו הנקודה הכי קרובה למישור מחוץ אליו. מדובר בקו ישר שמחבר את y למישור בזווית של 90° ויפגוש את $\hat{y}$ בזווית זו. המרחק הזה (הקו שמחבר בין השניים) הוא וקטור השגיאה $e = y - \hat{y}$. מעבר לאינטואיציה, נראה הוכחה פורמלית של הדבר:

נזכר - שני וקטורים מאונכים אמ"מ המכפלה הפנימית בניהם שווה לאפס. נרצה לבדוק את המכפלה בין וקטור השגיאה e לבין החיזוי $\hat{y}$. אם כן:

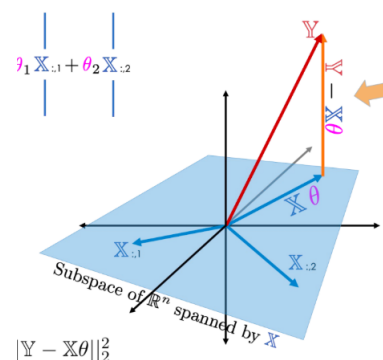
$$\langle y - \hat{y}, \hat{y} \rangle = (y - \hat{y})^T \hat{y} = (y - \hat{y})^T X \hat{w} = y^T X \hat{w} - \hat{y}^T \hat{y} =$$

נבחין כי $\hat{y}^T \hat{y}$ ניתן לפירוק כך: $\hat{y}^T = (X \hat{w})^T = \hat{w}^T X^T$ ומכאן: $\hat{y}^T \hat{y} = \hat{w}^T X^T X \hat{w}$. נציב מפורשות את $\hat{w}$ משמאל: $(y^T X (X^T X)^{-1}) X^T X \hat{w}$.
נראה כי קיבלנו $(X^T X)^{-1} X^T X = I$ ומכאן: $y^T X \hat{w}$.

$$= y^T X \hat{w} - y^T X \hat{w} = 0$$

כלומר הוכחנו כי $\langle y - \hat{y}, \hat{y} \rangle = 0$.

כלומר, הראינו כי וקטור השגיאה מאונך לוקטור החיזוי. וכיצד זה עוזר? זה גורר כי בהכרח מצאנו את הפתרון האופטימלי במסגרת הפתרון הלינארי - שכן המרחק הקצר ביותר בין נקודה (y) לבין מישור (מרחב ההשערות של המודל, X) הוא הקו המאונך אליו. כלומר הוכחנו פורמלית כי $\hat{y}$ הוא הנקודה במישור שמרחקה מ- y מינימלית.
במילים אחרות: **גרסיה לינארית היא הטלה אורתוגונלית על המרחב X .**



4.6 רגרסיה פולינומיאלית

כיצד נשתמש במודל לינארי על מנת ללמוד על קשרים שאינם לינאריים? נשתמש ברגרסיה פולינומיאלית, ובאופן מפתיע כלל לא נצטרך לשנות את המודל אלא בסך הכל להתאים לו את הנתונים. לכן, אנחנו ניצור תכונות חדשות ע"י העלאה בחזקה של הקלט המקורי שלנו. כלומר, אם קיבלנו קלט $x \in \mathbb{R}$ נתאים לו אחים $x^1, x^2, \dots, x^k$ וניצור פולינום מדרגה k :

$$\hat{y} = w_0 + w_1 x^1 + \dots + w_k x^k$$

למרות שהגרף נראה כעקומה, המודל **כן לינארי** ביחס לנתונים שכן x^2 מבחינת המודל הוא תכונה של הוקטור, ולא העלאה בריבוע של הקלט. נבנה מטריצה חדשה:

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ 1 & x_2 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^k \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{pmatrix}$$

זו מטריצת ונדרמונדה על הערכים $(x_1, \dots, x_k)$. כעת, וקטור התחזיות יקיים גם הוא $\hat{w} = (X^T X)^{-1} X^T Y$

4.7 Ridge Regression

קיים פתרון חיזוי $\hat{w}$ אם, ורק אם $X^T X$ היא מטריצה הפיכה. אם כן: מטריצה זו היא הפיכה אמ"מ עמודות המטריצה X ב"ת לינארית. כלומר X צריכה להיות מדרגת עמודות מלאה. (אין אף תכונה "מיותרת" שניתן להרכיבה משילוב תכונות אחרות). ישנם שני מקרים בהם לא קיימת הופכית:

- $d > n$, במקרה זה יש לנו יותר תכונות מאשר דטה אימון. לצורך ההמחשה - אם ננסה להעביר קו ישר דרך נקודה אחת. יש אנסוף קווים כאלו. יש לנו עודף של חופש - ואין פתרון.
- $n > d$ אך אחת התכונות היא שילוב מושלם של אחרות. זה נקרא **Multicollinearity**. למשל: אם יש לנו עמודה של גובה בס"מ וגובה באינץ'. אלו נתונים כפולים - שיוצרים במטריצה לאחר דירוג שורת אפסים - והופכים אותה ללא הפיכה.

לשם כך, יש לנו פתרון: Shrinkage. במקום לחפש את השגיאה הכי נמוכה (MSE), נוסיף "קנס" על גודל המשקלים. הוספת קנס שכזה גורמת למטריצה להפוך להפיכה תמיד - והבעיה הזו נפתרת לנו. זה כאילו שהוספנו ערך קטן לאלכסון המטריצה, מה שמבטיח שנוכל לחשב את $\hat{w}$. כמו כן - הקנס הנ"ל מושך את המשקלים כלפי מטה ומונע מהם לגדול ללא שליטה. זה הופך את המודל ל**רובוסטי**, הוא פחות רגיש לתנודות קטנות בנתונים. לפתרון זה קוראים Ridge Regression:

$$\hat{w}_{ridge} = \operatorname{argmin}_w [(y - Xw)^T (y - Xw) + \lambda \|w\|_2^2]$$

באשר, $\lambda > 0$ הוא היפר פרמטר ששולט בכמה שאנחנו "מכאיבים" למודל על משקולות גדולות. וכן, $\|w\|_2^2 = \sum_{i=1}^d w_i^2$, על מנת למזער ביטוי זה, נגזור אותו: כפי שראינו קודם הנגזרת ללא הל היא: $2X^T Xw - 2X^T y$ וכן

$$\nabla_w (\lambda \|w\|_2^2) = 2\lambda w$$

לכן הנגזרת:

$$X^T X w - X^T y + \lambda w = 0 \implies (X^T X + \lambda I) w = X^T y \implies \hat{w} = (X^T X + \lambda I)^{-1} X^T y$$

משפט 85. לכל מטריצה X ולכל $\lambda > 0$ מתקיים כי $X^T X + \lambda I$ הפיכה.
הוכחה: אנו יודעים כי הערכים העצמיים של $X^T X$ הם תמיד אי שליליים, שכן:

$$a \langle v, v \rangle = \langle av, v \rangle = \langle X^T X a, v \rangle = \langle Xv, Xv \rangle = \|Xv\|^2 \implies a = \frac{\|Xv\|^2}{\langle v, v \rangle}$$

אם $a = 0$, אזי כעת ע"ע תואם של $X^T X + \lambda I$ יהיה $\lambda > 0$. אחרת, נקבל כי בהכרח $\langle v, v \rangle > 0$ ולכן $a > 0$. סה"כ הע"ע של $X^T X$ הם אי שליליים.
 מכאן, כיוון ש $\lambda > 0$ בהכרח לכל ע"ע a של $X^T X$ מתקיים כי $a + \lambda$ הוא ע"ע של $X^T X + \lambda I$ וכן כעת $a + \lambda > 0$ ולכן הע"ע של המטריצה בהכרח אינם אפס, ולכן היא הפיכה.

■

4.8 בפרספקטיבה של SVD

- כל מטריצה (מעל המרוכבים) ניתנת לפירוק לשלוש מטריצות $X = UDV^T$ כך ש:
- U היא מטריצה שמורכבת מוקטורים עצמיים או"ג של $X^* X$. מטריצה אורתוגונלית, כלומר $U^T U = I$.
 - D היא מטריצה אלכסונית של ערכים סינגולריים σ_i שקובעים כמה כל כיוון בנתונים הוא חזק / משמעותי.
 - V היא מטריצת הכיוונים של הנתונים. מטריצה אורתוגונלית, כלומר $V^T V = I$.

נתבונן ב $X^T X$:

$$X^T X = (UDV^T)^T (UDV^T) = V D^T U^T U D V^T = V D^T D V^T = V D^2 V^T$$

כאשר D^2 היא מטריצה חדשה שמוגדרת להיות $D^2 = D^T D$ שכן היא אלכסונית ריבועית, ועל אלכסונה הערכים σ_i^2 . כעת, נסתכל על הפתרון לפי Ridge Regression:

$$X^T X + \lambda I = V D^2 V^T + \lambda I = V D^2 V^T + \lambda V I V^T = V (D^2 + \lambda I) V^T$$

כעת, נסתכל על המטריצה ההופכית למטריצה זו. לפי הכלל $(\prod_{i=1}^n A_i)^{-1} = A_n^{-1} A_{n-1}^{-1} \cdot \dots \cdot A_1^{-1}$ נקבל:

$$(X^T X + \lambda I)^{-1} = [V (D^2 + \lambda I) V^T]^{-1} = (V^T)^{-1} (D^2 + \lambda I)^{-1} V^{-1}$$

אך כיוון ש V אורתוגונלית מתקיים $V^T V = I$ ולכן $V^{-1} = V^T$ וכן $(V^T)^{-1} = V$ כלומר:

$$(X^T X + \lambda I)^{-1} = V (D^2 + \lambda I)^{-1} V^T$$

כעת,

$$\hat{y} = X \cdot (X^T X + \lambda I)^{-1} \cdot X^T y = U D V^T \cdot V (D^2 + \lambda I)^{-1} V^T \cdot V D^T U^T y =$$

$$U D (D^2 + \lambda I)^{-1} D U^T y$$

כעת, D ו- $(D^2 + \lambda I)^{-1}$ הן מטריצות אלכסוניות. לכן מכפלתן $D(D^2 + \lambda I)^{-1}D$ לפי הגדרה ישירה ולכל איבר על האלכסון מתקיים:

$$\sigma_i \cdot \frac{1}{\sigma_i^2 + \lambda} \cdot \sigma_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

מכאן, כיוון ש- U היא מטריצה של וקטורים אורתוגונליים שמהווים בסיס, אשר נסמנו $\{u_1, \dots, u_d\}$ נקבל:

$$\hat{y}_{ridge} = \sum_{i=1}^d u_i \left(\frac{\sigma_i^2}{\sigma_i^2 + \lambda} \right) u_i^T y$$

מה זה אומר בתכלס?

• הביטוי $u_i^T y$ הוא ההטלה של y על הכיוון u_i . המקדם, הוא "הווליום" בו אנחנו משאירים את המידע הנ"ל. אם σ_i גדול ביחס לג השבר קרוב לאחד (המידע כמעט יעבור במלואו), ואם σ_i קטן מאוד (רעש) השבר שואף לאפס. (המידע נמחק).

4.9 Binary classification

קלט: נניח סט של דוגמאות מתיוגות $\{(x_i, y_i)\}_{i=1}^n$. במקרה זה, נדון בבעיית סיווג. כלומר y_i איננו ערך מספרי כמו ברגסיה אלא שיוך לקבוצות. לכל $i \in [n]$, $x_i \in \mathbb{R}^d$ וכן, $y_i \in \{+1, -1\}$ (למשל: חולה, בריא וכו').

מטרתנו **פלסט** למצוא פונקציה $f: X \rightarrow Y$ כך ש:

1. f מסכימה עם נתוני האימון, כלומר עבור x_i שראינו תוציא y_i תואם לו.
2. תבצע הכללה - בהינתן $x \notin \{x_i\}_{i=1}^n$ תנחש נכונה אם $y = 1$ או $y = -1$ בהתאמה.

נבחין כי בסיווג, בניגוד לרגסיה שם חיפשנו פונקציה רציפה שתעבור בין כל הנקודות, כאן נרצה למצוא מעין "קו" שחותך את המרחב ל-2: כל מי שמעליו +1 ומתחתיו -1.

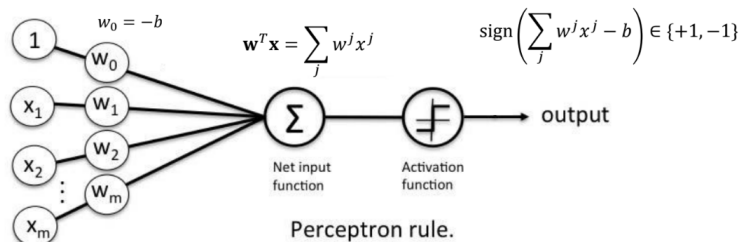
4.10 Linear-threshold neuron

כעת, נראה כיצד מערכת מיוחדת - הניורון הבודד, הופכת קלט מספרי להחלטה של כן או לא. התהליך מתבצע בשלבים הבאים:

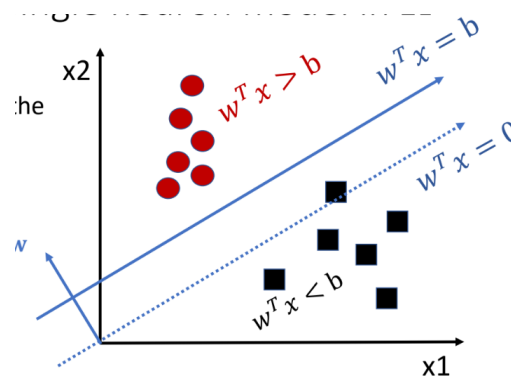
- קלט x_j : תכונות $(x_1, \dots, x_m)$.
- משקולות: כל תכונה מקבלת משקל כמה היא משפיעה על ההחלטה.
- $bias$: מתקיים $w_0 = -b$, שמתאים לערך $x_0 = 1$. מדובר בסף (Threshold): כמה חזק צריך להיות הקלט כדי שהניורון ירה.
- הניורון מחשב ממוצע משוקלל של התכונה כתלות במשקל כך: $w^T x = \sum_j w^j x^j$.
- על מנת לעבור ממס' רציף לסיווג בינארי משתמשים בפונקציית $sign$. שמוגדרת כך: $sign(\sum_j w^j x^j - b)$. אם תוצאת הסכימה גדולה מהסף b נקבל 1 ואחרת אפס. מעט יותר פורמלית:

$$h_{w,b}(x) := \begin{cases} +1 & w^T x > b \\ -1 & w^T x < b \end{cases} = \begin{cases} +1 & \sum_j w^j x^j - b > 0 \\ -1 & \sum_j w^j x^j - b \leq 0 \end{cases}$$

את הפונקציה $sign$ נגדיר באמצעות מרחב ההיפותזות $h_{w,b}(x)$. כך שבהינתן w, b ו- x נחזיר פלט בהתאם. גאומטרית: אנו מותחים קו ישר במישור, מעליו $(w^T x - b > 0)$ נסווג 1 ומתחתיו 0. חשוב להבחין כי w^j - הרכיב j בוקטור w . לא חזקה.



אם נתבונן ב- $\mathbb{R}^2$, הסיווג יראה כך:



נדמיין מצב של סוכריות ואדומות. אם נמצא קו ישר שנעבירו וכל מה שיהיה מתחתיו יסווג לצבע אחד ומתחתיו לצבע שני - הצלחנו. מתי מתחילה הבעיה? מה אם הסוכריות הכחולות נמצאות במרכז, והאדומות מקיפות אותן במעגל? לא משנה מה - לא נצליח להעביר קו ישר. מכאן שנקבל מסקנה: ניורון בודד יכול להגיע לדיוק מושלם אם ורק אם הנתונים פרידים לינארית. ניורון בודד לא מסוגל להתעסק, לצירי עיגול וכו'.

כעת, נבצע טריק מתמטי דומה לזה שביצענו ברגרסיה: נגדיר מעתה כי $x = (1, x_1, \dots, x_d)^T$ וכן $w = (-b, w_1, \dots, w_d)^T$. כעת, $w^T x$ מייצג את אותו הביטוי כמו קודם. כשהיה לנו b נפרד, קו ההפרדה היה יכול להיות בכל מקום במרחב. כעת - במרחב החדש שיצרנו: המישור המפריד תמיד עובר דרך הראשית.

4.11 The perceptron algorithm (אלגוריתם הפרספטרון)

מדובר באחד מן האלגוריתמים הוותיקים והאלגנטיים ביותר בבינה מלאכותית. נציג את הפסודו הבא:

Algorithm 1 Perceptron Algorithm

```

Initialize  $w$  randomly
while not all samples are classified correctly do
  for  $i = 1$  to  $n$  do
    Predict  $\hat{y}_i = \text{sign}(w^T x_i)$ 
    if  $y_i \neq \hat{y}_i$  then
      if  $y_i > 0$  then
         $w \leftarrow w + x_i$ 
      else
         $w \leftarrow w - x_i$ 
      end if
    end if
  end for
end while

```

מדובר באלגוריתם עבור הגרסה שבה $y_i \in \{+1, -1\}$. הגרסה שבה $y_i \in \{0, 1\}$ מעט שונה.

מה עושה האלגוריתם?

האלגוריתם מאתחל וקטור משקולות אקראי w , בשלב זה כנראה שנטעה המון. האלגוריתם עובר שוב ושוב על כל הדגימות - עד שכל הדגימות מסווגות נכון. עבור כל דגימה (x_i, y_i) :
 א. מנבאים את מה שהמודל יגיד כרגע
 ב. בודקים טעות: אם המודל צדק, ממשיכים הלאה. אם המודל טעה - מעדכנים בהתאם: אם היינו אמורים לקבל +1 וקיבלנו -1 נוסף x_i למשקולות. אם היינו אמורים לקבל -1 וקיבלנו +1 נחסיר את x_i מהמשקולות.

נבחין כי ניתן להבין את האלגוריתם משתי דרכים:

- כשמחסירים או מוסיפים x_i ל w - מסובבים את וקטור המשקולות w לכיוון הנקודה שבה טעינו. כלומר: מזיזים את קו ההפרדה כך שבפעם הבאה "יחבק" את הנקודה הזו בצד הנכון שלה.
- נסיון למזער פונקציית הפסד - כל פעם שיש טעות: מנסים לעשות צעד אחד בכיוון של הקטנת הטעות.

כעת ננסה לענות על השאלה הבאה:

מדוע הפעולה הפשוטה של חיבור או חיסור וקטורים גורמת למודל ללמוד?

נבחין שבזכות הקשר הבא ניתן להבין זאת:

$$w^T x = \|w\| \cdot \|x\| \cdot \cos(\theta)$$

כאשר θ היא הזווית בין הוקטורים x, w .

- אם הזווית בין הוקטורים קטנה מ-90° $\cos(\theta) > 0$ ואז $w^T x > 0$ ולכן החיזוי הוא +1.
- אם זווית זו קטנה גדולה מ-90° (קהה): $\cos(\theta) < 0$ ואז $w^T x < 0$ ולכן החיזוי הוא -1.

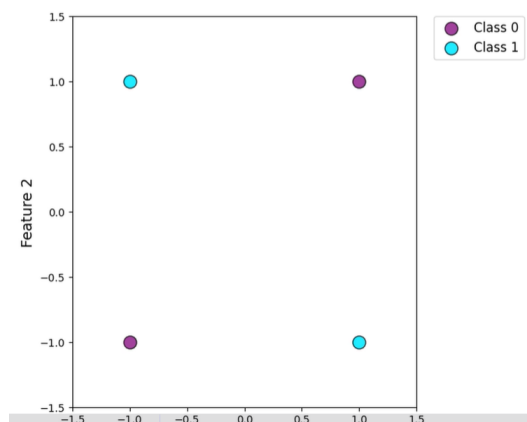
מה קורה כאשר בידינו טעות?

נניח כי ישנה נקודה x_i שהתווית האמיתית שלה $y_i = +1$ אך המודל חזה -1. משמעות הדבר: הזווית בין w ל- x גדולה מדי! לכן, אנחנו מעדכנים $w_{new} = w + x_i$. מה קורה למכפלה הסקלרית כעת?

$$w_{new}^T x_i = (w + x_i)^T x_i = w^T x_i + x_i^T x_i = w^T x_i + \|x_i\|^2$$

כיוון ש $\|x_i\|^2 > 0$ אנחנו תמיד מגדילים את ערך המכפלה הסקלרית. הגדלת המכפלה - לפי הקשר ששמנו לב אליו: הקטנת הזווית - כלומר - גורמים לחיזוי לשאוף כעת ל-1! הוקטור מסתובב לכיוון הנקודה שבה טעה על מנת שבפעם הבאה יהיה קרוב יותר לצד הנכון שלה. נכונות: לא נראה נכונות, אך ישנה הוכחה לכך שאם הנתונים אכן פרידים לינארית - האלגוריתם עוצר ומגיע ל-0 טעויות.

כעת, נתבונן בדוגמה מאוד פשוטה מתי האלגוריתם לא יעבוד. מתי? כאשר הנתונים לא פרידים לינארית. נתבונן ב- Xor : אי אפשר להעביר שום קו בין הנקודות ולכן הפרספקטון לא יתכנס - שכן הנתונים לא פרידים לינארית.



4.12 Gradient Descent

בסופו של דבר, למידה היא בסך הכל בעיית אופטימיזציה של מינימום. בפתרונות האנליטיים של אלגברה לינארית ניתן לגזור, למצוא נוסחה סגורה ע"י השוואה לאפס. אבל - כאשר המודלים הופכים למורכבים יותר ויותר בלתי אפשרי לפתוח זאת בדף ועט. לכן: עוברים לשיטה איטרטיבית של ניסוי וטעייה חכמים.

אך, על מנת שהמחשב יוכל ללמוד עלינו לתת לו שני דברים:

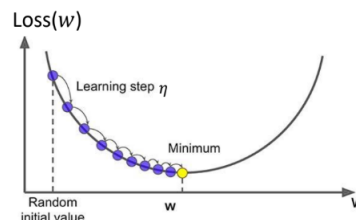
- פונקציית הפסד: פונקציה $L(w)$ שמקבלת את המשקולות ומחזירה מס' בודד. המס' הזה אומר לנו כמה המודל גרוע. מטרתנו - למזער את הערך הזה.
 - אלגוריתם חיפוש: שיטה למצוא את w שנותן את ה- L המינימלי.
- אם פונקציית ההפסד היא דיפרנציאלית - אזי ניתן לגזור אותה. ניתן להשתמש בנגזרות על מנת לדעת להיכן ללכת. הגרדיאנט מצביע לכיוון שבו הפונקציה עולה הכי מהר. ונעזר בכך. על מנת להגיע למינימום של פונקציית ההפסד, אנחנו נבצע עדכון איטרטיבי למשקולות. בכל שלב:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

נפרק את מה שכתוב כאן:

- $\frac{\partial L}{\partial w}$: זהו הגרדיאנט, וקטור הנגזרות החלקיות. הוא מצביע על הכיוון בו הפונקציה עולה הכי מהר.
- הסימון η - כיוון שאנחנו רוצים למזער ולהגיע למינימום, נרצה ללכת בכיוון ההפוך לכיוון הגרדיאנט! η הוא גודל הצעד - הוא קובע כמה אנחנו סומכים על הכיוון שמצאנו בכל פעם.

לצורך ההמחשה, ניתן לראות כמעין כדור שמתגלגל במורד פרבולה. אם η קטן מדי, נגיע למינימום - אך זה יקח הרבה מאוד זמן (צעדים). אם η גדול מדי - יתכן שנדלג מעל המינימום. בכל שלב אנחנו יורדים מכיוון הגרדיאנט, על מנת להגיע למינימום.



מתי זה בוודאות יעבוד לנו?

1. אם הפונקציה תהיה קמורה חזקה (כמו קערה), בפונקציות שכאלו אין בורות מקומיים שנתקע בהם ויש מינימום גלובלי יחיד.
2. חסם על גודל הצעד: $0 < \eta < \frac{1}{L}$ כאשר L הוא קבוע ליפשיץ' של הגרדיאנט. מדובר במדד לכמה מהר השיפוע משתנה. נרצה שלא להשתגע - לקחת צעדים קטנים מאוד כשי לא לאבד שליטה.

4.13 GD במימד גבוה $\mathbb{R}^d$

נזכר כי כאשר וקטור המשקולות מכיל הרבה פרמטרים $\{w_i\}_{i=1}^d$ לא נוכל להסתפק בנגזרת אחת ולשם כך מוגדר הגרדיאנט:

$$\nabla l = \left(\frac{\partial l}{\partial w_1}, \frac{\partial l}{\partial w_2}, \dots, \frac{\partial l}{\partial w_d} \right)^T$$

אם הגרדיאנט רציף, הוא מצביע לכיוון העלייה הכי חדה. לכן, באופן שקול, $-\nabla l$ הוא הכיוון היעיל ביותר להקטנת ההפסד באופן מקומי. כעת כלל העדכון יהיה:

$$w_{t+1} \leftarrow w_t - \eta \nabla l$$

באשר:

- w_{t+1} הם המשקולות החדשות.
- w_t הם המשקולות הנוכחיות.
- η קובע כמה "חזק" נדחוף את כל המשקולות יחד לכיוון החדש.
- נבחין כי התהליך מתבצע באופן איטרטיבי, והעדכון קורה לכל המשקולות בו זמנית.

4.14 Stochastic Gradient Descent (SGD)

ישנה בעיה: על מנת לבצע עדכון אחד בודד של המשקולות, עלינו לחשב את הגרדיאנט על פני כל n הדגימות במערכת:

- $TotalError: \sum_{i=1}^n l(w, x_i, y_i)$ - סכום כל ההפסדים של כל הדגימות.
- עדכון המשקולות: $w^{t+1} = w^t - \eta \nabla TotalError(w^t)$ (פחות הגרדיאנט של פונקציית ההפסד על המשתנה w^t).

כאשר n הוא גדול מאוד - חישוב הגרדיאנט הופך להיות מתיש חישובית עבור עדכון בודד. הפתרון: במקום לחשב את הגרדיאנט על כל המידע, אנחנו עושים קירוב באמצעות קבוצה קטנה ואקראית של דגימות שנקראת mini batch (גודל הקבוצה b יקיים $b < n$). כעת,

$$BatchError: \sum_{i=k}^{k+b} l(w, x_i, y_i) - \text{סוכמים רק את הפסד ה batch הנוכחי. ומעדכנים:}$$

$$w^{t+1} = w^t - \eta \nabla BatchError$$

כלומר, SGD מאפשר לנו להתקרב למינימום באמצעות צעדים מהירים הרבה יותר - כי כל צעד דורש הרבה פחות חישובים וללא מעבר על כל מסד הנתונים.

4.15 הטריידאוף בין GD לSGD

נדון בשני מקרי קצה.

- $Batch = n$ (GD) כאן אנחנו משתמשים בכל הדטה בכל צעד. יתרונות: מסלול יציב וחלק ישירות למינימום, קל לחישוב מקבילי ב GPU כי זו מכפלת מטריצות ענקית.

חסרונות: איטי בטירוף. אם הדטה גדול מדי - הוא לא יכנס בזיכרון המחשב (RAM) והאלגוריתם יקרוס.

- $Batch = 1$: כאן אנחנו מעדכנים באמצעות שימוש בדגימה בודדת בכל צעד. יתרונות: כל צעד הוא מידי ומהיר, ה"רעש" עוזר למודל שלא להכנס לבורות אלא לקפוץ מהם החוצה. חסרונות: המסלול שעושה האלגוריתם נראה כמו זיגזג פרוץ - קשה להגיע לדייק סופי מוחלט וזה לא מנצל היטב את הכוח של GPU .

הפתרון: Mini Batch SGD - נקח $1 < b < n$. זו הדרך הכי טובה:

- זיכרון: נכנס בקלות לזיכרון GPU .
- יעילות: מאפשרת לבצע כפל מטריצות מקבילי ומהיר מאוד.
- יציבות: מספקת גרדיאנט יציב מספיק כדי לא להתפרע אך משאירה מעט רעש שעוזרת ללמידה לא להתקע.

הערה 86. אז רעש זה טוב למודל? עד כה אמרנו שרעש בעייתי לנו ומהדיון כאן ניתן להבין שאנחנו צריכים להתפלל לרעש - ובכן: בתוך תהליך אופטימיזציה (חיפוש אחר מינימום) רעש הוא לא בהכרח רע לנו. ננסה לדמיין סיפור יפה להמחשה: נניח כדור שמתגלגל לתחתית של הר. אך ההר מלא בבורות קטנים לאורך הדרך. ב GD רגיל - אין רעש: הכדור מתגלגל בצורה חלקה ומושלמת - אם הוא נכנס לבור קטן, שאיננו התחתית האמיתית של ההר - הוא יעצור שם. אין לו כוחות לצאת מהבור, כי השיפוע בתוך הבור אומר לו להישאר במרכז. לעומת זאת: ב SGD בגלל שכל צעד מבוסס על קירוב, הכדור לא מתגלגל אלא רועד וקופץ תוך כדי תנועה: הרעידות הן הרעש. אם הכדור נתקע בבור קטן, אחת הרעידות האקראיות האלו יכולה לתת לו מספיק תנופה בכדי לצאת החוצה ולהמשיך להתגלגל מטה עד למינימום האמיתי - תחתית ההר. זה כמובן משל. מסקנה: רעש בגרדיאנט הוא טוב כי משמש בריחה ממלכודות מתמטיות בדרך לפתרון האופטימלי. אך: גם יותר מדי רעש זה לא טוב ($b = 1$) מהסיבות שצוינו. לכן נשאל את עצמנו: איך מוצאים גודל $batch$ טוב מספיק? וזה האומנות של למידת מכונה עליה נדון בהמשך.

האם SGD באמת מתכנס לפתרון?

• GD : ההתכנסות טבעית. כיצד? ככל שאנחנו מתקרבים לתחתית הקערה - המינימום, השיפוע הופך להיות מתון יותר. לכן: הגרדיאנט שואף לאפס והצעדים הופכים להיות קטנים יותר ויותר אוטומטית. מתי עוצרים? כאשר השינוי בהפסד אפסי ($|L_t - L_{t-1}| < \epsilon$) או שהגרדיאנט קרוב מאוד לאפס.

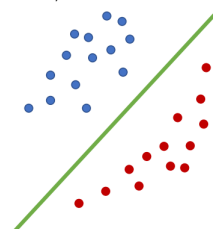
• SGD : באופן תאורטי - הוא לא באמת מתכנס! מדוע? בגלל שאנחנו דוגמים רק חלק מהנתונים, הגרדיאנט הטוכסטי כמעט לעולם לא יהיה אפס. אם אם הגענו למינימום הגלובלי - $Batchn$ הבא שנגריל יאמר לנו: "זו קצת ימינה" ואחריו יגיד "זו קצת שמאלה" וכו'. המודל ימשיך לקפץ לנצח מסביב למינימום - Variance ball. איך בכל זאת עוצרים? Learning Rate Decay

אנחנו מכריחים את הצעדים להצטמצם ע"י הקטנת η לאורך הזמן. למשל: $\eta_t = \frac{\eta_0}{t}$: קצב הלמידה באיטרציה t שווה לקצב ההתחלתי חלקי הזמן. ההגיון מאחורי זה הוא שבהתחלה אנחנו מרשים למודל להשתולל על מנת ללמוד את המרחב, אך ככל שהזמן עובר אנחנו מקטינים את הצעדים על מנת שייאלץ להתיישב בנקודה אחת יציבה. כלומר צריך להרגיע את המשימה לבד בכוח.

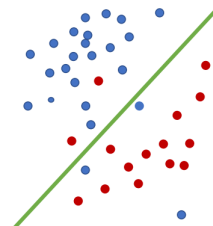
4.16 Linear separability

נתבונן במספר מקרים:

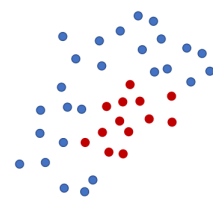
1. **מקרה ראשון:** Linearly separable data הנתונים ניתנים להפרדה לינארית. זהו המצב האידיאלי - קיים בהכרח וקטור משקולות w וסף b כך שהמודל שלנו יצדק ב-100% מהמקרים. זהו המקרה ה"ריאליזבלי". במקרה זה נוכל ונרצה להשתמש בפרספטרום.



2. **מקרה שני:** Measurement and label noise: רעש במדידה ובתיג. כאן אנחנו למעשה מתחילים להתקרב למציאות, אך הנתונים למרות שבבסיסם נראים לינארית ישנם כמה נקודות "סוררות". כנראה - שהסיבות לכך הם טעויות אנוש בתיג או מכשירי מדידה ישנים. הבעיה - קו ישר לא יכול להפריד בניהן בצורה מושלמת. אם ננסה בכוח להפרידם - נקבל מודל עקום מדי. במקרה זה נרצה לעבוד עם גישה הסתברותית להתמודד עם הרעש.



3. **מקרה שלישי:** נתונים לא לינאריים. זהו המקרה המורכב ביותר - הנקודות מסודרות בצורה ששום קו ישר, לא משנה היכן נשים אותו לא יצליח להפריד בניהן בצורה טובה. לכן במקרה זה המודל הלינארי לא מספיק טוב ונצטרך לעבור למודלים מורכבים יותר.



במקרה הראשון - ה-GS וה-SGD ימצאו את הקו בקלות. במקרים השני והשלישי, פונקציית ההפסד לעולם לא תגיע לאפס.

4.17 Correct Classification - הצלחה בסיווג

נגדיר כעת פורמלית מהי "הצלחה בסיווג". מטרת הלמידה: חיפוש וקטור משקולות אופטימלי w שמתאים בצורה הטובה ביותר לנתוני האימון שלנו. האסטרטגיה היא להגדיר פונקציית הפסד שתמדוד את הטעויות שלנו, ואז למזער את הפונקציה באמצעות GD. כיצד נראה חיזוי טוב? התנאי הוא פשוט: $y_i \cdot (w^T x_i) > 0$. נוכיח שזה עובד:

- מקרה ראשון: אכן צדקנו ו-1 $y_i = 1$, אזי, $1 \cdot 1 > 0$. בדומה, אם צדקנו ו-1 $y_i = -1$, נקבל $-1 \cdot -1 = 1 > 0$.
- מקרה שני: לא צדקנו. בה"כ $y_i = 1$ והחיזוי שגוי. אזי, $1 \cdot -1 < 0$, והתנאי לא מתקיים.

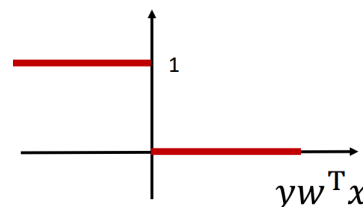
הערך $y_i(w^T x_i)$ יקרא *Margin*. *Margin* חיובי מעיד על חיזוי נכון ושלישי - על חיזוי שגוי.

4.18 דוגמאות

דוגמה 87. נפתח בדוגמה ראשונה - loss function 0-1: נגדיר את הפונקציה כך:

$$l_{0-1}(w; x, y) = \begin{cases} +1 & yw^T x < 0 \\ 0 & yw^T x > 0 \end{cases}$$

כלומר, 1 כהפסד אם טעינו בסיווג. השגיאה הכוללת הרי היא: $Err_w = \sum_{i=1}^n l_{0-1}(w; x_i, y_i)$. מה הבעיה? לא ניתן לחשב גרדיאנט עבור פונקציית 0-1. מדוע? ברוב המקומות מדובר בפונקציה שהיא קו ישר אופקי: ונגזרת קו ישר אופקי הוא אפס. וכן: בעיה נוספת - בנקודה 0 יש קפיצה, לכן הפונקציה לא דפרנציאלית. הבעיה היא שהגרדיאנט תמיד אפס: ולכן לא נבצע בכלל עדכונים. מסקנה: אי אפשר להשתמש ב-0-1 loss function ישירות עם GD כי היא לא נותנת לנו כיוון להיכן להשתפר.



דוגמה 88. נגדיר את פונקציית ה- $l_{hinge}(w; x, y)$:

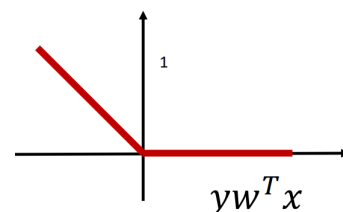
$$l_{hinge}(w; x, y) = \begin{cases} -yw^T x & yw^T x < 0 \\ 0 & yw^T x > 0 \end{cases}$$

כלומר אם צדקנו ההפסד הוא אפס, ואם טעינו נשלם $-yw^T x$. נבחין: ככל שהטעות גדולה יותר (רחוקים מהקו) הערך של ההפסד הופך לגדול יותר מתמטית. לכן ההפסד גדל. נחשב את הגרדיאנט:

$$\nabla l_{hinge} = 0 \quad \text{כאשר צודקים - מימין:}$$

$$\nabla l_{hinge} = -yx \quad \text{כאשר טועים, הגרדיאנט הוא}$$

כעת, הפונקציה רציפה וכן כעת ניתן להשתמש בגרדיאנט למען GD.



4.19 אלגוריתם לאימון ניורון בודד באמצעות SGD ו Hinge loss function

כעת, נדון באלגוריתם המלא לאימון ניורון בודד באמצעות SGD ו Hinge loss function:

1. מאתחלים וקטור משקולות w רנדומלית.
2. חוזרים על הפעולות הבאות עד שהמודל מפסיק להשתפר:
3. הגרל דוגמה אחת מהדטה (x_i, y_i) .
4. אם הסיווג נכון (ה Margin חיובי) אזי - אל תעשה כלום.
5. אחרת, עדכן: $w = w + \eta y_i x_i$

מדוע זה עובד? נתעכב על שלב 5. אם טעינו בדוגמה חיובית: כלומר $y_i = 1$, אזי בפועל $w = w + \eta x_i$. אם טעינו בדגימה שלילית: אזי $w = w - \eta x_i$. כעת, מדוע זה עובד עם פונקציית ההפסד?

$$w_{new} = w_{old} - \eta \nabla = w_{old} - \eta(-y_i x_i) = w_{old} + \eta y_i x_i$$

שכן, $\nabla l_{hinge} = -y x$, לכן אכן האלגוריתם מתיישב עם פונקציית ההפסד.

מסקנה 89. הפרספרקטון הוא אותו אלגוריתם בדיוק כמו זה, כאשר $\eta = 1$.

פסאודו:

Algorithm 2 Single Neuron Training via SGD with Hinge Loss

Require: Training set $\{(x_i, y_i)\}$, learning rate η

Initialize the weight vector w randomly

repeat

Sample a random example (x_i, y_i) from the dataset

if the classification is correct (positive margin) **then**

Do nothing

else

Update: $w \leftarrow w + \eta y_i x_i$

end if

until the model stops improving

4.20 רגרסיה לוגיסטית

נזכר בדוגמה השנייה שהופיעה קודם, כאשר הדטה יחסית לינארי - אך יש חריגות שמונעות מאיתנו להעביר קו ישר. עד כה עסקנו בהחלטות של סיווג לפי $\pm$. כעת נציע גישה רכה יותר שמבוססת על הסתברות. הבעיה עם פונקציית ה $sign$ היא שהיא בינארית לחלוטין - אם המרחק מהקו הוא 0.0000001 ו-109 הוא יגיד לנו '1'. נרצה כי המודל יוציא מס' בין 0 ל-1 שייצג את ההסתברות שהדגימה שייכת למחלקה מסויימת. למשל: $Pr(y = 'red'|x)$

הגדרה 90. (פונקציית הסיגמואיד). נגדיר את הפונקצייה הבאה:

$$\sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

נעזר כעת בתכונות שלה על מנת להשיג את מטרטנו. המכפלה $w^T x$ שיכולה להיות כל מס' ב $(-\infty, \infty)$ תומר לערך ב $[0, 1]$ באמצעות הפונקציה. מדוע?

$$\lim_{w^T x \rightarrow \infty} \sigma(w^T x) = \lim_{w^T x \rightarrow \infty} \frac{1}{1 + e^{-w^T x}} = \lim_{w^T x \rightarrow \infty} \frac{1}{1 + 0} = 1$$

$$\lim_{w^T x \rightarrow -\infty} \sigma(w^T x) = \lim_{w^T x \rightarrow -\infty} \frac{1}{1 + e^{-w^T x}} = \lim_{w^T x \rightarrow -\infty} \frac{1}{1 + \infty} = 0$$

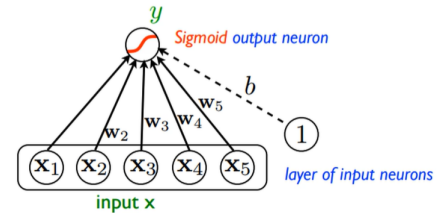
וכמו כן: אם $\sigma(w^T x = 0) = \frac{1}{1+1} = \frac{1}{2}$ (חוסר ודאות מוחלט).

מדובר בפונקציה גזירה $\nabla f(x) = \frac{w \cdot e^{-w^T x}}{(1+e^{-w^T x})^2}$, ובפרט נגזרתה מקיימת: $\nabla f(x) = f(x)(1-f(x))w$.

כיצד נראה ניורון לוגיסטי? כקלט, $x \in \mathbb{R}^d$ המחברים למשקולות $w \in \mathbb{R}^d$. המחשב כופל כל קלט במשקולתו וסוכם. $b \in \{0, 1\}$ הוא האיבר החופשי. ואז - הסכום שחישבנו נכנס לפונקציית הסיגמואיד - התוצאה מהפונקציה:

$$\hat{y} = Pr[y = 1|x]$$

כלומר, ההסתברות שהדגימה שייכת למחלקה 1.



מניין מקור השם? מדוע קוראים לזה 'רגרסיה' אם אנחנו מבצעים סיווג? ובכן - אנחנו מבצעים תחילה רגרסיה (חיזוי ערך רציף) על ההסתברות, ומשתמשים בתוצאה לבצע סיווג.

כעת, נשאל את עצמנו את השאלה הבאה: כיצד מודדים שגיאה כשהתוצאה היא הסתברות? כעת, נעזר בכלי שכבר השתמשנו בו קודם - Likelihood.

כיוון שהמודל מוצא הסתברות, נשאל את עצמנו: מהו הסיכוי שהמודל הנוכחי שלנו עם המשקולות w יסביר את הנתונים שראינו בפועל? נרצה למצוא את w שימקסם את ההסתברות שהתוצאות האמיתיות אכן יקרו. ובכן, ההסתברות להשתייך לדגימה חיובית $y = 1$ היא פשוט $\sigma(w^T x)$ ולדגימה שלילית $y = 0$ היא $1 - \sigma(w^T x)$. מכאן, תחת ההנחה כי הדגימות הן iid ההסתברות לראות את כל מסד הנתונים הוא מכפלת ההסתברויות:

$$\mathcal{L}(w) = \prod_{y_i=1} Pr(y = 1|x_i) \prod_{y_i=0} Pr(y = 0|x_i)$$

כלומר - עוברים על כל דוגמה (מפצלים למקרים שבו $y_i = 1$ או $y_i = 0$ וכדומה) ומכפילים בהסתברות כי אכן המודל היה קרוב/לא קרוב לחזות. ערך זה נותן ציון מספרי אחד למודל.

כעת - יש לנו ערך שאנחנו יכולים למקסם: נרצה למקסם את $\mathcal{L}(w)$. נמשיך לפשט את הביטוי:

$$\mathcal{L}(w) = \prod_{y_i=1} Pr(y = 1|x_i) \prod_{y_i=0} Pr(y = 0|x_i) = \prod_{i=1}^n Pr(y = 1|x_i)^{y_i} \prod_{i=1}^n Pr(y = 0|x_i)^{1-y_i} =$$

$$\prod_{i=1}^n \sigma(w^T x_i)^{y_i} \prod_{i=1}^n (1 - \sigma(w^T x_i))^{1-y_i} =$$

$$\prod_{i=1}^n \sigma(w^T x_i)^{y_i} (1 - \sigma(w^T x_i))^{1-y_i}$$

בדומה לעבר, כיוון ש $\log$ פונקצייה מונוטונית עולה, w שממקסם את $\mathcal{L}(w)$ ימקסם את $\log(\mathcal{L}(w))$. מכאן:

$$\log(\mathcal{L}) = \sum_{i=1}^n y_i \log[\sigma(w^T x_i)] + (1 - y_i) \log[1 - \sigma(w^T x_i)]$$

כעת נרצה למצוא את הנגזרת של הרגרסיה הלוגיסטית.

בתחילה, נרצה להפוך את הבעיה לבעיית אופטימיזציה מינימום שהרי לרוב אנחנו מטפלים באלגוריתמי אופטימיזציה שמוצאים מינימום - לכן נוסף מינוס.
 לפני שנתחיל בגזירה נזכר כי:
 • הנגזרת של סיגמואיד:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

• לפי כלל השרשרת:

$$\frac{d\text{low}(\mathcal{L})}{dw} = \frac{d\log(\sigma(w^T x_i))}{d\sigma(w^T x_i)} \cdot \frac{d\sigma(w^T x_i)}{dw^T x_i} \cdot \frac{dw^T x_i}{dw}$$

נסמן $\sigma_i = \sigma(w^T x_i)$. מכאן עבור דגימה בודדת (x_i, y_i) :

$$L_i = -[y_i \ln(\sigma_i) + (1 - y_i) \ln(1 - \sigma_i)]$$

כעת, לפי כלל השרשרת הרי שנקבל:

$$\nabla L_i = -\left(y_i \frac{1}{\sigma_i} \frac{d\sigma_i}{dw} + (1 - y_i) \frac{1}{1 - \sigma_i} \cdot \frac{d(1 - \sigma_i)}{dw}\right)$$

$$\nabla L_i = -\left(y_i \frac{1}{\sigma_i} \cdot \sigma_i(1 - \sigma_i)x_i + (1 - y_i) \frac{1}{1 - \sigma_i} \cdot (-\sigma_i(1 - \sigma_i)x_i)\right]$$

$$\nabla L_i = -\frac{y_i}{\sigma_i} \cdot \sigma_i(1 - \sigma_i)x_i - \frac{(y_i - 1)(1 - \sigma_i)x_i}{1 - \sigma_i} = -y_i(1 - \sigma_i)x_i - x_i(y_i - 1)$$

$$\nabla L_i = -[y_i x_i - \sigma_i x_i] = (\sigma_i - y_i)x_i$$

לכן סה"כ קיבלנו:

$$\nabla \mathcal{L}(w) = -\sum_{i=1}^n [x_i(y_i - \sigma(w^T x_i))]$$

מה קיבלנו כביטוי בכל איבר בסכימה? את האיבר האמיתי y_i ובצד השני מחוסר ממנו ההערכה שלו $\sigma(w^T x_i)$.
 אם כן, כיוון שמצאנו את הגרדיאנט של הרגרסיה הלוגיסטית נוכל לעדכן את נוסחת ה-GD:

$$w_{t+1} = w_t - \eta \sum_{i=1}^n x_i [\sigma(w^T x_i) - y_i]$$

ובאופן שקול ויפה יותר:

$$w_{t+1} = w_t + \eta \sum_{i=1}^n x_i (y_i - \hat{y}_i)$$

לסיכום: בפרספקטיון מתקיים $\hat{y}_i = \text{sign}(w^T x_i)$ וברגרסיה לוגיסטית $\hat{y}_i = \sigma(w^T x_i)$.

4.21 מדוע רגרסיה לוגיסטית נחשבת למסווג לינארי?

הרי, אנחנו משתמשים בפונקציה מעריכית כסיגמואיד, למה רגרסיה לוגיסטית נחשבת כמסווג לינארי?
 • כלל ההחלטה: אנחנו נחליט כי דגימה שייכת למחלקה 1 אם הסיכוי שהיא ב-1 גדול מהסיכוי שהיא ב-0:

$$Pr[y = 1|x] > Pr[y = 0|x]$$

זה קורה בדיוק כאשר הסיגמואיד גדול מחצי.
 • וכן כלל ההחלטה הנ"ל, שקול ל:

$$\frac{Pr[y = 1|x]}{Pr[y = 0|x]} > 1 \iff \frac{\sigma(w^T x)}{1 - \sigma(w^T x)} = e^{w^T x} > 1$$

ערך זה $(e^{w^T x})$ נקרא *odds*.
 • אם נפעיל $\ln$ על המשוואה הקודמת, נקבל:

$$\ln\left(\frac{\sigma(w^T x)}{1 - \sigma(w^T x)}\right) = w^T x > 0$$

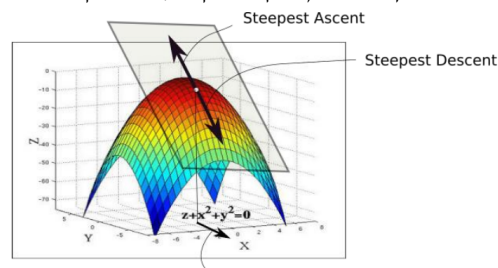
וכן $w^T x$ היא פונקציה לינארית לחלוטין של x .
 • בשורה התחתונה - למרות שהמודל מוציא הסתברויות עגולות ורכות: קו ההפרדה שבו המודל עובר מלהמר על 0 ללהמר על 1 הוא קו ישר.
 אם $w^T x > 0$ הוא מהמר על 1, אחרת הוא מהמר על אפס.
 זה בדיוק כמו בפרספטרום - רק שההבדל הוא שבפרספטרום הקפיצה היא מיידיית, וכאן המעבר ליד הקו הדרגתי.

4.22 תרגול 4

נרצה לשאול את עצמנו את השאלה הבאה. אם אנחנו משתמשים בכלי אינפי, מדוע שלא נגזור את פונקציית ההפסד ונשווה לאפס? מדוע יש צורך בתהליך איטרטיבי כמו GD ?
 • אם כן, ישנן פונקציות רבות שלא ניתן לגזור אותן וכן פונקציות מסובכות עם הרבה משתנים לרוב אין פתרון סגור. כלומר: לא ניתן לבדוד את x מהמשוואה שכן היא מסובכת מדי לפתרון אנליטי.
 • גם אם קיים פתרון סגור, לרוב הוא ידרוש פעולות מתמטיות כבדות מאוד. למשל - ברגרסיה קיים פתרון סגור הכולל היפוך של מטריצה. להפוך מטריצה כשיש מיליוני פרמטרים זה תהליך יקר מאוד חישובית.

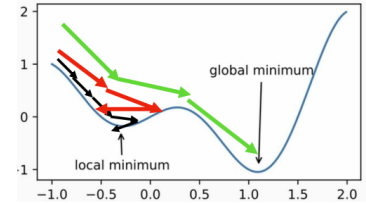
משפט 91. הגרדיאנט ∇f פניב לנו את כיוון העלייה הפהיר ביותר ואת קצב העלייה הפהיר ביותר. לכן, אם נבצע צעדים נגד כיוון הגרדיאנט נגיע למינימום המקומי של הפונקציה.

הערה 92. לצורך המחשה, החץ בכיוון מעלה מסמן את הגרדיאנט והחץ בכיוון מטה את הכיוון ההפוך לו - מורד הגרדיאנט:



הערה 93. מדוע ב- GD ישנו סימן המינוס $-\eta$? לפני שמגיעים למינימום, הפונקציה יורדת - הנגזרת שלילית. אחרי שמגיעים ועוברים את המינימום - הפונקציה עולה, הנגזרת חיובית. לכן, אם אנחנו משמאל למינימום אזי כיוון שהנגזרת שלילית, במכפלה במינוס התוצאה שה"כ תהיה חיובית ואנחנו למעשה נסיף w ונתקדם ימינה - לכיוון המינימום. אם אנחנו מימין למינימום, הנגזרת חיובית ובביצוע פלוס אנחנו שה"כ נוריד w ונחזור שמאלה לכיוון המינימום. שה"כ לא משנה מה - אנחנו נתקרב למינימום.

כעת נרצה לדבר על קצב הלמידה, η : הכמות / ערך שבה המשקולות מתעדכנות במהלך אימון נקרא η (קצב הלמידה).
 כאשר קצב הלמידה גבוה מדי, המודל קופץ מצד לצד. במקרה שכזה - הוא עלול לעולם לא להגיע לתחתית הקערה של הפונקציה - ואף לברוח החוצה. מצד שני, אם קצב הלמידה קטן מדי המודל מתקדם בבטחה בוודאות לעבר המינימום אך זה עלול לקחת זמן רב. כמו כן: אם נשתמש בצעדים קטנים מדי - כשנגיע למינימום מקומי ייתכן שנחשוב כי הגענו למינימום גלובלי - ונשגה.



כעת נרצה להוכיח כי אם בכל שלב נקח קבוע η קטן אזי אלגוריתם ה-GD יתכנס לכיוון המינימום. ראשית, נגדיר כי פונקציה f היא חלקה עם קבוע $\beta > 0$ אם מתקיים: $\|\nabla f(x) - \nabla f(y)\| \leq \beta \|x - y\|$ לכל $x, y \in \mathbb{R}^d$. במקרה זה נאמר כי הפונקציה מקיימת את תנאי ליפשיץ. כלומר - שיפוע הפונקציה לא יכול להשתנות באופן "מיידי" ופתאומי. קבוע β אומר לנו כמה הפונקציה יכולה להיות 'עקומה' לכל היותר. אם הפונקציה משתנה בפראות - היא לא תקיים תנאי זה. כעת:

משפט 94. תהי f פונקציה קמורה, גזירה, בעלת גרדיאנט שמקיים את תנאי ליפשיץ עם קבוע $\beta > 0$. אם נריץ GD במשך T איטרציות עם גודל צעד קבוע הפקיים $n \leq \frac{1}{\beta}$ אזי הפתרון שנקבל בסוף (x^T) יקיים את אי השוויון הבא:

$$f(x^T) - f(x) \leq \frac{\|x_0 - x^*\|^2}{2\eta T} \leq \varepsilon$$

כאשר x^* הוא הפתרון האופטימלי. כלומר קצב ההתכנסות הוא $O(\frac{1}{T})$.

הוכחה:

ראשית, נטען כי לכל פונקציה שמקיימת את תנאי ליפשיץ מתקיים:

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{\beta}{2} \|y - x\|^2$$

מה זה אומר? הלפה אופרת כי אם נבחר את הנק' הבאה y בצורה חכמה - נוכל להבטיח כי $f(y)$ יהיה קטן יותר מ- $f(x)$. כלומר - אם נלך בכיוון הנכון הפונקציה חייבת לרדת. אם כן כמובן ש- f תייצג את פונקציית ההפסד, x את המשקולות w^{t-1} וכן y את w^t . כעת - **אזהרה:** יתחיל כיף אלגברי: אנחנו יודעים כי מתקיים $w^t = w^{t-1} - \eta \nabla L(w^{t-1})$ ומכאן: $w^t - w^{t-1} = -\eta \nabla L(w^{t-1})$ (הפרש המשקולות הוא נגד כיוון הגרדיאנט). אם כן נציב בלפה ונקבל:

$$L(w^t) \leq L(w^{t-1}) + \nabla L(w^{t-1})^T (w^t - w^{t-1}) + \frac{\beta}{2} \|w^t - w^{t-1}\|^2 = L(w^{t-1}) + \nabla L(w^{t-1})^T (-\eta \nabla L(w^{t-1})) + \frac{\beta}{2} \|\eta \nabla L(w^{t-1})\|^2$$

$$= L(w^{t-1}) - \eta \|\nabla L(w^{t-1})\|^2 + \frac{\beta \eta^2}{2} \|\nabla L(w^{t-1})\|^2$$

$$\implies L(w^t) \leq L(w^{t-1}) - \eta \left(1 - \frac{\beta \eta}{2}\right) \|\nabla L(w^{t-1})\|^2$$

כעת, כדי שערך הפונקציה אכן יקטן אנחנו צריכים כי $L(w^t) < L(w^{t-1})$. זה יקרה אם"פ $1 - \frac{\beta \eta}{2} > 0$ שכן $\eta > 0$ וכן גם ערך הנורמה. כלומר נרצה $\frac{\beta \eta}{2} < 1$. אם נקח כבחירה טבעית את $\eta = \frac{1}{\beta}$ נקבל: $1 - \frac{\beta \eta}{2} = \frac{1}{2}$. כלומר:

$$\implies L(w^t) \leq L(w^{t-1}) - \frac{1}{2\beta} \|\nabla L(w^{t-1})\|^2$$

כעת כיוון שהביטוי $\frac{1}{2\beta} \|\nabla L(w^{t-1})\|^2$ אי שלילי, בהכרח Loss יורד בכל איטרציה. הוא יפסיק רק כאשר הגרדיאנט יהיה שווה ממש לאפס. אך, כל מה שהוכחנו עד כה זה שהפונקציה יורדת - לא שהיא מגיעה גם למינימום הגלובלי. היא יכולה להתקע בנקודת מינימום מקומי או אוקף. כעת נעזר בהנחה לפיה הפונקציה קמורה. אם כן, פונקציה קמורה היא זו שדומה לקערה. המשיק בכל נקודה נמצא מתחת לפונקציה! מכאן נבחין כי בהכרח הערך בפנימינום חייב להיות גדול או שווה לערך בנק' הנוכחית פלוס השינוי הליניארי לפי הגרדיאנט. כלומר הפונקציה נמצאת מעל הקו שמשיק לה:

$$L(w^*) \geq L(w^{t-1}) + \nabla L(w^{t-1})^T (w^* - w^{t-1})$$

לאחר סידור:

$$L(w^{t-1}) - L(w^*) \leq \nabla L(w^{t-1})^T (w^{t-1} - w^*)$$

כעת נבדוק כמה אנו קרובים לנק' המינימום w^* לאחר צעד אחד של GD:

$$\|w^t - w^*\|^2 = \|w^{t-1} - \eta \nabla L(w^{t-1}) - w^*\|^2 = \|(w^{t-1} - w^*) - \eta \nabla L(w^{t-1})\|^2 = \|w^{t-1} - w^*\|^2 - 2\eta \nabla L(w^{t-1})^T (w^{t-1} - w^*) + \eta^2 \|\nabla L(w^{t-1})\|^2$$

אבל האיבר האמצעי (זה שפותח ב -2η) לפי מה שראינו קודם מקיים $L(w^{t-1}) - L(w^*) \leq \nabla L(w^{t-1})^T (w^{t-1} - w^*)$ ולכן:

$$(**): \|w - w^*\|^2 \leq \|w^{t-1} - w^*\|^2 - 2\eta(L(w^{t-1}) - L(w^*)) + \eta^2 \|\nabla L(w^{t-1})\|^2$$

נזכר כי קודם קיבלנו $\|\nabla L(w^{t-1})\|^2 \leq 2\beta(L(w^{t-1}) - L(w^*))$ ובאופן שקול $L(w^t) \leq L(w^{t-1}) - \eta(1 - \frac{\beta\eta}{2})\|\nabla L(w^{t-1})\|^2$ נזכר כי קודם קיבלנו $\eta = \frac{1}{\beta}$ ולכן $2\beta = \frac{2}{\eta}$ ומכאן:

$$\|\nabla L(w^{t-1})\|^2 \leq \frac{2}{\eta}(L(w^{t-1}) - L(w^*))$$

נחזור ל(**): ונחסום עם אי השוויון שמצאנו כעת את האיבר האחרון ונקבל:

$$(\|w^t - w^*\|^2 \leq \|w^{t-1} - w^*\|^2 - 2\eta(L(w^{t-1}) - L(w^*)) + \eta^2 \frac{2}{\eta}(L(w^{t-1}) - L(w^*)) = \|w^{t-1} - w^*\|^2 - 2\eta(L(w^{t-1}) - L(w^*)) + 2\eta(L(w^{t-1}) - L(w^t)))$$

$$\implies \|w^t - w^*\|^2 \leq \|w^{t-1} - w^*\|^2 + 2\eta(L(w^*) - L(w^t))$$

כלומר, הפרחם הריבועי למינימום בצעד הבאה, קטן שווה לאותו מרחק מהמינימום בצעד הקודם ועוד ערך חיובי. כלומר - כל עוד לא הגענו למינימום הפרחם הגאומטרי שלנו לנק' האופטימום חייב להתקצר בכל צעד. כמעט סיימנו: נסדר את אי השוויון ונקבל:

$$2\eta(L(w^*) - L(w^t)) \leq \|w^{t-1} - w^*\|^2 - \|w^t - w^*\|^2$$

כעת נרצה לראות מה קורה לכל אורך הדרך מצעד 1 לצעד T :

$$2\eta \sum_{t=1}^T L(w^t) - L(w^*) \leq \sum_{t=1}^T (\|w^{t-1} - w^*\|^2 - \|w^t - w^*\|^2) = \|w^0 - w^*\|^2 - \|w^T - w^*\|^2$$

שכן מדובר בטור טלסקופי. לכן:

$$\sum_{t=1}^T L(w^t) - L(w^*) \leq \frac{\|w^0 - w^*\|^2 - \|w^T - w^*\|^2}{2\eta} \leq \frac{\|w^0 - w^*\|^2}{2\eta}$$

קודם הוכחנו כי הפונקציה מונוטונית יורדת בכל איטרציה. כלומר, ההפרד מהמינימום בצעד האחרון הוא הקטן ביותר מכל האיברים שסכמנו בשורה מעלה. לכן, אם נסכום T איברים סכום זה בהכרח גדול שווה מהאיבר האחרון כפול T (הרי הוא הכי קטן):

$$T \cdot (L(w^T) - L(w^*)) \leq \sum_{t=1}^T L(w^t) - L(w^*) \leq \frac{\|w^0 - w^*\|^2}{2\eta}$$

וכפרט:

$$L(w^T) - L(w^*) \leq \frac{\|w^0 - w^*\|^2}{2\eta T}$$

כנדרש.

■

4.23 רגרסיה לינארית עם GD

האלגוריתם:

א. התחל מוקטור משקולות אקראי w^0 .

ב. חזור:

ג. חזה את $\hat{y}_w(x_i)$.

ד. מחשבים את הגרדיאנט $(y_i - \hat{y}_w(x_i))(-x_i)$ $\frac{\partial \ell}{\partial w} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i))(-x_i)$

ה. מבצעים את צעד העדכון $w^{t+1} = w^t - \eta \nabla_w \ell$

כלומר: במקום לבצע את התהליך המקורי של רגרסיה לינארית - בה צריך לחשב מטריצה הפיכה (לא יעיל, סיבוכיות $O(n^3)$) מבצעים GD באופן איטרטיבי).

5 הרצאה 5

5.1 הקדמה

- מה למדנו עד כה? למדנו רגרסיה לוגיסטית - סיווג הסתברותי בין שתי תוצאות אפשריות בלבד - סיווג בינארי ($K = 2$) באמצעות פונקציית הסיגמואיד, על מנת למזער את שגיאת המודל השתמשנו בSGD. אבל - מה אם יש לנו יותר מזוג קטגוריות? לשם כך נשתמש בSoftmax regression. זה הפתרון כאשר מס' המחלקות $K > 2$, ומהווה מעין הכללה של המודל הקודם. לדוגמה: בהיתן תמונה ובה ספרה - סיווג של מהי הספרה. במקרה זה $K = 10$. נבחין שכל שמש' המחלקות גדל, הבעיה הופכת לקשה יותר שכן המודל צריך לבחור מבין כמה וכמה אפשרויות בו זמנית.

נגדיר פורמלית את הבעיה:

- קלט:** נתוני האימון - $S = \{(x_i, y_i)\}_{i=1}^n$, כך שכל דגימה $x_i \in \mathbb{R}^d$. לכל קלט תווית $y_i \in \{1, \dots, K\}$. K הוא מס' המחלקות. נתייחס למקרה בו $K \geq 3$ שכן אחרת מדובר בסיווג בינארי שכבר ראינו.
- המטרה:** פונקציית מיפוי $f: \mathbb{R}^d \rightarrow \{1, \dots, K\}$. הפונקציה הנ"ל צריכה לקחת נתונים חדשים שהמודל עוד לא ראה ולסווג אותם למחלקה הנכונה מבין K האפשרויות.

5.2 vs-All-1 (אחד כנגד כולם) - פתרון נאיבי

הרעיון המרכזי בגישה זו הוא במקום לבנות מודל אחד מורכב, מפרקים את הבעיה להרבה תתי בעיות קטנות:

אימון - מאמנים K מסווגים לינאריים בינאריים נפרדים: $w_1, \dots, w_k$.

כל מסווג מתמקד בשאלה בינארית אחת: האם הדגימה שייכת למחלקה k או שלא.

על מנת להחליט על דגימה מסוימת החלטה סופית, מריצים את הקלט על כל K המסווגים ובחרים את המחלקה שקיבלה את הציון הגבוה ביותר שאפשרות שהקלט שייך אליה:

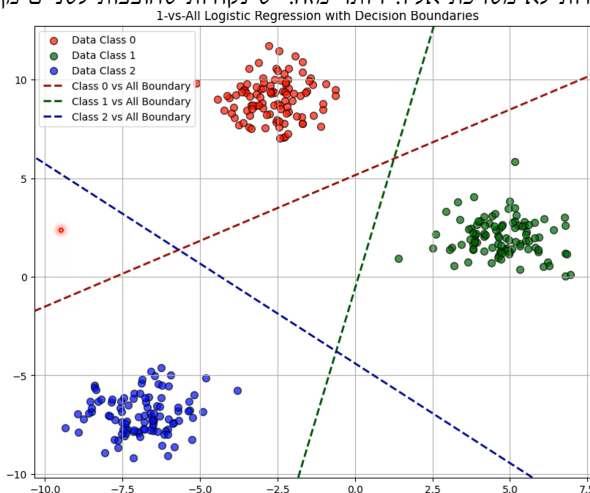
$$\hat{y} = \operatorname{argmax}_k \{w_k^T x\}$$

הפתרון מאוד טריוואלי. וגם עובד. אבל - יש הרבה בעיות:

- מאוד יקר חישובית! אנחנו צריכים לאמן K מודלים נפרדים לחלוטין.
- ציונים לא מנורמלים - כיוון שכל מודל התאמן בנפרד, הציון שמוציא w_1 לא בהכרח מדבר באותה השפה של המודל שהוציא את w_2 . לכן ההשוואה ביניהם (מי גדול יותר) לא תמיד נכונה מתמטית.

• מקרי קצה: מה אם אף אחד מהמסווגים לא חוזה אמת? מה אם אף אחד לא חוזה שקר ושניים מאוד בטוחים שהתשובה אמת?

נתבונן בדוגמה הבאה: קיבלנו קלט ויצרנו 3 מסווגים בינאריים. נבחין כי המשולש בין הקווים שקיבלנו - הוא שטח מת מבחינתנו. אף אחת מן הנקודות לא משויכת אליו. ויותר מזה: יש נקודות שחופפות לשניים מן האפשרויות.



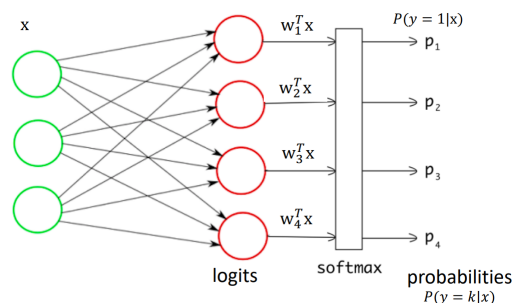
5.3 Softmax regression

במקום לבא מחלקה אחת, המודל יוציא מודל הסתברותי של פונקציית התפלגות הסתברות חוקית על כל פני K המחלקות. נרצה למפות קלט x לוקטור הסתברותי p . הוקטור הנ"ל חי בתוך מרחב מדגם הסתברות בעל $K - 1$ מימדים: Δ^{K-1} . פורמלית, נגדירו:

$$\Delta^{K-1} = \{p \in \mathbb{R}^K : p_j \geq 0 \text{ for all } j, \text{ and } \sum_{j=1}^K p_j = 1\}$$

כלומר, הרשת מוציאה וקטור $p = [p_1, \dots, p_K]^T$. כל איבר p_j מייצג את ההסתברות כי הדגימה שייכת למחלקה j בהינתן הקלט x . כלומר: $p_j = \Pr[y = j|x]$. נבחין כי הוקטור בעל $K - 1$ מימדים שכן כיוון שסכום ההסתברויות הוא 1, אנחנו יודעים אוטומטית את הערך של p_K בהינתן הערכים הקודמים.

כיצד זה עובד? נתבונן בתמונה:



העיוגולים הירוקים מייצגים את וקטור הקלט x . החצים - אומרים שכל רכיב x משפיע על הציון הסופי של כל אחת מהמחלקות. הקלט עובר דרך מערכת קשרים - שכבת $logits$ לעיוגולים אדומים. בכל עיגול אדום מחושב ציון שנקרא $Logit$. החישוב לכל מחלקה k הוא מכפלה פנימית בין וקטור המשקולות של אותה מחלקה לקלט: $w_k^T x$. כל ה- $Logits$ נכנסים לתוך קופסה אחת שנקראת $softmax$. התפקיד שלה הוא לקחת את הציונים האלו ולעבד אותם ליחידה אחת מנורמלת. כפלט - יוצאים ערכי ההסתברות הסופיים: $\{p_i\}_{i=1}^4$. נבחין כי הקלט מגודל 3, אך $K = 4$: שכן יתכן מאוד והקלט x הוא מאפיינים כלשהם (גובה, מין, משקל) והמחלקות: תת משקל, משקל תקין, עודף משקל קל, השמנת יתר. נבחין כי לכל מחלקה יש וקטור משקולות משלה.

כעת, נראה את תהליך ה- $softmax$ פורמלית:

- בהינתן ציונים גולמיים עבור כל אחת מהמחלקות $\{w_i^T x\}_{i=1}^K$.
- שלב ראשון: נהפוך את כל הציונים לחיוביים באמצעות פונקציית $\exp$ (e):

$$\exp(w_1^T x), \dots, \exp(w_K^T x)$$

- שלב שני: על מנת לדאוג כי הסכום יהיה אחד, נחלק כל ערך בסכום האקספוננציאלי כך נקבל את החלק היחסי:

$$P_i = \Pr(y = i|x) = \frac{\exp(w_i^T x)}{\sum_{j=1}^K \exp(w_j^T x)}$$

מודל פשוט מאוד. לדוגמה:

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \rightarrow \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \rightarrow \begin{bmatrix} 0.02 \\ 0.9 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

- מניין מקור השם *softmax*? מדובר בקירוב חלק - וגזיר לפונקציית *argmax*. בניגוד ל*argmax* רגיל, שבה פשוט בוחרים את המקסימום ומתעלמים מהשאר - ה*softmax* רך: הוא מדגיש את הערך הגבוה ביותר אך לא מאפס לחלוטין את שאר הערכים. העובדה שהוא לא מאפס את שאר הערכים מאפשרת לנו לשמור את המידע לצורך למידה ו*GDI*. אם הכל היה מתאפס פרט למקסימום - היה קשה למודל ללמוד מהשגיאות שלו.

5.4 הגדרה מטריציונית ל*softmax*

במקום להסתכל על דגימה אחת בודדת x , נרצה להסתכל על קבוצת דגימות ולהכליל את המודל באמצעות מטריצות.

בהינתן:

$X \in \mathbb{R}^{n \times d}$ כך ש- X היא מטריצת הנתונים - כל שורה היא דגימה (n שורות) וכל דגימה מממד d - d מאפיינים.
 $W \in \mathbb{R}^{d \times K}$ מטריצת משקולות, כל עמודה מייצגת וקטור משקולות של אחת מ- K המחלקות, וכן כל וקטור משקולות בעל d שורות - כמס'

המאפיינים.

הנוסחה לחישוב כל התחזיות יחד היא:

$$\hat{P} = \text{softmax}(XW) \in \mathbb{R}^{n \times K}$$

מטריצת התחזיות מחזיקה: הכניסה בשורה i ובעמודה k מחזיקה את $\hat{P}_{i,k} = \Pr[y_i = k|x_i]$.
 כל שורה במטריצה היא פונקציית התפלגות חוקית, עבור הדגימה i .

נעיר כי *softmax* מופעל בפועל שורה אחר שורה על הנתונים במטריצה, שכן הוא מקבל וקטור ומחזיר וקטור.

5.5 פרקטיקה - איזה בעיות יהיו לנו?

הנוסחה של *softmax* מחייבת אותנו לחשב אקספוננט על ציונים גולמיים. מה הבעיה? פונקציית האקספוננט גדלה מאוד מהר. אם ננסה לחשב e^{1000} המחשב יחזיר אינסוף, כשנחשב אנסוף חלקי אנסוף המודל יקרוס.

מה הפתרון? ישנה תכונה מתמטית מעניינת ל*softmax*. אם נוסיף/נחסר קבוע c מכל הציונים יחד - התוצאה הסופית של ההסתברות לא תשתנה. כלומר:

$$\frac{e^{z_k - c}}{\sum_{j=1}^K e^{z_j - c}} = \frac{e^{z_k} e^{-c}}{e^{-c} \sum_{j=1}^K e^{z_j}} = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

לכן, על מנת למנוע מהמספרים להיות גדולים מדי: לפני שעושים אקספוננט מגדירים:

$$c = \max\{z\}_{z \in \{w_1^T x, \dots, w_K^T x\}}$$

ומחסירים אותו מכל הציונים. כעת, הציון הגבוה ביותר יהיה אפס וכל שאר הציונים יהיו מספרים שליליים או אפס. כתוצאה: לעולם לא נקבל מספרים ענקיים שייבילו לקריסה.

5.6 כמה המודל שלנו טוב?

בדומה לרגרסיה לוגיסטית, על מנת להעריך כמה המודל שלנו טוב נעזר ב-likelihood. נניח כי ישנם סט דגימות $\{(x_i, y_i)\}_{i=1}^n$ המכיל דגימות $i.i.d$. נרצה למצוא את המשקולות W שהכי מתאימות לנתונים שנראו. נגדיר:

$$\mathcal{L}(W) = \prod_{i=1}^n \prod_{k=1}^K Pr(y_i = k | x_i)^{\mathbb{I}\{y_i=k\}}$$

באשר $\mathbb{I}\{y_i = k\}$ היא פונקציית האינדיקטור. הוא שווה 1 אם התנאי מתקיים - כלומר אם המחלקה k היא אכן התוויית הנכונה עבור דגימה i (הרי יש בידינו את התוויית, כלומר אם אכן $y_i = k$ אנחנו נכפיל בהסתברות זו, כי זה אכן המציאות. אם לא פונקציית האינדיקטור תניב אפס ולא נתייחס להסתברות שחושבה. כלומר נתייחס רק להסתברות שחושב על מה שאכן נכון). ואפס אחרת. סה"כ הנוסחה נותנת את ההסתברות לראות את כל הנתונים. כעת נחשב את ה-log-likelihood:

$$\log(\mathcal{L}(W)) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{I}\{y_i = k\} \cdot \log(Pr(y_i = k | x_i)) = \sum_{i=1}^n \sum_{k=1}^K \mathbb{I}\{y_i = k\} \cdot \log\left(\frac{\exp(w_k^T x_i)}{\sum_{j=1}^K \exp(w_j^T x_i)}\right)$$

כיצד המודל לומד?

- מגדירים פונקציית הפסד - מגדירים את ה-Negative log-likelihood (NLL) כפונקציית ההפסד, כיוון שלגוריתם של ערך בין אפס לאחד הוא שלילי - שמים מינוס ומקבלים את ה- $-\log(\mathcal{L}(W))$. קיבלנו מס' חיובי שאותו נרצה למזער (הרי רצינו למקסם את הפונקציה המקורית, שקול למזער את הפונקציה עם הסימן השלילי).
- משתמשים באלגוריתם ה-SGD (בוחרים קבוצה קטנה Batch של דגימות, בוחרים בכל שלב דגימה אקראית ומעדכנים את וקטור המשקולות נגד כיוון הגרדיאנט). כדי לעדכן את המשקולות צעד אחר צעד עד להגעה למינימום של פונקציית ההפסד.

5.7 Cross Entropy Loss

ה-NLL עליו דיברנו קודם הוא מקרה פרטי של Cross Entropy Loss. מה זה? זהו מדד מתמטי שמודד את המרחק (או ההבדל) בין זוג התפלגויות P, Q . P היא ההתפלגות האמיתית (במקרה שלנו: וקטור שיש בו 1 במחלקה הנכונה ואפס בשאר) ו- Q היא ההתפלגות שהמודל שלנו חזה (הפלט של Softmax). באופן כללי מתקיים:

$$L_{CE}(P, Q) = - \sum_{i=1}^K Pr(i) \log(Q(i))$$

מדוע זה עובד? כיוון ש- $Pr(i) = 0$ עבור כל המחלקות פרט לזו הנכונה, הסכום מתכווץ לאותו ביטוי שראינו קודם.

5.8 כלל העדכון ל-SGD

נרצה לחשב את כלל העדכון של SGD בכל צעד. ההסתברות שהמודל נותן למחלקה הנכונה y_i והמחיר היא

$$L = - \log\left(\frac{e^{w_{y_i}^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}}\right)$$

נשים לב כי לפי $\log(\frac{A}{B}) = \log(A) - \log(B)$ נקבל

$$L = \log\left(\sum_{j=1}^K e^{w_j^T x_i}\right) - w_{y_i}^T x_i$$

כעת נגזור ביטוי זה לפי w_c (וקטור משקולות של מחלקה כלשהי). נשים לב:

$$\nabla_{w_c}(-w_{y_i}^T x_i) := \begin{cases} -x_i & c = y_i \\ 0 & 0.w \end{cases}$$

האיבר השני נגזר באופן הבא:

$$\nabla_{w_c} \log \left(\sum_{j=1}^K e^{w_j^T x_i} \right) = \frac{1}{\sum_{j=1}^K e^{w_j^T x_i}} \cdot \nabla_{w_c} \left(\sum_{j=1}^K e^{w_j^T x_i} \right) = \frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} \cdot x_i$$

לכן סה"כ:

$$\nabla_{w_c} L = \left(\frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} - 1[c = y_i] \right) x_i$$

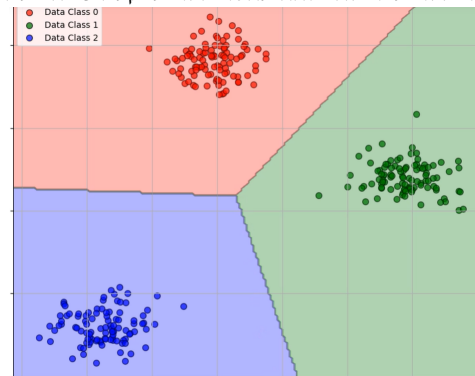
לכן נקבל סה"כ כי כלל העדכון:

$$w_c = w_c - \eta \left(\frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} - 1[c = y_i] \right) x_i$$

$$p_c = \frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} \text{ והרי } 1[c = y_i] = \text{target} \text{ נסמן}$$

• זמן טוב להעיר: בעת SGD בסיווג עם $K > 2$ אנחנו מעדכנים את כל K וקטורי המשקולות במקביל. שקול לומר: מעדכנים את כל עמודות המטריצה W . לכן, אנחנו בשלב העדכון כאן עוברים על כל וקטורי המשקולות, אם הסיווג שלנו אכן שייך למחלקה זו הטרנט יהיה אחד: אחרת אפס.

לבסוף, *softmax* חישבנו הסתברויות אמנם: ומסווג נקודה למחלקה שקיבלה את ההסתברות הגבוהה ביותר, אך נוצרים לבסוף "גבולות" לכל מחלקה. מדוע? הציונים הגולמיים הם פונקציות לינאריות, והמשוואות בהם $w_1^T x = w_2^T x$ הם משוואות ישר (מעל מרחב d מימדי) שבהם ההסתברות ל"תפוח" זהה להסתברות ל"קלמנטינה" למשל: וכך נוצרים מחלקות כמו בתמונה -



לסיכום:

- קובעים ערכים אקראיים לכל המשקולות במטריצה W .
- לולאת אימון:
 1. לכל דגימה (x_i, y_i) :
 - א. חוזים - מחשבים ציונים לכל k המחלקות - $z = W^T x_i$
 - ב. הופכים את הציונים להסתברויות באמצעות *softmax*, יהי וקטור p כפלט.
 2. מחשבים כמה טעינו עם פונקציית Loss entropy (נשאל את עצמנו מדוע: הרי המודל לא משתמש בזה אחרי זה - ובכן זה למעקב: לראות שהמודל משתפר והשגיאה יורדת. וכן, המודל כן משתמש בזה בפועל! הוא בודק אם Loss הגיע לערך נמוך מספיק בשביל להפסיק את האימון).
 3. לכל מחלקה c מ 1 עד K :

$$\begin{aligned} \text{א. מחשבים את הגרדיאנט } & \left(\frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} - 1[c = y_i] \right) x_i \\ \text{ב. מעדכנים את וקטור המשקולות של אותה מחלקה: } & w_c = w_c - \eta \left(\frac{e^{w_c^T x_i}}{\sum_{j=1}^K e^{w_j^T x_i}} - 1[c = y_i] \right) x_i \end{aligned}$$

וחוזר חלילה.

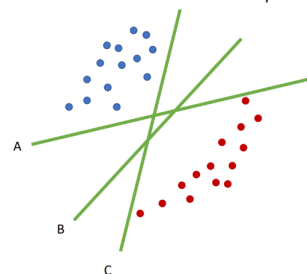
להלן פסאודו מפורט יותר:

Algorithm 3 Softmax Regression Training

Require: Training set $\{(x_i, y_i)\}$, number of classes K , learning rate η
 Initialize all weights in the matrix W with random values
repeat
 for each training example (x_i, y_i) **do**
 Predict: compute scores for all K classes: $z = W^T x_i$
 Convert scores to probabilities using softmax, obtaining a vector p
 Compute the cross-entropy loss (for monitoring: verify the model improves and the loss decreases; stop training once the loss is sufficiently low)
 for each class $c = 1$ **to** K **do**
 Compute the gradient: $(p_c - \text{target}) x_i$
 Update the class weight vector: $w_c \leftarrow w_c - \eta (p_c - \text{target}) x_i$
 end for
end for
until converged

5.9 SVM - הקדמה

כעת נלמד על אלגוריתם נוסף, SVM : Support vector machines. נתבונן בתמונה הבאה:



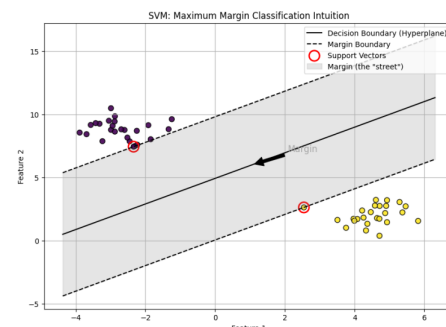
נשאל את עצמנו - מבין המפרידים הלינאריים, מיהו המפריד הנכון?

אם נריץ עליהם אלגוריתם כמו פרספקטרום שמחזיר קו לינארי - הוא יכול להחזיר כל אחד מהם. עם זאת - קו B נראה הכי הגיוני. ה- SVM לא מחפש סתם מפריד - הוא יחפש מפריד עם $Margin$ הכי גדול. כלומר - המרחק בין הקו המפריד לבין הנקודות הכי קרובות אליו משני הצדדים. האלגוריתם יעבוד על נתונים פרידים לינאריים, וכן על נתונים שיש בהם רעש בתיוגים. בהמשך - נראה כי הוא גם יעבוד לנתונים שאינם פרידים לינאריים.

5.10 Maximum margin

נגדיר:

- קו ההפרדה (הקו השחור הרציף בתמונה): מדובר בקו שקובע את הסיווג בפועל - מדובר בפונקציה לינארית, אך נרצה למקם אותה במיקום אופטימלי.
 - אזור השוליים (האזור האפור): לאזור זה נקרא $Street$: המטרה של SVM היא להפוך את הרחוב הזה לכמה שיותר רחב. רוחב $Street$ למעשה הוא פי 2 מה- $Margin$ (לשני הכיוונים).
 - וקטורים תומכים: נראה כי ישנן נקודות שמוקפות באדום על הקווים המקווקים - אלו הנקודות הכי חשובות במודל. הן הנקודות שיושבות בדיוק על שפת המדרכה של הרחוב, ונקראות כך כי הן "תומכות בקו ההפרדה". אם נזיז נקודה צהובה שנמצאת רחוק מהרחוב - שום דבר לא ישתנה. אך אם נזיז אחת מהנקודות שמוקפות באדום, כל הקו והרחוב יזוז יחד איתה.
- המשמעות: כל שאר הנקודות ב- $Dataset$ לא מעניינות את האלגוריתם ברגע שהוא מצא את הוקטורים התומכים. אפשר למחוק אותם והמודל ישאר זהה.



נשאל את עצמנו - מדוע המטרה היא למקסם את השוליים?

ובכן, הנקודות בתמונה לעיל הן מה שהן: נקודות נתונות. בעתיד יגיעו דגימות חדשות שהן לא בדיוק באותו מקום - אלא ליד. אם הקו המפריד שלנו צמוד מדי לנקודות הקיימות (כמו קווים A, C קודם) מספיק שהנקודה החדשה תזוז במילימטר והיא תיפול בצד הלא נכון של הקו. לכן: ככל שהשוליים רחבות יותר - יש למודל יותר מקום לטעות לפני שהוא נכשל על נתונים חדשים. וכן, מיקום השוליים מכריח את המודל שלא ללמוד את הנתונים הקיימים ולבצע overfitting. אנחנו נבחר את הפתרון הכי יציב גאומטרי. כלומר ישנן כמה סיבות:

- חסינות לרעש - אם יש טעות קטנה במדידה, או שהנתון החדש שונה בקצת מהמוצע, השוליים הרחבים ידאגו לכך שהסטייה הנ"ל לא תגרום לנתון לחצות את הקו בטעות.
- הכללה טובה יותר - ומניעת overfitting. ככל שהשוליים רחבות יותר - יש פחות אפשרויות לקווי הפרדה שיכולים לעבור שם (ביחס לפרפסטרון, שם אין שוליים בכלל ויש אנסוף קווים שיכולים לעבור): זה מגביל את המודל ומונע ממנו להיות יצירתי ולבחור עקומה שתתאים לנתונים.
- כשהגבול רחוק מהנתונים, אנחנו הרבה יותר בטוחים בתוצאה. אם נקודה נמצאת ממש על הגבול: המודל מתלבט ולא מקבל וודאות.

כעת נגדיר מתמטית את הבעיה. המשוואה של הקו היא $w^T x - b = 0$. w הוא וקטור המשקולות, והוא תמיד מאונך לקו ההפרדה. הוא קובע את כיוון הקו.

המרחק הניצב של כל נקודה מהקו הוא: $\frac{|w^T x - b|}{\|w\|}$. מדוע זה קשור אלינו? כאשר הנקודה x תשב בדיוק על קו המדרכה נקבל כי $w^T x - b$ יהיה בדיוק 1 בערכו המוחלט. ולכן, המרחק מנקודה מסוימת x שנמצאת על שפת המדרכה - ($Margin$ - הקו המפריד) הוא:

$$\gamma = \frac{1}{\|w\|}$$

ולכן, על מנת למקסם את γ עלינו למזער את $\|w\|$. כלומר - נחפש וקטור שהוא כמה שיותר "קצר".

נשאל את עצמנו כעת - למה כדי למקסם את המרחק המינימלי עבור נקודה כלשהי, מישור ההפרדה $w^T x - b$ חייב לעבור בדיוק באמצע?

נניח כי המרחק בין הנקודה הכחולה הקרובה ביותר לאדומה הקרובה ביותר הוא D . המישור המפריד מחלק את המרחק הזה לשני חלקים - d_1 ו- d_2 . אנחנו רוצים למקסם את המרחק המינימלי עבור נקודה כלשהי. לכן: אם הקו קרוב יותר לאדומים - d_2 קטן והמרחק המינימלי הוא d_2 . אם הקו קרוב יותר לכחולים - d_1 קטן והוא המרחק המינימלי. לכן - בשביל למקסם את המרחק המינימלי (להגדיל את השוליים, הרי "המרחק המינימלי" הוא המרחק מנקודה אל הקו) נרצה כי $d_1 = d_2 = \frac{D}{2}$. ולכן: מישור ההפרדה חייב לעבור בדיוק באמצע השוליים.

כעת נרצה לפתור למעשה את בעיית האופטימיזציה שבה $Margin$ יהיה מקסימלי. לשם כך,

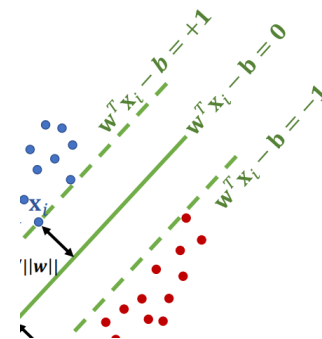
נעזר בטריק: Scaling Trick.

נניח ובידינו משוואה $2x + 4y - 2 = 0$. המשוואה הנ"ל שקולה לה: $x + 2y - 1 = 0$. לכן - כל עוד אנחנו שומרים על היחס בין הוקטור w לב נוכל לכווץ אותו איך שנרצה. לכן: ננרמל את w ואת b כך שעבור הנקודות הכי קרובות יתקיים:

- בצד החיובי, +.
- בצד השלילי, -.

כעת: לכל נקודה מהמחלקה החיובית: היא חייבת להיות מהצד החיובי של המדרכה (מעל הקו) - כלומר $w^T x_i - b \geq 1$. לכל נקודה מהמחלקה השלילית: חייבת להיות מהצד השני כלומר $w^T x_i - b \leq -1$.

המרחק מהוקטור התומך עד לקו האמצע ($w^T x - b = 0$) הוא בדיוק $\gamma = \frac{1}{\|w\|}$. נרצה שוליים רחבים ככל הניתן - לכן צריך למקסם את $Margin$. כלומר, את המרחק $\frac{1}{\|w\|}$. שקול: למזער את $\|w\|$. נבחין שזה הרבה יותר קל לבצע בעיית מינימום על $\|w\|$. יתרה מכך - זה שקול למזער את $\frac{1}{2} \|w\|_2^2$ כי זה גזיר ונוח לאופטימיזציה. וכן: נבחין שאנחנו מקבעים את העל מישור שנוצר (הקו הישר) ומזערים את $\|w\|$.



5.11 Hard SVM

עבור נתונים פרידים לינארית ותוויות בינאריות ($y_i = \pm 1$) - נרצה לפתור את בעיית האופטימיזציה הבאה:

$$\min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \frac{1}{2} \|w\|^2 \text{ s.t } \forall i : y_i(w^T x_i - b) \geq 1$$

כלומר, למזער את $\|w\|$ שקול למזער את $\|w\|^2$, ונוסיף בעיית אילוף - נבחין שהאילוף מסתדר עם מה שראינו קודם. שכן y_i היא תווית (± 1). לכן, אם $y_i = 1$ נדרוש אכן כי $w^T x_i - b \geq 1$, ואחרת נדרוש (עם הפיכת האי שוויון) כי $w^T x_i - b \leq -1$. נבחין שזו פונקציה קמורה, וכן קיים לה מינימום גלובלי יחיד. לכן - אם מצאנו מינימום: הוא בוודאות המינימום ולא בור.

כיצד המחשב פותר את הבעיה הזו?

- פתרון ישיר - The primal QP: הדרך הנאיבית. לתת למחשב את בעיית המינימום הנ"ל, איך זה עובד? המחשב מחפש את הערכים האופטימליים עבור כל תכונה שיש לנו בדטה ($d+1$ משתנים) - אך הסיבוכיות ($O(d^3)$).
- SGD: משתמשים בגרדיאנט דיסנט, הוא סופר מהיר ודורש מעט זיכרון. נרחיב עליו בהמשך.
- The Dual formulation: כאן מבצעים טריק מתמטי - זו הדרך הכי מוכרת. מבצעים אופטימיזציה של הנקודות עצמן (n נקודות). למה זה טוב? כי לפעמים $n < d$ וכן זה מאפשר להפריד נתונים שאינם לינאריים - כמו עיגול בתוך עיגול וכו'.

5.12 The penalty method

על מנת לפתור את ה-SVM, נרצה לעבור מבעיה עם אילוצים לבעיה שאפשר לפתור בצורה חופשית הרבה יותר. במקום לומר למחשב - אסור לך לעבור את הקו (האילוצים) משנים את פונקציית המטרה להיות:

$$\min_x f(x) + \sum_i \text{Penalty}(g_i(x))$$

כך שהאילוצים $g_i = 1 - y_i(w^T x_i - b) \leq 0$ וכן $f(x) = \frac{1}{2} \|w\|^2$. נגדיר את העונש:

$$\text{Penalty}(g_i(x)) = \begin{cases} 0 & g_i(x) \leq 0 \\ \infty & g_i(x) > 0 \end{cases}$$

כיוון שהמחשב מפחד מקנס אינסופי - הוא ייאלץ לפתור את הבעיה הזו תחת האילוצים הקיימים.

אבל, זה די רע להגדיר "אנסוף": מתמטית פונקציית אנסוף לא גזירה ונשברת בנק' האפס. למחשב קשה לעבוד עם פונקציות כאלו באופטימיזציה כי אין גרדיאנט. לכן: משתמשים בקירוב לינארי. כעת הקנס יוגדר:

$$\text{Penalty}(g_i(x)) = \alpha_i g_i(x)$$

כאשר α_i הוא משקל שאנחנו לומדים. לכל נקודה i בדטה, הצמדנו מעין "שומר ראש" בדמות α_i : תפקידו לקבוע כמה חמור הקנס על חריגה מהאילוף הנוכחי עבור נקודה זו. ערכו תמיד אי שלילי, והאלגוריתם לומד כמה גדול צריך להיות α_i על מנת לאכוף את האילוף $g_i(x) \leq 0$. כעת נגדיר:

הגדרה 95. (The Lagrangian Formulation) נגדיר את נוסחת הלגראזיאן כך:

$$\mathcal{L}(x, \alpha) = f(x) + \sum_i \alpha_i g_i(x)$$

באשר $f(x) = \frac{1}{2} \|w\|^2$, $g_i(x)$ הם האילוצים α_i הם הקנס.

נבחין שקיבלנו פונקציה בשני משתנים:

משתנה ראשון, α . הפונקציה רוצה למקסם את הקנס באמצעותו.

משתנה שני, x . הפונקציה מנסה למזער את הערך על מנת למקסם את השוליים (שכן נזכר כי השוליים $\frac{1}{\|w\|}$).
לכן קיבלנו את בעיית המינימום:

$$\max_{\alpha \geq 0} \min_x \mathcal{L}(x, \alpha)$$

משפט 96. משפט KKT . באופטימום (בפתרון הכי טוב) צריך להתקיים: לכל נקודה i $\alpha_i g_i = 0$.

כלומר, לכל נקודה i יקרה אחד מההבאים:

1. $\alpha_i = 0$, הנקודה לא משפיעה על הקנס. אלו הנקודות שנמצאות עמוק בתוך הפחלקה שלהן - רחוק מהשוליים.
2. $g_i(x) = 0$. הנקודה היא נמצאת בדיוק על המזרקה. אם הוקטורים התומכים.

מסקנה מ KKT : רק הוקטורים התומכים קובעים את הקו (הרי אנחנו מחפשים w, b עבורם $w^T x - b = 0$).

משפט 97. הפתרון הזואלי של SVM יהיו $W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$ **הוכחה:** נרצה לגזור את

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i (1 - y_i (w^T x_i - b))$$

הרי כי

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^n \alpha_i y_i x_i \implies w^* = \sum_{i=1}^n \alpha_i x_i y_i$$

כלומר, הוקטור w^* שמקסם הוא צ"ל של הדטה. נבחין כי לפי KKT רוב הנקודות יקיימו $\alpha_i = 0$ ולכן ה SVM חסכוני כל כך.
כעת נגזור את הגרדיאנט לפי b ונקבל את האילוץ:

$$\nabla_b \mathcal{L} = \sum_{i=1}^n \alpha_i y_i = 0$$

כעת נציב ב $\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i (1 - y_i (w^T x_i - b))$ בשלבים: 1.

$$\frac{1}{2} \|w\|^2 = \frac{1}{2} w^T w = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i x_i y_i \right)^T \left(\sum_{i=1}^n \alpha_i x_i y_i \right) = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i x_i^T y_i \right) \left(\sum_{i=1}^n \alpha_i x_i y_i \right) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

2. נסדר את $\sum_{i=1}^n \alpha_i (1 - y_i (w^T x_i - b))$. נראה כי שקול לכתוב:

$$\sum_{i=1}^n \alpha_i - b \sum_{i=1}^n \alpha_i y_i - \sum_{i=1}^n \alpha_i y_i w^T x_i$$

ראשית, $-b \sum_{i=1}^n \alpha_i y_i = 0$ מהאילוץ שראינו. שנית,

$$\sum_{i=1}^n \alpha_i y_i w^T x_i = \sum_{i=1}^n \alpha_i y_i \left(\sum_{i=1}^n \alpha_i x_i y_i \right)^T x_i = \sum_{i=1}^n \alpha_i y_i \left(\sum_{i=1}^n \alpha_i x_i^T y_i \right) x_i = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j$$

לכן סה"כ

$$W(\alpha) = \sum_{i=1}^n \alpha_i + \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j) - \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

$$W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

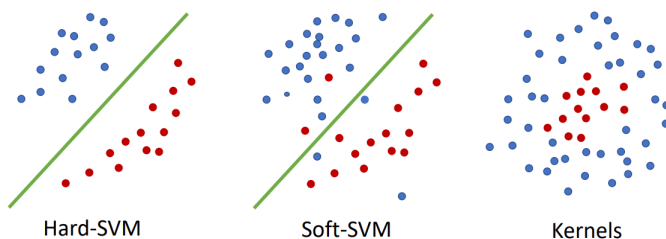
ולכן קיבלנו את בעיית האופטימיזציה הבאה:

$$\max_{\alpha} W(\alpha) \text{ s.t. } \sum_{i=1}^n \alpha_i y_i = 0$$

אם נסכם את בעיית ה-SVM:

- זו בעיית אופטימיזציה קמורה - ולכן מובטח מינימום גלובלי.
- זמן הריצה לרוב $O(n^2)$ או $O(n^3)$ באשר זה תלוי במס' הנקודות n בלבד.
- כשיש הרבה פיצ'רים, d גדול אך ניתן לראות מהזמן ריצה שזה לא תלוי בו ולכן הופך לעיל מאוד כאשר מס' הפיצרים גדול.
- בזכות KKT , רוב ה- α_i הופכים לאפס. לכן לאחר האימון המודל תופס מעט מאוד זיכרון.

אם נסכם, עד כה טיפלנו ב-Hard-SVM, ובהמשך:



5.13 Soft SVM

היכן נשבר ה-Hard-SVM? כאשר הנתונים אינם פרידים לינארית. אך גם את המחלקה הזו ניתן לסווג - למקרה השני בדוגמה מעלה. שיש מעט חריגות - בודדות, בגלל רעש או טעויות תיוג. בזה נטפל כעת.

על מנת שלא כל נקודה תהרוס לנו את המודל, נגדיר לכל נקודה דטה x_i משתנה אישי בשם ξ_i , אי שלילי. המשתנה הנ"ל מאפשר למודל להרפות מהאילוץ הקשוח של המרחק מהקו. ישנם כמה אפשרויות:

1. $\xi_i = 0$, אזי הנקודה נמצאת בדיוק במקום הנכון. או על שפת המדרכה באחד הכיוונים - או רחוק ממנה.
2. $\xi_i \in (0, 1]$: הנקודה נמצאת בתוך הרחוב, אך עדיין בצד הנכון של הקו המפריד.
3. $\xi_i > 1$: טעות בסיווג. הנקודה נמצאת בצד הלא נכון וחצתה את הקו המפריד.

כעת ניצור את האילוץ הבא: $y_i(w^T x_i - b) \geq 1 - \xi_i$. אם $\xi_i > 0$ אזי הנקודה אישור להכנס לתוך המרווח. בשביל שהאלגוריתם לא יתן לכל הנקודות ξ_i אינסופי ויתעלם מכולן ככה, מוסיפים קנס לפונקציית המטרה:

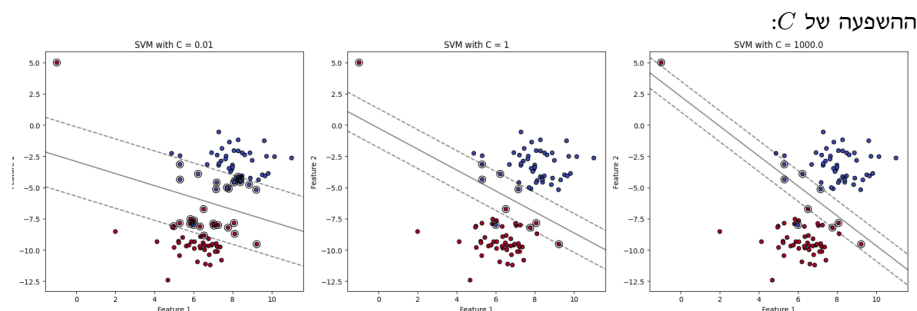
$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

יש לנו כאן שני מחוברים שלוקחים לכיוונים שונים - $f(x)$ שואפת למקסם את הרווח - השוליים, $C \sum \xi_i$ שואף למזער את כמות וגודל הקנס. את C קובע המתכנת עצמו.

• גדול: C הקנס על כל חריגה עצום! המודל ילחם על כל נקודה שתהיה בצד הנכון. גם אם זה אומר שיתפשר על השוליים ויהיו שוליים צרים. מתנהג דומה ל-Hard SVM.

• קטן: C המודל סבלני יותר ויותר מקבל טעויות. יותר טעויות אימון. סכנה ל-underfitting.

אם המודל יסבול ל-overfitting: נרצה להקטין את C - שכן נרצה לשאוף להגדיל את השוליים וכך השוליים הרחבים יותר מייצגים קו הפרדה כללי יותר שלא מנסה להתאים את עצמו לנתונים בכוח.



במקום להחזיק משתנה נפרד ξ_i לכל נקודה, נשאל את השאלה הבאה: מיהו ξ_i המינימלי שפותר את האילוף?
 אם הנקודה מסווגת נכון - היא לא צריכה שום גמישות ו $\xi_i = 0$.
 אם הנקודה לא מסווגת נכון - נרצה להעניש אותה בכמה היא רחוקה מלהגיע לאחד. כלומר: $\xi_i = 1 - y_i(w^T x_i - b)$.
 מכאן מקבלים:

$$\xi_i = \max\{0, 1 - y_i(w^T x_i - b)\}$$

כעת:

$$\min_{w,b} \left[\|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i - b)) \right]$$

ביטוי זה מורכב גם הוא מזוג חלקים שמופיעים כמעט בכל אלגוריתם למידה:
 1. Empirical Risk: כמה המודל טועה על נתוני האימון, מימין.
 2. רגולציה: משמאל. שומר על המודל פשוט.

5.14 סיכום

למה אנחנו אוהבים את SVM ?

- דלילות - המודל חסכוני מאוד בזיכרון. אחרי האימון אפשר לזרוק 99% מהדטה ולשמור רק את הוקטורים התומכים. רק הם קובעים את קו ההפרדה.
- בניגוד לפרספטרון שרק מוצא קו, ה SVM מוצא את הקו עם השוליים הרחבים ביותר מה שמאפשר לו לבצע הכללה באופן מעולה.
- עובד מעולה גם שיש יותר פיצ'רים מדגימות.

5.15 אופטימיזציה תחת אילוצים

עד כה בקורס פתרנו בעיות אופטימיזציה אך לא תחת אילוצים - בנוסחאות סגורות (רגרסיה), באמצעות שיטות גרדיאנט (GD) וכו'. בחיים האמיתיים זה לא ככה - לרוב אכן יש מגבלות.

הגדרה 98. הגדרה: בעיית אופטימיזציה מוגדרת להיות מהצורה הבאה:

$$\min_x f(x)$$

$$s.t \ g_i(x) \leq 0, \forall 1 \leq i \leq m$$

תחת תנאים מסויימים, ניתן להמיר את בעיית המינימום עם האילוצים למבנה של פונקציה אחת חדשה (הלגראזיאן):

$$\max_{\lambda} \min_x f(x) + \lambda_i g_i(x)$$

במקום לחפש מינימום תחת מגבלה - מחפשים נקודת אוכף (מינימום לפי x ומקסימום לפי λ). λ נקרא כופל לגראנז'.

משפט 99. (תנאי KKT). על מנת שנקודה x^* תהיה הפתרון האופטימלי של הלגראזיאן היא חייבת לקיים את התנאים הבאים:

1. הגרדיאנט בה חייב להתאפס - $\nabla_x L(x^*, \lambda^*) = 0$.
2. $\lambda g(x^*) = 0$ (או שהאילוץ שווה לאפס, או שהכופל הוא אפס).
3. $g(x^*) \leq 0$.
4. $\lambda \geq 0$.

כאשר תנאים אלו מתקיימים, x^* הוא המינימום של פונקציית הלגראנז'.

נרצה לראות דוגמה. נתבונן בפונקציה הפשוטה $f(x, y) = x + y$ תחת האילוץ $x^2 + y^2 \leq 1$ - כלומר: מעגל היחידה. נקבל כי $g(x, y) = x^2 + y^2 - 1 \leq 0$ במקרה זה נקבל:

$$L(x, y, \lambda) = x + y + \lambda(x^2 + y^2 - 1)$$

את הפונקציה גוזרים לפי כל אחד מהמשתנים ומשווים לאפס:

$$\frac{\partial L}{\partial x} = 1 + 2\lambda x, \quad \frac{\partial L}{\partial y} = 1 + 2\lambda y, \quad \frac{\partial L}{\partial \lambda} = x^2 + y^2 - 1 = 0$$

ממשוואה ראשונה ושנייה מקבלים כי $1 + 2\lambda x = 1 + 2\lambda y$ ולכן $2\lambda x = 2\lambda y$ כלומר $x = y$ (תחת ההנחה כי $\lambda > 0$) ונקבל $2x^2 = 1$ כלומר $y = x = \pm \frac{1}{\sqrt{2}}$. ישנן 2 נקודות חשודות אם כן: $(\pm \frac{1}{\sqrt{2}}, \pm \frac{1}{\sqrt{2}})$. מטרתנו למצוא את $x + y$ ולכן בבירור הנקודה שתמצא את הסכום היא $(-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}})$.

5.16 maximum entropy

בהינתן מידע חלקי בלבד, כיצד נבחר פונקציית התפלגות הסתברות? כאשר ידועות לנו רק עובדות ספורות על מערכת מסוימת - קיימות הרבה התפלגויות הסתברות שונות שיכולות להתאים לעובדות הללו. מתוך כל האפשרויות שמתאימות למה שאנחנו יודעים - מיהי זו שהכי נכונה לבחור? עקרון maximum entropy קובע שעלינו לבחור את ההתפלגות שמתיישבת עם המידע הקיים אך עושה את מס' ההנחות הנוספות המועט ביותר. כלומר - אל תניח דבר שלא אמרו לך באופן מפורש. אם לא אמרו לך: תשאיר אותו אקראי או לא מוטה - הכי שאפשר. **לדוגמה:** קוביה, נתון שיש בה 6 פאות. אם נסמן p_i כהסתברות לפגוע בפאה ה- i : נקבל כי $p_i = \frac{1}{6}$ לפי maximum entropy שכן לא אמרו לנו שלאף פאה יש סיכוי טוב יותר מהאחרות. כעת נגדיר זאת בצורה פורמלית. כיצד נמצא את אותה התפלגות הכי פחות מוטה? התפלגות זו היא ההתפלגות בעלת האנטרופיה הגדולה ביותר. אנטרופיה גבוהה מעידה על אי ודאות מקסימלית, וזה גורר מינימום הנחות מוקדמות. נגדיר את האנטרופיה בצורה הבאה:

$$H(p) = - \sum_{i=1}^n p_i \log_2(p_i)$$

באשר p_i היא ההסתברות של המאורע ה- i לפי פונקציית ההתפלגות p . כיוון ש $p_i \in (0, 1)$ אזי $\log(p_i) < 0$ ולכן כל ה"ל קטן מאפס. המינוס לפני - מגיע בשביל להפוך את הביטוי לחיובי. אנחנו מעוניינים לבצע מקסימיזציה של האנטרופיה תחת אילוצים:

1. $\sum_i p_i = 1$
2. $p_i > 0$ לכל $1 \leq i \leq n$
3. אילוצים נוספים שתלויים במקרה.

כעת נחזור לדוגמת הקוביה. אין לנו מידע כלל על הקוביה. נרצה למקסם את $H(p) = - \sum_{i=1}^6 p_i \log_2(p_i)$ תחת האילוצים: $\sum_{i=1}^6 p_i = 1$ וכן $p_i \geq 0$ לכל $i \in [6]$. אם כן הלגראזיאן הוא:

$$\mathcal{L}(p, \lambda_0, \lambda_1) = - \sum_{i=1}^6 p_i \log_2(p_i) + \lambda_0 \left(\sum_{i=1}^6 p_i - 1 \right) + \sum_{i=1}^6 \lambda_i p_i$$

באשר λ_i הם הכופלים שמאלצים אי שליליות λ_0 אוכף את הסכום. אם גוזרים לפי p_i מקבלים כי (לאחר שמתעלמים מהאילוץ $p_i \geq 0$ שהוא ברור מאליהו):

$$p_i = 2^{\lambda_0 - \frac{1}{\ln(2)}}$$

באופן כללי, מדובר בערך קבוע. נסמנו C . ידוע כי צריך להתקיים $\sum_{i=1}^6 p_i = 1$ ובדומה $6C = 1$ ולכן $C = \frac{1}{6}$. כלומר, הראינו מתמטית שהדרך למקסם את האנטרופיה ולהיות הכי פחות מוטים כשלא יודעים כלום - הוא להניח שלכל פאה יש סיכוי שווה בדיוק.

5.17 Evaluation Metrics (מדדי הערכה)

נניח תרחיש מסוים: יש לנו דטה-סט של מאה חולים שיש לאבחן. נתון שהמודל שבידינו גרוע: הוא חוצה לכל המטופלים שיהיו בריאים. התוצאה - המודל אכן צדק לגבי 19 מטופלים. כלומר הדיוק (Accuracy) של המודל הוא 19%. אז נשאלת שאלה למחשבה - האם זה מודל טוב? על הנייר 91% נשמע מעולה. אך המודל הנ"ל מסוכן וחסר תועלת! הוא החטיא את כל החולים - וכן יש כאן חוסר איזון. המודל ניחש ולא למד כלום.

עבור בעיית סיווג בינארי - נגדיר את Confusion matrix: יש לנו תמיד 4 מצבים אפשריים.

1. TP - המטופל חולה - והמודל אכן צדק ואמר שהוא חולה. (מעולה).

2. TN - המטופל בריא, והמודל צדק ואמר שהוא בריא. (מעולה).

3. FP - המטופל בריא, אך המודל אמר שהוא חולה. (אזעקת שווא).

4. FN - המטופל חולה, אך המודל אמר שהוא בריא. (זו הטעות הכי גרועה!).

חשוב להבחין - בסטטיסטיקה ובלמידת מכונה, $Positive$ זה לא "טוב" אלא "קיים".

כיצד קוראים את השמות? מימין לשמאל. למשל: FN - המילה $Negative$: המודל אמר! המודל חזה שלילי. אבל $False$! הוא שיקר: כלומר

אלו אנשים שעבורם המודל חזה בריא (לא חיובי למחלה) בעוד המטופל לא בריא.

		Prediction		
		pos	neg	Total
Actual	p	True Positive	False Negative	P'
	n	False Positive	True Negative	N'
Total		P	N	

קודם דיברנו על בעיית הדיוק המטעה - עם 91% הדיוק. כעת נטפל בזה בצורה שנבין לא רק "כמה המודל צדק" אלא "איך הוא צדק".

הגדרה 100. נגדיר את Precision להיות מספר התחזיות החיוביות הנכונות מתוך כל התחזיות שהמודל סימן כחיוביות. כלומר:

$$\text{Precision} = \frac{TP}{TP + FP}$$

כלומר, מתוך כל הפעמים שהמודל סימן כחיובי - כמה מהם הוא באמת צדק.

כמו כן, נגדיר את Recall להיות מספר התחזיות החיוביות הנכונות מתוך כל המקרים שהם אכן חיוביים במציאות. כלומר:

$$\text{Recall} = \frac{TP}{TP + FN}$$

כלומר, מתוך כל החולים שקיימים: כמה המודל תפס. כיצד זה מסתדר? FN זה המודל טעה על אנשים שהם לא N . כלומר הם $Positive$ בפועל, וכן TP : אנשים שהם $Positive$ בפועל והמודל צדק עליהם.

הגדרה 101. קשה להשוות שני מודלים כשאחד טוב בPrecision (חזרה טוב מבין מה שהוא סימן) ואחד טוב בRecall (חזה טוב את הנכונות). לכן, מגדירים את ה-F-Score (F_1) להיות מדד שנותן ציון אחד משוקלל לשניהם.

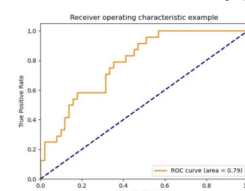
$$F_1 = \frac{2}{\text{Recall}^{-1} + \text{Precision}^{-1}} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

משתמשים בממוצע הרמוני שכן הוא מעניש חזק ערכים קיצוניים - אם אחד המדדים שואף לאפס, כך גם ה- F_1 כולו ישאף לאפס. מחייב את המודל להיות מאוזן (בשני המדדים) על מנת לקבל ציון גבוה.

עקומת ROC-Curve: המודל לא תמיד אומר 0 או 1 בצורה נחרצת. אלא - כמו ברגרסיה לוגיסטית, נותן הסתברות. אנחנו אלו שבוחרים את Threshold אם נקבע אותו להיות 0.5 או 0.9. העקומה הנ"ל מציגה את הביצועים של המודל על פני כל הספים (Threshold) האפשריים ומשלבת שני מדדים:

1. TPR - True positive Rate: ערך הRecall, כמה מהחיוביים תפסנו. (ציר ה-y).

2. FPR - False Positive Rate: אמרנו חיובי וטעינו - כלומר כמה מהשליליים "לכלכנו" ואמרנו שהם חיוביים. $\frac{FP}{FP+TN}$ (ציר ה-x).



ROC Curve שמופיע מטה (0.79) הוא השטח שמתחת לעקומה. אם $AUC = 1$ (Area under the curve) אזי המודל מושלם ואם $AUC=0.5$ מדובר בניחוש אקראי. למה הגרף עוזר לנו? אם אנחנו בבית חולים - נעדיף Recall גבוה ולכן נבחר נקודה גבוהה על ציר ה-y, גם אם נוזז ימינה בציר ה-x (יותר אזהקות שווא).

הגדרה 102. ה-AUC הוא השטח הכלוא מתחת לעקומת ה-ROC (השטח בין הקו הכתום לציר ה-x). הוא נע תמיד בטווח בין 0 לאחד. ככל שה-AUC גבוה יותר, המודל חכם יותר בזיהוי ההבדלים בין המחלקות. אם $AUC = 1$ אזי מדובר במודל מושלם, שתמיד נותן לחיוביים ציון גבוה יותר מלשלילים.

6 הרצאה 6

6.1 SVM kernels

נזכר בתמונה המפורסמת שמלווה אותנו כבר כמה הרצאות - ונדון כעת במצב שבו הנתונים מגיעים ללא הפרדה לינארית כלל, לא במקרה של זליגה בשל רעש והפרעה בתיוג. אלא: אין הפרדה לינארית בכלל.

נדון כעת בבדיקה אמיתית - מודלים לינאריים (למשל: רגרסיה לינארית, SVM לינארי רגיל) סובלים High bias (הטיה גבוהה) כאשר מיישמים אותם על סט נתונים מורכבים. אם גבול ההחלטה האמיתי בסוף לא לינארי - מסווג לינארי טהור יבצע תמיד Underfitting לנתונים (שהרי, המודל אותו לקחנו פשוט מדי ולא מצליח להתמודד איתם). מה ניתן לעשות אם כן?

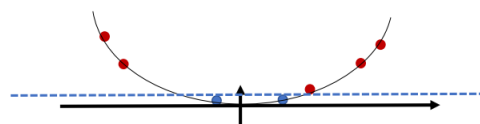
1. ובכן, פשוט להפוך את הנתונים לניתנים להפרדה לינארית. ישנה דרך לבצע טרנספורמציה (התמרה) לנתונים כך שיהיו ניתנים להפרדה לינארית - על זה נלמד היום. הרעיון יהיה ראשית לבצע התמרה של הנתונים למרחב שבו הם כן יהיו ניתנים להפרדה לינארית, ושם להפעיל אלגוריתם לינארי כמו שכבר ראינו.

2. להשתמש באלגוריתמים של סיווג לא לינארי, כמו רשתות ניורונים רב שכבתיות וכו'.

אם כן, כיצד ממירים נתונים לניתנים להפרדה לינארית? נניח ואנחנו עובדים תחת מימד אחד, $\mathbb{R}$. יהי קלט $x \in \mathbb{R}$ של נתונים. נניח כי קלט הנתונים שלנו לא ניתן להפרדה לינארית (באמצעות קו אחד על הישר). לא משנה היכן נעביר את הגבול - בהכרח נפספס חלק מהנקודות. למשל:

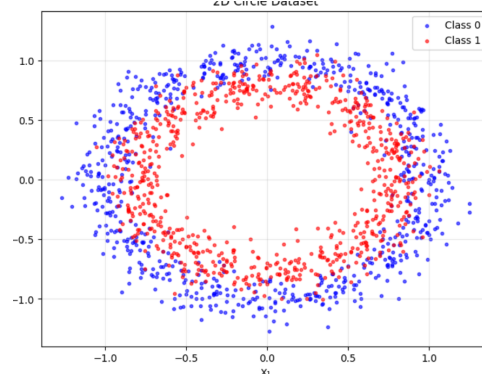
[■, ■, □, □, ■, ■, ■]

כעת נתבונן בהתמרה הבאה: $x'_i \rightarrow [x_i, x_i^2]$. **נשים לב:** זו לא העתקה לינארית ולא הומומורפיזם! למעשה: מעבירים את הערך ב- $\mathbb{R}$ לערך ב- $\mathbb{R}^2$. נקבל כעת שכל נקודה היא וקטור, וקלט הנתונים הקודם יהיה כעת:



כעת הנתונים כן ניתנים להפרדה לינארית. בפרט: הוקטור הכחול מפריד ביניהם. זה יהיה הבסיס לכל מה שנלמד השיעור.

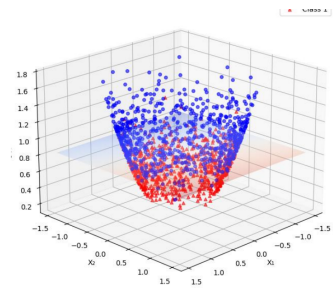
דוגמה נוספת: נתונים ב- $\mathbb{R}^2$ שלא ניתנים להפרדה לינארית!



עם זאת, נגדיר את ההתמרה הבאה:

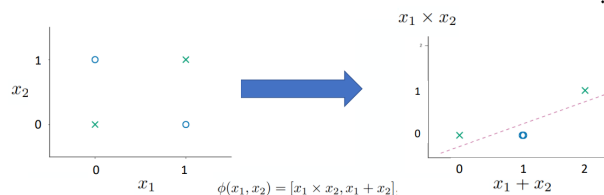
$$\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$$

מדוע? נבחין כי קודם לכן השתמשנו ב- x^2 על מנת שהנקודות שרחוקות מהראשית יעלו למעלה יותר בציר - ולכן נוכל להפריד ביניהם לינארית. כאן, מחפשים רכיב שלישי (מימד נוסף) שיגרום לנקודות להתרומם בצורה שתאפשר הפרדה. נבחין כי כל הנקודות האדומות, קרובות יותר לראשית באשר לכוחות שרחוקות ממנה. ולכן: המימד השלישי ייצג את המרחק מהראשית ויצליח (אולי) להפריד בין הנתונים. נקבל:



נראה כי כעת כן יש מישור שיכול להפריד בין הנקודות. עם זאת - אכן יש נקודות שחופפות לקטעים שלא שלהם, ומדובר ברעש. אנחנו יודעים להתמודד עם רעש באמצעות Soft SVM - ולכן מצבנו כעת מעולה.

נעיר, כי לא חייבים בכלל לעבור מימד. למשל ההתמרה: $\phi(x_1, x_2) = (x_1 \cdot x_2, x_1 + x_2)$ מצליחה להפוך את הבעיה של XOR לפרידה לינארית:



6.2 הגדרה פורמלית

אם סט נתונים אינו ניתן להפרדה לינארית במרחק הקלטים המקורי שלו $\mathbb{R}^d$, מבצעים טרנספורמציה (התמרה) של הנתונים למרחב תכונות חדש (Feature Space) שיוסמן ב- $\mathbb{R}^D$ בו נוכל למצוא גבול החלטה לינארי. נגדיר את פונקציית המיפוי הבאה:

$$\phi: \mathbb{R}^d \rightarrow \mathbb{R}^D$$

לרוב $D > d$ (מרחיבים את כמות התכונות על מנת למצוא את ההפרדה).

תהליך העבודה עובד בצורה הבאה:

1. התמרה: לכל נקודת אימון $x_i \in S_n$ נבצע התמרה $x_i \rightarrow \phi(x_i)$
2. מציאת מפריד: מוצאים על מישור לינארי (אם המימד הוא d , על מישור הוא במימד $d-1$. לכן על מישור ב- $\mathbb{R}^2$ הוא קו ישר וב- $\mathbb{R}^3$ הוא מישור) במרחב החדש, משוואת המסווג היא $f(x) = w^T \phi(x) + b$
3. וקטור המשקולות w חי כעת בתוך מרחב התכונות $\mathbb{R}^D$.

6.3 ומה עושים לאחר ההתמרה?

לאחר ההתמרה, נרצה למצוא מפריד (על מישור לינארי) במרחב החדש. נוכל להשתמש באלגוריתמים הלינאריים שכבר ראינו.

1. פרפסטרון עם $x \rightarrow \phi(x)$

אימון -

- אתחל את w אקראית.
- חזור על התהליך עד התכנסות:
- לכל $1 \leq i \leq n$:
- אם $y_i w^T \phi(x_i) < 0$ אז $w^{t+1} = w^t + y_i \phi(x_i)$
- החיזוי - $y_i = \text{sing}(w^T \phi(x_i))$

הערה. להלן האלגוריתם לאחר הטריק הדואלי

Algorithm 4 Kernel Perceptron Algorithm**Require:** Training set $D = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$, kernel $K(\cdot, \cdot)$, number of iterations T **Ensure:** Coefficients $\alpha_1, \dots, \alpha_n$ Initialize $\alpha_1, \dots, \alpha_n \leftarrow 0$ **for** $t = 1$ **to** T **do****for** $i = 1$ **to** n **do** $f(\mathbf{x}_i) \leftarrow \text{sign}\left(\sum_{j=1}^n \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i)\right)$ **if** $f(\mathbf{x}_i) \neq y_i$ **then** $\alpha_i \leftarrow \alpha_i + 1$ **end if****end for****end for**

2. SVM:

בעל פונקציית המטרה:

$$\min_{w, b} \left[\|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T \phi(x_i) - b)) \right]$$

6.4 The curse of explicit mapping (קללת המיפוי המפורש)

עד כה זה נראה מעולה - כמו קסם. הכל עובד. אבל זה לא באמת כל כך פשוט. כאשר מתקיים $d \gg D$ נוצרות שתי בעיות משמעותיות:

- 1. בעיית ההכללה:** ניסינו לפתור בעיה של High bias (הטיה גבוהה, כי המודל פשוט ולא הצליח ללמוד את הנתונים) אך אנחנו עלולים למצוא את עצמנו עם שונות גבוהה - כלומר עם מודל שמבצע Overfitting. עברנו מUnderfitting to Overfitting. הרי, שבממד גבוה קל מאוד למצוא על מישור שמפריד את נתוני האימון בצורה מושלמת אך הוא עלול להיות תפור מדי לרעש הספציפי ולא להצליח לסווג נתונים חדשים. כיצד נפתור זאת? באמצעות רגולציה, שתקטין *Overfitting* (המודל לא יוכל להתפרע עם משקולותיו) או שנוכל גם להגדיל את נתוני האימון.
- 2. בעיית החישוביות:** אם אנחנו מבצעים מיפוי מפורש - כלומר ממש מחשבים לכל נקודה בדטה את $\phi(x_i)$ זה יעלה המון! להפוך כל וקטור $\phi(x_i)$ ל- SVM (אותו לא למדנו) עולה $O(d^3)$. אם d גדול - המחשב פשוט לא יעמוד בזה. כלומר, יש כאן באמת בעיה שקשה לפתור.

6.5 Kernel Trickבעיית החישוביות אכן מקשה עלינו. אך ננסה להזכר בנוסחאות ה-*SVM*:

• The Dual Soft-SVM:

• Maximize:

- $\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
- subject to $0 \leq \alpha_i \leq C, \sum_{i=1}^n \alpha_i y_i = 0$

• The Primal Soft-SVM:

- $\min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \left[\frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i - b)) \right]$
- Rewriting using: $w = \sum_{j=1}^n \alpha_j y_j x_j$:
 $\min_{\alpha \in \mathbb{R}^n, b \in \mathbb{R}} \left[\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j) + C \sum_{i=1}^n \max(0, 1 - y_i(\sum_{j=1}^n \alpha_j y_j (x_i^T x_j) - b)) \right]$

אם נסתכל על הבעיה הדואלית, בה נרצה למקסם את המקדמים α_i השקולה, נראה כי בצהוב נקבל $x_i^T x_j$. כלומר בבעיה הדואלית הנתונים מופיעים אך ורק כשהם כבר בתוך מכפלה סקלרית בין שני וקטורים. אין לנו באמת צורך במקרה זה בחישוב $\phi(x_i), \phi(x_j)$ בנפרד. אלא רק בנתון: $\phi(x_i)^T \phi(x_j)$. למה זה כל כך חשוב? אם נמצא פונקציה $K(x_i, x_j)$ שיוצאת לחשב את המכפלה הנ"ל במרחב הגבוה, אך עושה זאת באמצעות חישוב פשוט במרחב הנמוך (בלי לחשב ϕ לכל וקטור) חסכנו את כל קללת המימד.

מטרה: לחשב לכל x, z את $\phi(x)^T \phi(z)$ אך לעשות זאת בעלות $O(d)$ - בעלות המימד הנמוך.

הגדרה 103. פונקציית קרנל, המסומנת ב- $K(x, z)$ היא פונקציה שמקבלת שני וקטורים במרחב המקורי ומוציאה ישירות את המכפלה הסקלרית שלהם במרחב התכונות. כלומר: $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^D$ ע"י:

$$K(x, z) = \phi(x)^T \phi(z)$$

עבור מיפויים מסויימים ϕ ניתן לחשב את פונקציית הקרנל ישירות - מבלי לחשב לעולם את $\phi(x), \phi(z)$ באופן מפורש. ואז: אנחנו מבצעים חישוב בעלות $O(d)$ בדיוק כמו ב SVM רגיל, וחשכנו את קללת המימד.

דוגמה 104. נגדיר מיפוי באופן הבא: $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ כך:

$$\phi(x_1, x_2) = \begin{pmatrix} x_1^2 \\ \sqrt{2}x_1x_2 \\ x_2^2 \end{pmatrix}$$

נראה כי:

$$\phi(x)^T \phi(z) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)(z_1^2, \sqrt{2}z_1z_2, z_2^2) = x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = (z_1x_1 + x_2z_2)^2$$

והרי, $z_1x_1 + x_2z_2$ זו המכפלה הסקלרית הרגילה במרחב המקורי. לכן מתקיים:

$$K(x, z) = (x^T z)^2$$

ולכן עבור פונקציית הקרנל הנ"ל, אנחנו לא נדרשים לחשב את $\phi(x)\phi(z)$ בכלל אלא רק את $x^T z$ (בעלות $O(d)$ באשר אצלנו $d = 2$) ולהעלות בריבוע.

כעת, נראה כי אם נרחיב את ההגדרה לכל דרגה פולינומיאלית p ונגדיר:

$$K(x, z) = (x^T z + 1)^p$$

אזי מדובר בפונקציית קרנל.
(ההוכחה מתבססת על הטענות הבאות שלא הראינו בהרצאה אך נראה לי פשוטות להראות:
בהינתן פונקציות קרנל K_1, K_2 וערך a אזי:
א. $K_1 K_2$ היא קרנל.
ב. $K_1 + K_2$ היא קרנל.
ג. $K_1 + a$ היא קרנל)

6.6 The radial Basic function (RBF)

נגדיר את הקרנל הבא, שהוא מהנפוצים ב SVM :

$$K(x, z) = e^{-\frac{\|x-z\|^2}{2\sigma^2}}$$

באשר σ^2 היא השונות (פרמטר חופשי שנקבע).
הקרנל הנ"ל לא סתם מכפלה. אלא יש לו משמעות - הוא בודק כמה הנקודות קרובות זו לזו:
1. אם $z \approx x$ המרחק בניהם קרוב לאפס ולכן הערך של הפונקציה הוא קרוב ל1.
2. אם x ו z רחוקים זה לזה - האקספוננט שואף למינוס אנסוף, והקרנל שואף לאפס.

כאשר משתמשים ב SVM עם RBF המודל לא עובד כמו במודל הרגיל - שם הוא לומד וקטור w ושוכח לאחריו את נקודות האימון. בגלל שהקרנל מודד מרחק, המודל עובד בצורה הבאה:

1. שומרים את ה $Support\ vectors$
2. כשמוגיעים נקודה חדשה, שואלים "כמה הוא דומה לוקטורים התומכים?"
3. כל וקטור תומך מקבל ציון לפי המרחק מהנקודה החדשה באמצעות הקרנל. אם הנקודה החדשה קרובה מאוד לוקטור תומך של מחלקה א', היא תקבל ציון גבוה ותסווג לשם.

פונקציית הקרנל מאפשרת להמיר וקטור ממימד d לוקטור ממימד אינסופי. כלומר, $\phi(x)$ ממימד אינסופי. כיצד? נתבונן בפיתוח טור טיילור של האקספוננט e^a :

$$e^a = 1 + a + \frac{a^2}{2!} + \dots$$

כשפותחים את ביטוי הקרנל, מקבלים מכפלה של שני טורים כאלו. כל איבר בטור האינסופי הזה מייצג מימד חדש ב $\phi(x)$. במקום שנצטרך לבחור פולינום מדרגה מסוימת, הקרנל נותן לנו את כל הדרגות בבת אחת. בפועל משתמשים בנוסחה פשוטה של אקספוננט, אבל מבחינה מתמטית, המחשב הרגע הריץ הפרדה במרחב עם אינסוף ממדים.

6.7 מתי קרנל נחשב חוקי?

משפט 105. פונקציה $K(x, z)$ היא קרנל חוקי אם ורק אם קיימת הטלה ϕ כלשהי כך ש:

$$K(x, z) = \phi(x)^T \phi(z)$$

על מנת לבדוק את הקיום הנ"ל משתמשים במטריצת גראם. בפרט, נקרא לה מטריצת הקרנל G . זו מטריצה בגודל $n \times n$ לפי מס' הדגימות, שמקיימת:

$$G_{ij} = K(x_i, x_j)$$

משפט 106. תנאי פרסר: קרנל K הוא חוקי אם ורק אם מטריצת גראם הפתאיה לו G היא PSD .

מדוע זה טוב לנו? אם מטריצת הקרנל אכן PSD אזי בעיית האופטימיזציה של Dual SVM היא קמורה. ולכן: קיים לנו פתרון אופטימלי אחד.

משפט 107. עבור כל קרנל פולינומי מהצורה $K(x, z) = (x^T z + 1)^d$ כאשר n הוא פימד הקלט הפקורי d הוא דרגת הפולינום, פימד ההטלה יהיה $\binom{n+d}{d}$.
דוגמה: אם נתבונן על $K(x, z) = (1 + x^T z)^3$ כאשר $x, z \in \mathbb{R}^2$ אזי: $n = 2, d = 3$ ונקבל $\binom{3+2}{3} = 10$ ואכן:

$$\phi(x) : \mathbb{R}^2 \rightarrow \mathbb{R}^{10}$$

ע"י

$$\phi((x_1, x_2)^T) = \begin{pmatrix} 1 \\ \sqrt{3}x_1 \\ \sqrt{3}x_1^2 \\ \sqrt{3}x_2 \\ \sqrt{6}x_1x_2 \\ \sqrt{3}x_2^2 \\ x_1^3 \\ \sqrt{3}x_1^2x_2 \\ \sqrt{3}x_1x_2^2 \\ x_2^3 \end{pmatrix}$$

6.8 Decision Trees

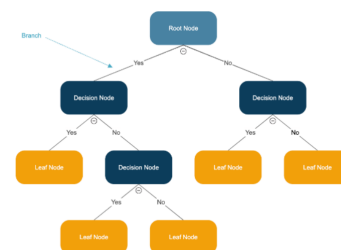
מהם עצי החלטה?

- למידה מונחית (supervised learning): עצי החלטה הם מודלים שמשמשים ללמידה מונחית. כלומר הם לומדים מנתונים מתויגים. ניתן להשתמש בהם גם למשימות סיווג, וגם למשימות רגרסיה.
- המודל איננו מניח הנחות מראש על צורת הקשר בין המשתנים (בניגוד למודל לינארי למשל) והוא לא מוגדר ע"י קבוצה קבועה של פרמטרים. כלומר הוא לא לינארי ולא פרמטרי. למשל: הוא לא מניח כי $y = ax + b$ (וכו'). ברגרסיה לינארית (פרמטרי) יש סט קבוע של משקולות w . כאן מס' הפרמטרים (הפיצולים בעץ) לא קבוע מראש וגדל ומשתנה ככל שיש יותר דטה. המודל גמיש והמבנה שלו נבנה לפי הנתונים עצמם.
- המודל מבצע חלוקה של מרחב התכונות למלבנים המקבילים לצירים - כל החלטה היא למעשה חיתוך ישר בניצב לאחד הצירים.

- מרחב כל העצים האפשריים הוא עצום בגודלו! הגמישות הזו מאפשרת למודל ליצור כל פונקציה בוליאנית.
- היתרון במודל זה הוא השקיפות - ניתן להבין את הלוגיקה שהובילה לכל החלטה.
- המודל יודע לעבוד גם עם נתונים קטגוריאליים וגם עם נומריים.

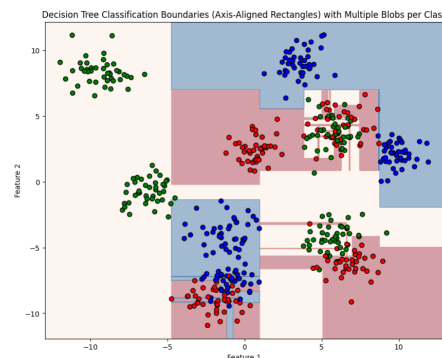
כיצד נראה עץ החלטה?

- מדובר במבנה של מודל שמדמה תרשים זרימה. נועד למדל החלטות ואת תוצאותיהן בצורה היררכית.
- צומת ראש - Root node: הצומת העליון ביותר בעץ. מייצג את התכונה הראשונה שלפיה מבצעים את הפיצול הראשוני של הדטה.
- צמתים פנימיים - Internal nodes: הצמתים שנמצאים באמצע העץ. כל אחד מהם מייצג בדיקה על תכונה ספציפית. לפי התשובה ממשיכים לענף המתאים.
- צמתי עלה - Leaf nodes: אלו הקצוות של העץ. צומת עלה לא מתפצל יותר: מייצג את התוצאה הסופית. בין אם זו תווית סיווג או ערך מספרי.
- בשלב האימוון (בהמשך נרחיב) העץ נבנה לפי כל מיני אלגוריתמים קיימים - ובשלב החיזוי מבנה העץ נקבע כבר מראש. נתונים חדשים נכנסים לעץ ומסווגים למחלקה מסויימת.



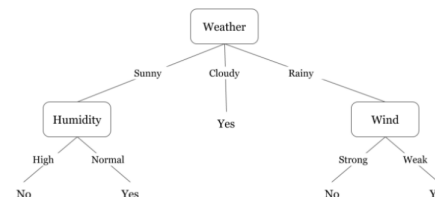
כיצד המודל הנ"ל מחלק את המרחב בפועל?

נתבונן בגרף שימחיש לנו. העץ מצליח ליצור גבולות הפרדה באמצעות מלבנים שמקבילים לצירים. העץ יוצר שרשרת חיתוכים ישרים שמבודדים את גושי הדטה. אם ניתן לעץ להיות עמוק יותר (ולהכיל יותר פיצולים) הוא יוכל לייצר מלבנים קטנים ככל שנדרש על מנת שיוכל להפריד בין סוגי הנקודות.



- כל הפרדה בגרף היא קו אופקי או אנכי. זה מה שקורה בכל צומת כאשר העץ שואל על פיצור בודד למשל: $Feature1 > 5$ - אז הוא מעביר קו אנכי בערך 5 על ציר ה-x: משמאלו הולך לענף שני, ומימינו לענף ראשון.
- עם זאת המלבנים לא תמיד עובדים, מדוע?
- Overfitting: אם ננסה לסווג כל נקודה ונקודה בצורה מושלמת, נקבל מודל שמשנן את הרעש בדטה במקום ללמוד את המגמה הכללית. זה ייצור עץ עמוק מדי, שלא יצליח להכליל.
 - נתונים עם הפרדה אלכסונית - אם ההפרדה האופטימלית בין הנקודות היא קו אלכסוני: למשל $x = y$ העץ יצטרך המון מדרגות מלבנים קטנים על מנת לנסות לדמות את האלכסון הנ"ל. זה לא יעיל ביחס למודל לינארי.

להלן דוגמה למודל כנ"ל. נבחין שגם בהינתן קלט מסויים חדש יתכן שהמודל לא יצטרך לקרוא את כל הפיצורים. למשל: יתכן כפי שקורה כאן שאם *cloudy* אזי התשובה תמיד כן.



6.9 כיצד מוצאים את עץ ההחלטה הכי טוב?

פורמלית, בהינתן דטה D נרצה למצוא עץ T כך ש:

$$\min_T \text{Loss}(T, D)$$

אם כן, אם יש לנו d פיצ'רים בוליאניים אזי ישנן 2^d פונקציות בוליאניות אפשריות. מס' עצי ההחלטה האפשריים אפילו גדול מזה. עבור דטה $X \in \mathbb{R}^{n \times d}$ (n דגימות עם d פיצ'רים) מרחב ההיפותזות האפשרי בגודל $(dn)^n$ - זהו מס' ענקי. ולכן: מציאת העץ האופטימלי היא בעיה NP -קשה.

מה הפתרון אם כן? נתפשר על חיפוש היוריסטי. במקום לנסות את כל השילובים נשתמש באלגוריתמים חמדניים שבונים את העץ צעד אחר צעד ומקבלים החלטה מספיק טובה בכל פיצול בלי להסתכל קדימה על כל העץ.

כיצד אלגוריתמים כאלו עובדים בפועל?

- Top-Down, Greedy: בונים את העץ מלמעלה למטה בצורה חמדנית. בכל צומת אנחנו בוחרים את הפיצול שהוא אופטימלי מקומית - כלומר זה שנותן את ההתקדמות הגדולה ביותר באותו רגע: בלי לבדוק כיצד זה ישפיע על פיצולים עתידיים בהמשך העץ.
- תהליך רקורסיבי: מחלקים את סט האימון לתתי קבוצות, ומחילים את אותה הלוגיקה על כל תת קבוצה בנפרד עד להגעה לתנאי עצירה.
- הומוגניות: נרצה כי העלים של העץ יהיו כמה שיותר טהורים. מה זה אומר? עלה טהור הוא כזה שבו כל הדגימות שבו שייכים לאותו Label בבעיית הסיווג. במצב כזה המודל בטוח ב-100% בסיווג עבור נתוני האימון.
- המטרה היא לחלק את מרחב התכונות כך שההסתברות המותניתית $Pr(y|x)$ תהיה קרובה ככל האפשר ל-0 או ל-1 (ודאות מוחלטת).

אם כן, כיצד בוחרים את הפיצ'ר הכי טוב לפיצול?

המטרה הרי שכמה שיותר עלים יהיו טהורים. נצטרך להשתמש בפונקציה היוריסטית שיאמר לנו כיצד לבחור את הפיצ'ר הכי טוב בקבוצה שנוצרה לאחר הפיצול - ובאמצעותו לפצל שוב. לכן נגדיר: פיצול גרוע - אם אחרי שחתכנו את הדטה קיבלנו קבוצה שהיא תערובת מבולגנת של תוויות: סימן שהפיצול לא קידם אותנו הרבה לקראת החלטה ברורה. פיצול מעולה - אם הפיצול יצר קבוצות טהורות. כלומר קבוצה שכולה אדומה או כחולה. סימן שמצאנו פיצ'ר חזק שמפריד את הדטה בצורה הכי עילה.

6.10 אלגוריתם ה- $ID3$ TDIDT

נתבונן בפסאודו הבא:

BuildTree(Date, Features)

א. יצירת שורש: יוצרים צומת Root.

ב. אם כל הדגימות בדטה הן Yes (כולן חיוביות) תחזיר את השורש עם התווית Yes.

ג. אם כל הדגימות בדטה הן No (כולן שליליות) תחזיר את השורש עם התווית No.

ד. אם אין יותר פיצ'רים לבדיקה אך הדטה עדיין מעורבב - החזר את השורש עם התווית הנפוצה ביותר בדטה.

ה. אחרת:

1. בחירת הפיצ'ר הטוב ביותר: בוחרים את הפיצ'ר A שמפצל את הדטה בצורה הכי טהורה. (בהמשך נדון כיצד).

2. יצירת ענפים: לכל ערך אפשרי v של A (למשל: sunny, cloudy, rainy וכו')

• יוצרים ענף חדש מתחת לשורש.

• מגדירים תת קבוצה של דטה $Data_v$ שכוללת רק את הדגימות שבהן הפיצ'ר $A = v$.

• אם תת הקבוצה ריקה: מוסיפים עלה עם התווית הנפוצה ביותר בדטה המקורי.

• אחרת: קוראים ברקורסיה לאלגוריתם שוב עבור תת הקבוצה והפיצ'רים שנותרו (ללא A). כלומר BuildTree(Data_v, Features \ {A}).

ו. בסוף הרקורסיה: העץ מוכן.

נבחין כי בסוף אכן האלגוריתם תמיד יעצור כי תמיד קבוצת הפיצ'רים קטנה באחד בכל שלב. האלגוריתם בוחר מקומית את הפיצ'ר שמפצל את הדטה הכי טוב כרגע - ולכן חמדני.

הערה 108. להלן פסודו פורמלי יותר:

Algorithm 5 Decision Tree Construction via BuildTree(Data, Features)**Require:** Training data $Data$, feature set $Features$

```

Create a root node  $Root$ 
if all examples in  $Data$  are positive (Yes) then
    return  $Root$  with label Yes
end if
if all examples in  $Data$  are negative (No) then
    return  $Root$  with label No
end if
if  $Features$  is empty then
    return  $Root$  with the most common label in  $Data$ 
end if
Choose the best feature  $A$  that splits  $Data$  most purely
for all possible values  $v$  of  $A$  (e.g., sunny, cloudy, rainy) do
    Create a new branch below  $Root$  for  $A = v$ 
    Let  $Data_v \subseteq Data$  be the subset of examples where  $A = v$ 
    if  $Data_v$  is empty then
        Add a leaf with the most common label in  $Data$ 
    else
        Recursively call BuildTree( $Data_v, Features \setminus \{A\}$ )
    end if
end for
return  $Root$ 

```

6.11 בחירת הפיצ'ר הכי טוב - Entropy

גם באלגוריתם שראינו, סעיף 11 התבסס על "בחירת הפיצ'ר הטוב ביותר". כיצד קובעים מיהו הפיצ'ר הטוב ביותר? לשם כך נחזור קצת למתמטיקה.

אנתרופיה היא הדרך שלנו למדוד כמה חוסר ודאות (או חוסר טוהר כמו אצלנו) יש באוסף דגימות.

- אנתרופיה נמוכה: מצב של ודאות גבוהה מאוד. הסט טהור.
- אנתרופיה גבוהה: מצב של חוסר ודאות מקסימלי. הסט תערובת מבולגנת. למשל - 50% כן ו-50% לא.

עבור קבוצה S המכילה דוגמאות חיוביות ושליליות האנתרופיה מוגדרת באופן הבא:

- סיווג בינארי (זוג מחלקות):

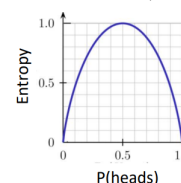
$$H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

- סיווג רב מחלקתי:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

באשר p_i היא ההסתברות לדגימה ממחלקה i מכלל הדגימות ב S (פורפורציה).

לדוגמה: גרף האנתרופיה עבור סיווג בינארי. ערכי p_+ (נניח $head$ נחשב לחיובי) שנעים מ-0 עד 1. האנתרופיה מקסימלית ב $p_+ = \frac{1}{2}$.



6.12 Information Gain - כלל הבחירה

כעת נבין איך ה-ID3 מחליט בה-1 באלגוריתם איזה פיצ'ר הוא הטוב ביותר. Information Gain (IG) הוא הירידה באנתרופיה שנגמרת כתוצאה מחלוקת הדגימות לפי תכונה מסוימת A . כלומר: הוא מודד כמה וודאות הרווחנו מהפיצול. פורמלית -

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

כלומר, מחסירים מהאנתרופיה המקורית את האנתרופיה (משוקללת) של כל אחת מהקבוצות שנוצרו בעקבות הפיצול של תכונה A . המטרה בבחירת הפיצ'ר A בכל שלב הוא מקסום $IG(S, A)$. מדוע מקסום? נשאף כי $\sum$ יהיה קטן ביותר - כך נבטיח כי חוסר הוודאות לאחר הפיצול יהיה קטן ככל הניתן.

לכן: בתחילה האלגוריתם יבצע את התהליך על כל הפיצ'רים הקיימים. הוא יבחר בשורש העץ להיות זה שמקסם את $IG(S, A)$. כלומר, בתכונה A שמקיימת:

$$A = \arg \max_{A^* \in \text{Features}} IG(S, A^*)$$

בהמשך, בשלב ה-1 יעשה את אותו התהליך. פועלים בגישה זו עם אנתרופיה על מנת שבכל שלב הוודאות תהיה מקסימלית והעץ לא יהיה גדול מדי.

6.13 CART Algorithm

בניגוד ל-ID3 שיכול להתמודד רק עם משתנים קטגוריאליים, CART יודע לעבור גם עם משתנים קטגוריאליים וגם עם משתנים נומריים. ב-CART מדובר בעץ בינארי - כל צומת פנימי מתפצל ל-2 ילדים בלבד. (כן או לא, או גדול קטן מערך מסויים).

הגדרה 109. (מדד ג'יני) נגדיר את מדד ג'יני באופן הבא: $Gini = 1 - \sum_{i=1}^c p_i^2$

- אם כל הדגימות מאותה מחלקה, $Gini = 0$
- הערך המקסימלי של $Gini$ הוא $\frac{1}{2}$.
- $Gini$ מהיר יותר לחישוב כי לא דורש פונקציה לוגריתמית.
- אם $Gini$ גבוה, מדובר בערבוב של מחלקות שונות ולכן זה מעיד על טוהר נמוך.

6.14 סיכום

כעת נרצה לסכם ולהבדיל בין השלב שבו המודל לומד בעץ החלטה לבין השלב שבו הוא חוזר עבור דוגמאות חדשות.

- בניית המודל: האלגוריתם: משתמשים ב-ID3 על מנת לבנות את העץ על סמך הדטה סט הקיים. בכל איטרציה של התהליך, בוחרים את הפיצ'ר שיפחית את האנתרופיה במידה המקסימלית. מפצלים עד להגעה לעלים.
- שימוש במודל: בהינתן שמגיעה דוגמה חדשה ולא מוכרת.
- 1. המודל מקבל סט תכונות של דוגמה חדשה
- 2. המודל מנווט בעץ עבור כל צומת מהשורש עד להגעה לעלה ומחזירים את התוית שמופיעה בו.

נשים לב שבעוד הבניה עלותה יחסית גבוהה, השימוש במודל מאוד פשוט ועלותו כגובה העץ שבנינו. נרצה לשים לב להבדל שקשור במה שדנו בו היום: בלמידה סטטיסטית ישנם שני סוגים: למידה עם פרמטרים, וללא פרמטרים. בלמידה עם פרמטרים מבצעים תהליך איטרטיבי (למשל GD) על מנת לעדכן את הפרמטרים. בלמידה ללא פרמטרים, המבנה של המודל עצמו נקבע מהדטה ישירות - כמו כאן.

ישנו חסרון גדול למודל שעלינו לקחת בחשבון:

- האלגוריתם הוא מנוע רקורסיבי עוצמתי. אם לא נשים לו גבולות, הוא ימשיך לפצל את המידע שוב ושוב. העץ מסוגל להגיע למצב שבו כל עלה טהור לחלוטין. במצב זה הטעות על נתוני האימון היא אפס. עץ החלטה הרי יכול לייצג כל שילוב תכונות שקיים - ולכן הוא יכול לבנות מסלול לכל שילוב שכזה. הבעיה ברורה - Overfitting: כשהעץ נהיה עמוק מדי הוא הופך להיות מודל שמאבד את היכולת לבצע הכללה על נתונים חדשים, והופך להיות מודל עם שונות גבוהה: כל שינוי קטן בנתוני האימון ישנה את כל מבנה העץ.

אז מה הפתרון? **גזיז - Pruning**:

נעדיף תמיד עצים קטנים: עדיף את ההסבר הפשוט ביותר שניתן למצוא לנתונים.

בעולם ML , מדובר במעין רגולציה. הגיוס מצמצם את מרחב ההיפותזות שהעץ יכול להגיע אליו וכך מונע ממנו לייצג פונקציות מורכבות מדי שהן כנראה רק רעש. ישנן שתי גישות גיוס:

- **Pre-pruning:** גיוס מראש. עוצרים מראש את צמיחת העץ בשלב האימון לפני שהוא נהיה מורכב מדי. כיצד? קובעים חוקי עצירה. למשל: הגבלת הגובה המקסימלי של העץ, קביעת מס' מינימלי של דגימות שחייבות להיות בצומת על מנת ליצור פיצול, עצירה אם $IG(S, A)$ של פיצול מסוים נמוך מסף שקבענו.

כיצד נקבע את הערך בעלה? בעת שגזמנו ענף והפכנו אותו לעלה לפני שהפך לטהור - העלה יכול תערובת של Yes/No. התווית תקבע לפי רוב הדגימות בענף שגזמנו.

הגישה הזו מאוד יעילה חישובית - חוסכים זמן יקר כי לא בונים חלקים מהעץ מראש.

- **Post-pruning:** נותנים לעץ לגדול עד להגעה לשיא הגודל שלו. ואז מתחילים לחתוך ענפים שנראים מיותרים. כיצד עושים זאת? בודקים ענפים ספציפיים: אם הסרת ענף והפיכתו לעלה לא פוגעת (ואולי אף משפרת) את הדיוק של המודל על סט הולידציה (נתונים שהמודל לא ראה באימון) נגזום אותו.

שיטה זו נחשבת לטובה יותר - הגישה הקודמת עלולה לעצור מוקדם מדי בגלל פיצול שנראה לא מבטיח אך כאן רואים את התמונה המלאה. גוזמים מה שהוכחנו כמיותר. אך כמובן, שהיא דורשת יותר משאבי חישוב.

6.15 Boosting

אם כן, לעצי החלטה ישנם חסרונות רבים ובעיקר מספר מגבלות - שינויים קטנים בנתוני האימון יכולים להוביל לשינוי קיצוני בנקודות הפיצול - מה שגורם לשונות גבוהה בעצים עמוקים. כמו כן, עצים מלאים נוטים ל-Overfitting ואם מגבילים את גובה העץ נקבל Bias גדול.

Weak learner: מודל שהוא Weak learner הוא כל מודל שמציג ביצועים טובים יותר מניחוש אקראי. הוא לא צריך להיות מדויק מאוד אלא רק לתפוס אות (חוקיות) מסויים בנתונים. למשל: נניח סיווג בינארי, כאשר התגיות הן כחול ואדום. ניחוש אקראי הוא דיוק של 50%. לומד חלש - דיוק שגובה מ-50%. לומד חזק יותר: דיוק גבוה יותר, למשל 80% וכן לומד חזק מאוד הוא דיוק של 95% וכו'. אם כי הוא עלול להתאים לרעש ויש סיכון של Overfitting.



Boosting היא השיטה שמשלבת לומדים חלשים רבים על מנת ליצור לומד חזק. במקום לנסות לבנות ישירות מודל אחד מורכב מאוד שעלול לסבול Overfitting גישה Boosting אומרת - נקח הרבה לומדים פשוטים וחלשים ונשלב אותם על מנת ליצור לומד חזק אחד. כלומר:

$$\text{Strong Model} = \sum_{m=1}^M \text{weight}_m \times \text{weak learner}_m$$

באשר weak learner_m זו פונקציה שמקבלת דגימה ומחזירה ניבוי ועוד Weight_m הוא ערך סקלרי קבוע שמוצמד לכל לומד. המס' הזה מייצג את רמת האמינות והחוזק של אותו לומד.

ישנם מודלים פשוטים רבים שיכולים לשמש כ-Weak learners:

1. **מסווגים חלשים** - מודלים פשוטים שמשמים למשימות סיווג. למשל: מודלים לינאריים (מהירים אך לא מסוגלים לקלוט קשרים על לינאריים), עצי החלטה בעלי מגבלת עומק קטנה (שלא מפצלים יותר מדי את הנתונים).
2. **Weak regressors:** מודלים פשוטים שמשמים למשימות רגרסיה, בהן הפלט הוא ערך רציף. למשל: רגרסיה לינארית, מודל ה"ממוצע" - תמיד מחזיר את הערך הממוצע של משתנה המטרה מתוך נתוני האימון.

ישנה פילוסופיה שלמה שעומדת מאחורי עקרון Boosting:

1. **Ensemble learning:** עקרון יסודי זה עומד מאחורי הקונספט הבא - שילוב מס' מודלים עצמאיים, מודלי הבסיס (או מודלים חלשים) לתוך מודל אחד מקיף יחיד. במקום להסתמך על מודל יחיד - בונים קבוצת מודלים שעובדים יחד.
2. למידה סדרתית: בשיטות אנסמל אחרות מאמנים מודלים במקביל. כאן הלמידה היא סדרתית. חברים חדשים באנסמל מתווספים אחד אחד. לכל לומר חדש שנוסף יש משימה ספציפית - לתקן את הטעויות של האנסמל שנוצר לפניו.
3. חכמת ההמונים: במקום לנסות לייצר מודל יחיד שיהיה מומחה גדול - נשלב מאות לומדים חלשים, שיחד יצרו מודל חזק.
4. הפחתת Bias: מודל זה מתמקד בעיקר בהפחתת ההטיה של המערכת. הוא עושה זאת בצורה איטרטיבית ומניע את המודל לכיוון ההתפלגות האמיתית ע"י כך שהוא מתאים את עצמו ומנסה ללמוד דגימות שקשה לחזות אותן - וכך הוא מפחית את Bias.

6.15.1 אלגוריתם XGBoost

נלמד על אלגוריתם בוסטינג שמבוסס גרדיאנט. האלגוריתם הנ"ל נחשב כיום לזה עם הביצועים הטובים ביותר עבור עבודה עם נתונים טבלאיים, בסיסי נתונים וכו'. כעת נגדיר פורמלית את הבעיה:

בהינתן קלט של דגימות אימון מתויגות $\{(x_i, y_i)\}_{i=1}^n$ כך ש $x_i \in \mathbb{R}^d$ והדגימות $i.i.d$ מהתפלגות נתונים $\mathcal{D}$, וכן $y_i \in \mathbb{R}$ נגדיר את פונקציית ההפסד הנקודתית $\ell(\text{true}, \text{predicted})$ המודדת את המרחק (או השגיאה) בין הערך האמיתי למנובא. ההפסד הכולל הינו:

$$\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i))$$

המטרה של ERM הוא למצוא פונקציה f המקיימת:

$$\arg \min_f \mathcal{L}(f)$$

מטרתנו תהיה למצוא מודל בעל דיוק גבוה מאוד ע"י שילוב הניבויים של הרבה מודלים פשוטים. לכן המודל הסופי שישומו ב $f(x)$ יוגדר:

$$f(x) = \sum_{m=1}^M \gamma_m h_m(x)$$

כל M הוא מס' המודלים השונים באנסמבל שלנו. לרוב: אלו יהיו עצי החלטה פשוטים. $h_m(x)$ הוא המודל החלש. וכן $\gamma_m \in \mathbb{R}$ הוא המשקל הסקלרי של אותו מודל.

במקום לנסות לחזות את ערך המטרה הסופי מאפס בכל שלב, כל עץ חדש שנוסף (כל מודל שנוסף) מנסה לחזות אך ורק את הטעויות (השאריות - Residuals) שביצע האנסמבל הנוכחי. לשם כך נתבונן בדוגמה:

- נניח כי המחירים האמיתיים של שלושה מוצרים הם: $y = [100, 120, 150]$
- נניח כי המודל הקיים הנוכחי ניבא: $f_{old} = [90, 130, 140]$
- שלב ראשון: מחשבים את הטעויות - השאריות. בודקים בכמה המודל הנוכחי פספס בכל דגימה $(y - f_{old})$ ומקבלים: $[+10, -10, +10]$.
- שלב שני: מאמנים את העץ הבא, המודל h_{next} : המטרה של העץ הנ"ל אינה לחזות את המחירים המקוריים y אלא לחזות את הטעויות שחישבונו כעת. כלומר ערכי y החדשים שעל פיהם העץ הנ"ל מתאמן הם קבוצת השאריות $[10, -10, 10]$.
- שלב שלישי: לאחר שהמודל h_{next} התאמן, נגדיר $f_{new} = f_{old} + h_{next}$. אם העץ החדש עשה עבודה טובה, הוא ניבא ערך שקרוב ל-10 למשל עבור המוצר הראשון. כאשר נחברו עם התחזית של 90 יחד יתקברו ל-100. המודל הנ"ל עובד די טוב כי הוא מתמקד רק בטעויות של המודל הקודם - וילמד רק אותן.

קצת נציג את הפסאודו לאלגוריתם Gradient Boosting:

- אתחול: קבע לכל דגימה i את $\hat{y}_i^{(0)} = 0$ ולכן $f_1(x) = 0$
- לכל $1 \leq m \leq M$:
- חשב את השארית עבור הדגימה: $r_i = \hat{y}_i^{(m-1)} - y_i$ (הניבוי עבור הערך הקודם, פחות הערך האמיתי)
- אמן עץ חדש h_m על פני מערך נתוני האימון החדש: $\{(x_i, r_i)\}_{i=1}^n$
- עדכן את הפונקציה בתוספת קבוע α קטן: $f_m(x) = f_{m-1}(x) + \alpha h_m$ (משמש כקצב למידה, ומחליף את γ_m שקודם ראינו).
- חשב ניבויים מעודכנים לכל הדגימות באמצעות הפונקציה החדשה - $\hat{y}_i^{(m)} = f_m(x_i)$
- לאחר סיום ריצת הלולאה, מתקיים לבסוף $f_M(x) = \sum_{m=1}^M \alpha h_m(x)$.

לרוב בלמידת מכונה, אנחנו ממזערים את פונקציית ההפסד ע"י כוונון קבוצה קבועה של פרמטרים (מעדכנים וקטור משקולות וכו') או משקלים של המודל. עם זאת - Boosting אנחנו מבצעים ירידת גרדיאנט ע"י שינוי ישיר של הפונקציה עצמה. לכן, נרצה להשוות בין GD הרגילה, לבין GD במרחב הפונקציות:

- מה אנחנו משנים? ב GD אנחנו משנים את הפרמטרים או משקלי המודל, θ . ב GD במרחב הפונקציות אנחנו משנים את פונקציית הניבוי עצמה f .
- מה המרחב שבו אנחנו פועלים? ב GD אנחנו פועלים במרחב הפרמטרים $\mathbb{R}^{|\theta|}$ בעוד ב GD במרחב הפונקציות אנחנו פועלים במרחב פונקציות אינסופי.
- הצעד שמבצעים: ב GD מזיזים את הפרמטרים θ לאורך ועם כיוון וקטור הגרדיאנט בעוד אצלנו - מוסיפים פונקציה (עץ) חדשה שמצביעה ומכוונת אל עבר כיוון הגרדיאנט.

כאשר אנחנו מעדכנים פונקציה באופן ישיר בצעדי אופטימיזציה, חוק העדכון האיטרטיבי יראה לרוב מהצורה הבאה עם קצב הלמידה α :

$$f_{t+1} = f_t - \alpha \frac{\partial \mathcal{L}(f)}{\partial f}$$

על מנת לחשב את הגרדיאנט הזה, נרצה להעריך את f בנקודות האימון x_i ולכן נגזור את ההפסד ביחס לכל ניבוי $f(x_i)$. כלומר:

$$\frac{\partial \mathcal{L}(f)}{\partial f} \sim \nabla_f L = \left[\frac{\partial \mathcal{L}}{\partial f(x_1)}, \dots, \frac{\partial \mathcal{L}}{\partial f(x_n)} \right]$$

$$\frac{\partial \mathcal{L}}{\partial f(x_k)} = \frac{\partial \left[\frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)) \right]}{\partial f(x_k)} = \frac{1}{n} \cdot \frac{\partial \ell(y_k, f(x_k))}{\partial f(x_k)}$$

עבור הרכיב k נקבל כי MSE הפסד פונקציית הפסד $\ell(y_i, f(x_i)) = \frac{1}{2}(y_i - f(x_i))^2$ נקבל כי:

$$\frac{\partial \ell(y_k, f(x_k))}{\partial f(x_k)} = \frac{1}{2} \cdot 2 \cdot (y_i - f(x_i)) \cdot (-1) = f(x_i) - y_i$$

ולכן:

$$f_{t+1} = f_t - \alpha \frac{\partial \mathcal{L}(f)}{\partial f} = f_t - \alpha [-(y_i - f(x_i))] = f_t + \alpha(y_i - f(x_i))$$

מכאן אם נסמן $r_i^{(t)} = y_i - f_t(x_i)$ כשארית נקבל כי: $f_{t+1} = f_t + \alpha r_i^{(t)}$ (נשים לב כי ה- $\frac{1}{n}$ נעלם לנו ונכנס כחלק מהקבוע α)

הערה 110. למעשה החלק לעיל נועד להראות כי הגרדיאנט **לפי הפונקציה ולא לפי וקטור המשקולות** הוא ההצדקה לאלגוריתם, אנחנו מבצעים דבר דומה ל-GD רק שגוזרים לפי פונקציה ולא לפי וקטור משקולות.

נבחין כי מתמטית היינו רוצים פשוט להוסיף לכל נקודת אימון את ערך הגרדיאנט המדויק שלה (r_i) אך אם נעשה את זה המודל ישנן את נקודות האימון ולא ידע להכליל לנקודות חדשות. כלומר הנוסחה שקיבלנו בשלב הקודם בעייתית לנו - נכונה מתמטית אך פרקטית תגרום ל-overfitting. וגם: אם מגיעה נקודה חדשה איננו יודעים את השארית שלה! לכן אמנם נוסחה זו תאורטית עובדת - במציאות לא. היא לא עובדת על נקודות חדשות.

לכן, נבנה עץ החלטה חדש בכל שלב שישמש כפרוקסי לוקטור הגרדיאנט ע"י פתרון MSE :

$$h_m = \arg \min_h \sum_{i=1}^n (r_i - h(x_i))^2$$

כלומר: בונים עץ החלטה רציף ומבני h ומאמנים אותו לפתור משימה חדשה - פלט העץ עבור הנקודות x_i יהיה קרוב ככל הניתן ל- r_i . כלומר הפונקציה הנ"ל היא זו שתחזה את השארית! מכאן המודל הוא $f_{t+1}(x) = f_t(x) + \alpha h_m(x)$.

סיכום דבר:

- המודל הזה הוא המודל הטוב ביותר עבור נתונים טבלאיים, ולפעמים אף מנצח בביצועים אפילו רשתות ניורונים עמוקות.
 - כשמריצים את האלגוריתם צריך לקבוע חמישה פרמטרים -
1. קצב הלמידה α , נרצה קצב קטן על מנת להפוך את המודל לאמיד יותר בפני Overfitting (פחות נשנן נתונים) אך זה ידרוש יותר עצים באנסמבל.
 2. מס' האיטרציות M - נקבע זאת באמצעות עצירה מוקדמת. נעקוב אחרי השגיאה על נתוני הוולידציה וברגע שהשגיאה שם מפסיקה לרדת נעצור על מנת שלא נשנן overfitting.
 3. עומק העץ - לרוב בין 3 ל-8 לכל עץ. עומק גבוה מדי עלול לגרום למודל לשנן רעש בדטה.
 4. סוגי הנומרים החלשים: לרוב עצי החלטה הם הסוג הנפוץ והסטנדרטי ביותר.
 5. פונקציית הפסד. לפעמים נשתמש ב- MSE אך היא רגישה מאוד לנקודות קיצון וחריגים. לכן לפעמים נשתמש בפונקציות אחרות.
- AdaBoost: לפני ה- GB אלגוריתם AdaBoost הצג לעולם את הקונספט של למידת אננסמבל סדרתית. הרעיון שלו מבוסס על העקרונות הבאים - במקום לחשב שאריות מתמטיות של רגרסיה הוא פועל בבעיות סיווג באמצעות המנגנונים הבאים:
1. בכל איטרציה - האלגוריתם בוחר אילו דגימות המודל הנוכחי פספס. נקודות שסווגו לא נכון, מקבלות משקל גבוה יותר באיטרציה הבאה. זה מכריח את הלומד החדש להתמקד בהן.
 2. האלגוריתם משתמש בעצי החלטה בעלי עומק אחד בלבד - עץ שמבצע פיצול יחיד על תכונה בודדת ומקבלת החלטה.
 3. בניגוד ל- GB בו לכל העצים משקל קבוע α הוא נותן לכל לומד משקל משלו α_m . המשקל הנ"ל נקבע ישירות על פי מידת הדיוק הספציפית של אותו לומד.
- לכן סה"כ כיוון שזו משימת סיווג, בסוף מתקיים כי פונקציית הניבוי של המודל הסופי היא $H(x) = \text{sign} \left(\sum_{i=1}^M \alpha_m h_m(x) \right) \in \{+1, -1\}$.

Naïve Bayes 6.16

בהינתן וקטור תכונות קלטים $x = (x_1, \dots, x_d)$ וקבוצת מחלקות אפשריות $C = \{c_1, \dots, c_k\}$. נרצה לקבוע לאיזו מחלקה הכי סביר שהוקטור שייך, כלומר: $c^* = \arg \max_{i=1, \dots, k} p(c_i|x)$. לפי חוק בייס והעובדה שבמכנה נקבל $p(x)$ נקבל שזה שקול ל:

$$c^* = \arg \max_{i=1, \dots, k} p(x|c_i)p(c_i)$$

הבעיה שנוצרת היא שבכדי לאמוד את Likelihood המשותף אנו זקוקים לכל שילוב אפשרי של ערכי התכונות. אם יש d תכונות ולכל אחת m ערכים מס' הפרמטרים שנדרש לאמוד הוא m^d שגדל אקספוננציאלית. לכן נניח אי תלות מותנית של התכונות בהינתן המחלקה. מכאן נקבל כי:

$$Pr(x|c_i) = p(x_1, \dots, x_d|c_i) = \prod_{j=1}^d p(x_j|c_i)$$

לכן סה"כ הנוסחה הסופית לחיזוי הינה:

$$c^* = \arg \max_{i=1, \dots, k} p(c_i) \prod_{j=1}^d p(x_j|c_i)$$

- עם זאת, זה לא תמיד עובד. המודל נקרא נאיבי כי ההנחה שהתכונות ב"ת היא לא מציאותית - למשל עבור ניתוח מילים בטקסט המילים חנים ומבצע קשורות ותלויות זו בזו. לכן כאשר יש תלות חזקה ומבנית בין הרכיבים המודל נכשל. אך הוא בכל זאת פופולרי כי בפועל הוא מצליח לתפוס מספיק תבניות מועילות, הוא פשוט למימוש ומהיר.
- כיוון שכעת ישנה מכפלה, נראה כי אם ערך מסויים של תכונה x_j לא הופיע מעולם בנתוני האימון תחת מחלקה ספציפית c_i האומדן שלו $Pr(x_j|c_i) = 0$ וכיוון שהמודל מבוסס על מכפלה איבר אפסי בודד יאפס את החישוב כולו. לכן, מוסיפים קבוע חיובי קטן α (לרוב 1) כך:

$$Pr(x_j|c_i) = \frac{\text{count}(x_j, c_i) + \alpha}{\text{count}(c_i) + \alpha|V|}$$

- בשלב האימון של המודל אנחנו מחשבים את ההסתברות המותנית של כל פרמטר בהינתן מחלקה מסוימת. כלומר $P(x_j|x_i)$. וכן אומדים את הסתברויות $Priors: Pr(c_i)$.
- בשלב המבחן אנחנו מעריכים בהינתן ערך חדש עם פרמטרים $x = (x_1, \dots, x_d)$ את המחלקה לה הוא שייך, c^* . בוחרים זו עם ההסתברות המקסימלית.

(K nearest neighbors) KNN 6.17

- אלגוריתם זה הוא אלגוריתם סיווג שמבוסס על השכנים הקרובים ביותר. הנחת היסוד היא כי הדגימות אשר בעלות מאפיינים דומים נוטות להשתייך לאותה מחלקה. באלגוריתם נבחר לפי כמה ערכי k (שכנים) נרצה להתבסס. נציג את הפסודו הבא:
- הגדר מטריקה d (פונקציית מרחק)
 - הגדר את k (מס' השכנים לבדיקה)
 - בהינתן דוגמת מבחן x : מצא את k הדוגמאות הקרובות ביותר ל- x לפי המטריקה d .
 - חזה לפי מחלקת הרוב (חזה את המחלקה של x לפי המחלקה שרוב השכנים של x בה).

Algorithm 6 KNN (K Nearest Neighbors)

Require: metric d (distance function), number of neighbors k , test sample x
 Find the k training examples closest to x according to the metric d
return the majority class among the k nearest neighbors of x

הגדרה 111. מטריקה היא פונקציית מרחק d שצריכה לקיים את התנאים הבאים:

- $\forall x, y : d(x, y) \geq 0$
- $\forall x : d(x, x) = 0$
- $\forall x, y : d(x, y) = d(y, x)$
- אי שוויון המשולש: $\forall x, y, z : d(x, y) + d(y, z) \geq d(x, z)$

כיצד נבחר את המטריקה? זה כמובן תלוי בבעיה.
 א. במרחב $\mathbb{R}^n$ המרחק האוקלידי: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
 ב. מדד המודד את זווית הכיוון בין זוג הוקטורים, ללא תלות בגודלם: $s(x, y) = \frac{x \cdot y}{\|x\| \|y\|} = \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum_{i=1}^d x_i^2} \cdot \sqrt{\sum_{i=1}^d y_i^2}}$
 ג. כיוון שאלגוריתם KNN דורש פונקציית מרחק (ערך נמוך מייצג קרבה) ודמיון קוסינוס ($s(x, y)$) מחזיר ערך הפוך (ערך גבוה מייצג קרבה) ניתן להפוך את המרחק כך: $d(x, y) = \frac{1}{s(x, y)}$ או $d(x, y) = \frac{1-s(x, y)}{2}$.

זמן ריצת האלגוריתם: על מנת למצוא את הנקודה הקרובה ביותר בכל פעם יש לעבור על פני n הדגימות וכל מרחק עלותו d תכונות ולכן $O(nd)$. כיוון שישנן k נקודות נקבל $O(nkd)$.
סיבוכיות מקום: המודל נדרש לשמור בזיכרון את n הדגימות מסט האימון על פני d תכונותיהם ולכן $O(nd)$.

סיכום דבר:

- מדובר באלגוריתם למידה מונחית - הוא משתמש בנתונים מתוגים לביצוע משימה.
- יש לקבוע את K מראש - זה היפר פרמטר שלא משתנה.
- ישנה רגישות למטריקה - מטריקות שונות יניבו תוצאות שונות.
- הוספת דוגמאות חדשות לסט הנתונים משנה באופן ישיר את המסווג (אינו אלגוריתם למידה קלאסי!).
- לאלגוריתם אין שלב אימון מפורש.
- האלגוריתם מקבל ערך הפסד של אפס כאשר $K = 1$.
- האלגוריתם רגיש לרעש וערכים חריגים, אם K קטן נראה זאת יותר.
- שלב החיזוי עולה יחסית הרבה זמן ומקום.
- האלגוריתם סובל מקללת המימד - הביצוע דל במימד גבוה.
- כאשר k נמוך השונות גבוהה. כאשר k גבוה ההטיה גבוהה.

7 הרצאה 7 - Deep Learning

7.1 Multi Layer Perceptrons

נתבונן בבעיה פשוטה - שמדגימה את המגבלות של הפרספטרון. בעיית הזוגיות מוגדרת באופן הבא: בהינתן קבוצה של n ביטים, נרצה לקבוע אם מס' האחדות בקבוצה הוא זוגי או אי זוגי. אלגוריתם פשוט שפותר את זה עובר בלולאה על הערכים וסוכם כמה ערכי 1 יש. זמן הריצה של אלגוריתם שכזה הוא $O(n)$, וסיבוכיות המקום $O(1)$. אם כן, כאשר $n = 2$ הפרספטרון לא מסוגל ללמוד את בעיית הזוגיות, שכן כאשר $n = 2$ הבעיה היא בעיית XOR המפורסמת (הפלט הוא 1 כאשר אחד מהביטים הוא 1 ואפס אחרת). לכן - פרספטרון בודד שהוא הרי מסווג לינארי נכשל בבעיה שכזו פשוטה. אך כפי שראינו אם מגדירים פונקציית $\phi(x_1, x_2) = (x_1 x_2, x_1 + x_2)$ כעת נתוני ה XOR כן ניתנים להפרדה לינארית ע"י הפרספטרון. אך במקום לבצע מניפולציה ידנית על הקלטים, ניתן להוסיף רשת של שכבות עם ניורונים שיבצעו בעצמם פעולות לא לינאריות. נגדיר:

הגדרה 112. ניורון מלאכותי הוא יחידת חישוב מתמטית פשוטה. הוא מקבל מס' קלטים, מבצע עליהם פעולה מתמטית ומחזיר פלט יחיד. הניורון מבצע חישוב שמורכב משלושת השלבים הבאים:
 1. שקלול הקלט (סכימה) - הניורון מקבל כקלט ערכים מספריים, כאשר לכל קלט מוצג משקל w . הניורון מכפיל כל קלט במשקל שלו ומחבר את המשקלות יחד. לתוצאה זו הוא מוסיף את ההטיה b .
 2. פונקציית אקטיבציה: את הסכום המשוקלל מכניס הניורון לתוך פונקצייה לא לינארית (למשל הסיגמואיד). תפקיד הפונקציה הוא להחליט האם ואיך הניורון יידלק או יירגע - לקבוע את עוצמת האות שיעביר הלאה.
 3. הפקת הפלט: הערך שיוצא מפונקציית האקטיבציה הוא הפלט הרשמי של הניורון - הפלט הזה מועבר כקלט לניורונים בשכבה הבאה ברשת.

הגדרה 113. רשתות שמורכבות משכבות של ניורונים כאשר המבנה הבסיסי שלהן בנוי בצורה הבאה - **שכבת קלט** (שכבה ראשונה אשר מקבלת את הנתונים), **שכבות נסתרות** (עמוקות ומחוברות לחלוטין, נקראות כך כיוון שכל ניורון בשכבה מסוימת מקושר לכל הניורונים בשכבה הבאה אחריה). **ושכבת פלט** - השכבה האחרונה שמוציאה את התוצאה הסופית של הרשת.

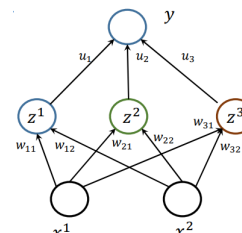
נשוב לדוגמת XOR : נציג כעת רשת ניורונים שמטרתה לפתור את פונקציית XOR - כל הניורונים בשכבות הנסתרות משתמשים בפונקציית sig כלומר: הניורון

$$z^i = \text{sign}(w^i x - b^i)$$

דהיינו, הפלט z^i של הניורון i נקבע לפי סימן המכפלה המשוקללת פחות ערך הסף b^i . שכבת הקלט תורכב משני רכיבים x_1 ו x_2 . המטרה היא שהרשת תדע לחזות ולממש את הטבלה הבאה:

y	x^1	x^2
0	0	0
1	0	0
1	1	0
0	1	1

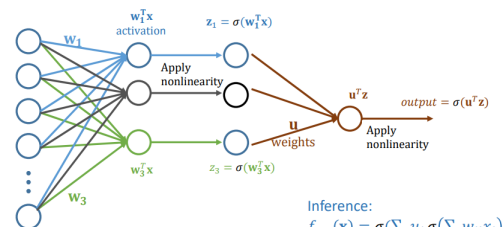
לכן הרשת תהיה מורכבת משכבת קלט - ערכי x_1 ו- x_2 : השכבה הנסתרת תהיה בודדת ותכיל שלושה נייורונים שונים - z_1, z_2, z_3 : הם מקבלים משקולות ייעודיות להם כאשר x_1 משקלל שלושה משקולות w_{11}, w_{12}, w_{13} ובדומה גם x_2 . בשכבת הפלט - נייורון פלט יחיד, y , שמקבל כקלט את הפיצ'רים החדשים שנוצרה בשכבה הנסתרת דרך המשקולות u_1, u_2, u_3 . נדגיש כי נייורון y לוקח את הפלטים של השכבה z ומכפיל אותם ב- w המתאים ומחבר את כל התוצאות יחד.



את הרעיון הזה, ניתן להכליל לבניית פונקציות מאוד מורכבות שיוכלו לפתור בעיות שפרפסטרון לא יכול לפתור. את הפונקציות המאוד מורכבות הנ"ל נבנה באמצעות שכבות של פרפסטרוניס. לפני כן, נחדד את ההבדל בין למידה והסקה:

- בהסקה - נתון לנו מודל מוכן שכבר עבר אימון ויש לו משקולות קבועות, f_w . נתון קלט חדש x שנרצה לבדוק. נרצה למצוא את הפלט המספרי y עבור הקלט הנתון כלומר לחשב $y = f_w(x)$. הפעולה היא חישובית פשוטה - החישוב זורם קדימה ברשת.
- בלמידה - נתונה קבוצת אימון שמורכבת מזוגות דוגמאות ותגיות, ונרצה למצוא את המודל עצמו - וקטור משקולות w שידע לנבא ערכים שקרובים ככל הניתן לתגיות האמיתיות. כלומר $f_w(x_i) \sim y_i$. לשם כך, יש לבצע מינימיזציה של פונקציית ההפסד מעל מרחב פונקציות.

אם נכליל את הרעיון, נקבל רשת נייורונים - מורכבת מכמה וכמה נייורונים שונים - כמו למשל כאן:



כל הנייורונים בשכבה השנייה מקבלים וקטור תכונות x , הנייורון הראשון מחזיק וקטור משקולות ששייך לו: w_1 והוא מפעיל את פונקציית הסיגמואיד על המכפלה הסקלרית. הנייורון השלישי - מחזיק וקטור משקולות משלו w_3 ומבצע פעולה דומה. כל הנייורונים מסתכלים על אותו קלט - אך משקללים את הפיצ'רים של הקלט באמצעות וקטור משקולות שונה. מן השכבה הראשונה - יוצא וקטור $z_i = \sigma(w_i^T x)$ וקטורים אלו הם עכשיו וקטורי התכונות, עבור השכבה הבאה - וכופלים אותם בוקטורי משקולות ייעודיים להם u : ערכים אלו מקבל הנייורון האחרון שמניב את התוצאה של המודל.

כיצד נקט מנבאים ברשת זו תוצאה? לאחר שכבר בנינו את הרשת -

$$f_{w,u}(x) = \sigma\left(\sum_j u_j \sigma\left(\sum_i w_{ji} x_i\right)\right)$$

כלומר קודם מבצעים את חישובי $\sigma(w_{ji} x_i)$, סוכמים תוצאות אלו ומשקללים בוקטורי המשקלים u ואת הפלט הנ"ל שולחים לפונקציית הסיגמואיד.

נעיר, כי יתכן ונבנה רשת עם כמה וכמה נייורוני פלט - בבעיות סיווג מחלקות כאשר $K > 2$. כלומר בבעיות סיווג בהן $y_i \in \{1, \dots, K\}$. במצב זה נשמור נייורון פלט לכל מחלקה.

מדוע שלא נשתמש בפונקציה לינארית? מדוע רצינו להשתמש בסיגמואיד? אם נוותר עליו ונשתמש בקשר לינארי בו שכבה אחת מיוצגת ע"י מטריצת משקולות W והשכבה הבאה ע"י וקטור משקולות u^T , הפונקציה עבור קלט x היא $f(x) = u^T W x = A x$ - כלומר מכפלה של מטריצות ווקטורים. כלומר, מה שיקרה במצב זה שהרכבה (הכפלה) של הרבה פונקציות לינאריות שקולה למכפלה אחת לינארית - ולמעשה איבדנו את כל היתרון של מערכת נייורונים - שהיא באמצעות המון פונקציות פותרת בעיות מורכבות.

מדוע הסיגמואיד ולא פונקציית הסימן? פונקציית $sign$ מקיימת שהנגזרת שלה בכל נקודה פרט לאפס שם אינה גזירה, היא בדיקה 0. על מנת לאמן רשתות נייורונים עמוקות משתמשים באלגוריתמים מבוססי GD - אם הנגזרת היא אפס, לא ניתן לעדכן את המשקולות. לכן נבחר פונקציות לא לינאריות וגזירות.

7.1.1 פונקציות אקטיבציה לא לינאריות וגזירות

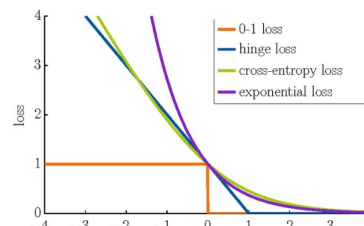
- ישנן כמה פונקציות שנוכל להשתמש בהן פרט לסיגמואיד $(\sigma(x) = \frac{1}{1+e^{-x}})$:
1. $\tanh(x)$: דומה בצורתה לסיגמואיד, והטווח שלה חסום בין -1 ל- 1 . היתרון שלה ביחס לסיגמואיד היא שהיא מקיימת $\tanh(0) = 0$.
 2. $ReLU: f(x) = \max(0, x)$: היא מחזירה 0 עבור קלטים שליליים ומשאירה את הקלט כפי שהוא עבור חיוביים. הנגזרת הקבועה שלה בחלק החיובי הוא 1 וזה עוזר למנוע את דעיכת הגרדיאנטים.
 3. $Leaky ReLU: f(x) = \max(0.1x, x)$: וריאציה של הפונקציה הקודמת שנועדה לפתור את בעיית הניורונים המתים - כאשר ניורון נתקע על קלט שלילי ומחזיר גרדיאנט אפס קבוע. כאן, עבור ערך שלילי יש שיפוע מתון שמאפשר לגרדיאנט להמשיך לזרום גם בתחום השלילי.
 4. $Maxout: f(x) = \max(w_1^T x + b_1, w_2^T x + b_2)$: הפונקציה מחשבת שני סכומים משוקללים לינאריים שונים ולוקחת את המקסימום ביניהם.
 5. ELU : הפונקציה מוגדרת באופן הבא:

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

עבור ערכים חיוביים מדובר ב- $ReLU$. עבור ערכים שליליים, משתמשים בפונקציית אקספוננט חלקה ששואפת לערך חסום $(-\alpha)$.

7.1.2 פונקציות הפסד

נרצה כמובן להעריך את ההפסד הכולל: $\sum_{i=1}^n loss_w(f(x_i), y_i)$. ישנן פונקציות הפסד רבות שלמדנו בקורס.



פרט לפונקציית $0-1$, כל הפונקציות הנ"ל גזירות. כיצד נבחר את הפרמטרים השונים? עלינו לקבוע היפר פרמטרים מראש - שאינם נלמדים מעצמם ע"י האלגוריתם אלא נקבעים מראש:

- כמה שכבות נסתרות? נרצה לקבוע את עומק הרשת.
- כמה ניורונים נשים בכל שכבה נסתרת? כאן נקבע את הרוחב של הרשת.
- באיזה פונקציית אקטיבציה נשתמש?
- באיזה פונקציית הפסד נבחר? נרצה לקבוע קריטריון שגיאה חלק וגזיר שנוצח למזער בהתאם לסוג המשימה.
- קביעת האופן שבו נעדכן את משקולות הרשת במהלך ריצת האלגוריתם - גודל הצעד שנלקח בכל עדכון בכיוון הפוך לגרדיאנט η , מנגנון שיאיץ את הלמידה ויעזור להתגבר על אזורי פלאטו ומינימום מקומי, רגולציה שתוסיף עונש על גודל המשקולות לפונקציית ההפסד על מנת למנוע overfitting.

7.1.3 פעפוע לאחור של הגרדיאנט (Backpropagation)

כיצד נלמד את הפרמטרים? נרצה לדעת כיצד שינוי מזערי בכל משקולת w בשכבות הפנימיות ישפיע על ההפסד הסופי - כדי שנוכל לעדכן אותה בכיוון שמקטין את השגיאה. לפי כלל השרשרת הרי שמתקיים:

$$\frac{d}{dw} loss_w(f(x_i), y_i) = \frac{\partial loss}{\partial f_w} \cdot \frac{df_w}{d\sigma(w^T x)} \cdot \frac{d\sigma(w^T x)}{dw}$$

- כלומר השינוי בהפסד כתוצאה מהשינוי במשקולות השכבה הראשונה מורכב ממכפלה של שלושה שלבים עוקבים - הזורמים מהסוף להתחלה:
1. $\frac{\partial loss}{\partial f_w}$: רכיב הפסד (פלט): הנגזרת של פונקציית ההפסד לפי פלט הרשת בפועל. רכיב זה מודד כמה השגיאה רגישה לשינוי בתחזית הסופית.
 2. $\frac{df_w}{d\sigma(w^T x)}$: רכיב פלט - שכבה נסתרת: הנגזרת של פונקציית הפלט לפי ערכי האקטיבציה של השכבה הנסתרת z . נקבעת ישירות ע"י משקולות שכבת הפלט u .
 3. $\frac{d\sigma(w^T x)}{dw}$: רכיב אקטיבציה, משקולות: הנגזרת של פונקציית האקטיבציה הלא לינארית לפי המשקולות הפנימיות שלה. תלוי ישירות בערך הוקטור x .

לבסוף הלמידה הולכת כך:

- מזינים קלט x לרשת, מחשבים את הפלט $f(x)$ דרך השכבות.

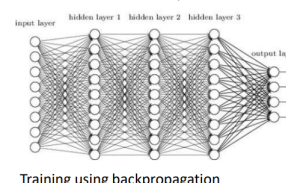
- מחשבים את ההפסד - משווים את הפלט לתגית האמיתית y ומחשבים את ערך פונקציית ההפסד.
- משתמשים בכלל שהרשרת כדי ללכת מהסוף להתחלה ולחשב עבור כל משקולת w_{ij} את הנגזרת החלקית שלה ביחס להפסד.
- מעדכנים את כל המשקולות ברשת בכיוון הפוך לסימן הגרדיאנט: $w_{ij} = w_{ij} - \eta \cdot \frac{\partial Loss}{\partial w_{ij}}$

7.2 Convolution Neural networks

MLPs: רשתות שמורכבות משכבות של ניורונים כאשר המבנה הבסיסי שלהן בנוי בצורה הבאה - **שכבת קלט** (שכבה ראשונה אשר מקבלת את הנתונים), **שכבות נסתרות** (עמוקות ומחוברות לחלוטין, נקראות כך כיוון שכל ניורון בשכבה מסוימת מקושר לכל הניורונים בשכבה הבאה אחריה). **ושכבת פלט** - השכבה האחרונה שמוציאה את התוצאה הסופית של הרשת. אימון של רשת מסוג זה נתקל בכמה בעיות מרכזיות:

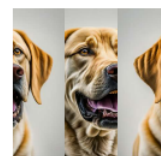
1. דעיכת גרדיאנטים - פונקציית הסיגמואיד הולכת ומשתטחת בערכים גבוהים או נמוכים. לכן הנגזרת שלה שואפת לאפס באזורים אלו. אלגוריתם שמאמן את הרשת לרוב מסתמך על מכפלות של נגזרות הלו - ולכן הגרדיאנטים הולכים וקטנים - דועכים ככל שחוזרים אחורה בשכבות, מה שעוצר את הלמידה של השכבות המוקדמות.
2. בגלל מורכבות הרשת, פונקציית ההפסד מכילה הרבה נקודות מינימום מקומיות - ולא גלובליות. וכן במצבים אלו אלגוריתם הלמידה עלול להתקע ולא להגיע לפתרון הטוב ביותר.
3. ריבוי פרמטרים: ברשתות מחוברות לחלוטין כמות המשקולות והפרמטרים גדלה בצורה עצומה ומגיעה בקלות למיליונים.

דוגמה לרשת כנ"ל עם 3 שכבות מוסתרות:



אם כן, ישנן מספר בעיות מרכזיות במבנה של רשתות ניורונים עמוקות מחוברות:

1. כאשר מנסים להזין לרשתות כאלו קלטים בעלי מבנה מרחבי כמו תמונות - נניח שאנחנו מתבוננים בתמונה של כלב. מה יקרה אם נשנה את סדר הפיקסלים בקלט? למשל: נחתוך את התמונה לשלושה טורים אנכיים ונערבב את הסדר שלהם, באופן הבא:



למעשה - כלום! החישוב הבסיסי של ניורון מבוסס על מכפלה פנימית $w^T x$. פעולה הסכימה היא קומוטטיבית - חילופית. אם נשנה את סדר איברי הקלט ובמקביל נשנה את סדר איברי וקטור המשקולות ערך המכפלה הפנימית ישאר זהה לחלוטין. לכן הרשת אדישה לחלוטין לעובדה שהתמונה המבולבלת כבר אינה נראית כמו כלב הגיוני. רשת MLP מסוגלת ללמוד ולזהות באותה מידה הצלחה של כלבים שעברו שינוי - וכלבים שלא עברו שינוי. אם כן, נוכל לחסוך כמות אדירה של פרמטרים אם נגביל את הארכיטקטורה של הרשת: במקום לאפשר קשרים מלאים, נכפה על הרשת לייצר רק פונקציות שמשמרות תכונות מרחביות שאכפת לנו מהן: למשל - אדישות להזזה: היכולת לזהות כלב בתמונה בלי קשר לשאלה אם הוא נמצא בצד ימין, שמאל או במרכז. **לכן:** נרצה לעבור מרשתות MLP מלאות לרשתות קונבולוציה עליהן נדבר בהמשך.

2. בעיה נוספת היא שרשתות MLP אינן אדישות להזזה: נתבונן בזוג התמונות הבאות -



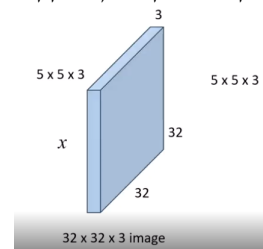
הן אולי נראות זהות, אך בפועל בתמונה הימנית הכלב הוזז מס' פיקסלים ימינה לעומת התמונה השמאלית. רשת MLP שלמדה לזהות את הכלב במיקום המקורי שלו לא תדע להשליך ולזהות אותו בתמונה הימנית. מדוע? כי הרשת מלאת קשרים - כל פיקסל קלט קשור למשקולת ייחודית משלו בשכבה הראשונה. ברגע שמסיתים את האובייקט בקצת, המידע שלו נופל על ניורוני קלט אחרים לחלוטין שמחוברים למערכת משקולות אחרת לגמרי. לכן ישנה כאן מגבלה של הכללה. לכן יש לתכנן מבנה שיהיה אדיש להזזות.

רשת קונבולוציה היא הפתרון לשתי הבעיות המרכזיות של ה- MLP בהן דנו. למעשה מדובר בארכיטקטורה חדשה - רשת ניורונים קונבולוציונית. מה שונה?

- השינוי אינו בפעולות המתמטיות, אלא באקלים הקישוריות. במקום חיבור מלא בין הניורונים כך שכל ניורון בשכבה הקודמת מחובר לכל ניורון בשכבה הבאה, המשקולות מסודרות במבנה של פילטרים קטנים - קרנלים. פילטרים אלו סורקים את הקלט ומבצעים שיתוף משקולות - מה שמאלץ את הרשת ללמוד תכונות מקומיות ומקנה לה את התכונה הקריטית של אדישות להזזה.
- מה שנשאר זהה הוא שהרשת עדיין מבוססת על פעולה לינארית, ומיד אחריה פונקציית אקטיביציה לא לינארית. אלגוריתם הלמידה ועדכון המשקולות נותר אותו אלגוריתם גזירה לאחור המבוסס על כלל ההרשרת - כיוון שפעולת קונבולוציה מורכבת מהמון מכפלות וסכומים מקומיים

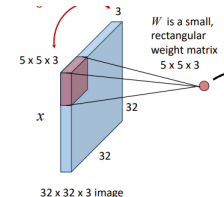
עצמאיים, קל להריץ אותה על מעבדים גרפיים.

נתבונן בדוגמה הבאה. תמונה בגודל 32×32 עם 3 ערוצי צבע (RGB). עבור Fully connected layer כדי להזין אותה לשכבה מלאת קשרים משטחים את התמונה לוקטור באורך $32 \times 32 \times 3 = 3072$ רכיבי קלט. ישנם 5 תכונות ולכן כדי שכל אחד מהניורונים יהיה מחובר לכל אחד מקלטי הפיקסלים, נזדקק למטריצת משקולות בגודל $15,360 = 5 \times 3072$! זה המון משקולות עבור תמונה כל כך קטנה.



במקום שכבה מלאת קשרים, נוכל להסתכל על ניורון בשכבת קונבולוציה שיתמקד בכל פעם רק באזור קטן ומקומי. כפילטר, נגדיר קוביה קטנה של משקולות (בורד). נבחין כי עומק הפילטר חייב להיות זהה לחלוטין למס' ערוצי הצבע של קלט השכבה. לכן הפילטר הוא קוביה תלת מימדית בממדים של $5 \times 5 \times 3 = 75$ פרמטרים. הפילטר מונח על תת אזור מקומי מהתמונה המלאה, נסמנו x' : האקטיבציה הלינארית מחושבת באמצעות מכפלה פנימית $w^T x'$: מכפילים איבר את 75 המשקולות של הפילטר ב-75 ערכי הפיקסלים שנמצאים כעת וסוכמים הכל לסקלר בודד. כאן ניורון בודד דורש 75 משקולות בעוד קודם הוא דרש $15K$. הפילטר הנ"ל יחליק על פני התמונה משמאל לימין, למעלה ומטה על מנת לייצר את מפת התכונות תוך שימוש באותן 75 משקולות בדיוק - שיתוף משקולות.

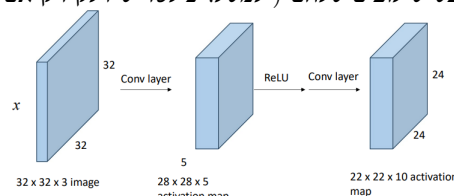
נשאל את עצמנו שאלה פשוטה מאוד: הפינה השמאלית למעלה לא נראית תמיד כמו הפינה הימנית למטה. אז איך פילטר משקולות אחד עם אותן משקולות אמור להתאים לשתייה? התשובה פשוטה גם כן - פילטר ספציפי לא מנסה להבין את כל התמונה. אלא מחפש פיז'ר ספציפי. למשל - אם הפילטר למד לזהות קשת חצי עגולה (אוזן של כלב). אם נריץ את הפילטר על הפינה השמאלית למעלה ויש שם שמיים חלקים - המכפלה הפנימית $w^T x'$ תניב ערך קרוב ל-0. אם נריץ אותו באזור אוזניו של הכלב - המשקולות שלו יסתדרו עם הפילטרים, והמכפלה תתן ערך גבוה מאוד. המטרה היא שאם הכלב יזוז ימינה - אותו פילטר יזהה בדיוק את האוזן שלו במקום אחר.



מטרת הפיצ'ר: חיפוש דפוס בתמונה. אך נראה כי נרצה ליצור קבוצה של מאפיינים שונים במקביל. הרי פילטר בודד לא מספיק כדי להבין תמונה שלמה. על מנת שהרשת תוכל לחפש גם "קימור צהוב" (בננה) וגם "גבול" (תפוח) באותו הזמן - מפעילים מערכת של פילטרים שונים על אותה תמונת קלט.

בסופו של דבר, שכבת קונבולוציה אמיתית מחזיקה עשרות או מאות פילטרים עצמאיים כאלו. אם נפעיל, למשל, 64 פילטרים שונים בגודל זהה, אנחנו נקבל בסוף השכבה 64 מפות אקטיבציה נפרדות.

מה התוצאה לבסוף? מפת אקטיבציה - תמונת מפה שמראה איפה הפיצ'ר שחיפשנו. נניח שבידינו פילטר אחד שלמד לזהות צבע צהוב מעוקל. הפילטר הנ"ל סורק את התמונה המקורית משמאל לימין ומלמעלה למטה. בכל נקודה שהוא עוצר בה, הוא מחשב מספר: אם באותה נקודה יש רקע לא קשור הוא יחזיר 0, וגם באותה נקודה יש את הקליפה יחזיר ערך גבוה. בסוף הסריקה: הפילטר הנ"ל מייצג מטריצה חדשה (לוח משבצעות): אם נצבע למשל באור בהיר את המשבצות שקיבלו ערכים גבוהים ובשחור את אלו שקיבלו אפס - נקבל מפה שמדליקה באור בהיר את המקומות שהבננו נמצאת בתמונה. היא מראה באיזה מקום הניורונים הופעלו מהקלט. אך שוב, ישנם כמה וכמה פילטרים. לכן לבסוף נוצרות k מפות כנ"ל שהרשת שותלת אותן אחת מאחורי השנייה (כמו ערימת דפים): בגודל $x \times x \times k$. המיקום (x_1, y_1) אומר את המיקום המרחבי - איפה זה נמצא בשטח התמונה. k_1 אומר איזה מאפיין נמצא שם. הקובייה הזו היא כבר לא פיקסלים של צבע. היא תמונה חדשה לחלוטין של רמזים ויזואליים. עכשיו, השכבה הבאה של הרשת (שתהיה עוד שכבת קונבולוציה) הולכת לקחת פילטרים חדשים משלה, להריץ אותם על קוביית הרמזים הזו, ולחפש שילובים שלהם (למשל: פילטר שידלק רק אם הוא מוצא מפת "עיגול אדום" שמעליה יושב "גבעול ירוק" - שזה כבר ממש תפוח).



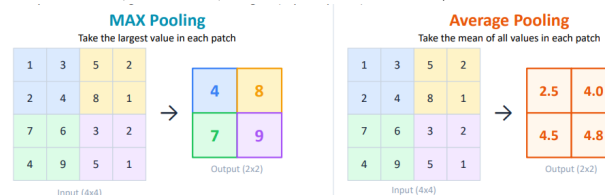
7.2.1 Max and Average pooling

ברשתות קונבולוציה לאחר פעולת הקונבולוציה - ישנו שלב נוסף: שכבות Pooling. מדובר בפעולת דיגימה מחדש שנועדה להקטין את הממדים המרחביים של מפות האקטיבציה. מדוע צריך מפות אלו?

- הקטנת מימדים מרחביים - כיווץ הלוח מפחית את עומס החישוב בשכבות הבאות.
- הפחתת פרמטרים: הקטנת המפה מקטינה בעקיפין גם את מס' המשקולות שנצטרך.
- מאחר שהתמונה מתכווצת, הפילטרים בשכבות הבאות יסתכלו בפועל על אזור גדול יותר מהתמונה המקורית. הרעיון המרכזי הוא לקחת ערך בודד מתוך אזור מוגדר. ישנם שני סוגים שונים:

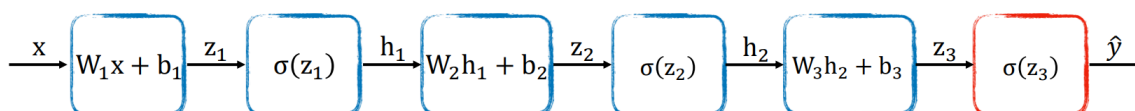
1. Max pooling: בכל חלון 2×2 (למשל) שומרים אך ורק את הערך המקסימלי ומקבלת פלט קטן יותר. זה משמר את האקטיבציה החזקה ביותר (ההתרגשות הגבוהה ביותר של הפילטר), מתאים לזיהוי קצוות ומרקמים, מעניק חסינת גבוהה יותר להזזות קטנות בתמונה.

2. Average pooling: בכל חלון שומרים את הערך הממוצע האריתמטי של הערכים שבתוך החלון. זה מחליק את מפת המאפיינים, תופס את הנוכחות הכללית של המאפיין באזור ופחות רגיש לאקטיבציות קיצוניות.



Multilayer perceptron 7.3

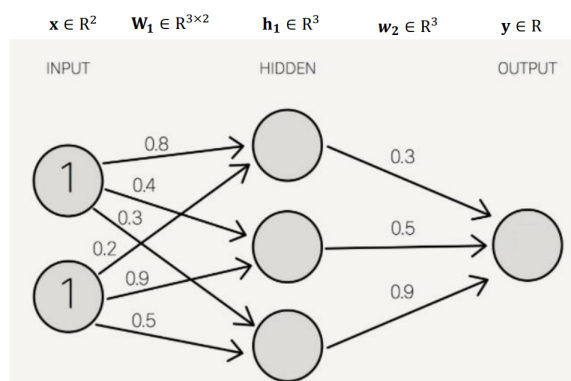
- כעת נציג כיצד קלט מתקדם בשכבות הרשת עד לקבלת פלט סופי. התהליך מתחיל מחישוב לינארי של השכבה הראשונה:
- x הוא וקטור הקלט, W_1 היא מטריצת המשקולות הנתונה על השכבה הראשונה (וקטור משקולות לכל איבר בשכבה) ו b_1 הוא וקטור ההטיה של השכבה הראשונה. הקלט x עובר טרנספורמציה לינארית בשכבה הראשונה לקבלת הערך $z_1 = W_1 x + b_1$.
 - הערך z_1 מוזן לתוך פונקציית האקטיבציה σ הלא לינארית, והפלט שמתקבל הוא וקטור $h_1 = \sigma(z_1)$.
 - הוקטור h_1 כעת משמש כקלט לשכבה הבאה - מוכפל במטריצת המשקולות W_2 בתוספת ה b_2 הטיה, ושוב התהליך חוזר על עצמו.
 - לבסוף תוצאת הפעלת פונקציית האקטיבציה על z_3 הוא הפלט $\hat{y}$.



כמו כן, דוגמה לרשת עם 2 שכבות - נראה את חישוב התהליך:

$$x = [1, 1]^T, W_1 = \begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.9 \\ 0.3 & 0.5 \end{pmatrix}$$

$$h_1 = \sigma(W_1 x) \Rightarrow \sigma \left(\begin{pmatrix} 0.8 & 0.2 \\ 0.4 & 0.9 \\ 0.3 & 0.5 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) = \begin{pmatrix} 0.73 \\ 0.78 \\ 0.68 \end{pmatrix}$$



באופן דומה ניתן להמשיך את החישוב עד לקבלת פלט סופי ברשת. אם כן: מאיפה אנחנו ידענו ברשת הקודמת את הערכים של המשקולות W_1, w_2 ? על מנת למצוא את הערכים הללו שימזערו את ההפסד נבצע *Backpropagation*: ראשית, נחשב את פונקציית ההפסד. ואז נשתמש בכלל השרשרת על מנת לנבא את הגרדיאנטים. בהתחלה מתחילים בחישוב הנגזרת של פונקציית ההפסד ביחס לניבוי $y - \frac{\partial L}{\partial \hat{y}}$. בצעד השני, עוברים דרך פונקציית האקטיבציה האחרונה $\sigma(z_3)$ וגוזרים את ההפסד לפיה - על מנת לחשב את ההפסד לפיה נעזרים בכלל השרשרת:

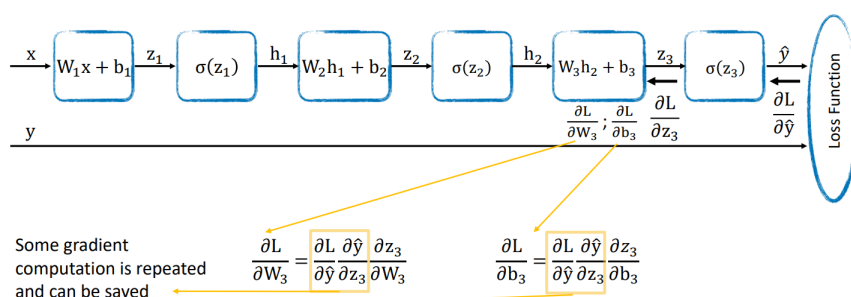
$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3}$$

שכן בשביל להגיע לערך הלינארי של z_3 יש לעבור קודם דרך פונקציית האקטיבציה האחרונה. בהמשך, אם אנחנו רוצים להבין כיצד שינוי ב W_3 משפיע על המשקולות נקבל:

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial W_3}$$

נבחין שישנן כאן מכפלות של חישובי נגזרות וגרדיאנטים שמתחילים לחזור על עצמם - וזה טוב לנו ליעילות על מנת לחשב חלק פעם אחת, לשמור אותו בזיכרון ולהשתמש בו בהמשך. עבור W_1 נקבל:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_3} \frac{\partial z_3}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial W_1}$$



כעת, בדומה ל GDD נוכל להגדיר כלל עדכון. נסמן ב t את אינדקס הצעד הנוכחי, ו $t-1$ יהיה מצב הפרמטרים באיטרציה הקודמת. נקבל:

$$W_3^t = W_3^{t-1} - \eta \frac{\partial L}{\partial W_3}$$

$$b_3^t = b_3^{t-1} - \eta \frac{\partial L}{\partial b_3}$$

- כמובן, שמבצעים זאת על כל מטריצות המשקולות ווקטורי ההסטה. ולא רק על אלו שהצגנו. התהליך המלא שקורה בכל איטרציה בודדת -
- הקלט x זורם מההתחלה לסוף ברשת.
 - פעפוע לאחור - הגרדיאנט זורם מהסוף להתחלה דרך כל השכבות באמצעות כלל השרשרת ומחשבים את הגרדיאנטים עבור כל הפרמטרים ברשת. כלומר $\frac{\partial L}{\partial W_i}, \frac{\partial L}{\partial b_i}$.
 - עדכון הפרמטרים - מבצעים את חוק העדכון לכל הפרמטרים בבת אחת.

סה"כ היתרון הגדול הוא להבחין כי בכל שלב במקום לחשב מחדש שרשראות גזירות, המחשב מבצע מכפלה מטריציונית קטנה בין הגרדיאנט שהגיע מימין לבין הקלט המקומי של השכבה משמאל.

אם נסכם - כך נראה אלגוריתם האימון המלא בפועל בעזרת SGD :

בצע k סבבים על כל הקלט (Epoch):

1. עבור כל פעם על דגימה אחת מהקלט ושלוף דוגמה x_i מהרשימה:

- העבר את הקלט x_i דרך שכבות הרשת על מנת לקבל $\hat{y}_i = f(x_i)$

- חשב את פונקציית $Loss$ ע"השוואת הניבוי $\hat{y}_i$ ל y_i .

- גזור את הגרדיאנט לאחור באמצעות כלל השרשרת.

- עדכון המשקולות: עדכן את המשקולות כולן באמצעות כלל העדכון.

8 הרצאה 8

ממResNets לConvNets

נזכר כי בהרצאה קודמת ראינו הן MLP - רשת ניורונים פשוטה, כשהאימון הוא עם Backpropagation - פעפוע לאחור. משתמשים בכלל השרשרת על מנת להבין איפה הייתה טעות מקומית - ומתקנים (מעדכנים משקולות). נזכר כי ראינו כי:

- $\mathcal{H}$ - מרחב ההיפותזות של הפונקציות מעליו אנחנו עובדים.
- $f_{\mathcal{H}}$ - הפונקציה הטובה ביותר שקיימת במרחב ההיפותזות $\mathcal{H}$ (הפונקציה שמזעזעת את השגיאה ההכללתית המתאימה בכל המרחב)
- $f_{\mathcal{H},S}$ - הפונקציה שמזעזעת את השגיאה על מדגם האימון הספציפי שלנו S .
- $f_{\mathcal{H},S}^*$ - הפונקציה שהאלגוריתם שלנו הגיע אליה בפועל בסיום האימון על המדגם S .

ראינו בהרצאה קודמת $ConvNets$ - רשתות קונבולוציה, שכבת קונבולוציה עליה מפועל מסנן (פילטר) שיוצר מפת אקטיבציה. רשתות קונבולוציה נהנות מתכונה של שיתוף משקולות - אותו פילטר W סורק את כל חלקי התמונה, לכן אם פילטר מסוים למד לזהות מאפיין ספציפי הוא יזהה אותו באותו אופן בין אם האובייקט נמצא בפינה השמאלית העליונה או בין אם הוא במרכז התמונה.

8.1 שיפור האופטימיזציה וההכללה

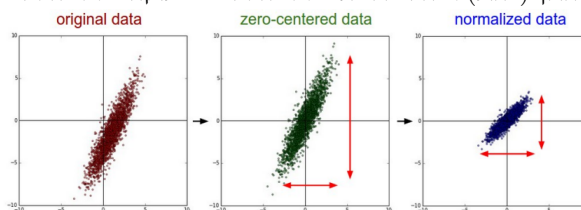
פונקציית ההפסד של רשת עמוקה היא ארוכה ומלאה בפעולות לינאריות ולא לינאריות. שינוי קטן במשקולות השכבה הראשונה מתגלגל דרך כל הרשת. המבנה הנ"ל יוצר משטח הפסד לא קמור - כלומר מלא בנקודות מינימום מקומיות. מרחב החיפוש מוגדר ע"י שילוב כל הערכים האפשריים עבור הפרמטרים של המודל. לרוב מדובר במיליונים ואף מיליארדי פרמטרים.

נרמול נתונים: מטרתנו היא לעזור לאלגוריתם (למשל SGD) למצוא את המשקולות הנכונות כמה שיותר מהר ובצורה יציבה - בלי להתקע. נתונים שמגיעים מהעולם האמיתי יכולים להיות בכל קנה מידה, לדוגמה אם ננסה לחזות מחיר דירה לפי מס' חדרים ($x \in [1, 5]$) ושטח הדירה ($S \in [40, 200]$) אם נזרוק את המספרים הללו כמו שהם לרשת - אלגוריתם האופטימיזציה ישתגע. צעדי הגרדיאנט יזגזגו בפראות! נרמול הוא תהליך מתמטי בו לוקחים את כל המאפיינים השונים ומביאים אותם לשפה משותפת - סקאלה זהה. ישנם שני דברים שצריך לעשות על מנת להפוך את הנתונים לטובים עבור רשת ניורונים:

1. תוחלת אפס - ניתן לראות בגרף כאן למטה, שמימין נקבל את אותם נתונים לאחר שמרכז המסה שלהם הוזז - המרכז יושב בדיוק בראשית $(0, 0)$. מתמטית לוקחים כל ערך x_i ומגדירים: $x_i = x_i - \mathbb{E}[x_i]$. מדוע שנרצה זאת? לרוב משתמשים ב $ReLU = \max(0, x)$. נרצה שהערכים שנכנסים יהיו קטנים ככל הניתן, אם הדטה כולו למשל היה שלילי $ReLU$ יכול להחזיר תמיד אפס או לחלופין להשאיר תמיד חיובי - ואז זה דומה לפונקציה לינארית.

2. שונות 1 - אם למאפיין מסוים שונות ענקית - תהיה לו השפעה דומיננטית מדי ולא פופרציונלית על המכפלה $w^T x$ - רק כי מספריו גדולים יותר, ולא כי הוא יותר חשוב. לכן נרצה שזה לא יקרה.

בשלב האחרון (כחול) אנחנו מנרמלים את הנתונים - לוקחים את הנתונים הממוכזים ומחלקים כל אחת בסטיית התקן שלו.



הלבנת נתונים ($Whitening$): בגרף למעלה הכחול, ניתן לראות שישנו מתאם מסוים בין x ל y . מהו תהליך ההלבנה? מסובבים את הדטה על מנת לבטל את המתאם בין המשתנים (המטריצה נשכבת אופקית על הציר) ולאחר מכן מותחים את הצירים באמצעות כפל במטריצה $A = inverse(Cov(X))$ לקבלת Ax . התוצאה - ענן נקודות עגול וסימטרי לחלוטין, כך שכל המאפיינים ב"ת זה בזה ובעלי שונות זהה.

למה זה לא טוב? מדובר בפעולה חישובית יקרה מאוד, וכן היא נוטה להגביר דרסטית את רעש הרקע שבנתונים - מה שעלול לפגוע ביכולת ההכללה של הרשת ולגרום לה ללמוד תבניות שונות. לכן לרוב, ברשתות עמוקות נסתפק בגרף הכחול לעיל, ולא נבצע הלבנה.

8.2 BatchNorm

קודם ראינו כי נרמול נתוני הקלט עוזר לאופטימיזציה בתחילת הרשת (מונע ערכים קיצוניים). אך מה קורה כאשר מעמיקים פנימה? במהלך האימון, משקולות השכבות הראשונות משתנות כל הזמן. שינוי זה גורם לכך שההתפלגות של הערכים שמגיעים לשכבות העמוקות יותר - משתנה ללא הרף. השכבות הפנימיות סובלות בדיוק מאותה בעיה שהקלט המקורי סבל ממנה: הן מקבלות ערכים לא ממוכזים, עם סקאלות שונות, מה שגורם שוב לבעיות אופטימיזציה (כמו גרדיאנטים דועכים). אם נרמול הקלט עבד טוב בהתחלה - נעצור באמצע הרשת וננרמל את הנתונים מחדש.

בהינתן מטריצה $X \in \mathbb{R}^{N \times D}$ שמגיעה לשכבת ביניים מסוימת - כך N הוא מס' הדגימות בBatch הנוכחי, D הוא מס' המאפיינים. מבצעים כך:

1. לכל מאפיין $j \in \{1, \dots, D\}$ מחשבים את הממוצע $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$

2. לכל מאפיין $j \in \{1, \dots, D\}$ מחשבים את שונות $\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$
3. מנרמלים - $\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$ (זה קורה לכל תא במטריצה, בהתאם למאפיין j). ϵ קבוע חיובי קטן שמונע לחלק באפס. (נבחין שזה דומה לנרמול שעושים בהסתברות).

על פניו, נראה שסימנו. בפועל, נרמול הנתונים ל- $N(0,1)$ מכריח פלט של שכבה ברשת להיות עם ממוצע 0 ושונות של 1. אנו מגבילים את הערכים להתרכז בצורה ליד האפס. עבור פונקציות לא לינאריות רבות - האזור ליד האפס לינארי לגמרי. למעשה נגרום לכך שהפונקציה הפכה ללינארית - ונרכיב פונקציות לינאריות ונאבד את המורכבות של הרשת. לכן, מציעים שדרוג. מאפשרים לרשת ללמוד כמה נרמול היא באמת צריכה. מגדירים לכל מאפיין j זוג פרמטרים:

1. γ_j - פרמטר סקאלה.

2. β_j - פרמטר הסטה.

שני הפרמטרים הללו הם וקטורים באורך D (מס' המאפיינים בשכבה) ומתעדכנים במהלך האימון כמו שאר הפרמטרים של הרשת. כעת, לאחר שהאלגוריתם מחשב את $\hat{x}_{i,j}$ המנורמל הוא מבצע טרנספורמציה לינארית נוספת:

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

זהו הפלט שמועבר לכל $i \in [N], j \in [D]$ לשכבה הבאה. נבחין כי אם במהלך הלמידה הרשת מגלה כי הנרמול מזיק לה והיא מעדיפה את הערכים המקוריים ללא נרמול - יש לה דרך קלה לבצע זאת: הגרדיאנט יעדיך את γ_j להיות סטיית התקן המקורית σ_j , את β_j להיות הממוצע המקורי μ_j , ואם נציב זאת במשוואה נקבל בדיוק את $x_{i,j}$ המקורי. כלומר BatchNorm לא יכול להרע את המצב - אלא רק לשפר.

אנחנו נכניס את שכבות ה-BN לתוך רשת הניורונים שלנו - כאשר אנחנו נכניס אותה תמיד לאחר פעולה לינארית או שכבת קונבולוציה - ותמיד לפני ההפעלה הלא לינארית (פונקציית אקטיביזציה) שכן שוב נרצה לייצב את הערכים שמגיעים מהפונקציה הלינארית לפני שנכנסים לפונקציה הלא לינארית.

היתרונות של BN:

1. בזכות העובדה שכל שכבה מקבלת קלטים מנורמלים ומאוזנים - המשטח הןף לפחות מעוות. זרימת הגרדיאנטים לאחור חלקה בהרבה מה שמונע כמעט לחלוטין תופעה של דעיכת כרדיאנטים. לכן אפשר להשתמש בצעדי למידה η גדולים בהרבה ולכן המודל מתכנס לפתרון אופטימלי בקצב מהיר משמעותית
2. כיוון שהממוצע והשונות מחושבים מתוך הבאץ' הספציפי בו הדגימה נמצאת, הייצוג הסופי של כל תמונה תמיד מושפע ומשתנה מעט בהתאם לחברותיה בבאץ'. השינויים הקטנים הללו מתפקדים כמו סוג של רעש בריא, הרשת לא יכולה לבצע Overfitting (כי יש רעש) מוחלט על תמונה בודדת - דבר שמשפר את יכולת ההכללה שלה על נתונים חדשים.

נתבונן כעת בדוגמה מספרית:

נקח דוגמה פשוטה לצורך העניין עם $D = 2$ פיצ'רים $N = 21$ כגודל Batch. נקבל:

$$X = \begin{pmatrix} 2 & 10 \\ 8 & 20 \end{pmatrix}$$

נחשב ממוצעים -

$$\mu_1 = \frac{2+8}{2} = 5, \mu_2 = \frac{10+20}{2} = 15 \implies \mu = \begin{pmatrix} 5 \\ 15 \end{pmatrix}$$

באופן דומה שנויות -

$$\sigma_1^2 = \frac{(2-5)^2 + (8-5)^2}{2} = 9, \sigma_2^2 = \frac{(10-15)^2 + (20-15)^2}{2} = 25 \implies \sigma^2 = \begin{pmatrix} 9 \\ 25 \end{pmatrix}$$

כעת ננרמל עם $\epsilon = 0$ כך ש $\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}$ ונקבל:

$$\hat{X} = \begin{pmatrix} -1 & -1 \\ 1 & 1 \end{pmatrix}$$

ResNet 8.3

עם הזמן גילו כי ברשתות עם שכבות עמוקות מאוד קשה לאמן. למשל, כמו בדוגמה כאן מטה:



רשת עם 56 שכבות מצליחה הרבה פחות מזו עם 20 שכבות. נשאלת השאלה - למה? על פניו ניתן לומר "אוברפיטינג" - אז עם יותר שכבות היא עם יותר פרמטרים" - אך זו לא התשובה הנכונה. שהרי גם באימון הרשת העמוקה מבצעת פחות טוב. אז מה הסיבה לכך? קשה לחלחל גרדיאנטים דרך הרבה מאוד שכבות - ולכן בדרך כלל הם נעשים מאוד קטנים. בפרט, אם מאתחלים בהתחלה רשת עם משקולות רנדומיות. לוקח הרבה זמן "לחנך" את המשקולות הללו. בשנת 2015 הבינו כיצד ניתן לאמן רשתות בגודל עמוק יותר - עם המצאת *ResNet*. אם כן, מהו *ResNet*?

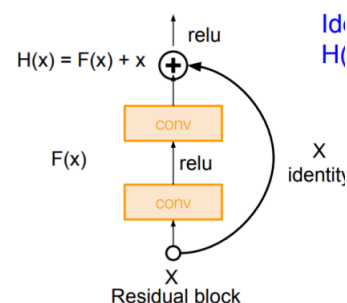
עד היום, ידענו כי מידע מגיע לשכבה מסוימת, עובר דרך קונוולוציה, וכן הלאה מתקדם לשכבה הבאה ומייצג פלט $H(X)$. כעת השכבה הבאה תקבל כקלט $H(X)$ ותתקדם. אם הרשת עמוקה מדי - קשה ללמוד את המיפוי הנ"ל בצורה יציבה. לכן משתמשים בפתרון מעקף - לוקחים את הקלט המקורי X ויוצרים לו מעקף פיזי - שיועד לדלג מעל שכבות. המידע עובר במקביל בשני נתיבים:

1. הנתיב הטיפוסי בין השכבות (עובר דרך קונוולוציות ופונקציות אקטיביציה) - נסמנו $F(X)$.

2. נתיב הזהות - מעביר את X כפי שהוא.

בסוף הבלוק, המערכת מבצעת חיבור פשוט רגע לפני *ReLU* האחרון:

$$H(X) = F(X) + X$$

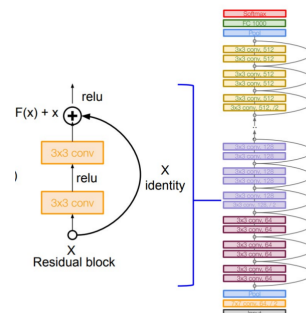


אם כן, מדוע השינוי הקטן הזה פותר את הבעיה?

- ניתן לשחזר את פונקציית הזהות בקלות - קודם לכן אמרנו שלרשת מבוססת משקולות קשה להעביר את הקלט ללא שינוי. במבנה של *ResNet* המצב הפוך: אם השכבות הניחושיות לא תורמות כלום והמודל פשוט רוצה להעביר את המידע הלאה, הוא צריך לעשות $F(X) \rightarrow 0$ (באמצעות הגרדיאנט) ואז המשוואה ישירות תהיה $H(X) \rightarrow 0 + X = X$.
- פתרון ישיר לבעיית הגרדיאנט הנעלם - מתקיים:

$$\frac{\partial H(X)}{\partial X} = \frac{\partial F(X)}{\partial X} + 1$$

כעת בהכרח כל הגרדיאנטים יהיו גדולים מאוד, ונמנע מהבעיה של דעיכת הגרדיאנט. להלן דוגמה לרשת (ResNet):



8.3.1 אתחול המשקולות

אתחול המשקולות משפיע מאוד על היכולת שלנו ללמוד - מדוע הבחירה הזו קריטית?

1. מניעת דעיכת גרדיאנטים
2. עבור פונקציות אקטיביציה כמו $\tanh x$ או סיגמואיד - אם נאתחל משקולות בערכים גבוהים מדי הקלטים שיגיעו לפונקציה יהיו חיוביים מאוד או שליליים קיצוניים, באזורים אלו הפונקציות שואפות לקצוות - הגרף יהפוך לשטוח והנגזרת לאפסית.
3. משטח Loss של רשת עמוקה הוא הררי, מפותל ומורכב מאוד, המטרה של אתחול חכם היא למקם את נקודת המוצא של המודל באזורים "טובים" ונוחים של המרחב - כאלו שיש בהם שיפועים ברורים ולא אזורים שטוחים או נקודות אוסף עמוקות, כדי לאפשר ל-SGD להתחיל להתקדם מיד לכיוון המינימום הגלובלי.

כאשר אנו מסתכלים על שכבת MLP כלשהי l , מטריצת המשקולות שלה מוגדרת להיות $W^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$, כך ש- n_{l-1} הוא מס' הניורונים שנכנסים אל תוך השכבה הנוכחית מהקודמת ו- n_l הוא מס' הניורונים שיוצאים משכבה זו לשכבה הבאה. נבחין כי אם נשים ערך קבוע וקטן מאוד בכל מטריצת המשקולות באתחול - ניצור בעייה של סימטריה. אם כל המשקולות בשכבה מסוימת זהות, כל הניורונים באותה שכבה יחשבו בדיוק את אותו פלט עבור קלט נתון, בזמן הפעפוע לאחור הם יקבלו את אותו הגרדיאנט ויעודכנו בצורה זהה. חייבים אקראיות על מנת לשבור סימטריה (ד"ש לצבי).

גישת Le Cun - נרצה שהשונות של האקטיביציות (פלט השכבות) לא תשתנה ולא תדעך ככל שמעמיקים ברשת גם אם השכבות משנות את גודלן כלומר $n_{l-1} \neq n_l$. נניח שרכיבי הקלט x ורכיבי המשקולות w הם משתנים אקראיים ב"ת בעלי תוחלת 0. נראה כי:

$$Var(W^T x) = \sum_{i=1}^{n_{l-1}} Var(w_i x_i) = \text{Independent} \sum_{i=1}^{n_{l-1}} Var(w_i) Var(x_i)$$

אם נניח גם כי כל המשקולות מגיעות מאותה התפלגות (בעלי וריאנס זהה) וכל רכיבי הקלט בעלת שונות שונה נקבל כי:

$$Var(W^T x) = n_{l-1} Var(w) \cdot Var(x)$$

אנחנו נהיה מעוניינים כי $Var(x) = Var(W^T x)$ (שונות השכבות זהה) לכן, נבחין כי אם נדרוש שהשונות שבה נאתחל את המשקולות תקיים $Var(w) = \frac{1}{n_{l-1}}$ בהמשך, גילו כי לעקוב אחרי ההתקדמות קדימה - זה לא מספיק. חייבים לוודא שגם במעבר אחורה הגרדיאנטים לא דועכים או מתפוצצים. לכן גילו כי צריך להתקיים $Var(w) = \frac{1}{n_l}$. אי אפשר לקיים את שני התנאים בו זמנית אם השכבות משנות את גודלן, לכן ניתן למצוא פשרה: הממוצע האריתמטי שלהן -

$$Var(w) = \frac{2}{n_l + n_{l-1}}$$

כלומר נרצה לאתחל את המשקולות כך שיתקיים $W \sim N(0, \frac{2}{n_l + n_{l-1}})$. אם נרצה מתוך התפלגות אחידה, הרי שיתקיים:

$$W \sim U(-\sqrt{\frac{6}{n_l + n_{l-1}}}, \sqrt{\frac{6}{n_l + n_{l-1}}})$$

8.4 Generalization in deep networks (הכללה ברשתות עמוקות)

בלמידת מכונה קלאסית דנו על הרגולציה - הוספת $\lambda ||w||^2$ על מנת להוריד Overfitting (זינוק המשקולות) ולהגביר את יכולת ההכללה. הוספת ערך זה נקראת Weight Decay. אם כן, נבחין כי: הגרדיאנט של איבר הענישה הוא $2\lambda w$ - אם נבלע את הקבוע 2 לקצב הלמידה η , נקבל כי הוספת האיבר לפונקציית ההפסד שקולה להוסיף את האיבר $-\lambda w$ לגרדיאנט. נבחן כעת את צעד העדכון של המשקולות בזמן t ביחס לרגולציה בלבד (ללא גרדיאנט הנתונים) לצורך ניתוח המבנה. נראה כי -

$$w^t = w^{t-1} - \lambda w^{t-1} = (1 - \lambda) w^{t-1}$$

כיוון של מס' חיובי קטן מאוד, $1 - \lambda$ קטן בקצת מ-1. אם נפתח את הנוסחה נקבל כי:

$$w_2 = (1 - \lambda) w_1$$

$$w_3 = (1 - \lambda)w_2 = (1 - \lambda)^2 w_2$$

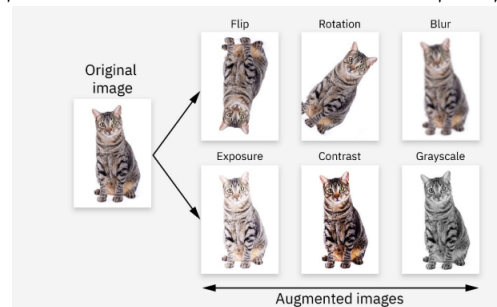
ובאופן כללי יותר:

$$w^t = (1 - \lambda)^{t-1} w^1$$

כלומר - ישנה דעיכה אקספוננציאלית של ערך המשקולות לכיוון האפס (כי $1 - \lambda$ קטן מ-1 בהכרח) ככל שהאיטרציות מתקדמות. במשך שנים, הניחו כי בלי רגולציה מפורשת מודל עם קיבולת גדולה בהכרח יסבול מ-Overfitting חמור. התעלומה הגדולה של למידת מכונה היא שרשתות ניורונים מודרניות מכילות לעיתים קרובות מיליוני פרמטרים - ולמרות זאת: אם אנחנו מוותרים על הרגולציה הרשתות עדיין מצליחות להציג ביצועי הכללה מעולים על נתוני הטסט. התופעה הזו נקראת Implicit Regularization ("רגולציה משתמעת").

8.4.1 Implicit Regularization - הרחבת נתונים

דוגמה ראשונה לרגולציה משתמעת היא הרחבת נתונים. בזמן האימון, במקום להזין לרשת שוב ושוב רק את התמונה המקורית המערכת מפעילה עליה טרנספורמציות אקראיות כדי לייצר תמונות חדשות. ישנן כמה טרנספורמציות נפוצות: הפיכת התמונה, סיבובה בזווית, הפחתת חדות התמונה, שינוי רמת התאורה, שינוי נגודיות בין בהיר לכהה, הסרת הצבע וכו'. מטרת הטרנספורמציות הללו היא להגדיל את מגוון הנתונים בצורה מלאכותית - ובכך לאלץ את הרשת ללמוד מאפיינים שלא תלויים במיקום המדויק, בצבע או ברמת התאורה של האובייקט (ליצור אי תלות).



שאלה שנרצה לשאול - האם זה מתבצע ב-CPU או GPU?

- אם מבצעים ב-CPU: המעבד טוען את התמונות מהדיסק, מפעיל עליהן את הטרנספורמציות, ורק אז מעביר את Batch המוכן ל-GPU. המטרה היא לנצל את המעבד לעיבוד מקדים, כדי שה-GPU יהיה פנוי אך ורק לחישובי רשת הניורונים.
- אם מבצעים ב-GPU: התמונות הגולמיות מועברות ישירות ל-GPU, והוא זה שמבצע את הטרנספורמציות הגיאומטריות והמתמטיות במקביל על הפיקסלים. זה קורה מהר מאוד, אך גוזל כוח עיבוד וזיכרון שיכלו לשמש לאימון שכבות המודל עצמו.

נבחין שצעד זה מונע מהרשת לשנן דטה - ובפרט גורם ל-Overfitting לרדת - כלומר ההכללה יותר טובה. בדיוק כמו המטרה של רגולציה.

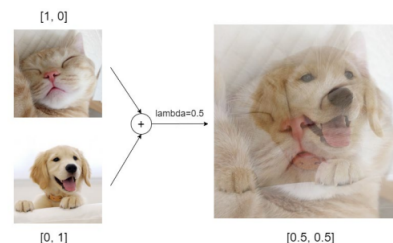
8.4.2 MixUp - Implicit Regularization

טכניקה זו מבצעת שילוב לינארי בין שתי דגימות שונות לחלוטין - ובין שתי התוויות שלהן בו זמנית. התהליך המתמטי עובד כך: האלגוריתם בוחר באקראי שתי דוגמאות חדשות מתוך ה-Batch: דוגמה i ודוגמה j . הוא מגדיר מקדם ערבוב בשם $\lambda \in [0, 1]$ ומייצר דוגמה סינטטית חדשה: $(\tilde{x}, \tilde{y})$ כך:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j$$

$$\tilde{y} = \lambda y_i + (1 - \lambda)y_j$$

אם נבחר $\lambda = \frac{1}{2}$ מקבלים תמונה משולבת שבה רואים שקיפות משולבת של שתי הדוגמאות. למשל, כמו בדוגמה כאן עם הכלב והחתול:

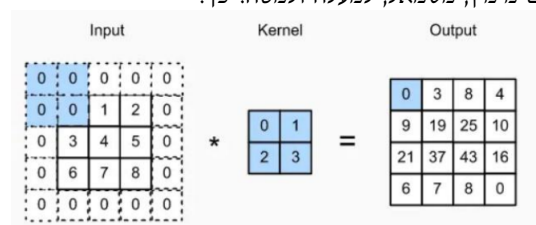


כיצד בפועל בוחרים את λ ? במקום לבחור ערך קבוע בכל איטרציה מגרילים את λ מתוך התפלגות בטא כלומר $\lambda \sim \text{Beta}(\alpha, \alpha)$ כאשר α נבחר לרוב להיות 0.2.

מדוע גישה זו עוזרת להכללה? האינטרפולציה שמבצעים כאן מאלצת את הרשת להתנהג בצורה לינארית ובריאה באזורים שבין המחלקות השונות. במקום שהרשת תפתח גבולות החלטה חדים, מפותלים וקיצוניים מדי סביב הדוגמאות הקיימות (יגדיל Overfitting), היא לומדת ששינוי הדרגתי בפיקסלים צריך להוביל לשינוי הדרגתי ותואם בהסתברות הפלט.

8.5 CNN - תרגול

ישנן שתי בעיות שנוצרות בקונבולוציה. בכל פעם שמבצעים פעולת קונבולוציה גודל תמונת הפלט קטן ומתכווץ בהשוואה לגודל תמונת המקור, וכן ישנו איבוד של מידע בקצוות - כאשר הקרנל נע על פני תמונת המקור הוא נוגע בשוליים ובקצוות של התמונה פחות פעמים מבמרכז התמונה. הפתרון לכך הוא שימוש ב-Padding - ריפוד, מאפשר לשמור את הגודל של תמונת המקור. לדוגמה - עבור $P = 1$ אנחנו מוסיפים שכבת אפסים מימין, משמאל, למעלה ולמטה. כך:



Stride: במהלך תנועת הקונבולוציה, הפילטר מחליק על פני תמונת הקלט משמאל לימין ולמעלה למטה עד שהוא מכסה את כל הערכים במטריצת הקלט. $S > 0$ יהיה גודל הצעד שהפילטר לוקח בכל שלב עד שהוא זז למיקום הבא. עבור $S = 1$ הפילטר זז בכל פעם משבצת אחת (פיקסל אחד) הצידה. עבור $S = 2$ הפילטר זז שתי משבצות בכל שלב (מדלג על אחת). כעת נציג נוסחה לחישוב:

בהינתן -

- $W_{in} \times H_{in}$ מימדי תמונת הקלט
- $K \times K$ - מימדי הפילטר (הקרנל)
- S גודל הפסיעה, P גודל הריפוד. יתקיים:

$$W_{out}/H_{out} = \left\lfloor \frac{W_{in}/H_{in} - K + 2P}{S} \right\rfloor + 1$$

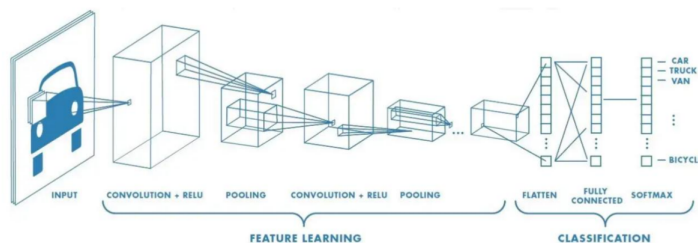
נבחין כי אם נרצה שגודל התמונה לא ישתנה לאחר שכבת הקונבולוציה ובהינתן כי $S = 1$ נרצה כי $P = \frac{K-1}{2}$.

8.6 Flatten

נגדיר את הפעולה כפעולת השטחה, כלומר המרה של מפות המאפיינים הדו מימדיות שנוצרו ברשת הקונבולוציה לוקטור חד ממדי. המרה זו הכרחית כדי שיהיה ניתן להזין את המידע לתוך שכבה Fully-connected אשר מקבלת כקלט וקטורים בלבד ולא מטריצות. דוגמה להשטחה:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \Rightarrow \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{pmatrix}$$

דוגמה לרשת CNN מלאה - לקראת הסוף משתמשים בהשטחה:



8.7 Weight Decay

נרצה קצת מוטיבציה לבעיה. נבחין כי פונקציה מורכבת מאוד (!) ככל הנראה לא תצליח להכליל טוב על נתונים חדשים שלא ראתה בזמן האימון - מה שיובייל Overfitting. לפי עקרון חשוב - אנחנו תמיד נעדיף מבין שני מודלים H, H' ששניהם מצליחים לפתור את אותה בעיה, את המודל הפשוט יותר. בהקשר של למידת מכונה - אנחנו רוצים למנוע מצב של דעיכת המשקולות. אנחנו נכפה את הפשטות הזו על ידי כך שנרסן את המשקולות של המודל ונמנע מהן למנוע יתר על המידה - מה שישמור על פונקציית הלמידה פשוטה יותר. כפי שכבר ראינו בעבר - הפתרון הוא ע"י הוספת L2 Norm - רגולריזציה. מדוע זה נקרא דעיכת משקולות? נבחין כי בGD נקבל כלל עדכון כעת:

$$W^{t+1} = W^t - \eta \left(\frac{\partial L_{original}}{\partial W} + \lambda W^t \right)$$

אם נפתח את הנוסחה נקבל:

$$W^{t+1} = (1 - \eta\lambda)W^t - \eta \frac{\partial L_{original}}{\partial W}$$

כיוון של η, λ הם ערכים קטנים, המקדם $1 - \eta\lambda$ הוא מספר שקטן בקצת מ-1. המשמעות היא שעוד לפני שמתחילים את העדכון של הגרדיאנט המקורי - המשקולות הנוכחיות קטנות ועוברות כיווץ ודעיכה אל עבר האפס!

8.8 Optimizers - אלגוריתמי אופטימיזציה

כיצד נראה צעד אימון פשוט?

1. בחירת קלט x מסט האימון.
2. חיזוי התוויית $\hat{y} = f(x)$
3. חישוב ערך ההפסד $L(\hat{y}, y)$
4. חישוב הנגזרת לכל פרמטר $\frac{\partial L}{\partial w}$ (באמצעות פעפוע לאחור)
5. עדכון כל פרמטר (כיצד?)

כיצד אנחנו מעדכנים כל פרמטר בשלב 5? האופטימיזר הוא אלגוריתם שקובע את אסטרטגיית העדכון. למשל - בכמה לנוע נגד הגרדיאנט, האם לקחת בחשבון צעדים קודמים וכו'. עד היום חשבנו שבשלב 5 תמיד מתבצע GD - אך יש עוד אלגוריתמים, המשותף לכולם הוא שכולם מסתמכים על הגרדיאנטים שחושבו בשלב 4.

8.8.1 אופטימיזר ראשון - SGD

נוסחת העדכון של הSGD שכבר ראינו היא פשוט:

$$\Theta_i = \Theta_i - \eta \frac{\partial L}{\partial \Theta_i}$$

משימה קשה של SGD היא בחירת קצב הלמידה η - קצב למידה גבוה מדי עלול להוביל את האלגוריתם לזגזג מצד לצד ולא להגיע לעולם לנק' המינימום בעוד קצב למידה קטן מדי יגרום לכך שהאלגוריתם יתקדם לאט מאוד למינימום מה שידרוש כוח גדול חישובי. נעיר כי ההבדל בין SGD לGD הוא שבGD בכל שלב עוברים על כל הדטה סט ואילו בSGD נהוג כי לרוב נבחרת קבוצה קטנה בכל איטרציה. בעיה נוספת שאנו מתמודדים איתה היא שהאלגוריתם עדכון כופה עלינו אותו קצב למידה לכל הפרמטרים.

8.8.2 אופטימיזר שני - Momentum

האלגוריתם שואב השראה מעולם הפיזיקה - כדור מתגלגל במורד גבעה וצובר מהירות - על מנת להתגבר על נקודות אוסף ומינימום לוקלי וזאזוגים מצד לצד. המומנטום משפר את ה- SGD הסטנדרטי בכך שהוא מוסיף משקל נע של הגרדיאנטים מהצעדים הקודמים אל צעד העדכון הנוכחי. באופן פורמלי, נעזרים במשתנה עזר v_t (מהירות בזמן t) ומגדירים אותו כך:

$$v_t = \gamma v_{t-1} + \eta \frac{\partial L}{\partial \Theta_i}$$

באשר γ נקרא מקדם המומנטום והוא היפר פרמטר, הוא קובע כמה מהמהירות הקודמת אנו משפרים. עדכון הפרמטר מתבצע באופן הבא:

$$\theta_i = \theta_i - v_t$$

8.8.3 אופטימיזר שלישי - AdaGrad

כעת נציג אלגוריתם שפותר את הבעיה שכל הפרמטרים חולקים את אותו קצב הלמידה. האינטואיציה מאחורי האופטימיזר הנ"ל היא שלא כל הפרמטרים נולדו שווים - יש להתאים לכל פרמטר קצב למידה משלו. כך מעדכנים זאת באופן פורמלי: האלגוריתם מחזיק משתני עזר בשם $c_{t,i}$ אשר צובר את היסטוריית הגרדיאנטים של הפרמטר i לאורך זמן. מעדכן כך:

$$c_{i,t} = \sum_{k=0}^t \left(\frac{\partial L}{\partial \theta_{k,i}} \right)^2$$

כלומר בכל צעד לוקחים את הנגזרת הרגעית בריבוע (של כל הזמנים הקודמים ביחס לפרמטר i) - מי שהצטבר מהצעד הראשון עד לנוכחי. הריבוע מבטיח שערך ה- $Loss$ שנצבר יהיה תמיד חיובי ויגדל מונוטונית. עדכון הפרמטר מתבצע כך:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{c_{t,i} + \epsilon}} \cdot \frac{\partial L}{\partial \theta_{t,i}}$$

עבור פרמטר i אנחנו לא מכפילים בקצב למידה קבוע - אלא גם מחלקים את קצב הלמידה בקבוע c (שורש היסטוריית הגרדיאנטים הריבועיים שנצברה).

החסרון הגדול של האלגוריתם - מתישהו הגרדיאנט יהיה אפס. מדוע? $c_{t,i}$ הוא סכום של ערכים ריבועיים ולכן חייב לגדול ולהצטבר בכל צעד. לאורך זמן c יהיה עצום ולכן קצב הלמידה ישאף לאפס (כי מחלקים ב- c).

8.8.4 אופטימיזר רביעי - RMSprop

נגדיר פורמלית:

$$c_{t,i} = \gamma c_{t-1,i} + (1 - \gamma) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)^2$$

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{c_{t,i} + \epsilon}} \cdot \frac{\partial L}{\partial \theta_{t,i}}$$

מה ההבדל? הוספנו γ כמקדם דעיכה, הוא קובע כמה מההיסטוריה אנחנו משמרים. כעת כיוון ש- $c_{t,i}$ מורכב הן מחלק של γ והן מחלק של $1 - \gamma$ הוא מתפקד כממוצע משוקלל, וכעת כבר לא יכול לגדול לאנסוף. לכן כעת בנוסחה אנחנו לא נקבע כי הגרדיאנט ישאף לאפס ופתרנו את הבעיה.

8.8.5 אופטימיזר חמישי - Adam

נגדיר את כלל העדכון:

$$m_{t,i} = \gamma_1 m_{t-1,i} + (1 - \gamma_1) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)$$

$$v_{t,i} = \gamma_2 v_{t-1,i} + (1 - \gamma_2) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)^2$$

$$\theta_{t+1,i} = \theta_{t,i} - \eta \frac{m_{t,i}}{\sqrt{v_{t,i} + \varepsilon}}$$

זהו אלגוריתם האופטימיזציה הכי נפוץ כיום ומשלב את כל הטוב מהאלגוריתמים הקודמים שראינו. הוא שילוב ישיר של שתי בעיות שראינו לפתור בעיות בSGD:

מצד אחד, משלב את מומנטום עם רכיב ששומר על מהירות וכיוון על מנת להחליק זיגזגים ולעבור נקודות אוכף ולא להתקע בהן. מצד שני, משלב את RMSprop עם רכיב שמתאים קצב למידה ייחודי לכל פרמטר מבלי לסבול מקפאון מונוטוני.

8.8.6 אופטימיזר שישי - AdamW

כמעט זהה, רק שמוסיפים Weight decay: נגדיר את כלל העדכון:

$$m_{t,i} = \gamma_1 m_{t-1,i} + (1 - \gamma_1) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)$$

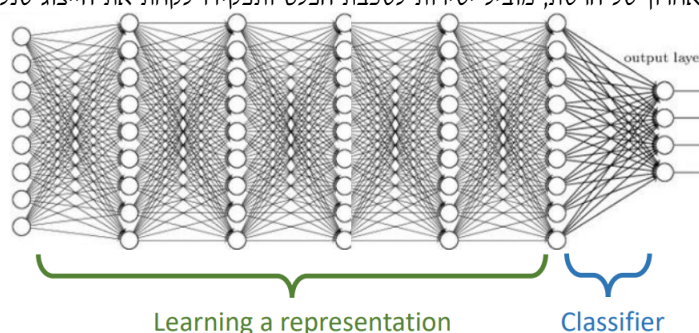
$$v_{t,i} = \gamma_2 v_{t-1,i} + (1 - \gamma_2) \left(\frac{\partial L}{\partial \theta_{t,i}} \right)^2$$

$$\theta_{t+1,i} = \theta_{t,i} - \eta \left(\frac{m_{t,i}}{\sqrt{v_{t,i} + \varepsilon}} - \lambda \theta_i \right)$$

9 הרצאה 9 -

9.1 Unsupervised Learning

רשתות עמוקות לומדות ייצוג של המידע כך שניתן יהיה לסווג אותו בקלות יחסית. ישנם שני חלקים עיקריים - **בירוק**: החלק הארוך ביותר ברשת, כולל את רוב השכבות הנסתרות, תפקידו לקחת את הנתונים הגולמיים ולתרגם אותם לייצוג מתמטי יעיל ומציאת פיצ'רים בדטה. **בכחול**: המסווג - החלק האחרון של הרשת, מוביל ישירות לשכבת הפלט ותפקידו לקחת את הייצוג שנלמד בשלב הקודם ולקבוע את הסיווג הסופי.



ברשת רגילה - הלמידה היא תלוית מטר. הרשת לומדת לייצג את המידע (ירוק) רק כדי להצליח במשימת הסיווג בסוף (כחול). מה אם אין לנו משימת סיווג? (אין לנו תוויות כמו כלב וחתול). אף אחד לא אומר לרשת מה נכון ומה לא - האם עדיין יש דרך לגרום למחשב להסתכל על המידע הגולמי, לחקור אותו וללמוד בעצמו ייצוגים מועילים ומבנים פנימיים שלו? שאלה זו מהווה בסיס ללמידה לא מונחית: למצוא מבנה וייצוג בנתונים מבלי להסתמך על משימת סיווג חיצונית.

מהי למידה לא מונחית? באופן אינטואיטיבי מהעולם האמיתי - תינוקות ובעלי חיים לומדים לתפוס ולהבין את העולם שמסביבם ללא הדרכה או פיקוח מפורש. כמו למשל שמות של קטגוריות או סיווגים. הם פשוט צופים ומבינים תבניות בעצמם. כלומר, בהינתן קלטים $\{x_i\}_{i=1}^n$ (ללא התוויות המתאימות y_i) - האם אנחנו יכולים ללמוד מזה? ישנן 3 מטרות של למידה לא מונחית:

1. להבין את מבנה הנתונים - לגלות כיצד המידע מאורגן באופן פנימי, אילו פריטים קרובים זה לזה או יוצרים קבוצות.
2. למדל כיצד המידע נוצר - להבין את מנגנון או התפלגות המקור שממנה הגיעו הנתונים.
3. ללמוד ייצוגים מועילים - למצוא דרך יעילה ומצמצמת יותר לתאר את המידע.

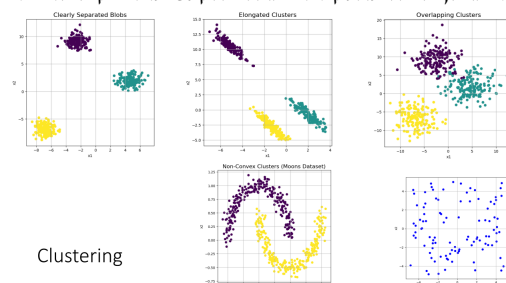
הגדרה 114. למידה לא מונחית היא פרדיגמה שבה אלגוריתם חושף מבנים חבויים, תבניות או קשרים בתוך דטה-סט. כל זה קורה ללא הדרכה של תוויות שנוצרו או תיגו ע"י בני אדם. בעוד שלמידה מונחית מתמקדת בחיזוי, למידה לא מונחית מתמקדת בגילוי ובייצוג. באופן פורמלי, בהינתן קלט נתונים $D = \{x_1, \dots, x_n\}$ כך ש $x_i \in \mathbb{R}^d$, $\forall 1 \leq i \leq n$, כך שהדיגמות בלתי תלויות ובעלות התפלגות זהה (i.i.d). המטרה: מציאת מבנה (נצטרך בהמשך לפרמל מה הכוונה ב"מבנה").

ישנן 3 תחומים מרכזיים בעולם הלמידה הלא מונחית:

1. הערכת צפיפות - מידול מפורש של פונקציית צפיפות ההסתברות האמיתית של הנתונים (למקסם את Log-likelihood של הנתונים הנצפים תחת פרמטרי המודל) - כפי שלמדנו בתחילת הקורס.
2. הורדת מימדיות: למידת פונקציית מיפוי $f: \mathbb{R}^K \rightarrow \mathbb{R}^k$ כך ש $K > k$ - הפונקציה דוחסת את הנתונים תוך שימור התכונות הגאומטריות. (נלמד זאת בהרצאה הבאה).
3. Clustering - חלוקה של מרחב הקלט לתתי קבוצות המבוססות על ידי קרבה גאומטרית. כלומר - נקודות שקרובות פיזית במרחב ישויכו לאותה קבוצה.

9.2 Clustering

כפי שאמרנו, נרצה לחלק את מרחב הקלט לתתי קבוצות באמצעות קרבה גאומטרית. ישנן כמה וכמה מקרים שנוכל להתקל בהם, בתמונה:



Clustering ישנן שימושים רבים בעולם האמיתי - למשל: אשכול הצעות למשתמשים בנטפליקס (או מערכת המלצות למשתמשים בוולט) - המערכת מזהה קבוצות של פריטים דומים או משתמשים בעלי טעם דומה.

נחלק את Clustering ל-3 גישות אלגוריתמיות מרכזיות:

1. Prototype-based clustering: כל אשכול מיוצג ע"י נקודת אב טיפוס מרכזית - נקודת הנתונים משויכות לאשכול שהאב טיפוס שלו הוא הקרוב ביותר גאומטרית. (על כך נרחיב היום בהמשך)
2. Hierarchical clustering: בנייה של היררכיה של אשכולות (מבנה מדורג) במקום לבצע חלוקה שטוחה ויחידה של המרחב. מיוצג לרוב ע"י עץ.
3. גישה מבוססת גרפים: הנתונים מיוצגים כגרף מעל הדגימות, שבו כל נקודת נתונים היא קודקוד והקשתות מייצגות את רמת הדמיון בין הנתונים.

נדון בסוג הראשון. האב טיפוס (הנקודה שמשייכת מחלקה) לא חייב להיות אחת מהנקודות שנמצאת בדטה-סט (הוא יכול להיות למשל ממוצע אריתמטי וכו'). המטרה הפורמלית היא לשייך כל דגימה x_i לאשכול מסוים מתוך k משקולות. פורמלית,

$$x_i \rightarrow \{1, \dots, k\}$$

כדי להשיג מטרה זו התהליך מורכב משני שלבים מרכזיים:

1. למצוא את אבות הטיפוס (הנקודות המרכזיות)
2. להגדיר את האשכולות: האשכול נקבע ע"י שיוך כל אחת מהדגימות לאב טיפוס הקרוב ביותר אליה - השיוך מתבצע באמצעות מדד מרחק או מטריקה מסוימת.

כעת נפרמלאת ההגדרה של אשכול. אנו עוסקים באשכול מבוסס אב טיפוס (נקודה) מעל $\mathbb{R}^d$, כאשר מס' האשכולות K ידוע מראש והמטרה היא מינימיזציה של מרחק L_2 .

המטרה: למצוא קבוצה של K אבות טיפוס שיסומנו כקטורים $\mu^* = \{\mu_1, \dots, \mu_K\}$ כאשר כל אב טיפוס נמצא ב- $\mathbb{R}^d$. קבוצת אבות הטיפוס הנ"ל צריכה למזער את סכום המרחקים של כל הנקודות ב- $\mathcal{D}$ אל אב הטיפוס הקרוב ביותר אליהן. כלומר למזער את:

$$\mu^* = \arg \min_{\mu} \sum_{i=1}^n \min_{j \in \{1, \dots, K\}} \|x_i - \mu_j\|_2^2$$

כלומר הוקטור $\mu = (\mu_1, \dots, \mu_K)$ שיזמער את הסכום. כל איבר בודד בסכום - צריך לעבור על כל וקטורי אבות הטיפוס, ולמצוא את זה שהמרחק אליו מינימלי (זה הקרוב ביותר אליו). לכל נקודה $1 \leq i \leq n$ נוכל להגדיר:

$$\text{cluster}(x_i) = \arg \min_j \|x_i - \mu_j\|_2^2$$

להיות האינדקס שנבחר עבור הנקודה x_i כאב הטיפוס הקרוב ביותר אליה.

אם כן, פונקציית המטרה הזו היא אי קמורה ולא רציפה. יש הרבה נקודות מינימום גרועות במרחב - אלגוריתמי אופטימיזציה פשוטים עלולים להתקע בפתרון לא מושלם ולא להגיע לפתרון הטוב ביותר האפשרי. כמו כן, על מנת למצוא את האופטימום הגלובלי המדויק נדרש זמן ריצה של $O(n^{kd+1})$ - זו סיבוכיות פולינומית גבוהה מאוד ביחס לפרמטרים! כיוון שלמצוא את הפתרון האופטימלי זה יקר מאוד, אנחנו נשתמש באלגוריתם היוריסטי שעובד באיטרציות חוזרות - ההיוריסטיקה הזו תבטיח התכנסות מונוטונית למינימום מקומי (נבחין: לא בהכרח גלובלי).

9.3 אלגוריתם K-Means (1957)

נציג פסאודו של האלגוריתם:

א. אתחול - בוחרים באופן אקראי K אבות טיפוס ראשוניים מתוך המרחב

$$\mu^0 = (\mu_1^0, \dots, \mu_K^0)$$

ב. נרוץ בלולאה עבור $t = 1, 2, \dots$ (עד להתכנסות למינימום מקומי)

1. E-Step: כל דגימה x_i משויכת לאשכול האב טיפוס הקרוב ביותר אליה בהתבסס על אבות הטיפוס מהאיטרציה הקודמת. כלומר:

$$\text{Cluster}^{(t-1)}(x_i) = \arg \min_j \|x_i - \mu_j^{t-1}\|_2^2$$

2. M-step: מחשבים מחדש את המיקום של כל אב טיפוס בתוך הממוצע (Mean) של כל הדגימות ששויכו לאותו אשכול בצעד הקודם. כלומר: - נגדיר לכל אשכול $1 \leq j \leq K$ את אוסף כל הנקודות שמשויכות לאשכול זה:

$$c_j = \{x_i \mid \text{Cluster}^{(t-1)}(x_i) = j\}$$

- נעדכן את האב טיפוס החדש μ_j^t (הממוצע) של האשכול j באיטרציה הנוכחית:

$$\mu_j^t = \frac{1}{|c_j|} \sum_{x \in c_j} x$$

האלגוריתם יעצור רק כאשר אין יותר שינוי במיקומי אבות הטיפוס ובשיוכים של הנקודות לאשכולות בין איטרציה אחת לבאה אחריה. (כלומר אם מתקיים לכל $1 \leq j \leq K$ כי $c_j^{(t-1)} = c_j^{(t)}$ נעצור). מדוע? אם הקבוצות לא זזו - גם כשנפעיל את ה- $step$ לחשב את המרכזים החדשים - הממוצעים של כל קבוצה יישארו בדיק אותם דבר.

משפט 115. אלגוריתם K -means מתכנס.

הוכחה: נוכיח שהאלגוריתם עוצר תוך מספר סופי של צעדים. תהי J פונקציית המטרה הגלובלית. פונקציה זו מודדת את הפחית של המודל ותלויה

בשני סוגי משתנים:

1. $\mu = (\mu_1, \dots, \mu_K)$ - וקטורי המרכזים כאשר $\mu_j \in \mathbb{R}^d$
2. $r \in \{0, 1\}^{n \times K}$ - מטריצת שיוך המשתנים הבינאריים - $r_{ij} = 1$ אפי"ה הנקודה x_i משויכת לאשכול j . נגדיר:

$$J(\mu, r) = \sum_{i=1}^n \sum_{j=1}^K r_{ij} \|x_i - \mu_j\|_2^2$$

נבחין כי באיטרציה ה- t המרכזים μ^t קבועים. האלגוריתם מעדכן את השיוכים לקבלת $r^{(t+1)}$ בצורה חשבונית כך שמתקיים:

$$r_{ij}^{(t+1)} = 1 \iff j = \arg \min_k \|x_i - \mu_k^{(t)}\|_2^2$$

(כלומר אם הנקודה i שייכה לאשכול j). נבחין, כיוון שלכל נקודה x_i בנפרד בחרנו j שממזער את $\|x_i - \mu_j^{(t)}\|_2^2$, הסכום הכולל תחת המטריצה החדשה $r^{(t+1)}$ חייב להיות קטן או שווה לסכום שהתקבל תחת המטריצה הקודמת $r^{(t)}$ (שכן רק יכולנו להקטין). כלומר,

$$J(\mu^{(t)}, r^{(t+1)}) \leq J(\mu^{(t)}, r^{(t)})$$

למת עזר. הפרמטרים $\mu_j^{(t+1)}$ המקיימים את חוק הממוצע של האלגוריתם הוא μ שממזער באופן גלובלי את J ביחס ל- $r^{(t+1)}$ הקבוע. **הוכחה:** נוכיח באמצעות גזירה.

$$\frac{\partial J(\mu^{(t)}, r^{(t+1)})}{\partial \mu_j} = \frac{\partial}{\partial \mu_j} \left[\sum_{i=1}^n \sum_{k=1}^K r_{ik}^{(t+1)} \|x_i - \mu_k^{(t)}\|_2^2 \right]$$

הנגזרת לפי μ_j , לכן כל $k \neq j$ מתאפסים ונותרים רק עם איבר זה. כלומר,

$$\sum_{i=1}^n r_{ij}^{(t+1)} \frac{\partial}{\partial \mu_j} \|x_i - \mu_j^{(t)}\|_2^2$$

לפי כלל גזירה של נורמה מתקיים

$$\nabla_y \|x - y\|^2 = 2(y - x)$$

לכן נקבל כי:

$$\sum_{i=1}^n r_{ij}^{(t+1)} \cdot 2(\mu_j^{(t)} - x_i) = 0$$

נשווה לאפס למצוא קיצון. נקבל,

$$\mu_j^{(t)} \sum_{i=1}^n r_{ij}^{(t+1)} - \sum_{i=1}^n r_{ij}^{(t+1)} x_i = 0 \implies \mu_j^{(t)} = \frac{\sum_{i=1}^n r_{ij}^{(t+1)} x_i}{\sum_{i=1}^n r_{ij}^{(t+1)}}$$

לכן, כמסקנה מהלפה, נבחין שקיבלנו בדיוק את כלל העדכון (ממוצע חשבוני על פני הנקודות ששייכות לאשכול j). כעת זה גורר כי החלפת הפרמטרים ל- $\mu^{(t+1)}$ הרי הקטינה את הפונקציה (או לא שינתה). לכן,

$$J(\mu^{(t+1)}, r^{(t+1)}) \leq J(\mu^{(t)}, r^{(t+1)})$$

מאי השוויון שראינו קודם נקבל כי:

$$J(\mu^{(t+1)}, r^{(t+1)}) \leq J(\mu^{(t)}, r^{(t)})$$

והרי ש $J \geq 0$ בהכרח כי סכום אי שליליים. מס' הדרכים השונות לשייך n נקודות ל- K אשכולות הוא K^n , כיוון שערך הפונקציה J קטן (או נשאר זהה) בכל צעד - האלגוריתם לא יכול לבצע מחזוריות ולחזור למטריצת שיוך r שכבר ביקר בה בעבר, כיוון שמס' המצבים הוא סופי (K^n) סדרת הערכים המונוטונית חייבת להיעצר ולכן האלגוריתם מתכנס.

■

כעת נעיר כמה הערות על האלגוריתם.

- אין הבטחה לאופטימטוס גלובלי - האלגוריתם מחייב התכנסות אך היא מובטחת אך ורק למינימום מקומי ולא דווקא לגלובלי (הכי טוב במרחב).
- רגישות גבוהה לאתחול - המינימום המקומי הסופי אליו יתכנס האלגוריתם תלוי בצורה קיצונית בבחירת המרכזיים הראשוניים. פתרון מקובל לכך הוא הרצה חוזרת ונשנית של האלגוריתם עם אתחולים שונים והחזרת הריצה שקיבלה את ערך ה- J הקטן ביותר.
- **דרך נוספת לאתחול - חלוקה אקראית:**
 1. בשלב הראשון אנחנו עוברים על כל נקודה במדגם ומשייכים אותה בצורה אקראית לחלוטין לאחד מ- K האשכולות.
 2. מיד לאחר השיוך - מדלגים על שלב העדכון של האלגוריתם. מחשבים את הממוצע הגאומטרי של הנקודות בכל קבוצה, ותוצאה זו היא שקובעת את מיקום המרכזים הראשוניים.
- (כלומר, קודם מחלקים לאשכולות בצורה אקראית. ואז לכל אשכול מחשבים ממוצע - זה הממוצע הראשוני של האשכול. ואז מריצים את האלגוריתם).
- האלגוריתם מוגבל מטבעו למזעור מרחקים אוקלידיים בריבוע. לכן - זה "אונס" את האשכול להיות בעלות מבני כדורי או סימטרי.
- האלגוריתם יכשל בדוגמאות עם מחלקות כמו חצאי ירחים.
- האלגוריתם רגיש מאוד לנקודות קיצוניות - שכן מחשבים ממוצע, ונקודה אחת חריגה מאוד יכולה להזיז את μ_j .
- קללת המימד: האלגוריתם לא אידיאלי עבור מרחבי מידע מעל מימד d כאשר d גדול, שכן במרחבים כאשר d גבוה מושג המרחק האוקלידי L_2 מתחיל לאבד ממשמעותו (המרחקים בין זוג נקודות במרחב הופכים לזהים) מה שמקשה על הבחנה גאומטרית מובהקת.
- פונקציית ההפסד מנוסחת כך:

$$\text{cost}(z_1, \dots, z_k, C_1, \dots, C_k) = \min_{C_1, \dots, C_k} \sum_{j=1}^k \sum_{x_i \in X_j} \|x_i - z_j\|_2^2$$

- הקשר בין בחירת K לפונקציית ההפסד: בכל פעם שאנחנו מגדילים את K אנחנו נותנים לנקודות אפשרויות בחירה קרובות יותר. המרחקים $\|x_i - z_j\|_2^2$ בהכרח יקטנו או ישארו זהים. אם $K = n$ כמס' הנקודות, האלגוריתם ישים ממוצע לבסוף על כל נקודה ונקודה. במצב זה המרחק של כל נקודה מהמרכז שלה יהיה 0 ופונקציית המחיר הכוללת תהיה 0 בדיוק. המסקנה שנובעת מכך - חלוקה שבה כל נקודה היא אשכול בפני עצמו היא חסרת תועלת לחלוטין. לכן המטרה שלנו היא לא למצוא את Costn המינימלי אלא למצוא נקודת איזון. כלומר ככל ש- K עולה ההפסד יורד, אך יכולת ההכללה נהרסת.

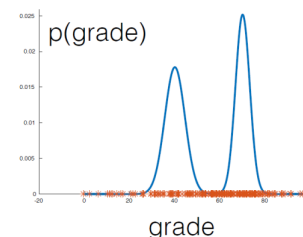


- יש לקבוע את K מראש שכן הוא היפר פרמטר - האלגוריתם לא באמת יודע לגלות באופן אוטונומי כמה אשכולות ישנן.

לכן, על אף היותו נחמד מאוד - האלגוריתם לא מוצא מבנים בדטה. אלא כופה עליהם להיות בצורה שהוא מעדיף - כדורים.

9.4 Gaussian Mixture Model (GMM) (מודל תערובת גאוסיאנים)

ננסה לתת קצת אינטואיציה. נניח מבחן באקדמיה - בקורס למידת מכונה. קיבלנו את הגרף הזה. זה אמנם לא גאוסיאן - אך נראה שניתן לסווג את הסטודנטים לשני גאוסיאנים - אחד שמתפלג סביב 40: סטודנטים שלא הבינו את החומר, ואחד שמתפלג סביב 70: סטודנטים שיותר הבינו את החומר. אם נדע לקחת את זוג המודלים הללו - לתת לכל אחד מהם משקל, ממוצע ושונות משל עצמו ונסכום אותם - נקבל מודל הסתברותי שמתאר את כלל האוכלוסייה. כלומר, נקח תתי קבוצות ומהם נבנה מודל.



כעת נפרמל זאת. נניח כי $\mathcal{D} = \{x_1, \dots, x_n\}$ הדטה-סט שלנו הם תוצר של תהליך אקראי מובנה כך $x_i \in \mathbb{R}^d$. המידע נוצר מתוך תערובת של K התפלגויות גאוסיאניות רב ממדיות שונות ומובחנות (לכל אשכול k ישנה זהות גאוסיאנית משלו). ההסתברות לראות נקודה x_i מוגדר להיות:

$$\Pr(x_i | \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

באשר: $\theta = (\pi, \mu, \Sigma)$

1. π - משקולות התערובת. ההסתברות האפריורית שנקודה תשויך לאשכול k . מתקיים:

$$\sum_{k=1}^K \pi_k = 1, \pi_k \in [0, 1]$$

2. μ - וקטורי הממוצעים - מרכז הגאוסיאן ה- k

3. מטריצת השונות המשותפת Σ_k

מטרתנו כעת היא לא למזער מרחקים, אלא למזער את נראות הנתונים. נחפש את סט הפרמטרים θ^* שיגרום לדטה הקיים שלנו להיות הכי הגיוני והכי סביר. לכן נמקסם את פונקציית הלוג-נראות תחת ההנחה שהדגימות $i.i.d$ נקבל:

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \right)$$

9.4.1 אלגוריתם EM

פונקציית המטרה אינה קמורה ולא ניתנת לפתרון אנליטי ישיר. לכן משתמשים בשיטה דומה ל-GD. בכל איטרציה ישנם שני שלבים מרכזיים:
1. E-Step: הפרמטרים של הגאוסיאן קבועים. כלומר $\theta^{(t)} = (\pi^{(t)}, \mu^{(t)}, \Sigma^{(t)})$. המטרה היא לחשב לכל נקודה x_i ואשכול k את ה"אחריות" שתחושב:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

למעשה זה בדיקת ההסתברות הפוסטריורית - בהינתן שראינו את x_i מה הסיכוי שרכיב הגאוסיאן ה- k יצר אותה.
2. M-Step: בשלב זה האחריות γ_{ik} שחישבנו בשלב הקודם מקובעות. מעדכנים כעת את פרמטרי המודל $\theta^{(t+1)}$ על מנת למקסם את הנראות:
א. מעדכנים את הממוצעים

$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} x_i}{\sum_{i=1}^n \gamma_{ik}}$$

כלומר הממוצע של הגאוסיאן ה- k (המחלקה שלו) יהיה הממוצע המשוקלל של כל הנקודות בדטה.
ב. עדכון מטריצת השונות המשותפת:

$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} (x_i - \mu_k^{(t+1)})(x_i - \mu_k^{(t+1)})^T}{\sum_{i=1}^n \gamma_{ik}}$$

ג. עדכון משקלי התערובת -

$$\pi_k^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}$$

המשקל האפריורי החדש של האשכול הוא פשוט סך כל ה"אחריות" שרכיב k לקח על עצמו מתוך n הנקודות הקיימות. האלגוריתם חוזר חלילה על שלבים אלו - עד שפרמטרי המודל או ערך הלוג-נראות מפסיקים להשתנות.

השוואה בין האלגוריתמים שראינו:

FEATURE	K-MEANS	GAUSSIAN MIXTURE (GMM)
Assignment	Hard (Binary {0,1})	Soft (Probabilistic γ_{ik})
Geometry	Spherical (Isotropic)	Elliptical (Anisotropic)
Parameters	Centroids (μ) only	μ , Covariance (Σ), Weights (π)
Algorithm	Lloyd's (Distance Minimization)	EM (Likelihood Maximization)
Flexibility	Rigid boundaries (Voronoi)	Adaptable to overlap & variance
Complexity (per iteration)	$O(n \cdot K \cdot d)$	$O(n \cdot K \cdot d^3)$

9.5 Principal Component Analysis (PCA)

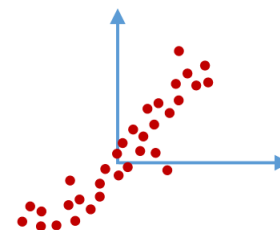
נחזור לשאלה ששאלנו קודם - האם בהינתן אוסף נתונים ללא תוויות יש דרך מתמטית לאסוף את המאפיינים של הדטה? אנחנו נשתמש ונציג את PCA - כלי שמחפש ייצוג כנ"ל בדטה. נבחין שבהינתן דטה ממימד גבוה (הרבה פיצ'רים) - התנודות וההתנהגויות של הדטה נובע ממס' קטן הרבה יותר של גורמים סמויים פיצ'רים שמסווגים אותו לקבוצות. למשל: מתוך 500 פיצ'רים למצוא קבוצה של 10 - 15 פיצ'רים שיסבירו את הנתונים.

למה שנרצה להוריד את המימד של הדטה?

1. דחיסה - הורדת המידע והמימד של הדטה מאפשרת לייצג את אותו המידע באמצעות הרבה פחות משתנים, מה שחוסך מקום ומייעל עבודה עם מטריצות.
2. ניקוי רעשים - דטה לרוב מגיע עם רעש, בעת הקטנת המימד אנחנו נפטרים מחלק גדול מהרעש.
3. ויזואליזציה - כבני אדם, קשה לנו לדמיין את $\mathbb{R}^d$. לעומת זאת - קל לנו לדמיין את $\mathbb{R}^2$ למשל - וזה יאפשר לנו לצייר את הנתונים על גרף.
4. מודלים מהירים יותר - יאפשר לאלגוריתמים של למידת מכונה לרוץ מהר יותר על הדטה המצומצם.

בכל פעם, לפני שנרץ PCA אנחנו ננרמל את הדטה. כלומר, $\hat{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$ (באשר j הוא אינדקס המאפיין i היא הדגימה). לכן, מעתה ואילך אנחנו נניח כי הממוצע של הדטה הוא 0 והשונות היא 1.

נביט בדוגמה למען המוטביציה: בגרף לעיל אנחנו רואים ענן נקודות ב $\mathbb{R}^2$. נבחין כי שני המשתנים נמצאים בקורלציה די גבוהה אחד עם השני - וכן הנתונים מפוזרים לאורך קו אחד מתוח, לכן מתנהגים הרבה יותר כמו קו ישר (מימד 1) מאשר כמו ענן שמתפרש במישור (מימד 2). לכן - ניתן להוריד את המימד של הדטה כך נמנע כפילות מיותרת של נתונים, מעבר למימד אחד יגרום אמנם לאובדן מידע אך זה יהיה שולי מאוד ביחס לתועלת - ייצוג חסכוני של הדטה. אנחנו נוריד את המימד - והפיצ'רים שישנם בדטה הם אלו שנותרו לאחר הורדת המימד.



כלומר - אמנם נאבד מידע, אך המשמעות בהקשר של PCA היא שאנחנו לא מחפשים מודל שיתאר במאה אחוזים את כל הניואנסים והתנודות הקטנות של המידע המקורי. המטרה שלנו היא לתפוס את ההיבטים החשובים ביותר של הנתונים, ולא את כל הפרטים שלהם.

הגדרת הבעיה: בהינתן מדגם נתונים $X = \{x_i\}_{i=1}^n$ כך ש $x_i \in \mathbb{R}^d$ המטרה שלנו היא לקרב את הוקטורים הללו באמצעות מרחב בעל מימד נמוך יותר $r < d$.

הגדרה 116. תת מרחב לינארי מדרגה r נפרס ע"י בסיס של r וקטורי עמודה $v_1, \dots, v_r, v_i \in \mathbb{R}^d$. אנחנו נניח כי:

1. הבסיס של תת המרחב הוא בסיס אורתונורמלי (הוקטורים מאונכים זה לזה ומנורמלים לגודל יחידה) - שכן תמיד ניתן לבצע גראם-שמידט ולקבל בסיס אורתונורמלי.
2. נסמן $V \in \mathbb{R}^{d \times r}$ להיות מטריצה כך ש: $C_i(V) = v_i$ (מטריצת וקטורי הבסיס). קל להוכיח כי $V^T V = I_{r \times r}$ (שכן מדובר בבסיס א"נ). תת המרחב עצמו הוא Span (המרחב הנפרש) של וקטורים אלו. כלומר -

$$\text{span}\{v_1, \dots, v_r\} = \sum_j \alpha_j v_j$$

בהינתן $z \in \text{span}\{v_1, \dots, v_r\}$ הוא מיוצג ע"י צ"ל של איברי הבסיס. נסמן $a = (a_1, \dots, a_r)^T$ והרי:

$$z = \sum_i a_i v_i = Va$$

המשמעות - אמנם הוקטור המקורי חי במימד d הגבוה, אך בהינתן שמתארים אותו באמצעות וקטור המקדמים a אנו צריכים רק r מספרים לדעת היכן הוא בתת המרחב.

תהליך הקידוד - לוקח וקטור ממימד גבוה ומכווץ אותו לוקטור ממימד נמוך. כקלט מקבלים $x \in \mathbb{R}^d$, הפלט a הוא וקטור המקדמים המקיים $a \in \mathbb{R}^r$. מתמטית:

$$a = V^T x$$

(כלומר כופלים את הוקטור x במטריצת הוקטורים). נבחין כי $V^T V = I$ ולכן $V^T = V^{-1}$. מהמשוואה קודם ראינו כי $x = Va$ וע"י הכפלה משמאל בהופכית אכן מקבלים כי $a = V^T x$.

תהליך הפענוח - לוקחים את הייצוג הדחוס ומנסים לשחזר מתוכו את הוקטורי המקורי במימד הגבוה. כלומר כקלט מקבלים את הוקטור a וכפלט את x הוקטור במרחב הגדול. מתמטית:

$$x = Va$$

מה קורה כשנקודה x לא נמצאת בתת המרחב, וכיצד נמצא את הקירוב הטוב ביותר שלה? כיוון ש V מורכב מבסיס וקטורים אורתונורמליים אזי הנקודה על תת המרחב שהיא הכי קרובה לדגימה המקורית x היא בדיוק: $a = V^T x$. במקרה זה, כל רכיב $a_i \in a$ הוא פשוט ההטלה הסקלרית של הוקטור x על הרכיב i . דהיינו,

$$a_i = v_i^T x$$

משפט 117. $VV^T x = x$ אפי"פ $x \in \text{Span}(V)$

נבחין: $VV^T \neq I, \in \mathbb{R}^{d \times d}$ אך $V^T V = I \in \mathbb{R}^{r \times r}$. אם הדגימה המקורית לא שייכת לתת המרחב - $x \notin \text{Span}(V)$, ואנחנו מקודדים באמצעות המטריצה V , הוקטור שמתקבל יסומן ב' x' ויתקיים:

$$x' = VV^T x$$

כיוון ש $x \notin \text{Span}(V)$, ממשפט קודם זה גורר כי $x' \neq x$. ההפרש בין הוקטור המקורי לוקטור החדש $x - x'$ מייצג את וקטור השגיאה שהטמענו במהלך הורדת המימד.

אם נסכם - הורדת המרחב זה פשוט למצוא תת מרחב לינארי (קו, מישור וכו') שעובר הכי טוב בתוך ענן הנקודות שלנו והשגיאה $(x - x')$ לכל קלט תהיה קטנה ככל הניתן (עבור $x \notin \text{Span}(V)$). אנחנו רוצים למצוא בסיס V אורתונורמלי כך שהטלת הנתונים עליו תגרום למינימום עיוות המידע, פורמלית:

$$\min_{V: V^T V = I} \sum_i \|VV^T x_i - x_i\|_2^2$$

הגדרה 118. בהינתן מטריצה $X \in \mathbb{R}^{n \times d}$ כך שכל שורה x_i היא וקטור עמודה במקור. נראה כי:

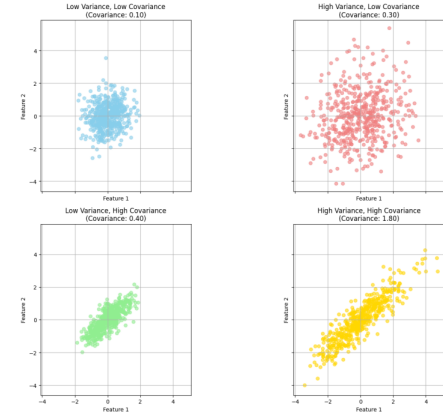
$$X^T X = \begin{pmatrix} | & | & | \\ x_1 & \dots & x_n \\ | & | & | \end{pmatrix} \begin{pmatrix} - & x_1 & - \\ - & \dots & - \\ - & x_n & - \end{pmatrix} = \sum_{i=1}^n x_i x_i^T$$

נגדיר את מטריצת השונות של X להיות $Cov(X) = \frac{1}{n} X^T X \in \mathbb{R}^{d \times d}$. מתקיים כי $Cov(X)_{ij}$ - כמה המשתנים x_i, x_j בקורולציה זה עם זה.

מטריצה זו $(Cov(X))$ היא אומדן לערך התוחלת $Cov(X) = \mathbb{E}_{x \sim \mathcal{D}}[xx^T]$ - כיוון שאין לנו גישה להתפלגות האמיתית, אנחנו נשתמש בממוצע המדגם האריתמטי.

ישנם 4 מקרים אפשריים:

1. שונות נמוכה, שונות משותפת נמוכה (תכלת) - הנקודות צפופות וקרובות למרכז (שונות נמוכה), הקשר האלכסוני בין המשתנים (שונות משותפת) נמוכה גם כן - ישנו ענן עגול. ידיעת משתנה אחד לא עוזרת לדעת את השני.
2. שונות גבוהה, שונות משותפת נמוכה (אדום) - הנקודות מפוזרות במרחב, אך עדיין אין קשר אלכסוני בין המשתנים.
3. שונות גבוהה, שונות משותפת גבוהה (צהוב) - גם קשר אלכסוני גבוה וגם נקודות מפוזרות במרחב.
4. שונות נמוכה, שונות משותפת גבוהה - נקודות צפופות, קשר אלכסוני.



כעת נוסיף לתזכר על אלגברה לינארית:

1. $Cov(X)$ היא מטריצה סימטרית.
2. לכל וקטור v מתקיים $v^T Cov(X) v \geq 0$, לכן היא PSD וכל הע"ע שלה אי שליליים.
3. הוקטורים העצמיים שלה $u_1, \dots, u_d$ מאונכים זה לזה ובעלי גודל 1 (אורתונורמליים).
4. באמצעות תכונות $(1 - 3)$, ניתן לפרק את המטריצה באופן הבא:

$$Cov(X) = U \Lambda U^T$$

כך U מטריצה אורתוגונלית ($U^T U = I$) בה העמודות הם הוקטורים העצמיים $\{u_i\}_{i=1}^d$ ו- Λ היא מטריצה אלכסונית שמכילה את הע"ע מהגדול ביותר לקטן.

משפט 119. הפתרון לבעיית האופטימיזציה של שגיאת השחזור:

$$\min_{V: V^T V = I} \sum_i \|V V^T x_i - x_i\|_2^2$$

מתקבל ע"י קביעת וקטורי הבסיס של V להיות הוקטורים העצמיים של מטריצת השונות המשותפת $Cov(X)$. כלומר, $v_i = u_i$ לכל $1 \leq i \leq r$.
הוכחה: אנחנו נוכיח את המשפט למקרה ספציפי בלבד. כאשר $r = 1$ - כלומר הורדת מימד לקו ישר אחד. המטריצה V כעת היא וקטור $v \in \mathbb{R}^d$ ותנאי האורתונורמליות $V^T V = I$ הופך לדרישה כי $v^T v = 1$, כלומר $\|v\|_2^2 = 1$. נציב בפונקציית המטרה:

$$\arg \min_{v \in \mathbb{R}^d: \|v\|_2^2=1} \sum_i (x_i - v v^T x_i)^T (x_i - v v^T x_i) =$$

$$\arg \min_{v \in \mathbb{R}^d: \|v\|_2^2=1} \sum_i x_i^T x_i - 2x_i^T v v^T x_i + x_i^T v v^T v v^T x_i$$

אנו יודעים כי $v^T v = 1$ ולכן $x_i^T v v^T v v^T x_i = x_i^T v v^T x_i$, כמו כן נבחין כי הביטוי $x_i^T x_i$ לא תלוי ב- v ולכן נקבל:

$$\arg \min_{v \in \mathbb{R}^d: \|v\|_2^2=1} \sum_i -x_i^T v v^T x_i$$

נבחין כעת כי ניתן לשנות את סדר המכפלה ל- $-v^T x_i x_i^T v$ (מדוע? $x_i^T v$ הוא סקלר וגם $v^T x_i$ הוא סקלר - ניתן להחליף את הסדר בניהם).

$$\arg \min_{v \in \mathbb{R}^d: \|v\|_2^2=1} \sum_i -v^T x_i x_i^T v$$

כעת, במקום למזער את הביטוי עם המינימום, נרצה למקסם את הביטוי ללא מינוס. נקבל

$$\arg \max_{v \in \mathbb{R}^d: \|v\|_2=1} v^T \left(\sum_i x_i x_i^T \right) v = \arg \max_{v \in \mathbb{R}^d: \|v\|_2=1} v^T \text{Cov}(X) v$$

נסמן $\text{Cov}(X) = C$ וזו מטריצה שלפי מה שראינו קיים לה פירוק ספקטרלי $C = U \Lambda U^T$ כאשר עמודות U הם הוקטורים העצמיים האורתונורמליים $u_1, \dots, u_d$. נוכל לייצג את v באמצעות וקטורים אלו $v = \sum_j \alpha_j u_j$. הרי שצריך להתקיים כי:

$$\|v\|_2^2 = 1 \implies v^T v = 1 \implies \sum_j \alpha_j^2 u_j^2 = 1 \implies \sum_j \alpha_j^2 = 1$$

נבחין כי אם נכפול את U ב v^T נקבל בדיוק את וקטור העצמים $v^T U = \alpha^T = [\alpha_1, \dots, \alpha_d]$. נציב ונקבל:

$$v^T C v = v^T U \Lambda U^T v = (v^T U) \Lambda (v^T U)^T = \alpha^T \Lambda \alpha$$

כיוון ש Λ מטריצה אלכסונית שמכילה ע"ע על האלכסון כפל שלה משני צדדיה בוקטור מניב:

$$= \sum_j \lambda_j \alpha_j^2$$

כלומר, כעת במקום לחפש וקטורים v אנחנו מחפשים מקדמים שימקסמו סכום:

$$\arg \max_{\alpha} \sum_j \lambda_j \alpha_j^2 \text{ s.t.: } \sum_j \alpha_j^2 = 1$$

נבחין שיש לנו סכום כולל של אחד. כיוון שהע"ע ממוינים מהגדול לקטן, λ_1 הוא הגדול ביותר. הדרך היחידה למקסם את הסכום היא למקסם את המקדם שלו. לכן נרצה כי $\alpha_1 = 1$ וזה גורר כי $\alpha_2 = \alpha_3 = \dots = \alpha_d = 0$. מכאן נקבל כי:

$$v = \sum_j \alpha_j u_j = 1 \cdot u_1 + 0 \cdot u_2 + \dots = u_1$$

כלומר הוקטור שמקסם את שגיאת השחזור הוא הוקטור u_1 - הוקטור העצמי של מטריצת הקובריאנס המשווית לערך העצמי הגדול ביותר. כנדרש. ■

כעת נסביר כיצד מבצעים הורדת מימד עם PCA -
1. עבור כל דגימה x אנחנו מבצעים:

$$a = V^T x$$

כאשר $V = [u_1, \dots, u_r]$ (עמודותיהם הם הוקטורים העצמיים של $\text{Cov}(X)$).
 a כפלט הוא וקטור החדש במימד הנמוך.

2. כמה איבדנו בהפחתת המימד?

$$\frac{1}{n-1} \sum_i \|V V^T x_i - x_i\|_2^2 = \sum_{j=r+1}^d \lambda_j$$

(מחלקים ב $n-1$ כי זה דרגות החופש של המדגם).

מדוע זה שווה לסכום הערכים העצמיים החל מאינדקס $r+1$ ועד אינדקס d ? אלו בדיוק הערכים העצמיים השייכים לציר (לוקטורים העצמיים) שנותרו בחוץ ולא נכנסו למטריצה V ! (נזכיר - אנחנו לוקחים את הגדולים ביותר בערכם).
לכן: השגיאה הכוללת שאנחנו מבצעים שווה ממש לסכום הערכים העצמיים שלא לקחנו.

3. צריך לבחור את r - כדי שהורדת המימד תהיה אפקטיבית נרצה כי r יהיה קטן ככל הניתן. מצד שני, נרצה לדאוג גם כי סכום הע"ע שלא לקחנו $\sum_{j=r+1}^d \lambda_j$ יהיה קטן ככל הניתן. הרי שאם נרצה r קטן - זה אוטומטית מגדיל את השגיאה. ולכן יש כאן טריידאוף בין הרצון להקטין את r לבין כמה שגיאה נקבל.

נבחין: הפתרון של הבעיה אנליטי לחלוטין, אין כאן תהליך איטרטיבי כמו GD . מחשבים מטריצה, מוצאים לה ערכים עצמיים ווקטורים עצמיים, ולוקחים בסיס של r וקטורים עצמיים שמתאימים ל r הערכים העצמיים הגדולים ביותר. נבחין - הדבר היחיד שיש לעשות שהוא לא דטרמיניסטי זה לקבוע את r .

משפט 120. המרחב החדש שנוצר ע"י PCA הוא מרחב עם ביטול מתאם - המשתנים החדשים (עמודות A) ענותקים לחלוטין מבחינה לינארית. השונות המשותפת ביניהם היא בדיוק אפס.

הוכחה: מטריצת הנתונים המקורית X בגודל $n \times d$, V היא מטריצת ההטלה בגודל $d \times r$. הרי $A = XV$ היא מטריצה בגודל $n \times r$ - יש בה n שורות, אותם דגימות, אך r עמודות בלבד (המאפיינים החדשים). ראשית נבחין כי הממוצע של הנתונים ב A עוזנו אפס -

$$\sum_i a_i = V^T \sum_i x_i = V^T 0 = 0$$

וכן, המאפיינים החדשים בלתי מתואמים זה עם זה. נרצה לחשב את מטריצת $Cov(A)$. הרי שהנוסחה אליה היא המכפלה $A^T A$. נפתח ונקבל:

$$A^T A = (XV)^T (XV) = V^T X^T X V = V^T Cov(X) V =_{Cov(X)=V \Lambda V^T} V^T V \Lambda V^T V = I \Lambda I = \Lambda$$

כלומר מטריצת Cov אלכסונית - בפרט לכל $i \neq j$, $Cov(a_i, a_j) = 0$, ולכן בין איברים שונים אין מתאם לינארי בין העמודות. כל עמודה חדשה שקיבלנו במטריצה A היא מאפיין חדש לחלוטין ש"גילינו" מתוך המבנה של הדאטה.

משפט 121. באופן שקול, PCA היא לא רק מזעור שגיאת השחזור אלא מקסום השונות של הנתונים המוטלים. הוכחה: נרצה להוכיח כי

$$\mathcal{L} = \arg \min_{V: x'_i = V V^T x_i} \sum_i \|x'_i - x_i\|^2 = \arg \max_{V: a = V^T x} \sum_{j=1}^r var(a_j)$$

נראה כי:

$$var(a_j) = \frac{1}{n} \sum_i (a_{ij} - \bar{a}_j)^2 =_{\bar{a}_j=0} \frac{1}{n} \sum_i a_{ij}^2$$

והרי ש:

$$\|x'_i - x_i\|^2 = \|x'_i\|^2 + \|x_i\|^2 - 2x_i'^T x_i$$

הרי ש $x'_i = V a_i$ ונקבל:

$$\|x'_i\|^2 = \|V a_i\|^2 = (V a_i)^T V a_i = a_i^T V^T V a_i = \|a_i\|^2$$

$$\|x_i\|^2 = const$$

$$-2x_i'^T V V^T x_i =_{x_i^T V = a_i^T} -2\|a_i\|^2$$

לכן נקבל:

$$\|x'_i - x_i\|^2 = \text{const} - \|a_i\|^2$$

כלומר הפונקציה אותה נרצה למזער היא:

$$\arg \min_{V: x'_i = VV^T x_i} \sum_i \|x'_i - x_i\|^2 = -(\sum_i a_{ij}^2) + \text{const}$$

בגלל שיש מינוס, זה שקול למקסם את $\sum_i a_{ij}^2$. וכפי שראינו זה בדיוק השוואת.

■

הערה 122. כל וקטור a כנ"ל מייצג עמודה חדשה במטריצה A . כלומר $a = V^T x$ (דגימה חדשה שהייתה x במקור ועברה להיות באורך r).

משפט 123. ערך פונקציית ההפסד של פתרון PCA ממימד r כאשר $V \in \mathbb{R}^{r \times d}$ היא מטריצת פתרון ה- PCA , מקיים:

$$\text{Loss}(r) = \frac{1}{n-1} \sum_{i=1}^n \|VV^T x_i - x_i\|_2^2 = \sum_{j=r+1}^d \lambda_j$$

כלומר, סכום הערכים העצמיים שלא לקחנו.

הוכחה:

הערה. ההוכחה לא נראתה בהרצאה.

נפתח את הנורמה:

$$\|VV^T x_i - x_i\|_2^2 = (VV^T x_i - x_i)^T (VV^T x_i - x_i) = x_i^T VV^T VV^T x_i - 2x_i^T VV^T x_i + x_i^T x_i$$

$$= x_i^T VV^T x_i - 2x_i^T VV^T x_i + x_i^T x_i = x_i^T x_i - x_i^T VV^T x_i$$

לכן:

$$\text{Loss}(r) = \frac{1}{n-1} \left(\sum_{i=1}^n x_i^T x_i - \sum_{i=1}^n x_i^T VV^T x_i \right)$$

נתבונן על האיבר השני בסכום. נשים לב שהוא סקלר, וניתן להכניסו לתוך Trace . כמו כן, Trace חילופי וע"י הכפלה ב- V, V^T מצדדיו לא נשנה אותו ($V^T V = I$). לכן:

$$\frac{1}{n-1} \sum_{i=1}^n x_i^T VV^T x_i = \text{Tr} \left(V^T \left(\frac{1}{n-1} \sum_{i=1}^n x_i^T x_i \right) V \right) = \text{Tr}(V^T C V)$$

כאשר $C = \text{Cov}(X)$. נסמן $C = U \Lambda U^T$ כפירוק הספקטרלי ונקבל:

$$\text{Tr}(V^T C V) = \text{Tr}(V^T U \Lambda U^T V) = \text{Tr}(\Lambda) = \sum_{j=1}^r \lambda_j$$

שכן האיברים מתבטלים כי Trace הוא חילופי, וכן הטרייס של מטריצה אלכסונית הוא סכום איבריה. באופן דומה נקבל כי $\text{Tr}(\frac{1}{n-1} \sum_{i=1}^n x_i^T x_i) = \sum_{i=1}^d \lambda_i$ לכן סה"כ

$$\text{Loss}(r) = \sum_{i=1}^d \lambda_i - \sum_{j=1}^r \lambda_j = \sum_{i=r+1}^d \lambda_i$$

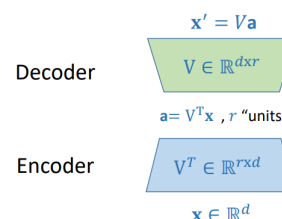
9.6 Autoencoder

נרצה לקשר את PCA לעולם של Deep learning. קידוד עצמי - הוא קונספט בו רשת ניורונים ומטרתה היא לקחת קלט, לדחוס אותו, ואז לשחזר אותו מתוך הדחיסה.

למעשה, PCA הוא Autoencoder לינארי.

ישנם שני שלבים:

1. המקודד - Encoder לוקח את הקלט המקורי x במימד הגבוה, כופל אותו במטריצה V^T ומקבל וקטור דחוס $a = V^T x$ שמכיל r יחידות.
2. המפענח - Decoder, לוקח את הייצוג הדחוס a , כופל אותו במטריצה V ומקבל את הוקטור המשוחזר $x' = Va = VV^T x$. פונקציית Loss של הרשת היא אותה פונקציה מ-PCA: $\|x' - x\|_2^2$. רשת הניורונים הנ"ל מבצעת פעולה לינארית.



מה יקרה אם $r > d$?

כלומר - יש לרשת תקציב מימדים גדול מהמקור. היא תבחר מטריצה כלשהי שתקיים $V^T V = I$ (למשל $V = I$) ואז השחזור יהיה מושלם $x' = x$ לכל x והטעות תהיה אפס. זהו מקרה שלא מעניין. על מנת להפוך מקרה זה לשימושי נרצה להוסיף רגולרזציה.

$$\mathcal{L} = \arg \min_{x' = VV^T x} \|x' - x\|_2^2 + \lambda \|a\|_2^2$$

האיבר הנ"ל קונס את הרשת על גודל המקדמים בתוך הייצוג הפנימי. כעת הרשת גם מנסה לשחזר טוב את x (שמאל) וגם לשמור על ערכי a קטנים יחסית.

9.7 תרגול - PCA

נציג את האלגוריתם בפסאודו קוד מפורט.

תהי $Cov(X) = U\Lambda U^T$ מטריצת Cov עבור הנתונים $\{x_i\}_{i=1}^n$. כל דגימה $x_i \in \mathbb{R}^d$ ולכן $X \in \mathbb{R}^{n \times d}$. נבחין כי Cov היא מטריצה סימטרית ולכן היא לכסינה אוניטרית. לכן ניתן לפרק אותה. U היא מטריצה עם הוקטורים העצמיים $\{u_1, \dots, u_d\}$ המתאימים לע"ע $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. נגדיר את הפסאודו באופן הבא:

function PCA(X)

1. חשב את ממוצע המדגם ואת מטריצת $Cov(X)$:

$$\mu_x = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\Sigma_x = \frac{1}{n-1} \sum_{i=1}^n (x_i - \mu_x)(x_i - \mu_x)^T$$

2. לכסן את המטריצה Σ_x וחשב את המטריצות U, Λ . המטריצה Λ היא מטריצה שבה $r < d$ ע"ע $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r$ (אלכסונית) ו- U היא מטריצה עם הוקטורים העצמיים התואמים.

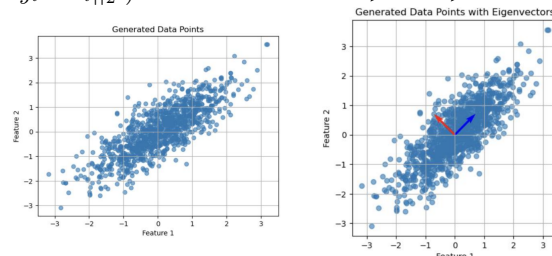
3. לכל נתון x_i ($1 \leq i \leq n$) חשב את $y_i = V^T x_i$ (הנקודה במימד r).

4. החזר את $Y = (y_1, \dots, y_n)$.

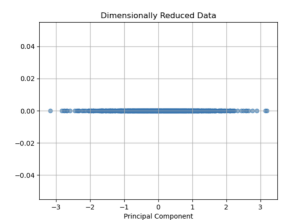
נבחין כי זה אלגוריתם שתואם להוכחות המתמטיות שראינו - הרצון למזער את מרחק השחזור הריבועי. וכן כדי לאבד כמה שפחות מידע, נרצה שהנקודות החדשות y_i יהיו מפוזרות ככל הניתן בתת-המרחב החדש - כלומר, נרצה למקסם את השונות שלהן.

נתבונן בדוגמה על $\mathbb{R}^2$. בגרף משמאל - ישנו מרחב דו מימדי עם זוג מאפיינים $(F1, F2)$. ניתן לראות שמרכז הנתונים הוא סביב האפס $(0, 0)$. כמו כן - יש מתאם (קורלציה) חיובית בין זוג המאפיינים. ככל שאחד עולה, גם השני נוטה לעלות. הגרף מימין - אותו הגרף שבשמאל. ישנם זוג וקטורים: אלו הוקטורים העצמיים של מטריצה $Cov(X)$. החץ הכחול - הוא הוקטור העצמי הראשי ששייך לערך העצמי הגדול ביותר. זהו הכיוון בו שונות הנתונים מקסימלית. החץ האדום - הוא הוקטור העצמי השני שמתאים לערך עצמי הקטן יותר. הוא אורתוגונלי לוקטור העצמי הראשי.

השני. אם נרצה להוריד את מימד הנתונים מ-2 ל-1, נבחר בוקטור העצמי המתאים לערך העצמי הגדול יותר - הכחול. הטלת הנתונים עליו תמזער לנו את שגיאת השחזור ($\|Vy_i - x_i\|_2^2$).



כעת, כיצד נראה את הורדת המימד ל-1? שלב האלגוריתם שבו מבצעים $y_i = V^T x_i$ כעת V מכילה רק את הוקטור העצמי הכחול. נבחין כי קיבלנו נתונים במימד אחד בלבד. קל לראות שהנתונים החדשים שמרו על השונות והמתיחה המקורית שהיו לנתונים בגרף המקורי.



9.8 סיווג ו-PCA

נבחין כי יתכנו מקרים של בעיית סיווג במימד גבוה $\mathbb{R}^d$ כשקודם לפני נרצה להוריד את המימד למימד r נמוך יותר ורק אז להפעיל אלגוריתם מוכר.

מתי הורדת מימד באמצעות PCA יכולה לשפר את יכולת הסיווג?

- שיפור יעילות חישובית - אימון מודלים במימדים נמוכים מהיר בהרבה וצורך פחות זיכרון.
- מניעת Overfitting: ככל שיש יותר פרמטרים למודל (d גדול) כך קל לו יותר לשנן את נתוני האימון במקום ללמוד חוקיות כללית. הפחתת המימד ל- r מקטינה בצורה דרסטית את מרחב החיפוש של המסווג, ומאצלת אותו למצוא משטח הפרדה פשוט יותר וכך נוריד את הסיכון לשינון הדטה.
- הפחתת רעש - במימדים גבוהים חלק גדול מהתכונות עשוי להכיל רעש אקראי או מידע שאינו רלוונטי שעלול לבלבל את המסווג. PCA מתמקד בכיוון השונות המקסימלי, ולכן אם הרעש מתאפיין בשונות נמוכה ה-PCA יסנן אותו וישאיר את הסיגנל העיקרי.

מתי הורדת מימד באמצעות PCA עלול לפגוע ביכולת הסיווג?

- איבוד מידע רלוונטי - אלגוריתם PCA הוא בלתי מונחה - הוא מסתכל אך ורק על פיזור הנתונים ללא התגיות ומחפש כיווני שונות מקסימלית. אם המידע הקריטי שמפריד בין המחלקות נמצא דווקא בכיוון בעל שונות נמוכה - שנזרק ע"י ה-PCA, יכולת ההפרדה של המסווג תהרס לחלוטין.
- PCA לא מתאים לבעיות לא לינאריות - PCA מבצע העתקה לינארית (סיבוב והטלה). אם המבנה הגאומטרי שמפריד בין המחלקות מורכב ואינו לינארי אזי הטלה לינארית פשוטה עלולה לערבב את המחלקות זו בזו.

10 הרצאה 10

10.1 Self-supervised learning

מתווה למידה של למידה מונחית (תוויות) - בפרט סוג הלמידה הנ"ל אחראי למהפכה של למידת מכונה. הדבר שאפשר לצאת מדטה-סט קטנים וללמוד בסקאלה אדירה - הוא מה שנלמד כעת.

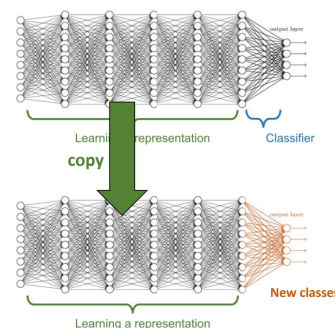
למידה מונחית עובדת טוב כשיש לנו מספיק דטה. מה הבעיה? ברוב הבעיות עשוי להיות די קשה ואף יקר להשיג דטה מתוויג. למשל יתכן שעבור תגיות מסוימות נצטרך לשכור מומחים. למשל בתחומי הרפואה - נצטרך הרבה דגימות שייצרו לנו רופאים. זה כמובן יקר מאוד. כמו כן, נניח ונרצה לסווג בעיות נדירות - לא רק נפוצות: עבור בעיות נדירות בכלל אין דרך להשיג הרבה דטה. כמו כן - אסיפת דטה מהרשת עלולה להיות שגויה: תוויות מוטות. אוקיי - אז איך אפשר ללמוד כשיש פחות דטה?

ניתן להסתכל על למידה עמוקה (רשת מלאת קשרים) בתור משהו שעושה שני דברים: ראשית, הוא לומד ייצוג, ושנית: הוא מבצע סיווג. הייצוג

אכן מוביל לסיווג, אך הייצוג עוזר להבין עוד על הדטה.

Transfer learning: נקח רשת עמוקה שיש לה רכיב ארוך שלומד ייצוג ובסוף מסווג.

הרעיון די פשוט - אנחנו נקח את הרשת העמוקה, ונקח את השכבות שלמדו את הייצוג שכבר נבנו לנו במודל העליון. אבל: את החלק של הסיווג - אנחנו נלמד כעת. נבחין שאכן הורדנו את כמות הדטה שנצרך במקרה זה.



הרעיון של Self-supervised learning הוא להשתמש בדטה לא מתוייג, אבל בשיטות של למידה מונחית. כלומר, אותם שיטות ללמידה מונחית שראינו שעובדות, נשתמש בהם לדטה שאינו מתוייג. אנחנו נשתמש בחלק מהדטה עצמו על מנת לייצר את התוויות. **הערה.** שאר נושא זה בהרצאה היו בעיקר דוגמאות ויזואליות - של תמונות וכו'. כדאי פשוט להסתכל בחומר ההרצאה. לא היה ממש עוד חומר תאורטי.

10.2 תרגול- Autoencoders

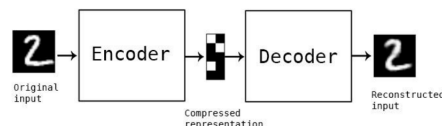
למה אנחנו זקוקים ל-Autoencoders? נתונים מהעולם האמיתי לרוב הם ממימדים גבוהים וכוללים יתירות (מידע מיותר או שחזור על עצמו). כדי להתמודד עם מורכבות זו, השאיפה שלנו להגיע ל:

1. ייצוג הנתונים בצורה מצומצמת יותר.
2. חסינות לרעש - היכולת לסנן רעשים לא רלוונטיים בתוך הנתונים.
3. מאפיינים למשימות המשך - הפקת תכונות שיהיו שימושיות לשלבים מאוחרים יותר בתהליך הלמידה.

מה זה בכלל Autoencoders? מדובר במודלים שמבוססים על רשתות נוירונים, המודלים הללו לומדים לייצוגים טובים של הנתונים על ידי תהליך דו שלבי:

1. קידוד: המרת הנתונים לייצוג פנימי
2. שחזור: נסיון לשחזר את הקלט המקורי מתוך אותו ייצוג פנימי דחוס.

המטרה הסופית היא שהעיצוב של מודלים אלו מכוון לכך שהייצוגים שיווצרו יחזיקו בתכונות רצויות כמו דחיסת המידע למימד קטן יותר. בלמידה מונחית רגילה, צריך מישור שגיד למודל מהי התווית. כאן אנחנו בלמידה לא מונחית - אין תוויות. אנחנו דוחסים את הדטה כך שלאחר הדחיסה הוא ישמור את הפרטים (הפיצ'רים) הכי חשובים של הדטה. כיצד לומדים אז בלי תווית? התווית היא הנתונים עצמם. התהליך פשוט - נדגים על הדוגמה שבתמונה: נכנס דטה גולמי (תמונה של '2'). המקודד דחס את התמונה לייצוג פנימי קטן - מכיל את תמצית המידע. המפענח מנסה לשחזר את הקלט המקורי על בסיס התמצית שקיבל. אם המודל הוציא פלט שדומה לקלט: הוא הצליח.



אם ככה - למה שהמודל פשוט לא יעתיק את הקלט לפלט? במקרה זה הוא היה מקבל שגיאת שחזור של אפס. אבל - זה רחוק ממושלם. הוא לא היה לומד שום דבר על התוכן של הנתונים: רק משנן אותם.

ה-Encoder מבצע פעולת מיפוי (mapping) של הקלט אל עבר קוד לטנטי המכונה z (הכוונה ב z הוא רשימת המספרים שיצאו לאחר הקידוד של הקלט המקורי - לטנטי הכוונה חבויה: למה חבויה? כי הנתונים האלו לא נמצאים בהכרח בתמונה הגולמית שראינו בתחלה. הם חבויים בתוך המודל ומייצגים את תמצית כל תמונה).

Bottleneck: אילוץ צוואר הבקבוק - מוגדר כשימוש במס' מימדים קטן יותר מאשר במס' המימדים בקלט המקורי. נבחין שבמקרה ובו מרחב הקידוד אינו מוגבל וגודלו שווה לגודל הקלט, המודל אינו נדרש לבצע למידה מהותית. במצב זה הוא לומד את פונקציית הזהות ולא לומד דבר על הנתונים. לכן אילוץ Bottleneck מכריח את המודל לבצע שני דברים - 1. לשמור רק מידע חיוני מהקלט. 2. לבצע הסרה של יתירות מהקלט.

מה עושה קוד לטנטי (Latent code) איכותי? כלומר, מה הקריטריונים לאיכות הייצוג הפנימי שנוצר?

- להיות קומפקטי, מצומצם במימדים אך בו זמנית להכיל מספיק מידע בשביל שהמפענח יוכל לבצע שחזור מדויק של הקלט המקורי.
- היכולת של הייצוג להיות מוצלח גם עבור דוגמאות שהמודל לא ראה מעולם בזמן האימון, מה שמעיד שהוא יודע לעשות הכללה.
- הוא לומד את המבנה של הנתונים ומתעלם מרעשים.

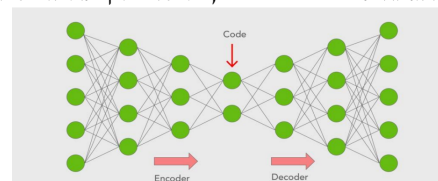
אם כן, למה שלא נשתמש ב-PCA פשוט? ובכן יש כמה הבדלים.

- Autoencoders מסוגלים למדל פונקציות לא לינאריות (מורכבות), הם מזכירים את PCA רק אם מגבילים אותם לשכבה חבויה אחת. במקרה כזה תתקבל תוצאה דומה ל- PCA בעוד, PCA מוגבל לפונקציות לינאריות בלבד.
- Autoencoders דורשים זמן חישוב רב יותר כיוון שהם מבוססים על רשתות נירונים, בעוד PCA מהיר וזול מבחינה חישובית בהשוואה.
- Autoencoders נוטים לאוברפיטנינג, לכן לרוב צריך להשתמש בגוליציה. בעוד PCA נוטה ל-Underfitting - לרוב לא מצליח לייצג מספיק טוב נתונים מורכבים.

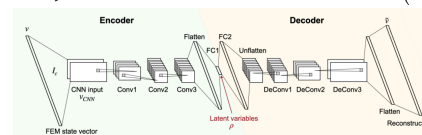
10.2.1 סוגים של Autoencoders

1. Convolutional: משתמש בשכבת קונבולוציה ושכבת Pooling כדי ללכוד היררכיות מרחביות - כלומר להבין קשרים בין פיקסלים בתמונה. שימוש אופטימלי - עיבוד תמונה.
2. Transformer - משתמש במנגנון של "תשומת לב עצמית" - מאפשר לשקול את החשיבות של כל חלקי הקלט בו זמנית, ללא תלות במיקומם היחסי. שימוש אופטימלי - בעיבוד שפות טבעיות (NLP)
3. Variational - במקום למפות קלט לנקודה בודדת, הוא ממפה את הקלטים להתפלגות הסתברותית שנלמדת.
4. Denoising - מנקה רעשים. המודל מקבל קלט שעבר הוספת רעש באופן מלאכותי ומתאמן לשחזר ממנו את הנתונים המקוריים והנקיים.

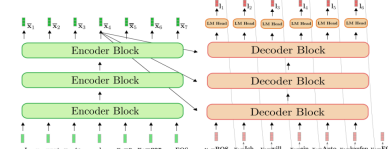
להלן דוגמה ל-Autoencoders, באשר ניתן לראות שמשמאל דוחסים את המידע, מקבלים את הקוד הלטנטי, ואז משחזרים אותו מימין.



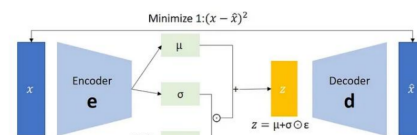
Convolutional Autoencoders (CAE): יעיל יותר מ- MLP , משמר קשרים מקומיים.



Transformer Autoencoders: יתרונות המודל הוא שבגלל המנגנון המיוחד שלו, אין צורך לעבד מילה מילה לפי הסדר אלא אפשר להזין משפט שלם למודל בבת אחת מה שמאץ את זמן הלמידה. הטרינספורמר מצטיין ביצירת קשרים בין מילים רחוקות. עם זאת, כמות הזיכרון וכוח החישוב הנדרשים גדלים בריבוע ביחס לאורך הרצף מה שהופך אותם למודלים כבדים.



Variational Autoencoders: Encoders לא מוציא וקטור בודד z אלא שני פרמטרים σ, μ של התפלגות נורמלית. רוצים לאפשר למודל להתאמן באמצעות פעופע לאחר, אך אנחנו לא דוגמים ישירות מתוך התפלגות. במקום זאת אנו לוקחים רעש אקראי מתוך ההתפלגות הנורמלית $N(0, I)$ ומחשבים את z לפי הנוסחה $z = \mu + \sigma \odot \epsilon$. המפענח מקבל את z הנ"ל ומנסה לשחזר ממנו את הקלט המקורי.



Denoising Autoencoders: איך עובד המודל? לוקחים את הקלט המקורי x ומשחיתים אותו בכוונה ע"י הוספת רעש אקראי. התוצאה היא הקלט המלוכלך $\tilde{x}$. הקלט המלוכלך נכנס למקודד (g_φ) שעובר דרך צוואר בקבוק וקוד לטנטי ומשם למפענח. למה זה טוב? כשהמודל מקבל קלט מלוכלך הוא לא יכול פשוט לשנן את הפיקסלים כי הרעש משתנה בכל פעם, הוא חייב ללמוד את המבנה האמיתי של הנתונים כדי להצליח לשחזר את התמונה הנקייה. זה מאלץ אותו להבין את המהות של האובייקט. אם כן ישנו טריידאוף: אם נוסף מעט מדי רעש, המודל לא ילמד מספיק טוב ואם נוסף יותר מדי רעש המודל יאבד את המידע המקורי ולא יצליח לשחזר אותו. אם נשתמש במפענח רגיל ללא אילוצים, המודל עשוי ללמוד פונקציית זהות פשוטה. לכן אם נשבש בכוונה את הקלט, הוא לא יוכל לעשות זאת. על מנת להצליח לשחזר נכון את הנתונים המודל חייב לעבור טרנספורמציה פנימית: הוא נאלץ להבין את חוקיות הנתונים ולומד לזהות מידע חשוב ורעש מיותר.

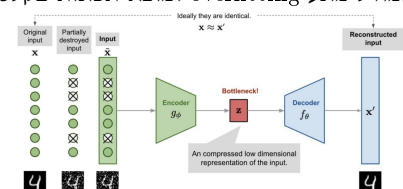
ישנן מס' שיטות נפוצות להוספת הרעש:

- הוספת רעש לבן (ערכים אקראיים לכל פיקסל)

- איפוס אקראי של פיקסלים
 - הסתרת חלקים מהקלט / טשטוש מכוון
 אם נרצה קצת לפרמל זאת מתמטית, המקודד יסומן ב $f(c(x))$ כאשר $c(x)$ היא פונקציית ההשחתה. המפענח מסומן ב $g(a)$. המטרה היא למזער את השגיאה הריבועית בין הקלט המקורי לשחזור:

$$\mathbb{E}_{x \sim \mathcal{D}} \|x - \hat{x}\|_2^2$$

המודל לומד ע"י עדכון המשקולות של המקודד והמפענח באמצעות SGD .
 המודל מונע overfitting ומוצא תכונות בקלט.



10.3 Diffusion Models

הרעיון של Diffusion Models הוא להוסיף לתמונה באופן מכוון רעש אקראי - באופן הדרגתי. כפי שניתן לראות שקורה לתמונת החתול מטה. מדוע שנעשה זאת? אנחנו רוצים לאמן את המודל לעשות בדיוק את הכיוון ההפוך - בהינתן תמונה עם רעש, לשחזר את המקור. ה Forward process הוא התהליך של מעבר מלמטה (תמונה מטושטשת) למעלה - התמונה המקורית. Deep Model הוא התהליך של מעבר מטה: ה"תלמיד". הוא מאומן ע"י השוואה של הניבוי שלו לבין המצב האמיתי של התמונה בשלבים השונים. מכניסים לו גם טקסט יחד עם התמונה. כעת נפרמל את הרעיון. בהינתן x^0 הנתון המקורי, יהי הרעש הסופי לאחר T צעדים.



התהליך הקדמי מסומן ב $q(x^{t+1}|x^t)$ - תהליך הסתברותי בו עוברים מצעד x^t לצעד x^{t+1} . התהליך האחורי הוא מסומן ב $p_\theta(x^t|x^{t+1})$ - θ זה הפרמטרים שהמודל שלנו לומד. נסמן את התפלגות הצעד הבא כך:

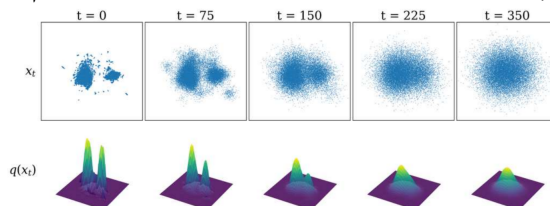
$$q(x^{t+1}|x^t) \sim \mathcal{N}(\sqrt{1 - \beta_t}x^t, \beta_t I)$$

כלומר התפלגות גאוסיאנית. β_t הם הערכים הקבועים מראש שקובעים לכל שלב $1 \leq t \leq T$ כמה רעש נוסף בכל צעד. על מנת לא לחשב בכל פעם צעד-צעד, נטען כי מתקיים:

$$q(x^T|x^0) = \mathcal{N}(\sqrt{\alpha_T}x^0, \sqrt{1 - \alpha_T}I) \text{ s.t. } \alpha_t = \prod_{s=1}^t (1 - \beta_s)$$

כלומר במקום לבצע את כל פעולות הללו T פעמים, ניתן לבטא את המצב של תמונה בכל זמן t ישירות כפונקציה של התמונה המקורית x^0 והרעש, ע"י הכפלת כל $(1 - \beta_s)$ שהצטברו עד אותו שלב. בתמונה ניתן לראות המחשה של התהליך. כאשר $t = 0$ יש לנו 2 קבוצות ברורות וכלל t עולה, הן מתפזרות באופן אחיד עד שנראה שהמידע המקורי אבד לגמרי. $q(x_t)$ עובד בדומה - בתחילה יש לנו 2 גבהות (התפלגות נורמלית), וכלל t עולה מקבלים גבעה אחת מרכזית - זה מעיד על כך שהמידע המקורי אבד והתפזר כרעש. בסיסם התהליך מקבלים גאוסיאן - זו המטרה של התהליך, לקחת תמונה ולהפוך אותה להתפלגות פשוטה שממנה נוכל להתחיל את התהליך ההפוך - תהליך היצירה.

נעיר כי זו למידה לא מונחית - בלמידה מונחית המודל מקבל תוויות, כאן אין תוויות. המודל לומד לבדו.



מהות התהליך היא למידה של רשת עמוקה - שתפקידה לחזות את הרעש מתוך מצב נתון של תמונה במצב x^t . הרשת העמוקה עובדת בכיוון ההפוך - מלמטה למעלה: תפקידה היא לזהות בכל שלב ברשת איזה רעש התווסף וללמוד אותו. מדוע זה חכם? אם המודל יודע איזה רעש נוסף בכל שלב מסוים - הוא יכול פשוט להסיר את אותו הרעש ולהתקרב בכל שלב לשחזור הנתונים האמיתיים. כעת נציג את האלגוריתם שמבצע כל מה שדיברנו עליו.

• שלב האימון: אנו רוצים לאמן רשת ניורונים ϵ_θ שתדע לחזות את הרעש שהוספנו לתמונה. אנו בוחרים תמונה מקורית $x_0 \sim q(x_0)$, דוגמים זמן צעד אקראי $t \sim U[1, T]$ ומייצרים רעש גאוסיאני $\epsilon \sim \mathcal{N}(0, 1)$. בשורה 5 - אנו מחשבים את המרחק הריבועי (MSE) בין הרעש האמיתי לבין הרעש שהמודל שלנו חזה. אנו מנסים למזער הפרש זה ולכן לוקחים גרדיאנט:

$$\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$$

עושים תהליך זה באופן איטרטיבי עד להתכנסות.

Algorithm 7 Training

```

repeat
   $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
   $t \sim \text{Uniform}(\{1, \dots, T\})$ 
   $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
  Take gradient descent step on
   $\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t)\|^2$ 
until converged
```

• שלב 2: תהליך היצירה. כאן משתמשים במודל המאומן על מנת ליצור תמונה חדשה מאפס. מתחילים עם רעש טהור לחלוטין $x_T \sim \mathcal{N}(0, I)$ ומבצעים לכל $1 \leq t \leq T$ את הצעד ההפוך. מחשבים את הצעד אחורה באמצעות הנוסחה:

$$z_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

באשר:

- $\epsilon_\theta(x_t, t)$: ניחוש המודל לפי רשת הניורונים שבנינו.
- α : הם היפר פרמטרים שנקבעו מראש, קובעים כמה חזק כל רעש.
- z הוא רעש אקראי שאנו מוסיפים על מנת להכניס אקראיות ונורמליות.
- σ_t מקדם שקובע כמה רעש אקראי יש להוסיף חזרה בצעד הנוכחי.
- x_0 : למעשה חוזרים על תהליך זה עד שמגיעים חזרה לתמונה המקורית - x_0 .

Algorithm 8 Sampling

```

 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
for  $t = T, \dots, 1$  do
   $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
   $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
end for
return  $\mathbf{x}_0$ 
```

DropOut 10.4

מדובר בטכניקה להפחתת Overfitting. ברשתות שסובלות מ-Overfitting, ניורונים עשויים לפתח תלות הדדית. כלומר, כל ניורון סומך על כך שאחרים יתקנו את שגיאותיו. תהליך זה נקרא Co-adaption, והוא פוגע ביכולת ההכללה של המודל על דטה חדש.

DropOut שובר את התלות הזו:

- במהלך האימון, ניורונים מושבתים בצורה אקראית ולכן כל ניורון חייב ללמוד לתפקד סכצמו, ללא הסתמכות על אחרים. כך, המודל לומד ייצוגים כלליים ועמידים יותר מה שמשפר את יכולת ההכללה שלו להתמודד עם דגימות חדשות.

- אימון: בכל איטרציה, כל ניורון "נכבה" (מושבת) בהסתברות p .

לכל שכבה של ניורונים יכולה להיות הסתברות p_t לכיבוי שונה.

11 הרצאה 11

11.1 Word Embeddings

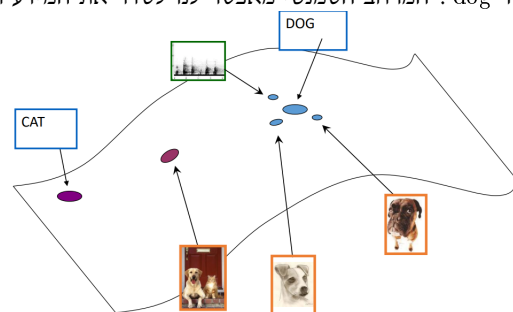
מודלים של למידת מכונה פועלים לרוב על וקטורים או טנסורים - קלט מספרי רציף. טקסט גולמי (משפטים פשוטים) אינם כאלו. בעבר, בתחום ה-NLP (עיבוד שפה אנושית) מילים נחשבו כיחידות אטומיות. כלומר מילה נחשבה ליחידה עצמאית של מידע, ללא קשר לוגי או מתמטי למילים אחרות. האתגר האמיתי הוא לא רק כיצד לייצג מילה - אלא איך נצליח לקודד את המשמעות שעומדת מאחורי המילה למבנה מספרי. ישנן כמה דרכים לעשות זאת:

1. ייצוג בו כל מילה מקבלת וקטור עם '1' במיקום 1 ו'0' בכל השאר: אין כאן שום מידע ומשמעות על דמיון בין מילים. בשלב הראשון אנו בונים אוצר מילים V שמכילה את כל המילים הידועות שבהן המודל ישתמש. לכל מילה באוצר המילים מצמידים מס' סידורי (אינדקס) למשל: $'cat' \rightarrow 12, 'dog' \rightarrow 123$. כל מילה מיוצגת ע"י וקטור במרחב $\mathbb{R}^{|V|}$ (גודל הוקטור הוא כגודל מס' המילים) - הוקטור מכיל אפסים בכל המקומות פרט ל1 שמופיעה במיקום המתאים של האינדקס של אותה מילה. לכל זוג מילים w_1, w_2 שונות הוקטורים v_{w_1}, v_{w_2} הם אורתוגונליים.

אז למה לא להשתמש בזה? ובכן אי אפשר למדוד דמיון בין מילים, וכן: קללת המימד: גודל אוצר המילים יכול לקיים $|V| = 10^6$ וכו', נקבל וקטורים עצומים עם המון אפסים 11 בודד. נקבל קלט דליל מאוד שלא יעיל לחישובים. כמו כן, לכל זוג מילים שונות $v_{w_1}^T v_{w_2} = 0$ ולכן המודל לא יכול לחזות דמיון בין זוג מילים.

2. ייצוג מילים על בסיס מילים אחרות שמופיעות לידן לעיתים קרובות:

מרחב סמנטי (Semantic spaces): במקום להתייחס למילים כסמלים בדידים, ממפים אותם לנקודות במרחב הגאומטרי. הרעיון הוא שדברים בעלי משמעות דומה יתקבצו קרוב זה לזה באותו אזור במרחב. למשל בתמונה מטה נראה שייצוג של כלבים שונים מגיעים לאותו איזור של המילה 'dog'. המרחב הסמנטי מאפשר לנו לסדר את המידע ולקבצו.



ההנחה הבסיסית היא כי משמעות של מילה נגזרת מהמילים שמופיעים מסביבה לעיתים קרובות. כדי לממש זאת אנו מגדירים חלון בגודל קבוע, אם מילה w מופיעה בטקסט ההקשר שלה הוא קבוצת המילים שמופיעים סביבה בתוך חלון זה.

לדוגמה, נתבונן על 4 המשפטים לעיל. נסתכל על חלון בגודל של 1. נתבונן על המילה 'cat'. במשפט הראשון היא מופיעה לצד 'the' ולצד 'chased'. לכן הם מקבלים 1 בטבלה. כמו כן מופיע לצד 'a' במשפט השני. סה"כ מקבלים מטריצה של דמיון של מילים לכל המילים האחרות סביב חלון בגודל קבוע (אצלנו 1).

	a	car	cat	chased	dog	passed	the	truck
a	0	1	1	1	0	1	0	0
car	1	0	0	0	0	0	1	0
cat	1	0	0	1	0	0	1	0
chased	1	0	1	0	1	0	1	0
dog	0	0	0	1	0	0	2	0
passed	1	1	0	0	0	0	1	1
the	0	1	1	1	2	1	0	2
truck	0	0	0	0	0	1	2	0

ובכן, למה גם הפתרון הזה לא טוב? כעת כן אפשר למדוד יחסית דמיון בין מילים. ובכן: הוקטורים נשארים ענקיים וגדולים מאוד ($|V|$). ניתן לנסות לצמצם את גודל הוקטורים באמצעות SVD , אך זה תהליך שדורש משאבים חישוביים יקרים מאוד. וכן: הטיה ענקית - מילים נפוצות מאוד כמו 'the', 'a' משתלטות על הספירות הגולמיות ומטביעות את הסיגנל של מילים נדירות בעלות משמעות גבוהה יותר. לכן, במקום לספור: נרצה לבצע למידה - ללמוד את הוקטורים הדחוסים הצפופים באופן ישיר.

3. **Embeddings:** הגישה המודרנית והצפופה. מילה מיוצגת ע"י וקטור של מספרים ממשיים שמקודדים את המשמעות שלה במרחב הרב ממדי. כלומר, אנחנו נאפשר בצורה זו למדוד דמיון בין מילים.

Word2Vec: הפתרון המודרני לבעיה. מסגרת עבודה ללמידת וקטורים של מילים. נסביר כיצד הרעיון עובד:

אנו משתמשים ב-corpus ("גוף" טקסט) גדול שהוא רשימה ארוכה של מילים. כל מילה באוצר המילים קבועה ומיוצגת ע"י וקטור משלה. התהליך: אנו עוברים על כל מיקום בטקסט. כאשר בכל רגע נתון יש לנו:

- מילת מרכז (c)

- מילים בהקשר (o)

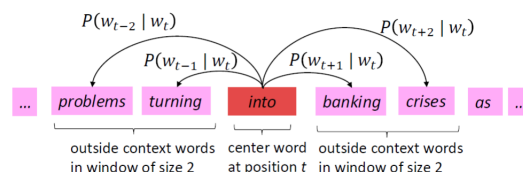
אנו משתמשים בדמיון בין הוקטורים של המילה המרכזית (c) והמילים בהקשר (o) כדי לחשב את ההסתברות $\text{Pr}[o|c]$. המטרה היא להמשיך ולכוון את הוקטורים של המילים כדי למקסם הסתברות זו. כלומר - ההסתברות שהמילים בהקשר יופיעו לצד מילת המרכז.

במילים פשוטות יותר, המודל לומד שמילים שמופיעות יחד לעיתים קרובות צריכות להיות בעלות וקטורים דומים במרחב הסמנטי, וכך הוא בונה את הייצוגים של המילים בעצמו ללא צורך בתיוג אנושי.

נתבונן בדוגמה. נניח כי מילת המרכז היא into. מילות ההקשר הן המילים שנמצאות סביב מילת המרכז בטווח חלון שנקבע מראש, כאן גודל החלון הוא 2. לכן אנו מסתכלים על המילים w_i כך $i \in \{t-2, t-1, t+1, t+2\}$. מטרת המודל היא לחשב את הסתברות מילות ההקשר בהינתן מילת המרכז:

$$\Pr[w_{t+j}|w_t]$$

כלומר, אם ראיתי את המילה into, מה הסיכוי כי תופיע לידה המילה turning, banking וכו'. זהו המנגנון שיוצר את Embeddings. מילים שמופיעות בהקשרים דומים - יקבלו וקטורים קרובים יותר במרחב הסמנטי כי המודל למד שהן מופיעות באותם הקשרים.



המטרה שלנו היא למצוא את הפרמטרים θ (שהם ערכי הוקטורים של כל המילים) שיגרמו לכך שהמודל יצליח לנבא הכי טוב את המילים שנמצאות בהקשר של כל מילה אחרת בטקסט. נקבל:

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m \wedge j \neq 0} \Pr[w_{t+j}|w_t; \theta]$$

במכפלה החיצונית אנו עוברים על כל מילה $t \in [1, T]$ בטקסט. במכפלה הפנימית עבור כל מילה מרכז w_t אנו מכפילים את ההסתברויות של כל מילות ההקשר שנמצאות בחלון בגודל m סביבה. אנו רוצים כי ההסתברות המשותפת של כל מילות ההקשר לכל המילים בטקסט תהיה מקסימלית. ע"י מעבר ל-Log-likelihood נקבל כי נרצה למזער את הנוסחה הבאה:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m \wedge j \neq 0} \log(\Pr[w_{t+j}|w_t; \theta])$$

ככל ש- $J(\theta)$ יהיה קטן יותר, המודל שלנו מאמין שהמילים שראינו בטקסט אכן שייכות להקשר זה של זה. מה שמוביל לוקטורים טובים ואמינים ומדויקים יותר.

כיצד נחשב את ההסתברות המותנית $\Pr[w_{t+j}|w_t; \theta]$?

כדי לבצע את החישוב המודל משתמש במנגנון הבא -

- לכל מילה w באוצר המילים יש שני ייצוגים וקטורים שונים - v_w משמש כאשר w היא מילת מרכז ו- u_w כאשר w משמשת כמילת הקשר.
- מדידת דמיון: משתמשים במכפלה סקלרית בין וקטור מילת הקשר לוקטור מילת המרכז כדי למדוד את מידת הדמיון ביניהם.
- על מנת להבטיח כי התוצאה תהיה התפלגות הסתברותית תקינה (סכום ההסתברויות שווה 1) משתמשים ב-Softmax:

$$\Pr[o|c] = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

המונה מייצג את ה"ציון" על הדמיון ביניהם, והמכנה מבצע את הנרמול שמבטיח כי הסכום יהיה שווה ל-1.

אימון המודל:

- כל הפרמטרים של המודל מאוגדים לוקטור אחד אורך θ .
- * לכל מילה באוצר המילים בגודל $|V|$ יש שני וקטורים (u_w, v_w) וכל וקטור באורך d . לכן הגודל הכולל של וקטור הפרמטרים θ הוא $2|V|d$.
- אתחול ואופטימיזציה:
- * מאתחלים את כל הפרמטרים ב- θ באופן אקראי.
- * המטרה למזער את פונקציית ההפסד $J(\theta)$.
- * משתמשים בגרדיאנט דיסנט ובכל שלב מבצעים:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

כאשר α הוא קצב הלמידה.

התוצר: בסיום האימון יש לנו "טבלת Look-Up". בכל שורה יש מילה ולידה וקטור באורך d שמייצג את המשמעות הסמנטית שלה. כאשר

אנו צריכים לבצע משימה (לסווג טקסט או לחפש) אנחנו לא מחשבים דבר מחדש. שולפים את הוקטור של המילים שמופיעות בטקסט שלנו מהטבלה המוכנה. המספרים הללו מוזנים כקלט למודלים אחרים (כמו רשתות ניורונים וכו'). בגלל שהוקטורים נלמדו מהקשרים - המודל החדש שמקבל אותם כבר יודע מראש שמילים דומות נמצאות קרוב אחת לשנייה במרחב. כמו שאמרנו, כבר לא מזינים למודל מילים (שהוא לא יודע איך להתמודד איתם) אלא וקטורים של מס' ממשיים שגם מייצגים הקשרים.

בעיות פרקטיות של המודל:

- במקום להשתמש ב- GD שדורש לעבור על כל הדטה על מנת לעדכן פרמטרים, נשתמש ב- SGD . העדכון נעשה בכל שלב על בסיס דגימה קטנה מהנתונים מה שהופך את האימון ליותר מהיר ויעיל.
- ייצוג המילים בסיים: לאחר שסיימנו את תהליך האופטימיזציה, אין צורך להחזיק יותר את שני הסטים של הוקטורים. ניתן לזרוק או להתעלם מהוקטורים של ההקשר u ולהשאיר רק את וקטורי המרכז כדי לייצג את המילים.
- בעיות נוספות שיש לפתור: איך מתייחסים למילים שנמצאות בתחילת או סוף משפט, שם החלון לא בגודל מלא כי חסרות מילים להקשר. בעיה נוספת - איך הופכים טקסט גולמי לרשימת מילים (האם באמת מספיק לפצל לפי רווחים, או שיש משמעות לסימני פיסוק וכדומה?). וכן מילים נדירות - מה עושים עם מילים שמופיעות לעיתים רחוקות ולא מאפשרות למידה טובה של הוקטורים שלהן?

11.2 Transformers

בבסיס כל משימת NLP (עיבוד שפות טבעיות) עומד רצף טוקונים. לכן כל שפה היא למעשה רצף. ישנן 4 סוגי משימות בהן המודל מעבד קלט רצף:

1. סיווג: קלט של רצף הופך לתוויית אחת.
 2. תיוג רצף: כל טוקן מקבל תוויית משלו (למשל "זיהוי שמות של אנשים", "ארגונים" וכו').
 3. רצף לרצף: רצף קלט מתורגם לרצף פלט חדש.
 4. מודלים גנרטיביים: רצף קלט (Prompt) משמש ליצירת רצף פלט חדש.
- למרות שהמשימות נשמעות שונות, השפה היא מיסודה רצף של מילים ומשמעות המילים תלויה במילים הסובבות אותן ובסדר שלהן. Transformers תוכננו כדי למדל את התלויות הללו ביעילות.
- בעוד כי Word2Vec סיפק לנו וקטורים עבור מילים בודדות, רוב המשימות החשובות ב-NLP מתבצעות על רצפים של מילים. לכן אנחנו נלמד בחלק זה של ההרצאה על ייצוגים תלויי הקשר של טקסט ועל רשתות ניורונים שמקבלות ומוציאות רצפים.
- עד כה המודלים שלמדנו הניחו כי הקלט בגודל קבוע ללא סדר, אך שפה דורשת גמישות רבה הרבה יותר. ישנם 3 אתגרים עיקריים שאריכקטורות קודמות לא יכלו להתמודד איתם: אורך משתנה (במציאות רצפים יכולים להיות קצרים מאוד או ארוכים מאוד), חשיבות הסדר (מודלים מסוגים קודמים שראינו זורקים את המידע על סדר המילים מה שמוביל לאובדן משמעות. יש הבדל גדול בין Dog bites man לבין Man bites dog), וכן תלויות לטווח הרחוק: במשפטים מורכבים, הנושא והפועל יכולים להיות רחוקים זה מזה משמעותית. בשורה התחתונה - מודל רצפים טוב חייב לתת מענה לשלושת האתגרים הללו ובו בזמן להיות כזה שניתן לאמן אותו בהיקפים גדולים.
- נחשוב על בעיה נוספת שקיימת במודל הקודם שראינו.** המילה fair יכולה להתפרש בשני אופנים שונים: הוגן, לעומת יריד. דוגמה נוספת היא bank - בנק כמסד פיננסי, או גדת נהר. מודלים כמו Word2Vec מקצים את אותו וקטור לכל הופעה של מילה, ללא קשר למשמעות שלה בהקשר. לכן אנחנו צריכים ייצוגים בהם Embedding של טוקן i תלוי בכל הרצף שמקיף אותו. ברגע שנצליח לבנות ייצוגים טובים יותר לטוקנים בתוך הקשר - משימות המשך יהפכו לקלות יותר.

היו מודלים רבים לפני שהגיעו transformers: אבל הם סבלו מבעיות רבות - של חוסר במקביליות (היה צריך לחשב את כל השלבים עד שלב t בשביל לחשב את שלב $t+1$) וסבלו מגרדיאנטים נעלמים או גדלים בקצב אקספוננציאלי. מודל ה Transformers הגיע לעולם בשנת 2017. תוך 5 שנים פרץ ומהווה כיום כשמיש ביותר בתחום. הוא לא הראשון שניסה לחבר רשתות ניורונים לרצף מילים, אך הראשון שהצליח.

Encoder-Decoder: האריכקטורה הבסיסית ביותר למשימות של תרגום מכונה.

Encoder תפקידו לקחת את רצף טוקוני הקלט $(x_1, \dots, x_n)$ דיסקרטיים (למשל: "I love Machine Learning") ולהפוך אותם לרצף של וקטורים רציפים ותלויי הקשר $z_1, \dots, z_n$. כל וקטור חדש z_i במרחב החדש מייצג את המילה שלו אך כבר מבין את ההקשר של המילה לשאר המילים במשפט. ההבדל כאן מ-Word2Vec היא ששם קיבלנו "מילון" של מילים, וכאן כל מילה תקבל וקטור דינמי שמתאים להקשר שלה באותו המשפט.

Decoder תפקידו לייצג את רצף הפלט המבוקש $(y_1, \dots, y_m)$ (למשל: המילים בעברית - אני אוהב למידת מכונה). Decoder מייצר את המילים אחת אחת. כדי לנבא את הטוקן y_i הוא מסתכל בו זמנית על 2 דברים: על הוקטורים הממופים מה-Encoder $(\{z_i\}_{i=1}^n)$ ועל כל הטוקונים שהוא עצמו הספיק לייצר בשלבים הקודמים $(\{y_j\}_{j=1}^{i-1})$.



11.2.1 Tokenization: המעבר מקלט טוקונים לוקטורים - קלט ל-Encoder

לפני שהמודל יכול לבצע פעולות מורכבות של Attention הוא חייב להפוך את הטקסט האנושי לשפה שהוא מבין - וקטורים. כיוון שה-Transformer הוא מודל שיעבד את כל המילים ברצף בו זמנית, הוא לא יודע מטבעו מהו סדר המילים. לכן מיד אחרי Embedding מתווסף מידע שנקרא

Positional Encoding - זהו מעין תיוג לכל וקטור על מנת שידע היכן הוא ממוקם במשפט. על מנת להבין מדוע זה עובד (טוקניזציה) יש להבין מדוע כל השיטות הישנות נכשלו:

- בעבר, חילקו טקסטים למילים פשוטות ע"י רווחים וסימני פיסוק. זה נראה פשוט אך בלמידה עמוקה מודרנית זה יצר 4 כשלים חמורים.
- אם המודל נתקל במילה שלא הופיע בקבוצת האימון, הוא לא יודע מה לעשות איתה. הוא מסמן אותה בתווית אחת קבועה UNK והמשמעות: המודל מאבד לחלוטין כל מידע סמנטי שהיה אמור להפיק ממילה זו.
- המערכות הישנות התייחסו למילים כאל אטומים נפרדים בלתי תלויים. אם יש לנו את המילים Run, runner, running המודל רואה אותם כמילים נפרדות ללא קשר ביניהם. הוא לא מבין שכולן מבוססות על אותו שורש.
- כדי לכסות את כל השפה נדרש אוצר מילים של מיליוני מילים. זה יוצר מטרצת Embedding ענקית מדי. הבעיה היא שבמטריצה זו מילים נדירות כמעט ולא מופיעות באימון מה שאומר שהוקטורים שלהן כמעט ולא מתעדכנים. התוצאה היא אימון איטי מאוד וחוסר יכולת של המודל להתכנס לתוצאות טובות.
- שיטת הפיצול לפי רווחים נכשלה כשהיא פוגשת מידע שלא נראה כמו משפט רגיל - היא מתייחסת לכתובות אינטרנט ולקטעי קוד ולסימון מתמטי ומדעי כמחרוזות טקסט ארוכות ומזירות במקום להבין שהם בנויים מתווים עם מבנה פנימי.
- לכן הפתרון הוא, במקום לנסות לעבוד עם מילים שלמות המודל מפרק מילים לחלקי מילים קטנים (תתי מילים). כך הוא יכול לבנות מילים חדשות מתוך חלקים מוכרים ובכך הוא פותר את רוב הבעיות שצוינו למעלה.

Sub-word tokenization: אנו מחפשים את "דרך האמצע" - תווים בודדים הם יותר מדי מפורטים מה שגורם לרצפים להיות ארוכים מאוד וקשים לעיבוד. מילים שלמות - נוקשות מדי מה שגורם לבעיות שדיברנו עליהן לעיל. הפתרון: לימור אוצר מילים שמורכב מחלקי מילים. אלגוריתם ה-BPE הוא האלגוריתם שבו השתמש ה-Transformer המקורי (2017): האלגוריתם מתחיל עם רשימת כל התווים הבודדים בשפה, ובאופן איטרטיבי הוא מחפש את זוג התווים הסמוך שמופיע בתדירות הגבוהה ביותר בטקסט ומאחד אותם לסימול אחד חדש. על פעולה זו חוזרים שוב ושוב. התוצאה: מילים נפוצות כמו the נשארות כטוקן אחד שלהם, כי הן מופיעות המון. מילים נדירות כמו unhappy מפורקות ליחידות משמעותיות קטנות יותר - היא תפרק ל-{un, happy}. מדוע זה כל כך מצליח? לעולם לא נתקל במילה שהמודל לא מכיר - כי אם הוא לא מכיר מילה שלמה הוא יכול לבנות אותה מהתווים הבסיסיים שלה. וכן: אוצר המילים מצטמצם בצורה משמעותית - במקום לנהל רשימה של 200,000 מילים המודל משתמש באוצר מילים של למשל לכל היותר 50,000 טוקונים.

באופן פורמלי, בהיתן קלט מחולק לטוקונים כבר ("I love Machine Learning") משתמשים ב-Token embedding lookup שמתרגמת כל טוקן לייצוג מספרי, התוצאה של התרגום היא שכל טוקן הופך לוקטור באורך d ($x_i \in \mathbb{R}^d$). אחרי שהטוקונים הפכו לוקטורים x_1, x_2, x_3, x_4 מעבירים אותם כקלט לתוך Encoder - זו השכבה הראשונה של Transformer.

חשוב לומר כי ה-Embeddings (הייצוגים הוקטוריים של הטוקונים) הם לא משהו ש"מביאים מבחוץ" אלא המודל לומד בעצמו. מדוע שלא נשתמש בוקטורים שמוכנים מבחוץ? (נעיר: הכוונה ב"מוכנים מבחוץ" היא להשתמש ב-Word2Vec למשל - ואז את הפלט של המודל להביא כקלט למודל זה. כלומר שימוש ב-2 מודלים בו זמנית). ובכן, מכמה סיבות: הראשונה היא שוקטורים מוכנים מראש בנויים ברמת המילה השלמה ואילו אנחנו בשיטה כמו BPE מעוניינים להשתמש בתתי מילים שעבורן פשוט אין וקטורים מקבילים מוכנים מראש. כמו כן, מרחב וקטורים קבוע עשוי להיות מותאם למדידת דמיון בין מילים אך ה-Transformer עשוי להזדקק למרחב וקטורי שמתאים ספציפית למשימת הניבוי שהוא צריך לבצע.

11.2.2 Self-Attention - הוקטורים שיצאו מהטוקניזציה נכנסים למנגנון זה.

נסה להבין את מנגנון ה-Self-Attention באמצעות קצת אינטואיציה. נחשוב על חיפוש רגיל בזיכרון המחשב. ישנה שאילתא q ואנו מחפשים מפתח k_i שמתאים לה במדויק. ניתן לחשוב על כך כפונקציית אינדיקטור, $I(q = k_i)$. אם ישנה התאמה, נקבל את הערך v_i המשוך למפתח k_i ואם לא, לא נקבל כלום. זהו מצב של הכל או כלום: או שהמפתח תואם, או שלא. ב-Transformer אנחנו לא מסתפקים בהתאמה מדויקת. אלא רוצים לדעת כמה כל מילה רלוונטית לכל מילה אחרת. במקום לבחור מפתח אחד, המנגנון מחשב משקל לכל המפתחות באמצעות פונקציית Softmax על הדמיון בין q ל- k_i . כך:

$$w_i = \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}}$$

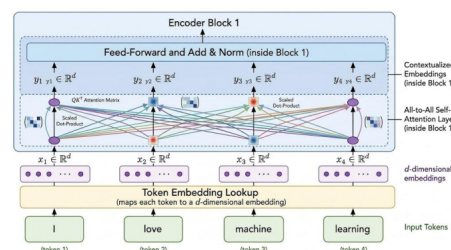
התוצאה הסופית - ממוצע משוקלל על כל הערכים v_i כאשר משקל כל ערך נקבע לפי מידת הדמיון בין השאילתא למפתחו:

$$\text{Result} = \sum_{i=1}^n w_i v_i$$

כלומר - במקום לחפש מילה אחת שמתאימה, ה-Self attention מאפשר למודל להסתכל על כל המילים במשפט בו זמנית ולתת לכל אחת מהן משקל (חשיבות) שונה ולבנות ייצוג חדש ומתוחכם יותר לכל מילה על בסיס הקשר שלה לשאר המילים.

אם נסכם: התהליך כזה - משפט $\Leftarrow$ טוקניזציה (הפרדת המשפט לטוקונים) $\Leftarrow$ LookUp - הטוקונים הופכים לוקטורים במיד d . זה Embedding (וקטורים מספריים בלבד, עם הייצוג של מילה fair למשל) $\Leftarrow$ וקטורים עוברים בשכבת Attention ויוצאים וקטורים עשירים יותר עם מידע על המילים שמשביב. יתכן גם כי הוקטורים יעברו בשכבות נוספות לפני שיצאו מה-Encoder סופית. שכבות עבור ייצוג הלמידה וכו' עליהן מיד נרחיב.

דוגמה שממחישה טוב את כל התהליך:



כעת נרצה לפרמל את הרעיון. במקום להשתמש בטרנספורמציה קבועה Self attention מאפשר לכל טוקן להתייחס באופן דינמי לטוקנים אחרים ברצף, לפי מידת הרלוונטיות שלהם. על מנת לבצע זאת אנו לוקחים כל וקטור קלט x_i ומטילים אותו לשלושה מרחבים וקטוריים נפרדים בעזרת מטריצות משקולות שנלמדות במהלך האימון - W^Q, W^K, W^V . עבור כל טוקן המודל מייצג שלושה וקטורים שונים:

1. q_i - Query: "מה אני מחפש?". זהו הוקטור שמשמש כדי לשאול שאלות על שאר הטוקנים.
 2. k_i - Key: "מה אני מציע לאחרים?". זהו הוקטור שמשמש כדי להיצא ע"י שאלות של טוקנים אחרים.
 3. v_i - Value: "התוכן שאני מכיל". המידע שבפועל יועבר הלאה אם תמצא התאמה בין Query של טוקן אחד לkey של טוקן אחר.
- המודל מקבל מטריצה X בגודל $n \times d$ שבה כל שורה היא וקטור של טוקן אחד מתוך רצף של n טוקנים. המודל מכפיל את מטריצת הקלט X במטריצות משקולות נלמדות כדי לקבל את שלוש המטריצות המרכזיות:

$$Q = XW^Q, K = XW^K, V = XW^V$$

המנגנון כולו מסתכם בנוסחה הבאה:

$$\text{Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

מדוע הנוסחה נראית כך? ראשית - QK^T - הכפלת השאלות במפתחות באופן של כפל מטריצות יוצר את מטריצת ההתאמות. היא קובעת כמה כל טוקן צריך להסתכל על טוקן אחר. שנית - $\sqrt{d_k}$ הוא חלוקה בשורש של מימד המפתחות, נעשה על מנת למנוע ערכים גדולים מדי. softmax הופך את כל התוצאות להסתברויות שסכומן 1, כך שכל טוקן יודע איזה אחוז של מידע לקחת מכל טוקן אחר. לבסוף, המשקלים שחישבונו מוכפלים בערכים (V) מה שיוצר את הייצוג החדש והעשירי של כל טוקן. המטריצה Attention שיוצאת היא בגודל $n \times d$ ומייצגת את הייצוג המעודכן (ההקשרי) של כל טוקן.

11.2.3 Feed Forward Layer - שלב עיבוד לאחר הattention

שלב נוסף שנמצא מיד לאחר הSelf-Attention בתוך בלוק Encoder. פועל כשלב עיבוד שתפקידו להעביר את הייצוג ההקשרי שקיבלנו מהAttention למרחב תכונות מורכב יותר. הוא מוסיף למוודד אינלינאריות שמאפשרות לו ללמוד דפוסים מורכבים יותר מרק קשרים לינאריים. השכבה מורכבת מ2 טרנספורמציות לינאריות כשביניהן יש פונקציית הפעלה מסוג $ReLU$. היא פועלת על הפלט של שכבת הattention ומבצעת:

$$FFN(x) = \max\{0, xW_1 + b_1\}W_2 + b_2$$

כלומר - אחרי ששכבת הattention אפשרה לטוקנים לדבר אחד עם השני ולהבין את ההקשר, שכבת FFN לוקחת את המידע הזה ומעבדת אותו עוד קצת באופן עצמאי עבור כל טוקן בנפרד כדי להעשיר את הייצוג שלו.

11.2.4 Add and Norm

שכבה נוספת שעובר המודל לפני שיוצא כפלט של Encoder. אלו שכבות שמאפשרות לטרנספורמר להיות כל כך עמוק (להערים בלוקים זה על גבי זה) מבלי לקרוס בתהליך הלמידה.

Add: חיבור שארית. הבעיה - ברשתות עמוקות מאוד גרדיאנטים עלולים להעלם או להתפוצץ בזמן האימון מה שמונע מהמודל ללמוד. לכן אנחנו מתחברים את הקלט x ישירות לפלט של תת השכבה (Attention או FFN). זה יוצר קיצור דרך שמאפשר לגרדיאנט לזרום בחופשיות ולמנוע את התדרדרות הביצועים.

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Layer Normalization: המנגנון מחשב את הממוצע והשונות לאורך מימד המאפיינים, עבור כל טוקן באופן עצמאי. זה מבטיח שהערכים בתוך הוקטור ישמרו על ממוצע ושונות יציבים מה ששומר על תהליך האימון חלק ומונע תופעה של שינוי פנימי בהתפלגות הנתונים בין השכבות. יחד, זוג הפעולות הללו Add and norm מופיעות בתוך כל בלוק בטרנספורמר - ומוודאות כי המידע והגרדיאנטים נשארים "בריאים" (לא מתפוצצים, נעלמים או משנים התפלגות) כשהם עוברים בין שכבות השונות.

11.2.5 בעיות נוספות בתהליך

מנגנון Self attention הוא אינוריאנטי לפרמוטציות. כלומר הוא מתייחס לקלט כאל קבוצה לא סדורה. מבלי פתרון נוסף, המודל יפיק תרגום זהה לחלוטין עבור משפטים כמו Dog bites man לבין Man bites dog. הפתרון - המודל מזריק לתוך Embeddings מידע על המיקום היחסי או המוחלט של הטוקנים. כלומר, רוצים להדביק לכל וקטור של מילה תג שם - תוספת שאומרת לו: אתה מס' 1, אתה מילה מס' 2 וכו'. בטרנספורמר המקורי, משתמשים בפונקציות סינוס וקוסינוס קבועות כדי לייצר את המידע הזה. Positional Encoding מוסף (איבר איבר) Token embeddings עוד לפני שהם נכנסים לבלוק Encodern הראשון.

$$p_i = \begin{pmatrix} \sin(i/10000^{2\cdot 1/d}) \\ \cos(i/10000^{2\cdot 1/d}) \\ \vdots \\ \sin(i/10000^{2\cdot d/2/d}) \\ \cos(i/10000^{2\cdot d/2/d}) \end{pmatrix}$$

Multi-Headed Self-Attention: במשפט פשוט למילה יש המון סוגים של קשים למילים אחרות - קשר תחבירי, קשר סמנטי (משמעות), קשר התייחסותי. מטריצה QK^T אחת פשוט לא יכולה לקודד את כל סוגי הקשרים הללו בבת אחת. לכן, במקום להשתמש בראש אחד משתמשים ב- h ראשים במקביל.

- כל ראש עצמאי, ולכל ראש יש את שלושת מטריצות המשקולות הייחודיות לו.
- חלוקת עבודה: כיוון שכל ראש מתמקד בהיבט אחד, המוכל יכול ללמוד קשרים שונים בו זמנית.
- יעילות חישובית: במקום לעבוד עם וקטורים ארוכים מאוד במימד d_{model} ניתן לחלקם ל- h חלקים קטנים יותר כך שכל ראש עובד על מימד $d_k = \frac{d_{model}}{h}$.

בסיום התהליך, הפלטים של כל הראשים משורשרים יחד על מנת ליצור ייצוג אחד עשיר ומלא. למעשה זה מומחים שונים שכל אחד מסתכל על המשפט מתחום שונה.

נסה להבין את ההבנה לגבי מנגנון Attention בדרך השלילה:

- הוא איננו קונובלוציה. בשונה מקונובלוציה, אין כאן חלון קבוע. אלא כל טוקן יכול לתת Attention (להתייחס) לכל טוקן אחר ברצף ללא תלות במרחק הפיזי ביניהם.
- הוא איננו RNN: אין כאן רקורסיה, כל המיקומים מעובדים במקביל ולא באופן סדרתי מה שמאפשר למידה יעילה יותר.
- איננו סימטרי. לא מתקיים $QK^T = KQ^T$
- חשיבות המיקום: ללא הזרקת מידע על המיקום (עם סינוס וקוסינוס) מנגנון Attention אינוריאנטי לסדר. הוא לא יודע את סדר המילים במשפט.

*כיוון שהמודל מחשב את כל זוגות האינטרקציות בין הטוקנים העלות החישובית גדלה באופן ריבועי ביחס לאורך הרצף. מטריצת Attention היא בעלת גודל של $n \times n$. אך למרות העלות הגבוהה, החישוב של המטריצה ניתן להרצה במקביל על גבי מעבד GPU.

11.2.6 סיכום

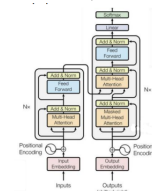
המטרה הייתה פשוטה: לעבד רצפי מידע כמו טקסט בצורה מקבילית ויעילה תוך הבנת הקשרים המורכבים בין איברי הרצף. שלבי התהליך:

1. טוקניזציה ו-Embeddings: פירוק הקלט לטוקנים והמרתם של הטוקנים לוקטורים מספריים בטבלת חיפוש.
2. הוספת מיקום: Positional Encoding: הזרקת מידע מיקומי לוקטורים באמצעות פונקציות טריגונומטריות כדי לפתור את בעיית האינווריאנטיות לסדר.
3. Encoder (עיבוד קלט): שכבות הכוללות Multi-Head attention עם חיבורי add and norm על מנת להפיק ייצוגים עשירים והקשריים, תוך שמירה על התפלגות הנתונים ומניעה של דעיכה או התפוצצות גרדיאנטים.
4. Decoder (יצירת פלט): שכבות משתמשות ב-Masked attention (לא הרחבנו על כך) ובדרכים על מנת להשתמש במידע מה-Encoder.
5. סיום: מעבר דרך שכבה לינארית ופונקציית softmax על מנת להפיק את ההתפלגות הסופית וההסתברות של הטוקנים.

בסיום שלב האימון - המודל משמש על מנת לבצע הסקה. נותנים למודל משפט קלט והדיקודר מקבל טוקן התחלה מיוחד. המודל מחשב הסתברויות ובוחר בכל שלב את הטוקן הכי סביר. בלולאה, לוקחים את הטוקן שיוצר ומשרשרים אותו לקלט הקיים ומכניסים הכל שוב למודל על מנת לייצר את הטוקן הבא. לדוגמה:

The cat sat $\rightarrow_{on}$ The cat sat on $\rightarrow_{the}$ The cat sat on the

וכן הלאה. למעשה משפט נכנס לאנדקודר והדיקודר, ויוצא לבסוף וקטור הסתברויות. לוקחים את המילה עם ההסתברות הגבוהה ביותר להיות הבאה בתור, וחוזר חלילה. זו כמובן הפשטה של הסיפור וזה הרבה יותר מורכב במציאות.



12 הרצאה 12

12.1 Biases, Fairness

מודלי בינה מלאכותית נמצאים בשימוש רבה בימינו. חשוב להיות מודעים להטיות שלהם.

- מתאם אינו סיבתי: הופעה משותפת של אירועים A ו- B לא מעידה על כך ש- A גרם ל- B . למשל: שלוליות שמופיעות יחד עם גשם אינן גורמות לגשם. למרות שההסתברות לגשם אכן עולה אם יש שלוליות. כלומר, $\Pr[Rain|Puddle] > \Pr[Rain]$.
- בדיקת סיבתיות באמצעות התערבות: על מנת לקבוע אם קשר הוא סיבתי, מבצעים התערבות. למשל יצירת שלולית באמצעות צינור מים. ובודקים את $\Pr[Rain|do(Puddle)]$.
- כמו כן, כאשר אירועים A ו- B מופיעים יחד לא בהקשר מדובר בקשר סיבתי ישיר של A ל- B או של B ל- A . יתכן כי קיים משתנה שלישי שגורם לשניהם.

כיצד זה קשור למשפט הראשון שפיתחנו בו אודות מודלי בינה מלאכותית?

מודלים עמוקים משתמשים לעיתים בתכונות בעלות יכולת ניבוי שאינן סיבתיות. למשל, בחיזוי סיכון למתן הלוואה יתכן כי מודל יזהה מתאם בין צבע עור ירוק לבין מעמק סוציו אקונומי נמוך. למרות שצבע העור אינה הסיבה לסיכון. כתוצאה משימוש בתכונות אלו, אדם בעל צבע עור "ירוק" המשתייך למעמד סוציו אקונומי גבוה, עלול לקבל יחד לא הוגן או שגוי מהמודל.

כפי שראינו כבר בעבר, ככל שמודל מתאמן על יותר נתונים הוא הופך ליותר מדויק והשגיאה יורדת באופן משמעותי. לכן ישנו קשר משמעותי בין כמות הדוגמאות שהמודל רואה לרמת הדיוק שלו. הבעיה נוצרת כי קבוצות מיעוט (בעלי צבע עור "ירוק" למשל) מיוצגות פחות בנתונים. כיוון שהמודל לא קיבל מספיק דוגמאות מאותן קבוצות, הוא לא מגיע לרמת דיוק גבוהה עבורן מה שגורם לכך שהשיגאות שלו על המיעוט יהיו גבוהות משמעותית מאשר על הרוב.

מודלים שמאומנים עם פונקציית הפסד של Cross entropy נוטים להיות בטוחים מדי בעצמם. מה זה אומר בפועל? המודל אמנם נותן הסתברות למשל 0.9 אך ההסתברות הזו לא באמת משקפת את רמת הדיוק האמיתית שלו. הוא לא אומן להגיד "אני בטוח ב-90% וגם צודק ב-90% מהמקרים". כלומר - יתכן שהמודל יהיה בטוח מאוד בעצמו, אך הדיוק שלו בפועל נמוך (לפעמים משמעותית) יותר מהבטחון שהוא מציג. זה אומר - שאיננו יכולים לסמוך באופן עיוור על ציון שנתן מודל למידת מכונה כזה או אחר. בשביל לפתור בעיה זו ישנם כמה פתרונות:

1. Histogram Binning: לחלק את התחזיות ל"סלים" ולשייך ציון מכויל לכל סל לפי רמת הדיוק הממוצעת של הדגימות בו. זוהי שיטה פשוטה, אך התוצאה היא פלט בדיד.
2. Isotonic Regression: לומדת פונקציה רציפה, עולה ומונוטונית שמתאימה את הפלט הלא מכויל למכויל.
3. Platt Scaling: למידת שני פרמטרים עבור פונקציית סיגמואיד כדי לבצע את הכויל:

$$acc = \text{sigmoid}(a \cdot y + b)$$

4. Temperature Scaling: שימוש בפרמטר סקלרי יחיד T כדי לרכך את פלט Softmax (ההסתברות). זוהי השיטה היעילה ביותר.

ישנם 3 קריטריונים להגדרת "הוגנות קבוצתית":

1. שוויון דמוגרפי - התחזית של המודל אינה תלויה בשייך הקבוצתי של הפרט, כך ששיעור התחזיות החיוביות שווה בכל הקבוצות.
2. סיכויים שווים - הן שיעור ה"חיובי האמיתי" והן שיעור ה"חיובי השגוי" זהים בכל הקבוצות.
3. כויל: עבור כל הפרטים שקיבלו ציון p , החלק היחסי שלהם שהוא חיובי בפועל שווה ל- p , ללא קשר לקבוצה שאלה הם שייכים.

בהצלחה לכולם!